



Data Science with Python 3



Prepared by:
Dr. Heba El Hadidi

2025-2026

Contents

Chapter One -Introduction to Data Science	3
Chapter Two-Data Acquisition and Processing	29
Chapter Three- Data Representation	69
Chapter Four- DATA ACQUISITION AND PROCESSING-2.....	96
Chapter Five- DATA WRANGLING COMBINING AND MERGING DATA SETS	140
Chapter Six- AGGREGATION AND GROUP OPERATIONS GROUP BY MECHANICS	178
Chapter Seven- Data Modelling	221
Chapter Eight- Data Science Applications	256
Chapter Nine- Hypothesis Testing	275
Chapter TEN - Data Transformation-Data Reduction-	288
Chapter Eleven – Statistics for Data Science	300
Chapter Twelve– Regression Analysis	349
References	364

Chapter One -Introduction to Data Science

In this course you will learn **dealing, manipulating data, visualize it, make decisions.**

You'll learn to take data:

- **Process it**
- **Visualize it**
- **Understand it**
- **Communicate it**
- **Extract value from it**

In Chapter 1:

Objectives

1.2 Data to Data Science

1.4 Foundation of Data Science

1.6 Applications of Data Science

1.8 Messy Data

1.10 Cleaning Data

1.1 Data Science-A discipline

1.3 Data Growth Issues and Challenge

1.5 Tools for Data Science

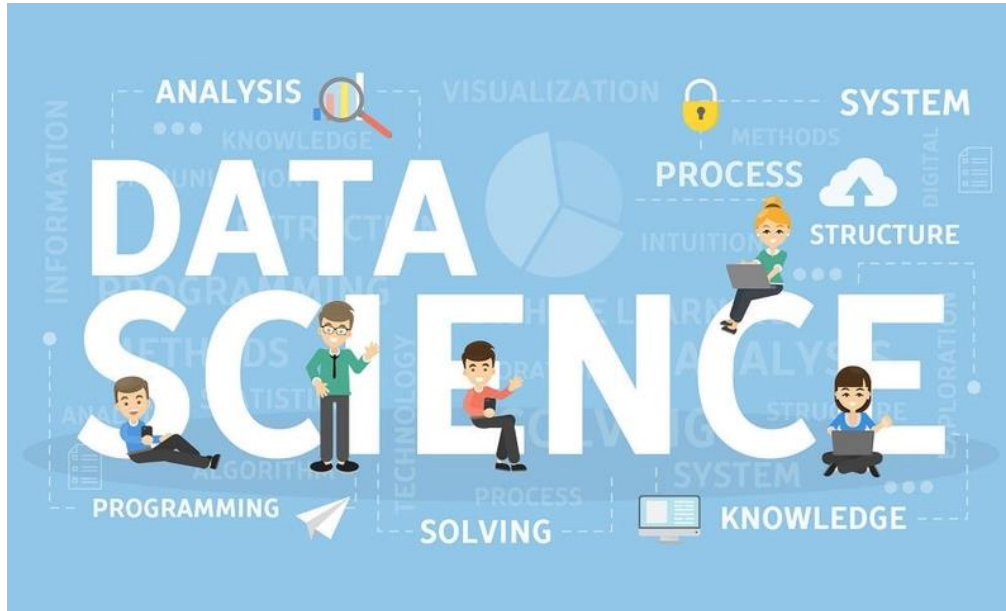
1.7 Data Science Process

1.9 Anomalies and Artefacts in Datasets

1.11 Questions

يتطور علم البيانات بشكل متسارع ليصبح واحداً من أهم المجالات في صناعة التكنولوجيا. يرجع الفضل للتقدم السريع في مجالات عدة كالاتصالات و الخدمات السحابية وكفاءة أجهزة الحاسب والذي جعل المستحيل ممكناً. من الممكن الآن تحليل مجموعات كبيرة من البيانات الضخمة والتي بدورها تمكننا و إلى حد غير مسبوق من اكتشاف حقائق و أنماط ورؤى ربما يستحيل اكتشافها بدون تحليل البيانات. مثال على هذه البيانات، سجلات زوار موقع إلكتروني بيانات المرضى.

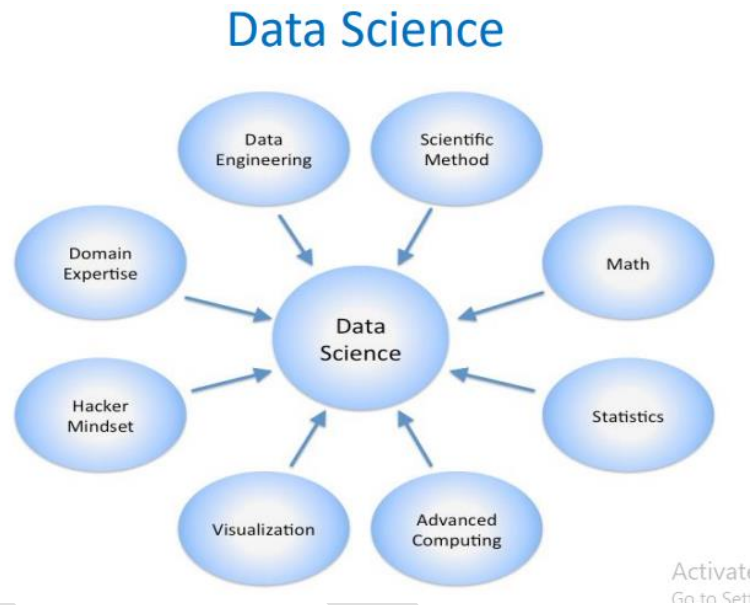
- محلل البيانات
- علماء البيانات
- مهندس البيانات
- مهندس تعلم الآلة



Essential Python Libraries: NumPy, pandas, matplotlib, SciPy, scikit-learn

Science Fields related to Data Science:

Statistics + mathematics + computer science + operations research + statistical and machine learning + data warehousing + data visualization + information science + Big data, ...



Before starting to learn Data Science, you need to learn some *terminologies*. You need to know **What is Data Science?**

*“The ability to take **data**—to be able to **understand it**, to **process it**, to **extract value from it**, to **visualize it**, to **communicate it**—that’s going to be a hugely important skill in the next decades, not only at the professional level but even at the educational level for elementary school kids, for high school kids, for college kids.”*

Data Science is

- An area that manages, manipulates, extracts, and interprets knowledge from tremendous amount of data
- Data science (DS) is a multidisciplinary field of study with goal to address the challenges in big data
- Data science principles apply to all data – big and small

Theories and techniques from many fields and disciplines are used to investigate and analyze a large amount of data to help decision makers in many industries such as science, engineering, economics, politics, finance, and education.

– Computer Science

- Pattern recognition, visualization, data warehousing, High performance computing, Databases, AI

– Mathematics

- Mathematical Modeling

– Statistics

- Statistical and Stochastic modeling, Probability.

Data All Around:

Lots of data is being collected and warehoused

– Scientific Experiments

– Internet of Things

– Web data, e-commerce

– Financial transactions, bank/credit transactions

– Online trading and purchasing

– Social Network

–many more!

We live in a world that's drowning in data.

- Sensing devices and sensor networks that can monitor everything 24/7 from temperature to pollution to vital signs

- Increasingly sophisticated smart phones

- Internet, social networks makes it easy to publish data

- Scientific experiments and simulations à astronomical data volumes

- Internet of Things

البيانات تحيط بنا:

يتم جمع الكثير من البيانات وتخزينها

- بيانات الويب ، التجارة الإلكترونية
- المعاملات المالية ، والمعاملات المصرفية / الإئتمان
- التداول والشراء عبر الإنترنت
- الشبكات الإجتماعية



How to handle that data? How to extract interesting actionable insights and scientific knowledge?

Websites track every user's every click. Your smartphone is building up a record of your location and speed every second of every day. "Quantified selfers" wear pedometers-on-steroids that are always recording their heart rates, movement habits, diet, and sleep patterns.

Smart cars collect driving habits, smart homes collect living habits, and smart marketers collect purchasing habits. The internet itself represents a huge graph of knowledge that contains (among other things) an enormous cross-referenced encyclopedia; domain-specific databases about movies, music, sports results, pinball machines, memes, and cocktails; and too many government statistics (some of them nearly true!) from too many governments to wrap your head around.

- تتعلم الشركات أسرارك وأنماط التسوق والتفضيلات الخاصة بك: على سبيل المثال ، هل يمكننا معرفة ما إذا كانت المرأة حامل ، من أنماط تسوقها
- الإعلانات المدفوعة على Facebook تستهدف مجموعات معينة .
- التصفح على Google



Data Science: Case Studies:

1- Cancer Research

Cancer is an incredibly complex disease; a single tumor can have more than **100** billion cells, and each cell can acquire mutations individually.

The disease is always changing, evolving, and adapting.

- Employ the power of big data analytics and high-performance computing.
- Leverage sophisticated pattern and machine learning algorithms to identify patterns that are potentially linked to cancer
- Huge amount of data processing and recognition

2- Health care

Healthcare is an incredibly complex issue;

3- Elections

In 2012, the Obama campaign employed dozens of data scientists who datamined and experimented their way to identifying **voters who needed extra attention**, choosing optimal donor-specific fundraising appeals and programs, and focusing get-out-the-vote efforts where they were most likely to be useful. And in 2016 the Trump campaign tested a staggering variety of online ads and analyzed the data to find what worked and what didn't.

قام مليون شخص بتنشيط تطبيق أوباما على Facebook والذي أتاح الوصول إلى المعلومات عن "الأصدقاء"

4- Internet of Things (IoT)

The Internet of Things is rapidly growing. It is predicted that more than 40 billion devices will be connected by 2025.

5- Customer Analytics

Customer Analytics is

```
users = [  
    {"id": 0, "name": "Hero" },  
    {"id": 1, "name": "Dunn" },  
    {"id": 2, "name": "Sue" },  
    {"id": 3, "name": "Chi" },  
    {"id": 4, "name": "Thor" },  
    {"id": 5, "name": "Clive" },  
    {"id": 6, "name": "Hicks" },  
    {"id": 7, "name": "Devin" },  
    {"id": 8, "name": "Kate" },  
    {"id": 9, "name": "Klein" }]  
  
friendship_pairs = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3),  
                    (3, 4), (4, 5), (5, 6), (5, 7), (6, 8), (7, 8), (8, 9)]
```

Data life cycle:

- Data collecting

تحديد مصادر البيانات المختلفة، والتي قد تشمل سجلات خادم الويب ، أو منشورات الوسائط الاجتماعية، أو البيانات من المكتبات الرقمية مثل مجموعات بيانات التعداد السكاني في الولايات المتحدة، أو البيانات التي يتم الوصول إليها من خلال المصادر الموجودة على الإنترنت عبر واجهات برمجة التطبيقات، أو المعلومات الموجودة بالفعل في جدول بيانات (Excel).

- Data Preparing/Processing

جمع بيانات مثل اختيار البيانات ذات الصلة، ودمجها عن طريق خلط مجموعات البيانات، وتنظيفها، والتعامل مع القيم المفقودة إما عن طريق إزالتها أو احتسابها مع البيانات ذات الصلة، والتعامل مع البيانات غير الصحيحة عن طريق إزالتها، وكذلك التحقق من القيم المتطرفة والتعامل معها.

- **Data Analysis**
- **Analysis hypothesis**
- **Decision**



1.0 OBJECTIVES

1. Introduction to Data Science
2. Familiarize with the issues and challenges in Data Growth
3. Familiarize with data Science Process
4. Familiarize with the concepts of data like **clean** and **messy** data, **data artifacts** and anomalies in data.

1.1 DATA SCIENCE-A DISCIPLINE

Data Science is a blend of various tools, algorithms and machine learning principles with the goal to discover hidden patterns from raw data. It is related to data mining, machine learning and Big Data.

It is an area that manages, manipulates, extracts, and interprets knowledge from tremendous amount of data.

Data science (DS) is a multidisciplinary field of study with a goal to address the challenges in big data. Big Data has given rise to Data Science and is a therefore, a multidisciplinary field of study with goal to address the challenges in big data.

علم البيانات هو مجال دراسي متعدد التخصصات يهدف إلى معالجة التحديات في البيانات الضخمة. وقد أدت البيانات الضخمة إلى ظهور علم البيانات، وبالتالي فهو مجال يهدف إلى معالجة التحديات في البيانات الضخمة.

1.2 DATA TO DATA SCIENCE

The growth of data has been seen since 2010, due to the growth in the number of data generating devices like smart phones, wearables, Internet of things, etc. The availability of more data publicly from **social media sites** like **Facebook**, **YouTube**, **twitter** etc, business transactions, sensors, audio, video, photos etc. This enormous growth of data led to the concept of **Big data**, which is a term used for collection of large and complex data sets.

As the **data** has **increased**, so did the need for its **storage**. Until 2010, the main focus was building framework and solutions to store data, which was successfully solved by **HADOOP** and other frameworks. In the present time, it has become difficult to process this large and complex data using traditional data management techniques such as, for example, the RDBMS (relational database management systems).

This rise in the use of data, sparked **حرض** the use of Data Science. Data science makes this possible, as it is a multidisciplinary study of data collection for analysis, prediction, learning and prevention. Data Science involves using methods to analyze massive amounts of data and extract the **knowledge** from the **raw data**.

1.3 DATA GROWTH ISSUES AND CHALLENGES

Big Data is characterized by five **V's** namely **volume**, **variety**, **velocity**, **veracity** and **value**. Consequently, the challenges these characteristics bring are being seen in **data capture, curation, storage, search, sharing, transfer**, and **visualization**.

تتميز البيانات الضخمة **بخمسة** خصائص هي الحجم والتنوع والسرعة والمصداقية والقيمة. وبالتالي، فإن التحديات التي تفرضها هذه الخصائص تظهر في التقاط البيانات وتنظيمها وتخزينها والبحث عنها ومشاركتها ونقلها وتصورها.

- i. **Volume** refers to the enormous size of data. Big data refers to data volumes in the range of exabytes and beyond e.g. In the year 2016, the estimated global mobile traffic was 6.2 Exabytes (6.2 billion GB) per month. Also, by the year 2020 we will have almost 40000 Exabytes of data. Such volumes exceed the capacity of current on-line storage systems and processing systems. Traditional database systems is not able to capture, store and analyse this large amount of data. Storage solutions have been provided by HADOOP framework, but represent long term challenges that require research and new paradigms.

يشير volume إلى الحجم الهائل للبيانات. تشير البيانات الضخمة إلى أحجام البيانات في نطاق الإكسابايت وما بعده على سبيل المثال في عام 2016، كان حجم حركة الهاتف المحمول العالمية المقدّر 6.2 إكسابايت (6.2 مليار جيجابايت) شهريًا. أيضًا، بحلول عام 2020 سيكون لدينا ما يقرب من 40000 إكسابايت من البيانات. تتجاوز هذه الأحجام سعة أنظمة التخزين عبر الإنترنت وأنظمة المعالجة الحالية. لا تستطيع أنظمة قواعد البيانات التقليدية التقاط وتخزين وتحليل هذه الكمية الكبيرة من البيانات. تم توفير حلول التخزين من خلال إطار عمل HADOOP، ولكنها تمثل تحديات طويلة الأجل تتطلب البحث والنماذج الجديدة.

- ii. **Velocity** refers to the high speed of accumulation of data. There is a massive and continuous flow of data from sources like machines, networks, social media, mobile phones etc. This determines the potential of data that how fast the data is generated and processed to meet the demands e.g. there are more than 3.5 billion searches per day are made on Google. Also, FaceBook users are increasing by 22%(Approx.) year by year. Data is streaming in at unprecedented speed and must be dealt with in a timely manner. RFID tags, sensors and smart metering are driving the need to deal with torrents of data in near-real time. The data has to be available at the right time to make business decisions accurately. Reacting quickly enough to deal with data velocity is a challenge for most organizations.

تشير velocity إلى السرعة العالية لتراكم البيانات. هناك تدفق هائل ومستمر للبيانات من مصادر مثل الآلات والشبكات ووسائل التواصل الاجتماعي والهواتف المحمولة وما إلى ذلك. وهذا يحدد إمكانات البيانات ومدى سرعة إنشاء البيانات ومعالجتها لتلبية المتطلبات على سبيل المثال، هناك أكثر من 3.5 مليار عملية بحث يوميًا يتم إجراؤها على Google. كما يتزايد عدد مستخدمي Facebook بنسبة 22٪ (تقريبًا) عامًا بعد عام. تتدفق البيانات بسرعة غير مسبوقة ويجب التعامل معها في الوقت المناسب. تعمل علامات RFID وأجهزة الاستشعار والقياس الذكي على دفع الحاجة إلى التعامل مع سيول البيانات في الوقت الفعلي تقريبًا. يجب أن تكون البيانات متاحة في الوقت المناسب لاتخاذ قرارات العمل بدقة. يعد الاستجابة بسرعة كافية للتعامل مع سرعة البيانات تحديًا لمعظم المؤسسات.

- iii. **Variety** refers to nature of data. Data is available in varied formats and heterogenous sources. It can be structured, semi-structured and unstructured data. It is a challenge to find ways of governing, merging and managing these diverse forms of data.

يشير التنوع variety إلى طبيعة البيانات. فالبيانات متاحة في أشكال متنوعة ومصادر غير متجانسة. ويمكن أن تكون بيانات مهيكلة وشبه مهيكلة وغير مهيكلة. ومن الصعب إيجاد طرق لإدارة ودمج هذه الأشكال المتنوعة من البيانات.

- iv. **Veracity** here means quality of data. It refers to inconsistencies and uncertainty in data. The available data may be messy and thus controlling the quality and accuracy becomes another challenge.

الصدق Veracity هنا يعني جودة البيانات. ويشير ذلك إلى التناقضات وعدم اليقين في البيانات. فقد تكون البيانات المتاحة غير منظمة، وبالتالي يصبح التحكم في الجودة والدقة تحديًا آخر.

- v. **Variability**: In addition to the increasing velocities and varieties of data, data flows can be highly inconsistent with periodic peaks. Big Data is also variable because of the multitude of data dimensions resulting from multiple disparate data types and sources e.g. Data in bulk could create confusion whereas less amount of data could convey half or Incomplete Information. Variability of data can be challenging to manage.

التباين: بالإضافة إلى السرعات المتزايدة وأنواع البيانات، يمكن أن تكون تدفقات البيانات غير متسقة. كما أن البيانات الضخمة متغيرة بسبب تعدد أبعاد البيانات الناتجة عن أنواع ومصادر بيانات متعددة ومتباينة، على سبيل المثال، يمكن أن تؤدي البيانات المجمعة إلى حدوث ارتباك في حين أن كمية أقل من البيانات يمكن أن تنقل نصف المعلومات أو معلومات غير كاملة. يمكن أن يكون إدارة تباين البيانات أمرًا صعبًا.

- vi. **Value**: The bulk Data having no Value is of no good to the company, unless you turn it into something useful. Data in itself is of no use or importance but it needs to be converted into something valuable to extract Information. The ability to transform the bulk data into business and make this enormous data of use to business to monetize. It is a challenge to connect and correlate relationships, hierarchies and multiple data linkages of the big data.

القيمة: إن البيانات الضخمة التي لا قيمة لها لا تقدم أي فائدة للشركة، ما لم تحولها إلى شيء مفيد. فالبيانات في حد ذاتها لا فائدة منها ولا أهمية لها، ولكن يجب تحويلها إلى شيء ذي قيمة لاستخراج المعلومات. والقدرة على تحويل البيانات الضخمة إلى عمل تجاري وجعل هذه البيانات الضخمة مفيدة للشركات لتحقيق الربح منها. إن ربط وترابط العلاقات والتسلسلات الهرمية وارتباطات البيانات المتعددة للبيانات الضخمة يمثل تحديًا.

1.4 FOUNDATION OF DATA SCIENCE

Data science involves using methods to analyse massive amounts of data and extract the knowledge it contains. Data Science is an interdisciplinary field focused on extracting knowledge from datasets which are large in size, applying the knowledge from data to solve problems in a wide range of application domains. Mathematics, statistics, computer science, and domain knowledge are the foundations of Data Science.

The main components of Data Science are given below:

- 1. Statistics:**

To analyze the large amount numerical data and to find the meaningful insights from it, knowledge of statistics is required

- 2. Domain Expertise:**

Data is available and applicable in various domains, therefore domain expertise or specialized knowledge of a specific area is required to get best results.

- 3. Data engineering:**

Data engineering is a part of data science, which involves acquiring, storing, retrieving, and transforming the data. Data engineering also includes metadata (data about data) to the data.

- 4. Visualization:**

Data visualization is meant by representing data in a visual context so that people can easily understand the significance of data. Data visualization makes it easy to access the huge amount of data in visuals.

- 5. Advanced computing:**

Advanced computing involves designing, writing, debugging, and maintaining the source code of computer programs. Machine Learning and Deep learning techniques are required for modelling and to make predictions about unforeseen/future data.

1.5 TOOLS FOR DATA SCIENCE

Some tools required for data science are as follows:

- **Data Analysis tools:** R, Python, Statistics, SAS, Jupyter, R Studio, MATLAB, Excel, RapidMiner.
- **Data Warehousing:** ETL, SQL, Hadoop, Informatica/Talend, AWS Redshift
- **Data Visualization tools:** R, Jupyter, Tableau, Cognos.
- **Machine learning tools:** Spark, Mahout, Azure ML studio.

1.6 APPLICATIONS OF DATA SCIENCE

Data Science is used by most organizations for predictive analysis, price optimization, and customer satisfaction. Some areas where data science is being used are Health Care, Finance, Security, Airline Routing, Manufacturing, Speech Recognition, Advertisement, Security, Fraud detection, Banking, Internet of Things etc.

Some use cases are given below:

- Genomic Data provides deeper understanding of Genetic issues and reactions to particular drugs and diseases. توفر البيانات الجينومية فهماً أعمق للقضايا الجينية وردود الفعل تجاه أدوية وأمراض معينة.
- Logistics companies like DHL, Fedex have discovered the best time and routes to ship cost effectively. اكتشفت شركات الخدمات اللوجستية مثل DHL و Fedex أفضل الأوقات والطرق للشحن بتكلفة فعالة.
- Predict employee attrition and understand the variables that influence employee turnover. التنبؤ بتسرب الموظفين وفهم المتغيرات التي تؤثر على دوران الموظفين.
- Airline companies can now easily predict flight delays and notify the passengers before time. تستطيع شركات الطيران الآن التنبؤ بسهولة بتأخيرات الرحلات وإخطار الركاب قبل الموعد المحدد.
- Banks can make better decisions by predict risk analysis, fraud detection and better customer management. يمكن للبنوك اتخاذ قرارات أفضل من خلال التنبؤ بتحليل المخاطر واكتشاف الاحتيال وإدارة العملاء بشكل أفضل.

1.7 DATA SCIENCE PROCESS

The structured approach to data science helps in maximizing the chances of success in a data science project at the lowest cost. **The data science process typically consists of the following steps:**

- I. **Business Understanding:** The main purpose here is making sure all the stakeholders understand the **what**, **how**, and **why** of the project. It involves two main steps namely defining research goals and preparing a project charter.
 - i) **Defining Research Goals:** It is very essential to understand the business goals to be able to define the research goal that states the purpose of assignment in a clear and focused manner. The data by for the same can be gathered by repeatedly asking questions and enquiring about business expectations, identifying the research goals so that everybody knows what to do and can agree on the best course of action. The outcome should be a clear research goal, a good understanding of the context, well-defined deliverables, and a plan of action with a timetable. This information is then best placed in a project charter.
 - ii) **Creating Project Charter:** A project charter has the information gathered while setting the research goal. The project charter must contain a clear research goal, the project mission and context, method for analysis, information about the resources required for project completion, proof that the project is achievable, or proof of concepts, deliverables and a measure of success and also a timeline for the project. This information is useful to make an estimation of the project costs and the data and people required for the project to become a success.
- II. **Data Acquisition or Data Collection or Data Retrieval:** This phase involves data gathering from various sources like web servers, logs, databases, API's and online repositories. It involves acquiring data from all the identified internal & external sources. One should start with data collection from internal sources. The data may be available in many forms ranging from simple text to database records. The data may be structured as well as unstructured. Data may be acquired from sources outside the organization also. Some sources may be available free of cost or some may be paid. Data collection is a tiresome and time-consuming task.
- III. **Data preparation:** Data collection is an error-prone process; in this phase you enhance the quality of the data and prepare it for use in subsequent steps. This

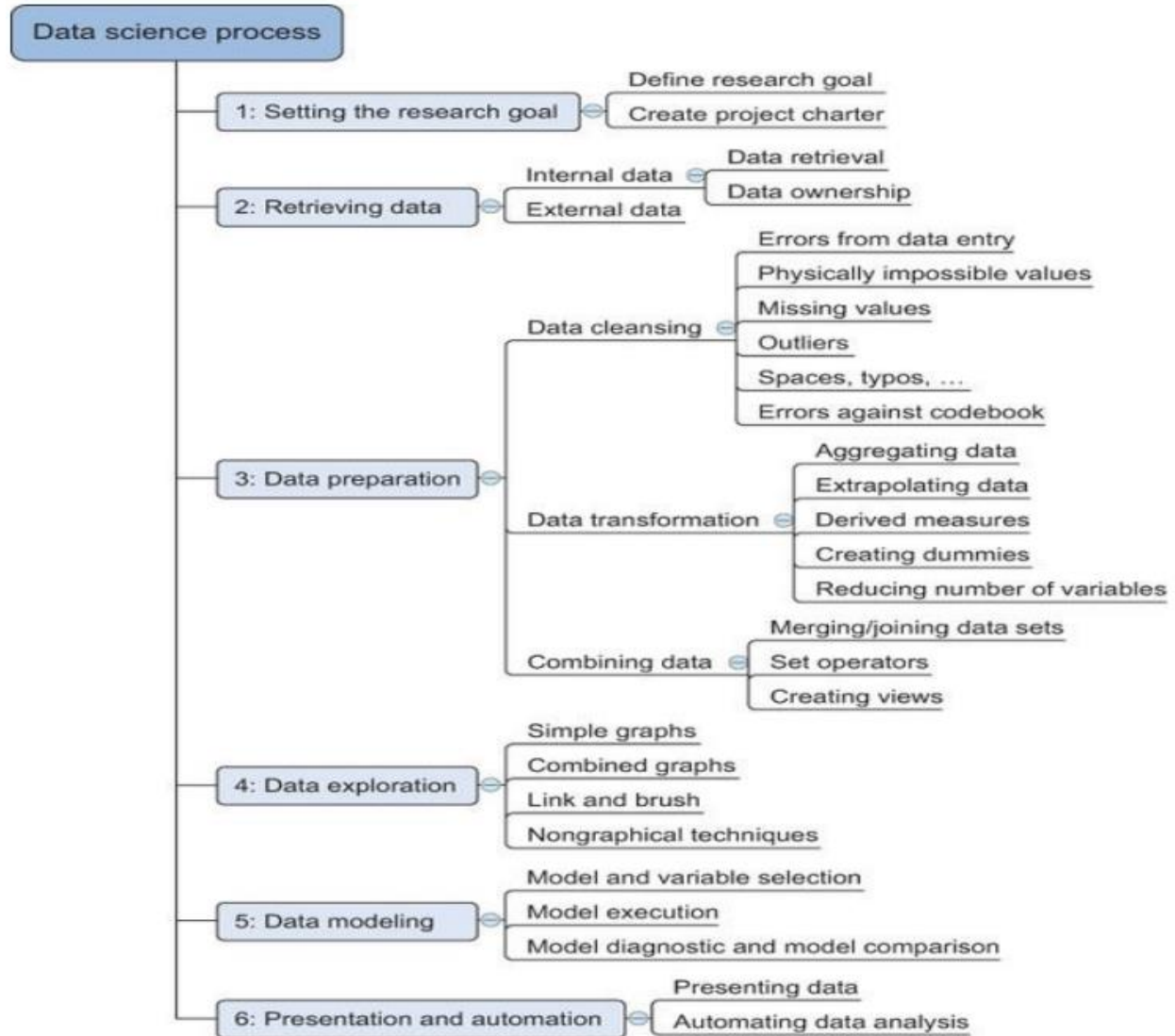
phase consists of three subphases: **data cleansing**, **data integration** **data transformation**.

- i) **Data Cleansing** involves cleansing of data collected in the previous phase. It involves removing false values from a data source, removing inconsistencies across data sources, checking misspelled attributes, missing and duplicate values and typing errors, fixing capital letter mismatches as most programming languages are case sensitive (e.g India and india), removing redundant white spaces, sanity checks (check for impossible values like six-digit phone number etc) and outliers (an outlier is an observation that seems to be distant from other observations). It should be ensured that most errors are corrected in this phase to make the data usable and get better results.
- ii) **Data Integration** enriches data sources by combining information from multiple data sources. Data comes from several sources and in this sub step the focus is on integrating these different sources. The data can be combined in the following ways:
 - **Joining Tables:** To combine information about some data in one table with the information available in another table. e.g a table may contain information about purchase of a particular product and the other table may contain information about people who have purchased that product. This can be done using join command in SQL.
 - **Appending or Stacking:** To add observations from one table to another table e.g. a table may contain the information about the purchase of a particular product in the year 2000 and the other contains the similar data in the year 2001, then appending means to add the records of 2001 to the table containing the records of 2000. This can be done using the union function in SQL.
 - **View:** View in SQL can be used to virtually combine two tables. This saves on the space requirement
 - **Aggregate Functions:** Aggregate functions may be used as per requirement for combining data.
- iii) **Data transformation** ensures that the data is in a suitable format to be used in the project model. Sometimes, the number of variables may have to be

reduced, It involves modification of data so that it takes a suitable shape. Tools like talend and informatica can be used for transformation.

- IV. **Exploratory Data Analysis (EDA) or Data exploration:** Data exploration is concerned with building a deeper understanding of the data. It involves the understanding of how variables interact with each other, the distribution of the data, and whether there are outliers. It involves defining and refining the selection of feature variables that will be used in model development. This is the most important step as it involves understanding of the data which will further be used for modelling. Various techniques like visualization, tabulation, clustering, and other modelling techniques can be used for exploratory analysis.
- V. **Data Modelling or Model building:** Model building is an iterative process. In this phase, domain knowledge, and insights about the data are used for modelling. It involves identifying the data model that best fits the business requirement. For this, various machine learning techniques like KNN, Naïve's, decision tree etc may be applied on data. The model is trained on the data set and the testing of the selected 7 model is done. Data modelling can be done using Python, R, SAS. Most models consist of the following main steps:
- i) **Selection of a modelling technique and variables** to enter in the model: After the exploratory analysis, the variables to be used are known, so in this phase, the variables can be used to build the model. A model that suits the project requirement has to be chosen.
 - ii) **Execution of the model:** The chosen model has to be implemented by coding. Python is most used language for coding as it has many inbuilt libraries.
 - iii) **Diagnosis and model comparison:** In this step, the best performing model or the model with lowest errors is chosen from among the multiple models that are built
- VI. **Presentation and automation:** In this stage, the results are presented. These results can take many forms, ranging from presentations to research reports.
- VII. **Deployment & Maintenance:** Before the final deployment in the production environment, testing is done in preproduction environment. After deployment, the reports are used for real time analysis.

The Data Science **life cycle** is an Iterative process, there is often a need to step back and rework certain findings. If the step 1(business understanding) is performed dedicatedly, rework can be prevented. The figure 1 below describes the Data Science model.



1.8 MESSY DATA فوضوية

Data that is not in usable form or it is impossible to obtain clearly interpretable information from it is messy data.

البيانات التي ليست في شكل قابل للاستخدام أو من المستحيل الحصول على معلومات واضحة قابلة للتفسير منها هي بيانات فوضوية.

Data science and its algorithms are **clean** and **precise**, but the data on which they operate come from the real world, are inherently messy i.e. the data which requires some preparation before you can use them effectively and it is not easy to find clean data. The quality of

insights you derive from data depends on the validity of that data, so some preparation is required. Some examples of messy data are missing data, unstructured data, multiple variables in one column, variables stored in wrong places, observations split incorrectly or left together against normalization rules, switched columns and rows, extra spaces etc.

إن علم البيانات وخوارزمياته نظيف ودقيق، ولكن البيانات التي تعمل عليها تأتي من العالم الحقيقي، وهي فوضوية بطبيعتها، أي البيانات التي تتطلب بعض التحضير قبل أن تتمكن من استخدامها بشكل فعال وليس من السهل العثور على بيانات نظيفة. تعتمد جودة الأفكار التي تستمدتها من البيانات على صحة تلك البيانات، لذا يلزم بعض التحضير. بعض الأمثلة على البيانات الفوضوية هي البيانات المفقودة، والبيانات غير المنظمة، والمتغيرات المتعددة في عمود واحد، والمتغيرات المخزنة في أماكن خاطئة، والملاحظات المقسمة بشكل غير صحيح أو المتروكة معاً ضد قواعد التطبيع، والأعمدة والصفوف المبدلة، والمسافات الإضافية وما إلى ذلك.

1.9 ANOMALIES AND ARTEFACTS IN DATASETS

Anomaly is a deviation in data from the expected value for a metric at a given point in time. Anomaly detection is any process that finds the outliers (items that do not belong to the dataset) of a dataset. The term anomaly is also referred to as outlier. Outliers are the data objects that stand out among other objects in the data set and do not conform to the normal behavior in a data set. There are three kinds of anomalies namely: point anomaly, contextual anomaly, and collective anomalies.

- **Point Anomaly:** If a single instance in a given dataset is different from others with respect to its attributes, it is called a point anomaly i.e. when a single instance of data is anomalous, it deviates largely from the rest of the set e.g. detecting credit card fraud based on “amount spent.”
- **Contextual anomaly:** If the data is anomalous in some context, it is called contextual anomaly. This type of anomaly is common in time-series data. In the absence of a context, all the data points look normal. E.g. if the context of the temperature is recorder in December and a high temperature reading is seen in December month, which is an abnormal phenomenon.
- **Collective anomalies** can be formed due to a combination of many instances i.e. a set of data instances collectively helps in detecting anomalies. For example, sequence data in network log or an attempt to copy data from a remote machine to a local host unexpectedly, an anomaly that would be flagged as a potential cyber-attack.

Anomaly detection refers to the problem of finding patterns in data that do not conform to expected behavior. It is a technique for finding an unusual point or pattern in a given set. These nonconforming patterns are often referred to as anomalies, outliers, discordant

observations, exceptions, aberrations, surprises, peculiarities, or contaminants in different application domains. Anomaly detection is commonly used for:

- Data cleaning
- Intrusion detection
- Fraud detection
- Systems health monitoring
- Event detection in sensor networks
- Ecosystem disturbances

Artifact It is a data flaw caused by equipment, techniques or conditions. Common sources of data flaws include hardware or software errors, conditions such as electromagnetic interference and flawed designs such as an algorithm prone to miscalculations. Some common data artifacts are:

- Digital Artifacts: Flaws in digital media, documents and data records caused by data processing errors e.g a distorted camera recording.
- Visual Artifacts: Flaws in visualizations such as user interfaces.
- Compression Artifacts: Flaws in data due to lossy compression.
- Statistical Artifacts: Flaw such as a bias in statistical data.
- Sonic Artifacts: Unwanted sound in a recording.

1.10 CLEANING DATA

Data cleaning is a key part of data science. Clean data increases overall productivity and allows for the highest quality information in decision-making. Data cleaning is the process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset. If data is messy, the outcomes and algorithms are unreliable. It can lead to poor business strategy and decision-making. The data can have many irrelevant and missing parts. To handle this part, data cleaning is done. It involves handling of missing data, noisy data etc. During cleansing, missing values may either be filled or removed depending on the data. Data cleaning is discussed later.

1.11 QUESTIONS

1. What are the foundations of data science?
2. Discuss the areas where data science is applicable?
3. Discuss the various challenges due to growth in data.
4. Explain briefly the data Science Process.
5. Define Messy data.

Python for Data Science

- Pandas
- Matplotlib
- Seaborn

Define a function:

```
def my_func(x, y):  
    if x > y:  
        return x
```

Define a function that returns a tuple:

```
def my_func(x, y):  
    return (x-1, y+2)  
  
(a, b) = my_func(1, 2)
```

```
a = 0; b = 4
```

```
len( ['c', 'm', 's', 'c', 3, 2, 0] ) #7
```

range: returns an iterable object

```
list( range(10)
```

enumerate: returns iterable tuple (index, element) of a list

```
enumerate( ["311", "320", "330"] )
```

```
# [(0, "311"), (1, "320"), (2, "330")]
```

map: apply a function to a sequence or iterable

```
arr = [1, 2, 3, 4, 5]  
map(lambda x: x**2, arr)
```

```
[1, 4, 9, 16, 25]
```

filter: returns a list of elements for which a predicate is true

```
arr = [1, 2, 3, 4, 5, 6, 7]  
filter(lambda x: x % 2 == 0, arr)
```

```
[2, 4, 6]
```

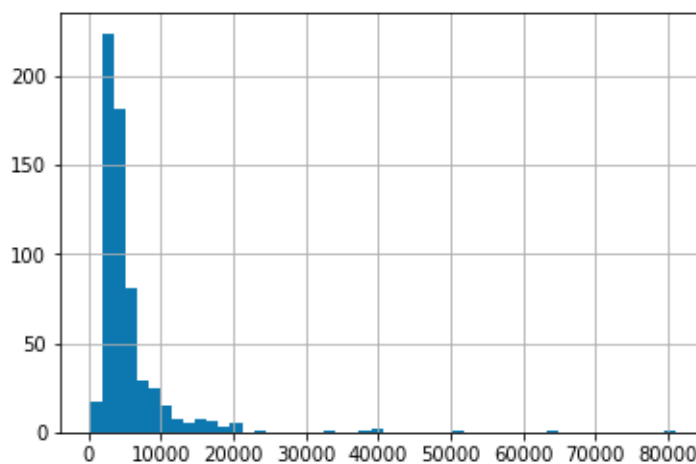
```
import pandas as pd  
import numpy as np  
import matplotlib as plt  
df=pd.read_csv(" ...")  
df.head(5)  
df.describe()
```

```
In [6]: df['Property_Area'].value_counts()
```

```
Out[6]: Semiurban    233  
Urban        202  
Rural        179  
Name: Property_Area, dtype: int64
```

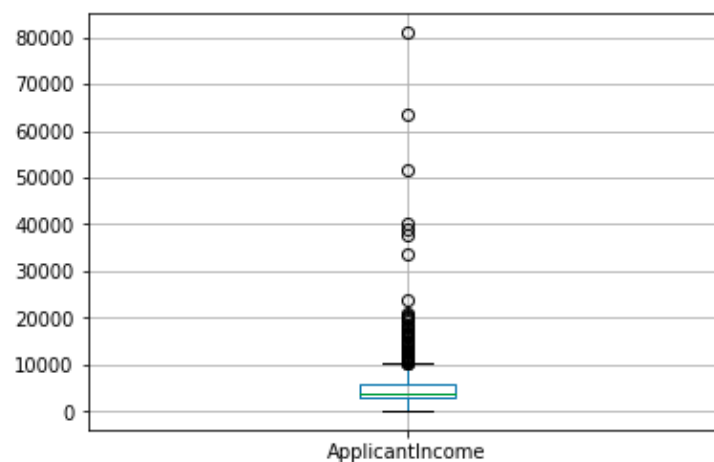
```
In [7]: df['ApplicantIncome'].hist(bins=50)
```

```
Out[7]: <matplotlib.axes._subplots.AxesSubplot at 0x7f5029ab7400>
```



```
In [8]: df.boxplot(column='ApplicantIncome')
```

```
Out[8]: <matplotlib.axes._subplots.AxesSubplot at 0x7f5029a7f080>
```



```
In [14]: temp1 = df['Credit_History'].value_counts(ascending=True)
temp2 = df.pivot_table(values='Loan_Status', index=['Credit_History'], aggfunc=lambda x: x.map({'Y':1, 'N':0}).mean())
print ('Frequency Table for Credit History:')
print (temp1)

print ('\nProbability of getting loan for each Credit History class:')
print (temp2)
```

Frequency Table for Credit History:

0.0 89

1.0 475

Name: Credit_History, dtype: int64

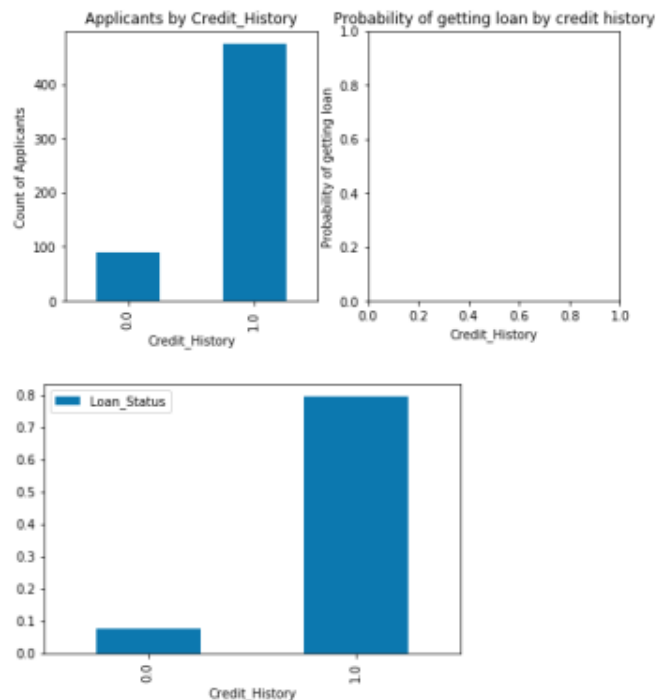
Probability of getting loan for each Credit History class:

Credit_History	Loan_Status
0.0	0.078652
1.0	0.795789

```
In [15]: import matplotlib.pyplot as plt
fig = plt.figure(figsize=(8,4))
ax1 = fig.add_subplot(121)
ax1.set_xlabel('Credit_History')
ax1.set_ylabel('Count of Applicants')
ax1.set_title("Applicants by Credit_History")
temp1.plot(kind='bar')

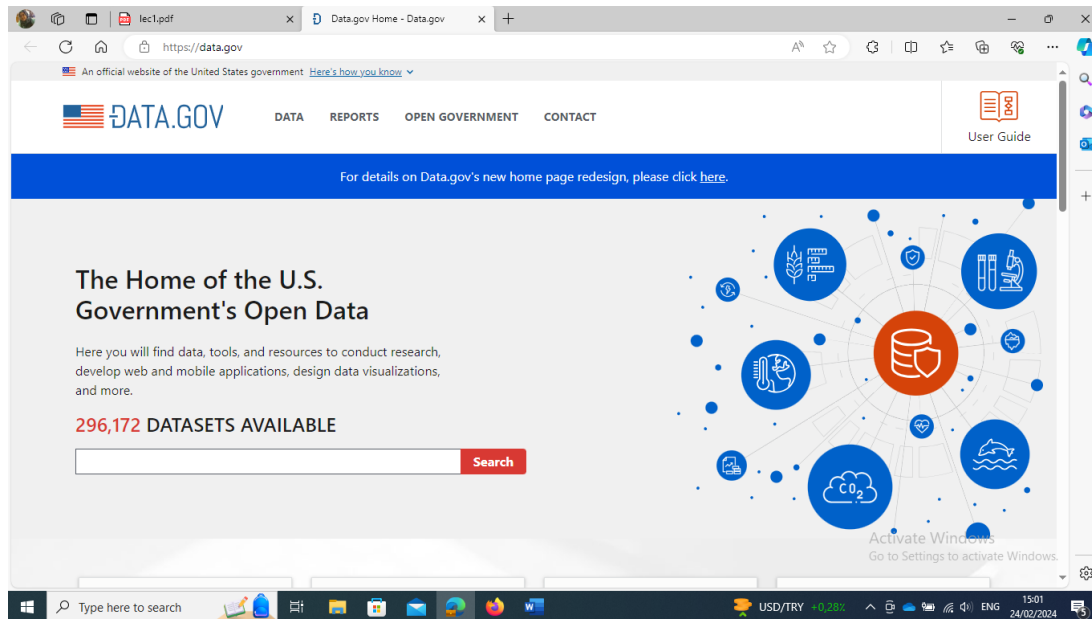
ax2 = fig.add_subplot(122)
temp2.plot(kind = 'bar')
ax2.set_xlabel('Credit_History')
ax2.set_ylabel('Probability of getting loan')
ax2.set_title("Probability of getting loan by credit history")
```

Out[15]: Text(0.5,1,'Probability of getting loan by credit history')



Datasets resources:

- 1- www.data.gov/
agriculture, climate, education, energy, finance, health, ...



- 2- <https://cloud.google.com/bigquery/public-data/>
BigQuery (google cloud) public datasets.

BigQuery public datasets

A public dataset is any dataset that is stored in BigQuery and made available to the general public through the [Google Cloud Public Dataset Program](#). The public datasets are datasets that BigQuery hosts for you to access and integrate into your applications. Google pays for the storage of these datasets and provides public access to the data via a [project](#). You pay only for the queries that you perform on the data. The first 1 TB per month is free, subject to [query pricing details](#).

Public datasets are available for you to analyze using either legacy SQL or [GoogleSQL](#) queries. Use a fully qualified table name when querying public datasets, for example `bigquery-public-data.bbc_news.fulltext`. If your organization restricts data access, for example with security perimeters, then you might need to contact your administrator for permission to access public datasets.

You can access BigQuery public datasets by using the [Google Cloud console](#), by using the [bq command-line tool](#), or by making calls to the [BigQuery REST API](#) using a variety of [client libraries](#) such as [Java](#), [.NET](#), or [Python](#). You can also [view and query public datasets through Analytics Hub](#), a data exchange platform that helps you discover and access data libraries.

[Go to Analytics Hub](#)

You can find more details about each individual dataset by clicking the dataset's name in the Datasets section of Cloud Marketplace.

3- www.kaggle.com/datasets

4- <https://aws.amazon.com/public-datasets/>

Registry of Open Data on AWS

The Registry of Open Data on AWS is now available on [AWS Data Exchange](#). All datasets on the Registry of Open Data are now discoverable on AWS Data Exchange alongside 3,000+ existing data products from category-leading data providers across industries. Explore the catalog to find open, free, and commercial data sets. [Learn more about AWS Data Exchange](#)

About

This registry exists to help people discover and share datasets that are available via AWS resources. See recent additions and learn more about sharing data on AWS.

Get started using data quickly by viewing all tutorials with associated SageMaker Studio Lab notebooks.

See all usage examples for datasets listed in this registry.

See datasets from Allen Institute for Artificial Intelligence (AI2), Digital Earth Africa, Data for Good at Meta, NASA Space Act Agreement, NIH STRIDES, NOAA Open Data Dissemination Program, Space Telescope Science Institute, and Amazon Sustainability Data Initiative.

Search datasets (currently 516 matching datasets)

Add to this registry

If you want to add a dataset or example of how to use a dataset to this registry, click [here](#).

The Human Sleep Project

[bioinformatics](#) [deep learning](#) [life sciences](#) [machine learning](#) [medicine](#) [neurophysiology](#) [neuroscience](#)

The Human Sleep Project (HSP) sleep physiology dataset is a growing collection of clinical polysomnography (PSG) recordings. Beginning with PSG recordings from ~15K patients evaluated at the Massachusetts General Hospital, the HSP will grow over the coming years to include data from >200K patients, as well as people evaluated outside of the clinical setting. This data is being used to develop CAISR (Complete AI Sleep Report), a collection of deep neural networks, rule-based algorithms, and signal processing approaches designed to provide better-than-human detection of conventional PSG...

Details

Use examples

- Decision modeling in sleep apnea: the critical roles of pre-test probability, cost of untreated OSA, and time horizon. *Journal of Clinical Sleep Medicine*. 2016 Mar;12(3):409-418. PMID: PMC4773629. by Moro M, Westover MB, Kelly J, Bianchi MT.
- The Human Sleep Project by Westover MB, Moura Junior V, Thomas R, Cash S, Nasiri S, Sun H, et al.
- Artificial intelligence in sleep medicine: Background and implications for clinicians.

Chapter Two-Data Acquisition and Processing

Contents:

2.1 Introduction to Data Acquisition

2.2 Data Preprocessing and Techniques

2.2.1 Data Cleaning

2.2.2 Data Integration

2.2.3 Data Transformation

2.2.4 Data Reduction

2.2.4.1 Dimensionality Reduction

2.2.4.2 Numerosity Reduction

2.2.4.3 Data Compression

2.3 Data Mining

2.3.1 Data Mining Applications

2.4 Data Interpretation

2.5 Question

OBJECTIVES

1. Introduction to Data Acquisition
2. familiarize with the different types of data
3. Provide the concept of data pre-processing and familiarize with the various reprocessing techniques
4. Familiarize with the concept of data mining and data interpretation

2.1 INTRODUCTION TO DATA ACQUISITION

The process of gathering data and making it useful by filtering and cleaning it as per the business requirement is termed as **data acquisition** it is the most important step of data science, but the data acquired must be suitable to the problem in hand, else the output may be inaccurate.

It is very important to acquire up to date data.

The characteristics of big data namely **volume**, **velocity**, **variety**, and **value** are very important for the acquisition of data.

→ In data science, there is a *variety of data* that we need to deal with.

The **data** is majorly **characterized** as:

- **Structured Data:** The data which is available in some stand **format** is called structured data.

Structured data is **organized** data.

It is stored in the row and column structure like in a relational database and excel sheets. Some examples of structured data are names, numbers, geolocations, addresses etc.

- **Unstructured Data:** This data is **unorganized** data.

There is **no particular format** for unstructured data.

It is available in a **variety of formats** like texts, pictures, videos etc.

Unstructured data is **more difficult** to search and requires processing to become understandable.

- **Semi- Structured data:** This data is basically a **semi-organised** data.

It is generally a form of data that do not conform to the formal structure of data.

Log files are the examples of this type of data.

1. Which of the following is an example of structured data?

- a) Images from CCTV cameras
- b) Data stored in a relational database table.
- c) Audio recordings
- d) Social media posts

(Data stored in a relational database table: Structured data follows a fixed schema like rows and columns.)

2. Which format is considered semi-structured data?

- a) CSV file
- b) JSON file.
- c) Plain text file
- d) Printed book

(JSON file: Semi-structured data has tags or markers but no strict table format.)

3. Emails are usually classified as:

- a) Structured
- b) Semi-structured.
- c) Unstructured
- d) Numerical

(Semi-structured: They have fields like sender, receiver, date, but the body is unstructured.)

4. Which of the following is unstructured data?

- a) Excel spreadsheet
- b) SQL table
- c) Video file.
- d) Customer database

(Video file: Unstructured data has no predefined model.)

5. Which characteristic best describes structured data?

- a) No predefined schema
- b) Easily searchable in relational databases.
- c) Stored as multimedia
- d) Difficult to query

6. XML files are considered:

- a) Structured
- b) Semi-structured.
- c) Unstructured
- d) d) Numeric

(XML uses tags but does not enforce a strict tabular format.)

7. Data stored in a MySQL database is typically:

- a) Unstructured
- b) Semi-structured
- c) Structured.
- d) Random

8. Which of the following is an example of semi-structured data in web applications?

- a) HTML document.
- b) Printed newspaper
- c) PNG image
- d) Audio file

9. Social media images and videos are classified as:

- a) Structured b) Semi-structured c) Unstructured. d) Tabular

10. Structured data is best described as:

- a) Data without any organization
- b) Data stored in rows and columns with a fixed schema.
- c) Multimedia content
- d) Random text

11. Semi-structured data differs from structured data because it:

- a) Has no organization
- b) Has flexible schema with tags or markers.
- c) Is always numeric
- d) Cannot be stored electronically

Data stored in a data warehouse is usually:

- a) Unstructured
- b) Semi-structured
- c) Structured.
- d) Multimedia

Which technology is mainly used to manage structured data?

- a) NLP
- b) RDBMS.
- c) Computer Vision
- d) Deep Learning

Which of the following is NOT structured data?

- a) MySQL table
- b) Excel spreadsheet
- c) PDF document.
- d) Customer database

Sensor data in CSV format is:

- a) Structured.
- b) Semi-structured
- c) Unstructured
- d) Audio

Log files are typically:

- a) Structured
- b) Semi-structured.
- c) Unstructured
- d) Binary

Big Data often contains:

- a) Only structured data
- b) Only tabular data
- c) Combination of structured, semi-structured, and unstructured data.
- d) Only images

Which file format is unstructured?

- a) PNG image.
- b) CSV
- c) Excel
- d) SQL

Medical MRI images are:

- a) Structured
- b) Semi-structured
- c) Unstructured.
- d) Tabular

A banking transaction table is:

- a) Structured.
- b) Semi-structured
- c) Unstructured
- d) Multimedia

Audio speech recordings are:

- a) Structured
- b) Semi-structured
- c) Unstructured.
- d) Relational

The main challenge of unstructured data is:

- a) Too organized
- b) Difficult to store
- c) Difficult to process and analyze automatically.
- d) Fixed schema

2.2 DATA PREPROCESSING AND TECHNIQUES

The data acquired need to be pre-processed to make it usable. The raw data may be inconsistent, erroneous, incomplete and not in the required format. These issues need to be resolved to make the data usable. Data Preprocessing is a collaborative term used for the activities involved in transforming the real-world data or raw data into a usable form to make it more valuable and to get it in the required format.

The preprocessed data is cleaner and more valuable and hence used as final training set. Data preprocessing is a very essential step, as more clean and inconsistent data we get, better shall be the final output. In other words, data processing improves the quality of data. The huge size of data collected from heterogenous sources leads to anomalous data. Data preprocessing has become a vital and most fundamental step considering the fact that high quality data leads to better models and predictions.

Data preprocessing techniques are majorly categorized in the following methods:

- i. Data Cleaning
- ii. Data Integration
- iii. Data Transformation
- iv. Data Reduction

2.2.1 Data Cleaning

- معالجة البيانات المفقودة **Processing missing data**
- اكتشاف القيم الشاذة **Detecting outliers**
- **Feature Engineering**

Data Cleaning involves removing duplicate data, filling missing values, identifying outliers, smoothening noise and remove data inconsistencies.

Why Is Data Cleaning Important?

- Improves model accuracy
- Reduces bias and errors
- Ensures consistency
- Makes analysis reliable
- Prepares data for Machine Learning algorithms

The Main steps for cleaning data:

◆ Handle missing data:

During analysis of data, it is important to understand why the missing values exist. Whether the missing value exist because they were not recorded or they imply that data does not exist for the missing values.

Missing data may be handled by using the following ways:

i) In case, the values were not recorded, it becomes essential to **fill up the values** by **guessing** based on the other values in that column and row. This is called **imputation**.

E.g. if a value is missing for gender, it is understood that the value was not recorded, so we need to analyse data and give it a value, we cannot leave the value blank in this case. Therefore, you can input missing values based on other observations; but there is a chance of losing integrity of the data because these values are being filled based on assumptions and not actual observations.

ii) If a value is missing because it doesn't exist e.g. the height of the oldest child of someone who doesn't have any children. In this case, it would make more sense to either leave it empty or to add a third value like *NA* or *NaN*. The *NA* or *NaN* values should be

replaced with 0. Panda's `fillna()` function to fill in missing values in a data frame can be used.

iii) In case, it is not possible to figure out the reason for the missing, then that particular value can be dropped. Pandas function, `dropna()` can be used to do this, but doing this can drop or lose information, so be mindful of what is being removed or dropped this before removing it.

Replace with:

- Mean
- Median
- Mode

◆ Remove duplicate or irrelevant observations

It is very important to remove inconsistencies in dataset like removing unwanted observations including duplicate and irrelevant data entries.

Duplicate values may be caused at the time of data collection e.g. the names of the countries may have repeated valued like *NewZealand* and *New Zealand*; *Pakistan* and *pakistan*.

Inconsistencies in capitalizations and trailing white spaces are very common in text data. The data set can be cleaned using available function in Python or R. functions like `unique`, `sort`, `lower()`, `upper()`, `strip()` can be used to handle inconsistencies.

◆ **Fix structural errors:** Structural errors are when you measure or transfer data and notice strange naming conventions, typos, or incorrect capitalization. These inconsistencies can cause mislabelled categories or classes. For example, you may find `—N/A` and `—Not Applicable` both appear, but they should be analysed as the

same category. Set the single date format in case there are multiple date formats in a single column.

- ◆ **Filter unwanted outliers** is essential to improve the performance of the data. If an outlier is found to be irrelevant it must be removed.

Detection methods:

- Boxplot
- Z-score
- IQR (Interquartile Range)

Example using IQR:

```
Q1 = df['salary'].quantile(0.25)
```

```
Q3 = df['salary'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df = df[(df['salary'] >= Q1 - 1.5*IQR) & (df['salary'] <= Q3 + 1.5*IQR)]
```

- ◆ **Standardizing Data Formats**

Examples:

- Convert dates to consistent format
- Standardize text values (e.g., "Male", "male", "M" → "Male")
- Convert strings to lowercase

```
df['gender'] = df['gender'].str.lower()
```

```
df['date'] = pd.to_datetime(df['date'])
```

- ◆ **Encoding Categorical Variables**

Machine Learning models require numerical data.

Techniques???

- ◆ **Noisy Data:**

Noisy data refers to data that contains errors, random variations, or irrelevant information that distort the true signal.

Random variance in the data is called noise. The following methods called **smoothing** techniques are used to handle noisy data:

- a) **Binning Method:** This method is also known as **discretization**, is used to smooth sorted data values by consulting the values around it i.e., the neighbouring values. It is a local smoothing method, since it refers to the neighbouring values.

This is a process of converting **continuous** data into **a set of data intervals**.

Continuous attribute values are substituted by small interval labels. This makes the data easier to study and analyze.

Data discretization can be classified into two types: **supervised discretization**, where the class information is used, and **unsupervised discretization**, which is based on which direction the process proceeds, i.e., 'top-down splitting strategy' or 'bottom-up merging strategy'.

For example, the values for the age attribute can be replaced by the interval labels such as (0-10, 11-20...) or (kid, youth, adult, senior).

In this method, the entire data is divided into equal segments called **bins** or **buckets**. Smoothing by binning is done by one of the following methods:

Smoothing by Bin Means: In smoothing by bin means, each value in a bin is replaced by the mean value of the bin.

- **Smoothing by Bin Median:** In smoothing by bin means, each value in a bin is replaced by the median value of the bin.

▪ **Smoothing by Bin Boundary:** In smoothing by bin boundaries, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.

Algorithm:

1. Sort the dataset.
2. Partition the dataset into n' segments. Each segment should contain approximately same number of data elements. Partitioning can be done using either of the following methods namely: equal width binning, equal frequency binning or entropy-based binning.
3. Calculate the arithmetic mean or media or replace by boundary (min and max value)
4. Replace each data element in each bin by the calculated mean/ median/ boundaries.

Example: Given data: 18, 22, 6, 6, 9, 14, 20, 21, 12, 18, 18, 16. Illustrate binning by mean, median and boundary replacement. Given bin depth=3

Step1: Sort the data: 6, 6, 9, 12, 14, 16, 18, 18, 18, 20, 21, 22,

Step 2: Partition the data into equal frequency bins of size of bin depth(n/d) where n = no. of elements and d = bin depth

$N/D = 12/3 = 4$ bins

Bin 1: 6, 6, 9

Bin 2: 12,14, 16

Bin 3: 18,18,18

Bin 4: 20, 21, 22

Step 3: Calculate Arithmetic mean

Arithmetic mean= Sum of observations ÷ number of observations

Bin 1= $(6+6+9)/3=21/3=7$

Bin 2= $(12+14+16)/3= 42/3=14$

Bin 3= $(18+18+18)/3= 54/3=18$

Bin 4= $(20+21+22)/3= 63/3=21$

Step 4: Replace each data element in each bin by the calculated mean

Bin 1: 7, 7, 7

Bin 2: 14,14,14

Bin 3: 18,18,18

Bin 4: 21, 21, 21

- **Binning using Median:** In this method, Step 1 and step 2 are same.

Step 3: Calculate Median (50% percentile)

Median is the observation 2 in each bin

Step 4: Replace each data element in each bin by the calculated mean

Bin 1: 6, 6, 6

Bin 2: 14, 14, 14

Bin 3: 18, 18, 18

Bin 4: 21, 21, 21

• **Binning using Boundary Values:** In this, we keep the minimum as well as maximum values.

Bin 1: 6, 6, 9

Bin 2: 12, 12, 16

Bin 3: 18, 18, 18

Bin 4: 20, 20, 22

b) Regression: Regression is used to find a mathematical equation to fit the data to smooth out the noise. Regression may be linear or multiple. Linear regression is used to find the best line that fits two variables such that one predicts the other. Multiple linear regression involves more than two variables are involved.

c) Clustering: This approach is useful for organizing similar data in groups or clusters. The outliers may be detected by clustering. Data Integration makes data more comprehensive and more usable.

Questions:

Data preprocessing is mainly performed to:

- a) Increase data size
- b) Improve data quality before analysis.
- c) Delete all missing values
- d) Build models directly

Which of the following is a data cleaning technique?

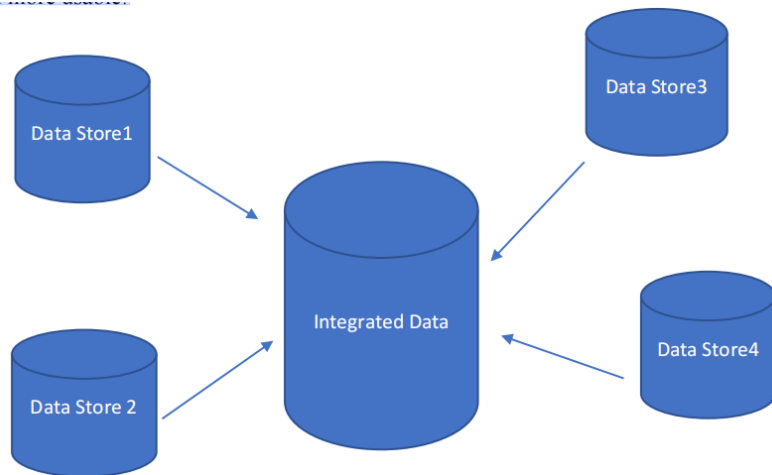
- a) Feature scaling
- b) Removing duplicates.
- c) Model evaluation
- d) Hyperparameter tuning

Removing extreme values using IQR method is used for:

- a) Feature scaling
- b) Encoding
- c) Outlier detection.
- d) Sampling

2.2.2 Data Integration

Data Integration is a vital step in which the data acquired from multiple sources is integrated into a single centralized location. Data Integration makes data comprehensive and more usable.



The most common approaches to integrate data are:

- a) Data Consolidation
- b) Data Propagation
- c) Data Virtualization
- d) Data Warehousing

a) Data consolidation means to consolidate data from several separate sources into one data store, so that it is available to all the stake holders to enable better decision making.

It involves eliminating redundancies, removing inaccuracies before consolidating to a single store. The most common data consolidation techniques are ETL (Extract, Transform, Load), Data virtualization and Data warehousing.

- **ETL** is the most widely used data consolidation technique. The data is first

extracted from multiple sources, then it is transformed into an understandable format by using various functions like sorting, aggregating, cleaning etc and then transfer it to a centralized store like another database or data ware house.

The ETL process cleans, filters, and transforms data, and then applies business rules before data populates the new source. ETL is further of two types namely Real time ETL used in real time systems and Batch processing ETL used for high volume databases.

- **Data Virtualization:** In this method, the data stays in the original location, but changes are made in a virtual manner and can be seen in a consolidated manner by the users. It is a logical layer that amalgamates data from various sources without performing actual ETL process. It is an abstraction such that only the required data is visible to the users without requiring technical details about the location or structure of the data source. It provides enhanced data security.

- **Data Warehousing** is the integration of data from multiple sources to a centralized source to facilitate decision making, reporting and query handling. A centralized source of data enables better decision making.

b) Data Propagation involves copying data from one location i.e., source to another location i.e., target location. It is event driven. These applications usually operate online

and push data to the target location. They are majorly useful for real time data movement such as workload balancing, backup and recovery. Data propagation can be done asynchronously or synchronously.

The two methods for data propagation are: Enterprise Application Integration (EAI) and Enterprise Data Replication (EDR). The key advantage of data propagation is that it can be used for real-time / near-realtime data movement and can also be used for workload balancing, backup and recovery. EAI is used majorly for the exchange of messages and transactions in real-time business transaction processing; whereas for transfer of voluminous data between databases, is used.

c) Data Virtualization: In this, data is not stored in a single location, but is abstracted and can be viewed as unified view of data from multiple sources. Data Federation is a form of data virtualization, supported by Enterprise information technology (EII). EII products have evolved from two different technological backgrounds – relational DBMS and XML, but the current trend of the industry is to support both approaches, via SQL (ODBC and JDBC) and XML (XML Query Language - XQuery - and XML Path Language - XPath) data interfaces.

d) Data Warehousing is the integration of data from multiple sources to a centralized source to facilitate decision making, reporting and query handling. A centralized source of data enables better decision making.

2.2.3 Data Transformation

After the data has been acquired, it is cleaned as discussed above. The clean data may not be in a standard format. The process of changing the structure and format of data to make it more usable is called data transformation. Data transformation may be constructive, destructive, aesthetic or structural.

- *Constructive Data Transformation* involves adding, copying, and replicating data.
- *Destructive Data Transformation* involves deleting fields and records.
- *Aesthetic Data Transformation* involves standardizing salutations or street names.
- *Structural Data Transformation* involves renaming, moving, and combining columns in a database.

Data Transformation may require smoothing, aggregation, discretization, attribute Construction, generalization and normalization to make data manageable.

Scripting languages like Python or domain-specific languages like SQL are usually used for data transformation.

Data Transformation Techniques

1. Data smoothing

2. Attribute construction

3. Data aggregation

4. Data normalization

5. Data discretization

6. Data generalization

2.2.4 Data Reduction

It is the process of reducing the volume of data such that data integrity is preserved. i.e., the volume of data is reduced but the results of data mining before and after mining are the same. Data reduction increasing the efficiency of data mining. It helps in reducing the storage requirement, reduced computation time and removal of redundancy. The various techniques used for data mining are:

Dimensionality reduction, numerosity reduction and data compression.

2.2.4.1 Dimensionality Reduction: is the transformation of data from high dimensional space to a low dimensional space. It is difficult to visualize data that has higher number of features. Sometimes, most of these features are correlated, and hence redundant. This is where the need of dimensionality reduction arises. In other words, dimensionality reduction is the process of reducing the number of random variables under

consideration, by obtaining a set of principal variables. e.g., 3-dimensional data may be reduced to a 2D data. The various methods used for dimensionality reduction include:

- **Wavelet Transformation** is mostly used in image compression. It is a lossy method for dimensionality reduction, where a data vector X is transformed into another vector X' , such that both X and X' represent the same length. The result of wavelet transform can be truncated, unlike its original, thus achieving dimensionality reduction. Wavelet transforms are well suited for data cube, sparse data or data which is highly skewed.

- **Principal Component Analysis (PCA)** is applied to skewed and sparse data. In this method, the entire data set is represented by few independent tuples with 'n' attributes. This method was introduced by Karl Pearson. It works on a condition that while the data in a higher dimensional space is mapped to data in a lower dimension space, the variance of the data in the lower dimensional space should be maximum. It involves the following steps:

- i. Construct the covariance matrix of the data.

- ii. Compute the eigenvectors of this matrix.

- iii. Eigenvectors corresponding to the largest eigenvalues are used to reconstruct a large fraction of variance of the original data.

Hence, lesser number of eigenvectors are left.

- **Attribute Subset Selection:** In this method, a subset of some selected attributes is created for reducing the volume of data. The goal of attribute subset selection is to find

a minimum set of attributes such that dropping of those irrelevant attributes does not much affect the utility of data and the cost of data analysis could be reduced.

Attribute Subset Selection is done by the following methods:

1. Stepwise Forward Selection.
2. Stepwise Backward Elimination.
3. Combination of Forward Selection and Backward Elimination.
4. Decision Tree Induction.

2.2.4.2 Numerosity Reduction: is the reduction of original data and its representation in a smaller form. It can be done in two ways: parametric and non-parametric numerosity reduction.

i) Parametric Numerosity Reduction: In this method, only the data parameters are stored, instead of the entire original data. The data is represented using some model to estimate the data, so that only parameters of data are required to be stored, instead of actual data. Regression and Log-Linear methods are used for creating such models.

ii) Non- Parametric Numerosity Reduction methods are used for storing reduced representations of the data include *histograms, clustering, sampling and data cube aggregation*.

- **Histogram** is the data representation in terms of frequency. It uses binning to approximate data distribution and is a popular form of data reduction.

- **Clustering** divides the data into groups/clusters. This technique partitions the whole data into different clusters. In data reduction, the cluster representation of the data are used to replace the actual data. It also helps to detect outliers in data.
- **Sampling** can be used for data reduction because it allows a large data set to be represented by a much smaller random data sample (or subset).
- **Data Cube Aggregation** involves moving the data from detailed level to a fewer number of dimensions. The resulting data set is smaller in volume, without loss of information necessary for the analysis task.

2.2.4.3 Data Compression is a technique in which the original data is compressed for reduction. This compressed data can again be reconstructed to form the original data. If the data is reconstructed without losing any information, then it is a 'lossless' data reduction.

2.3 DATA MINING

Data mining is the process that helps in extracting information from a given data set to identify trends, patterns, and useful data. The objective of using data mining is to make data-supported decisions from enormous data sets. Different types of data can be mined such as Data stored in database, data warehouse, transactional data and other types of data such as data streams, engineering design data, sequence data, graph data, spatial data, multimedia data, and more. In recent data mining projects, various major data mining techniques have been developed and used, including association, classification, clustering, prediction, sequential patterns, and regression.

- **Association** is a data mining technique useful to discover a link between two or more items. It uses if-then statements to show the probability of interactions between data items within large data sets. It finds a hidden pattern in the data set. It is commonly used to help sales correlations in data or medical data sets.
- **Clustering** is a data mining technique in which clusters are created of objects that share the same characteristics. Clusters relate to hidden patterns. It is based on unsupervised learning.
- **Classification** classifies items or variables in a data set into predefined groups or classes. It is based on unsupervised learning.
- **Prediction** is used in predicting the relationship that exists between independent and dependent variables as well as independent variables alone. It can be used to predict future trends. It analyses past events or instances in the right sequence to predict a future event.
- **Sequential patterns** is a data mining technique specialized for evaluating sequential data to discover sequential patterns. It comprises of finding interesting subsequence in a set of sequences, where the stake of a sequence can be measured in terms of different criteria like length, occurrence frequency, etc.

2.3.1 Data Mining Applications

Data mining is useful in predictions, classification, clustering and trends, which makes it applicable to many domains like health care, fraud detection, education, market basket

analysis, manufacturing, sales, customer relationship management, finance and banking etc.

2.4 DATA INTERPRETATION

The process of reviewing data through some predefined processes to be able to assign some meaning to the data and arrive at a relevant conclusion is called data interpretation. It involves taking the result of data analysis, making inferences on the relations studied, and using them to reach conclusions and develop recommendations. Data interpretation is done by analyst to make inferences from the data. There are two ways to interpret data namely **Qualitative** and **Quantitative**.

Qualitative Data Interpretation: Qualitative data also called categorical data, does not contain numbers, it consists of text, pictures, observations, symbols etc. The interpretation of patterns and themes in qualitative data is done using qualitative methods.

Quantitative Data Interpretation is done on quantitative data i.e. numerical data. It involves statistical methods such as mean, standard deviation, variance, frequency distribution, regression etc. Data analysis and interpretation, regardless of method and qualitative/quantitative status, may include the following characteristics:

- Data identification and explanation
- Comparing and contrasting of data
- Identification of data outliers

- Future predictions

Data Interpretation is helpful in improving processes by identifying problems. Data interpretations is used for predicting trends after studying the patterns in the data. It is helpful in better decision making.

Preprocessing Examples

Take a look at the table below to see how preprocessing works. In this example, we have three variables: name, age, and company. In the first example we can tell that #2 and #3 have been assigned the incorrect companies.

Name	Age	Company
Karen Lynch	57	CVS Health
Elon Musk	49	Amazon
Jeff Bezos	57	Tesla
Tim Cook	60	Apple

We can use data cleaning to simply **remove these rows**, as we know the data was improperly entered or is otherwise corrupted.

Name	Age	Company
Karen Lynch	57	CVS Health
Tim Cook	60	Apple

Or, we can perform **data transformation**, in this case, manually, in order to fix the problem:

Name	Age	Company
Karen Lynch	57	CVS Health
Elon Musk	49	Tesla

Name	Age	Company
Jeff Bezos	57	Amazon
Tim Cook	60	Apple

Once the issue is fixed, we can perform **data reduction**, in this case by descending age, to choose which age range we want to focus on:

Name	Age	Company
Tim Cook	60	Apple
Karen Lynch	57	CVS Health
Jeff Bezos	57	Amazon
Elon Musk	49	Tesla

2.5 QUESTIONS

1. What does the “NaN” value represent in a dataset?

- a) Not available
- b) Not a number
- c) Missing value
- d) All of the above

Answer: d) All of the above

While **None** = represent absence of some value in Python

2. Which method can be used to remove duplicate rows in a dataset?

- a) fillna()
- b) drop_duplicates()
- c) replace()
- d) groupby()

3. Which of the following is NOT a way to handle missing values?

- a) Dropping rows
- b) Imputation with mean/median/mode
- c) Filling with random values

d) Predicting missing values using ML

4. If a column has 90% missing values, what is the best practice?

- a) Fill with mean
- b) Fill with zero
- c) **Drop the column**
- d) Ignore it

5. Which pandas function can identify missing values?

- a) isnull() ☒
- b) notnull()
- c) info()
- d) All of the above ☒

Explain the difference between dropna() and fillna().

Answer:

- dropna() removes rows (or columns) with missing values.
- fillna() replaces missing values with a specified value, mean, median, or mode.

What are outliers, and why should they be handled?

Answer:

- Outliers are data points significantly different from others.
- They can skew statistics, affect model performance, and reduce accuracy.

Why is data type conversion important in cleaning?

Answer:

- Incorrect data types (e.g., string instead of numeric) prevent calculations and ML models from running correctly.
- Converting ensures proper operations and memory optimization.

Write code to Detect missing values in a dataset df.

```
import pandas as pd
# Check missing values
```

```
missing = df.isnull().sum()
print(missing)
```

Fill missing numeric values with median and categorical with mode.

```
# Numeric columns
num_cols = df.select_dtypes(include='number').columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Categorical columns
cat_cols = df.select_dtypes(include='object').columns
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

Remove duplicate rows from df.

```
df = df.drop_duplicates()
```

Remove outliers using IQR method for a column Revenue.

```
Q1 = df['Revenue'].quantile(0.25)
Q3 = df['Revenue'].quantile(0.75)
IQR = Q3 - Q1
# Filter data
df_clean = df[(df['Revenue'] >= Q1 - 1.5*IQR) & (df['Revenue'] <= Q3 + 1.5*IQR)]
```

Convert Date column to datetime format.

```
df['Date'] = pd.to_datetime(df['Date'])
```

Standardize column names (lowercase, replace spaces with underscores).

```
df.columns = df.columns.str.lower().str.replace(' ', '_')
```

- What are the various types of data?
- Explain the concept of data preprocessing. What are the techniques used for preprocessing of data?
- Write a short note on Data mining.
- What are the techniques used for data mining?
- What is the importance of data Interpretation?

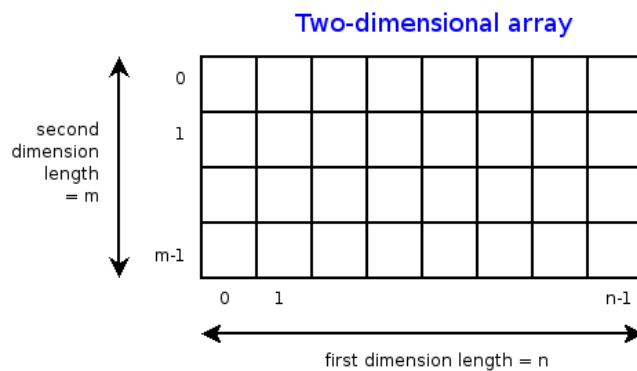
Data life cycle (continue)

1. **Data Representation**, i.e., what is the natural way to think about given data

One-dimensional Arrays, Vectors

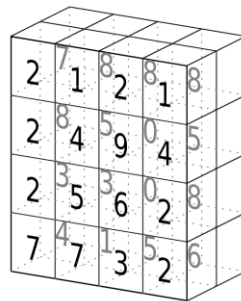
0.1	2	3.2	6.5	3.4	4.1
-----	---	-----	-----	-----	-----

"data"	"representation"	"info"
--------	------------------	--------



3	1	4	1
5	9	2	6
5	3	5	8
9	7	9	3
2	3	8	4
6	2	6	4

tensor of dimensions [6,4]
(matrix 6 by 4)



tensor of dimensions [4,4,2]

Indexing

Slicing/subsetting

Filter

'map' → apply a function to every element

'reduce/aggregate' → combine values to get a single scalar (e.g., sum, median)

Given two vectors: **Dot and cross products**

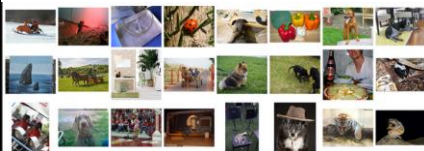
n-dimensional array operations

+

Linear Algebra

Matrix/tensor multiplication
Transpose
Matrix-vector multiplication
Matrix factorization

Sets: of Objects



Filter

Map

Union

Reduce/Aggregate

Given two sets, **Combine/Join** using "keys"

Group and then aggregate

Tables

Filter rows or columns

company	division	sector	tryint
00nil_Combined_Company	00nil_Combined_Division	00nil_Combined_Sector	14625
apple	00nil_Combined_Division	00nil_Combined_Sector	10125
apple	hardware	00nil_Combined_Sector	4500
apple	hardware	business	1350
apple	hardware	consumer	3150
apple	software	00nil_Combined_Sector	5625
apple	software	business	4950
apple	software	consumer	675
microsoft	00nil_Combined_Division	00nil_Combined_Sector	4500
microsoft	hardware	00nil_Combined_Sector	1890
microsoft	hardware	business	855
microsoft	hardware	consumer	1035
microsoft	software	00nil_Combined_Sector	2610
microsoft	software	business	1215
microsoft	software	consumer	1395

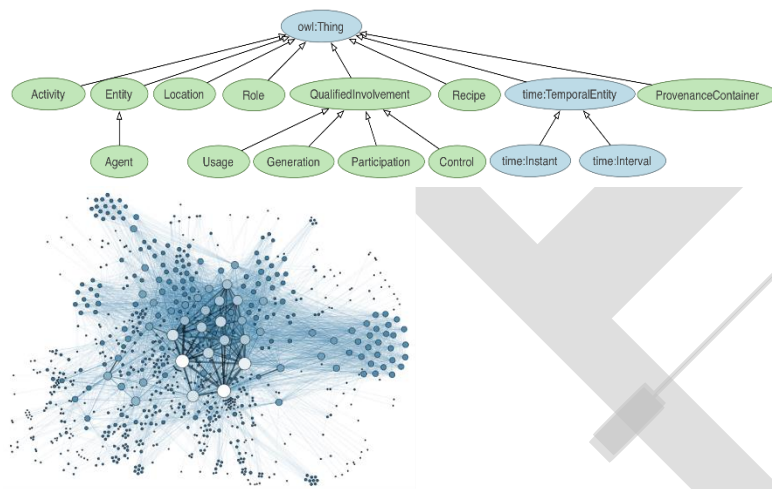
”Join” two or more relations
”Group” and **”aggregate”**
 them

Relational Algebra formalizes
 some of them

**Structured Query Language
 (SQL)**

**Many other languages and
 constructs, that look very
 similar**

Trees/Graphs



”Path” queries
**Graph Algorithms and
 Transformations**
Network Science

*Somewhat more ad hoc and
 special-purpose
 Changing in recent years*

2. **Data Processing Operations**, which take one or more datasets as input and produce one or more datasets as output

Python Data Manipulation libraries:

1. **NumPy**: Python Library for Manipulating nD Arrays

Multidimensional Arrays, and a variety of operations including Linear Algebra

2. **Pandas**: Python Library for Manipulating Tabular Data

Series, Tables (also called DataFrames)

Many operations to manipulate and combine tables/series

3. Relational Databases

Tables/Relations, and SQL (similar to Pandas operations)

4. Apache Spark

Sets of objects or key-value pairs

MapReduce and SQL-like operations

Types of Data Analytics:

descriptive analytics, **diagnostic** analytics, **predictive** analytics, and **prescriptive** analytics. Each has its nature and the type of tasks performed in it.

NumPy DataTypes:

Variety of data types than are built-in in Python by default. Defined by the `numpy.dtype` class and include:

- `intc` and `intp` (used for indexing)
- `int8`, `int16`, `int32`, `int64`
- `float16`, `float32`, `float64`
- `complex64`, `complex128`
- `bool_`, `int_`, `float_`, `complex_`

Example:

```
import numpy as np
x=np.float32(1.0)
print(x) # 1.0
y=np.int_([1,2,4])
print(y) #array([1,2,4])
z=np.arange(3, dtype=np.uint8)
print(z)      # array([0,1,2], dtype=uint8)
print(z.dtype) #dtype('uint8')
```


Example:

```
import numpy as np
x=np.array([2,3,1,0])
x=np.array([ [1, 2.0], [ 0, 0], [1+ 1j, 3.]])
x=np.array([ [ 1.0+0.j, 2. + 0.j], [0. + 0.j, 0.+0.j], [1. +1.j, 3.+0.j]])
```

zeros(shape)- creates an array filled with **0** values with the specified shape. The default **dtype** is **float64**.

```
import numpy as np
ex=np.zeros((2,3)) #array([[0., 0., 0.], [0., 0., 0.]])
```

ones(shape)- creates an array filled with **1** values with the specified shape.

arange()- same as **range** in Python

```
import numpy as np
ex=np.arange(10) #array([0,1,2,3,4,5,6,7,8,9])
y=np.arange(2, 10, dtype=np.float) #array([2., 3., 4., 5., 6., 7., 8., 9.])
z=np.arange(2, 3, 0.2) #array([2., 2.2, 2.4, 2.6, 2.8])
```

linspace()- creates arrays with a specified number of elements and spaced equally between the specified beginning and end values.

```
import numpy as np
ex=np.linspace(1., 4., 6) #array([1., 1.6, 2.2, 2.8, 3.4, 4.])
```

random.random(shape)- creates arrays with random floats over the interval [0, 1)

```
import numpy as np
ex=np.random.random((2,3)) #array([[ 0.75688597, 0.41759916, 0.35007419],
[0.77164187, 0.05869089, 0.98792864]])
```

Printing an array:

```
import numpy as np
a=np.arange(3)
print(a) #[0 1 2] or array([0,1,2])
b=np.arange(9).reshape(3,3)
print(b)
# [[ 0  1  2]
#   [ 3  4  5]
#   [ 6  7  8]]
c=np.arange(8).reshape(2,2,2)
print(c)
# [[ 0  1]
#   [ 2  3]]

# [[ 4  5]
#   [ 6  7]]
```

Indexing:

```
import numpy as np
a=np.arange(3)
x=np.arange(10)
print(x[2]) #2
print(x[-2]) #8
x.shape=(2,5) # now x is 2-dimensional
print(x[1,3]) #8
print(x[1,-1]) #9
#Sclcing

x=np.arange(10)

print(x[2:5]) #array([2,3,4])

print(x[:-7]) #array([0,1,2])

print(x[1:7:2]) #array([1,3,5])
```

```
y=np.arange(35).reshape(5,7)
print(y[1:5:2, :: 3]) #array([[7,10,13], [21,24,27]])
```

Array Operations

```
>>> a = np.arange(5)
>>> b = np.arange(5)
>>> a+b
array([0, 2, 4, 6, 8])
>>> a-b
array([0, 0, 0, 0, 0])
>>> a**2
array([ 0,  1,  4,  9, 16])
>>> a>3
array([False, False, False, False,  True], dtype=bool)
>>> 10*np.sin(a)
array([ 0.,  8.41470985,  9.09297427,  1.41120008, -
 7.56802495])
>>> a*b
array([ 0,  1,  4,  9, 16])
```

```

>>> a = np.zeros(4).reshape(2,2)
>>> a
array([[ 0.,  0.],
       [ 0.,  0.]])
>>> a[0,0] = 1
>>> a[1,1] = 1
>>> b = np.arange(4).reshape(2,2)
>>> b
array([[0, 1],
       [2, 3]])
>>> a*b
array([[ 0.,  0.],
       [ 0.,  3.]])
>>> np.dot(a,b)
array([[ 0.,  1.],
       [ 2.,  3.]])

```

There are also some built-in methods of ndarray objects.

Universal functions which may also be applied include exp, sqrt, add, sin, cos, etc.

```

>>> a = np.random.random((2,3))
>>> a
array([[ 0.68166391,  0.98943098,
         0.69361582],
       [ 0.78888081,  0.62197125,
         0.40517936]])
>>> a.sum()
4.1807421388722164
>>> a.min()
0.4051793610379143
>>> a.max(axis=0)
array([ 0.78888081,  0.98943098,
        0.69361582])
>>> a.min(axis=1)
array([ 0.68166391,  0.40517936])

```

ARRAY OPERATIONS

An array shape can be manipulated by a number of methods.

`resize(size)` will modify an array in place.

`reshape(size)` will return a copy of the array with a new shape.

```
>>> a =  
np.floor(10*np.random.random((3,4)))  
>>> print(a)  
[[ 9.  8.  7.  9.]  
 [ 7.  5.  9.  7.]  
 [ 8.  2.  7.  5.]]  
>>> a.shape  
(3, 4)  
>>> a.ravel()  
array([ 9.,  8.,  7.,  9.,  7.,  5.,  9.,  
        7.,  8.,  2.,  7.,  5.])  
>>> a.shape = (6,2)  
>>> print(a)  
[[ 9.  8.]  
 [ 7.  9.]  
 [ 7.  5.]  
 [ 9.  7.]  
 [ 8.  2.]  
 [ 7.  5.]]  
>>> a.transpose()  
array([[ 9.,  7.,  7.,  9.,  8.,  7.],  
       [ 8.,  9.,  5.,  7.,  2.,  5.]])
```

Linear Algebra

One of the most common reasons for using the NumPy package is its linear algebra module.

```
>>> from numpy import *
>>> from numpy.linalg import *
>>> a = array([[1.0, 2.0],
               [3.0, 4.0]])

>>> print(a)
[[ 1.  2.]
 [ 3.  4.]]

>>> a.transpose()
array([[ 1.,  3.],
       [ 2.,  4.]])

>>> inv(a) # inverse
array([[ -2. ,  1. ],
       [ 1.5, -0.5]])
```

```
>>> u = eye(2) # unit 2x2 matrix; "eye" represents "I"
>>> u
array([[ 1.,  0.],
       [ 0.,  1.]])
>>> j = array([[0.0, -1.0], [1.0, 0.0]])
>>> dot(j, j) # matrix product
array([[ -1.,  0.],
       [ 0., -1.]])
>>> trace(u) # trace (sum of elements on diagonal)
2.0
>>> y = array([[5.], [7.]])
>>> solve(a, y) # solve linear matrix equation
array([[ -3.],
       [ 4.]])
>>> eig(j) # get eigenvalues/eigenvectors of matrix
(array([ 0.+1.j, 0.-1.j]),
 array([[ 0.70710678+0.j, 0.70710678+0.j],
       [ 0.00000000-0.70710678j,
        0.00000000+0.70710678j]]))
```

SciPy

SciPy is a collection of mathematical algorithms and convenience functions built on the **NumPy** extension of Python. It adds significant power to the interactive Python

session by providing the user with high-level commands and classes for manipulating and visualizing data.

Basically, SciPy contains various tools and functions for solving common problems in **scientific computing.**

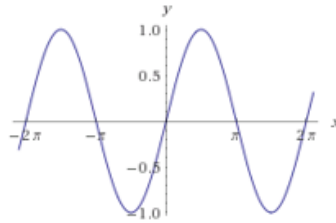
Some functionality:

- Special mathematical functions (scipy.special) -- elliptic, bessel, etc.
- Integration (scipy.integrate)
- Optimization (scipy.optimize)
- Interpolation (scipy.interpolate)
- Fourier Transforms (scipy.fftpack)
- Signal Processing (scipy.signal)
- Linear Algebra (scipy.linalg)
- Compressed Sparse Graph Routines (scipy.sparse.csgraph)
- Spatial data structures and algorithms (scipy.spatial)
- Statistics (scipy.stats)
- Multidimensional image processing (scipy.ndimage)
- Data IO (scipy.io) – overlaps with pandas, covers some other formats

SciPy Example: integration

Compute:

$$\int_a^b \sin x \, dx$$



scipy.integrate

np.sin function defines the sin function

to compute the definite integral from x:0 to pi using the quad function

```
>>> res = scipy.integrate.quad(np.sin, 0, np.pi)
>>> print(res)
(2.0, 2.220446049250313e-14) # 2 with a very small error
margin!
>>> res = scipy.integrate.quad(np.sin, -np.inf, +np.inf)
>>> print(res)
(0.0, 0.0) # Integral does not converge
```

```
>>> sample_x = np.linspace(0, np.pi, 1000)
>>> sample_y = np.sin(sample_x) # Creating 1,000 samples
>>> result = scipy.integrate.trapz(sample_y, sample_x)
>>> print(result)
1.99999835177

>>> sample_x = np.linspace(0, np.pi, 1000000)
>>> sample_y = np.sin(sample_x) # Creating 1,000,000
samples
>>> result = scipy.integrate.trapz(sample_y, sample_x)
>>> print(result)
2.0
```


Chapter Three- Data Representation

Objectives

3.1 Big Data Analytics

3.2 Data Science – working

3.3 Facets of Data

3.3.1 Natural Language Data

3.3.2 Machine Generated Data

3.3.3 Streaming Data

3.3.4 Acoustic, Video and Image Data

3.3.5 Sensor Data

3.3.6 Graph/ Network Data

3.4 Multiple Problems in Handling Large Datasets

3.5 General Techniques for Handling Large Data

3.5.1 Choosing the Right Algorithm

3.5.2 Right Data Structure

3.5.3 Choosing the Right Tools

3.6 Distributing Data Storage and Processing with Frameworks

3.6.1 Hadoop

3.6.2 Apache Spark

3.6.3 Apache Storm

3.6.4 Samza

3.6.5 Flink

3.7 Questions

OBJECTIVES

1. To familiarize with the data representation and types of data
2. To familiarize with the general techniques of handling data

3.1 BIG DATA ANALYTICS

The assortment of any kind of data which is large, highly complex and difficult to manage and handle in the real time scenario with the traditional management techniques becomes the Big Data.

The most extensively technique for the management of data was **Relational DBMS** which was a kind of dataset that fits in all the scenarios for the storage, manipulation and interpretation of data but it **failed** in case of **big data**.

A groundbreaking study in 2013 reported 90% of the entirety of the world's data has been created within the previous two years. In the previous two years researchers have composed and handled data which is 9 times the data being processed in the previous 1000 years of the survival. As per the statistical institute, already 2.7 zettabytes of data have been generated and by 2025 it might escalate to a limit beyond belief which ca be 100 zettabytes or even more.

What do we do with all of this data? How do we make it useful to us? What are its real-world applications? These questions are the domain of data science.

3.2 DATA SCIENCE - WORKING

In order to handle the query for complete in depth and a sophisticated approach for the exploration of the raw unprocessed data into the real time multiple disciplines with the expertise in machine learning and AI based dimensions, data science gives the solution with the most advanced means and methods.

The scrutiny of the relevant information from the irrelevant raw data and pass on or forward only the most vital data for the enhancing the computing efficiency with the revolution in the fields of engineering, mathematics, statistics, advanced computing and visualizations.

The data science is the basic field of exploration of data where the researchers or the data scientists also rely heavily on artificial intelligence for the creation of simulated models and then predicting the values of the parameters with the algorithms designed for manipulation of data into information. The working in the field of the data science is thereby based on the types of data being explored and manipulation needed e.g., if the simple data in form of tables is available then RDBMS is used and if image data is present then RDBMS fails.

3.3 FACETS OF DATA

In data science and big data, the data required for the manipulation and interpretation changes with the change in the types of tools and techniques applied in the algorithm.

The types of the data explored in the field of data science can be categorized as follows:

- Structured

- Unstructured
- Natural language
- Machine-generated
- Streaming
- Acoustic data, video, and images
- Sensor Data
- Graph-based/ Network data

As in the previous chapter we have explored the data categories structured versus unstructured data. Let's talk about the rest of the representation of data and the explore the characteristics. The structured data is the main type of data explored in a model and can be expressed as a record with a field of fixed length and dimension. The RDBMS and the Excel data is usually the structured data with known or used defined datatypes with structural information. But the unstructured data is data that may not fit any kind of mathematical or simulated model for the study of data due to the context-specific or varying nature of the stored raw data. One example of unstructured data is the email.

3.3.1 Natural Language Data

Another special type of unstructured data is the Natural language data with the challenging approach to progression the data and manipulate it. The handling of large amount of natural language data and processing in itself necessitates the data scientists

to acquire the knowledge of specific data science techniques and linguistics. The natural language processing community has had success in entity recognition, topic recognition, summarization, text completion, and sentiment analysis, but models trained in one domain don't generalize well to other domains. Even state-of-the-art techniques aren't able to decipher the meaning of every piece of text. The meaning of the same words can vary when coming from someone upset or joyous.

3.3.2 Machine Generated Data

The machine or the computer has the capability to generate the data based on the requirement of the developer or the application developed. The data generated by the machine or any robotic process without the intervention of a human is a fast process and also help to forecast certain information for the application, or other machine without human intervention. Machine-generated data is becoming a major data resource and will continue to do so. Wikibon has forecast that the market value of the industrial Internet (a term coined by Frost & Sullivan to refer to the integration of complex physical machinery with networked sensors and software) will be approximately \$540 billion in 2020. IDC (International Data Corporation) has estimated there will be 26 times more connected things than people in 2020.

This network is commonly referred to as the internet of things. The analysis of machine data relies on highly scalable tools, due to its high volume and speed. Examples of machine data are web server logs, call detail records, network event logs, and telemetry.

3.3.3 Streaming Data

The streaming data when being administered for the information can take any form and format which is an interesting property. This type of data only gets loaded into the server or the data warehouse when any relevant activity occurs or there's an interpretable change in the parameters. The data is not accessed in batch mode. This type of data is not actually a different category but the system for the processing of such varying formats needs to be adaptive to the minutest variations in the information to be handled.

Examples are the 'What's trending' on Twitter, live sporting or music events, and the stock market.

3.3.4 Acoustic, video and image data

The world of animation and digitization has developed multimedia datasets with audio, video and images. These types of datasets can be stored, handled and interpreted with the Object-Oriented Databases. The databases include the class inheritance and interface in a pre-specified format for handling the complexity of the data and finding its application in Digital libraries, video-on demand, news-on demand, musical database, etc. The other type of information in the multimedia datasets can be represented and reproduced as sensory experiences - touch, sense and hear which are typically leading to storing a huge amount of data handled by digitizing. Furthermore, data compression for images and sounds can exploit limits on human senses performed by throwing away

information not needed for good-quality experience which is performed by compression.

There are certain **limitations** when you deal with such a large amount of data are described below:

1. Range is limited and might lead to misinformation

- only certain pitches and loudness can be heard
- only certain kinds of light are visible, and there must be enough / not too much light.

2. Discrimination due to the descriptive features of the data

- pitches, loudness, colors, intensities can't be distinguished unless they are different enough (color1, color2)

3. Coding the information of the sensory data into digital world.

- nervous systems —encode|| experience, e.g., rods and cones in the eye

3.3.4.1 REPRESENTATION OF IMAGE DATA

The image data is expanding vastly and due to enhancement in the cameras the encoding is needed for computation and manipulation of image data. The techniques are vector array-based encoding and bit map-based encoding.

Vector graphics encode the image sequences as a series of lines or curves. The process is expensive in term of image computation but smoothly rescales.

Bit map encode the image as an array of pixels. The encoding process is cost effective in terms of computation but scales inefficiently leading to loss of image data.

The Basic idea of image data is the array of receptors where each receptor records a pixel by “counting” the number of photons that strike it during exposure. The Red, green, blue recorded separately at each point on image produced by group of three receptors where each receptor is behind a color filter.

Representation of Acoustic data: In recent years, the analysis of acoustic characteristics of speech and sound has been one of the areas that data mining has found its way through. The present research study is also related to this topic which aims to detect the gender of the speaker by using the acoustic feature of his voice. When an instrument is played or a voice speaks, periodic (many times per second) changes occur in air pressure, which we interpret as acoustics. For the representation of the acoustic data compression is needed. Codecs (compression/decompression) implement various compression/decompression techniques which are either lossy or lossless. The lossy compression of the acoustic data may lead to loss of certain information as it is non-repetitive: MPEG (like JPEG) a family of perceptually-based techniques are all lossy

techniques. While the other type of techniques is where the information of the acoustic data is preserved in which WMA Lossless, ALAC, MPEG-4 ALS methods are applied.

The encoding of data in form of the sounds heard or the visuals captured are highly challenging for the data scientists to process the information. Some tasks performed by the human brain have been of great difficulty to the computers for the recognition of the object in the images. MLBAM (Major League Baseball Advanced Media) announced in 2014 that they'll increase video capture to approximately 7 TB per game for the purpose of live, in-game analytics. The higher resolution cameras and acquiring sensors have the capability to capture the motion of ball and the player in a real time scenario e.g., the path taken by a defender relative to two baselines.

Recently a company called DeepMind succeeded at creating an algorithm that's capable of learning how to play video games. This algorithm takes the video screen as input and learns to interpret everything via a complex process of deep learning. It's a remarkable feat that prompted Google to buy the company for their own Artificial Intelligence (AI) development plans. The learning algorithm takes in data as it's produced by the computer game; it's streaming data.

3.3.5 Sensor Data

Any input acquired from the physical environment which is detected and responded from the devices as an output is the collaborative approach of the sensor data. The extracted data from the sensor devices act as an output for a real time system or as an input to another system for the performance of any activity. These sensors can be used

to perceive any type of physical element with the approach to detect the events or changes in the environment. A sensor is always used with other electronics, as simple as a lamp or as complex as a computer. Advanced chip technology makes it possible to integrate all the required functions at low cost, in a small volume and with low energy consumption. The number of sensors around us is increasing rapidly. Estimates vary, but many expect that by 2030 more than 500 billion sensors will be connected to each other via the Internet of Things (IoT).

The exponential growth of the IoT based systems leads to ever demanding rise in the input sensor devices which are responsible for the collection, storage and interpretation of the captured data. In addition, consumers, organizations, governments and companies themselves produce more and more data, for example on social media. The amount of data is growing exponentially. People speak of Big Data when they work with one or more datasets that are too large to be maintained with regular database management systems.

The pros of applying sensor data to the input devices is that the decisions of the IoT devices are subjected to information accessed from the evidences and not from any kind of irrelevant details and subjective experiences. This type of knowledge base makes the system cost effective with the enhanced streamlined processes, boosted product quality and better services. By combining data intelligently and by interpreting / translating, new insights are created that can be used for new services, applications and markets. This information can also be combined with data from various external sources, such as weather data or demographics.

3.3.6 Graph/ Network Data

“Graph data” is the type of data which can be represented as graph with a special characteristic of comprising the mathematical graph theory into the mined information.

“Graph” in the network data generally represents a model in the statistical and mathematical domain with pair wise relationship in the constituent objects. Graph or network data is, in short, data that focuses on the relationship or adjacency of objects. The representation of the graph models includes the nodes, edges, and relationships between the stored data in the nodes. Graph-based data is a natural way to represent social networks, and its structure allows you to calculate specific metrics such as the influence of a person and the shortest path between two people.

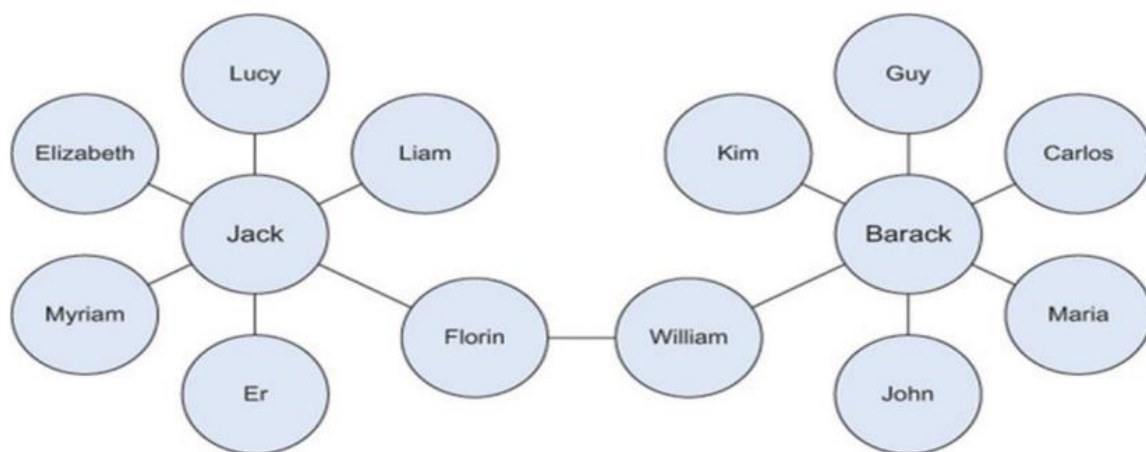


Fig. 3.6.3 Friends in social network are example of Graph Network

Examples of graph-based data can be found on many social media websites (figure 3.6.3). For instance, on LinkedIn you can see who you know at which company. Your follower list on Twitter is another example of graph-based data. Graph databases are

used to store graph-based data and are queried with specialized query languages such as SPARQ.

3.4 MULTIPLE PROBLEMS IN HANDLING LARGE DATASETS

The multiple problems in the large datasets are discussed by numerous data scientists with the need to understand the growing demand of the data and handling the mathematical as well as statistical operations for the manipulation of the data. In the last two years, over 90% of the world's data was created, and with 2.5 quintillion bytes of data generated daily, it is clear that the future is filled with more data, which can also mean more data problem in context to the following:

- Collecting, storing, sharing and securing data
- Creating and utilizing meaningful insights from their data.

Some common big data problems and the respective solutions have been discussed below.

1. Lack of Understanding

The lack of understanding of the mined data in the data science-based companies might lead to knocked down the performance in many areas. Many of the major areas for the data- based companies were: depreciate the expenses of mining information, innovate new ideas for interpretation, new product launching, enhance performance and so on. Despite the benefits, companies have been slow to adopt data for a data centric approach.

Solution: Follow a top-down approach for the introduction and manipulation of the data science based on the procedures followed up. In case of lack of a data science professional, the consultancy services or an IT proficient with data science knowledge should be hired to get a better understanding.

2. High Cost of Data Solutions

The companies have understood that buying and maintaining of necessary components make the system less cost effective. In addition to cost of the servers and the software-based storage, the high-end cost of the data science experts makes the system time consuming.

Solution: The solution is to understand the need and use of the data with a collaborative method to find a goal, conduct a research or solution and implement the execution with a plan.

3. Too Many Choices

Coined as the “paradox of choice” Schwartz explains how option overload can cause inaction on behalf of a buyer. In the world of data and data tools, the options are almost as widespread as the data itself, so it is understandably overwhelming when deciding the solution that’s right for the business, especially when it will likely affect all departments and hopefully be a long-term strategy.

Solution: Like understanding data, a good solution is to leverage the experience of your in-house expert, perhaps a CTO. If that’s not an option, hire a consultancy firm to assist

in the decision-making process. Use the internet and forums to source valuable information and ask questions.

4. Complex Systems for Managing Data

The systems to understand the data management and finding a relevant solution for the manipulation of data is in itself a problem. Due to the vast expanse of the different types of data with the IT teams creating their own data during the process of data handling results in increased complexity.

Solution: Find a solution with a single command center, implement automation whenever possible, and ensure that it can be remotely accessed 24/7.

5. Security Gaps

Another important aspect of the data science is the security of the data and the biasing of the large amount of data is always possible. In order to handle it the encryption and decryption must be performed with the data store with proper storage.

Solution: The data need to be handled with automated security updates of the data warehouse and automated backups.

6. Low Quality and Inaccurate Data

Having data is only useful when it's accurate. Low quality data not only serves no

purpose, but it also uses unnecessary storage and can harm the ability to gather insights from clean data.

A few ways that data can be considered low quality is:

- Inconsistent formatting (which will take time to correct and can happen when the same elements are spelled differently like —"US" versus —"U.S."),
- Missing data (i.e. a first name or email address is missing from a database of contacts),
- Inaccurate data (i.e. it's just not the right information or the data has not be updated).
- Duplicate data (i.e. the data is being double counted)
- If data is not maintained or recorded properly, it's just like not having the data in the first place.

Solution: Begin by defining the necessary data you want to collect (again, align the information needed to the business goal). Cleanse data regularly and when it is collected from different sources, organize and normalize it before uploading it into any tool for analysis.

Once you have your data uniform and cleansed, you can segment it for better analysis.

7. Compliance Hurdles

When collecting information, security and government regulations come into play.

With the somewhat recent introduction of the General Data Protection Regulation (GDPR), it's even more important to understand the necessary requirements for data collection and protection, as well as the implications of failing to adhere. Companies have to be compliant and careful in how they use data to segment customers for example deciding which customer to prioritize or focus on. This means that the data must: be a representative sample of consumers, algorithms must prioritize fairness, there is an understanding of inherent bias in data, and Big Data outcomes have to be checked against traditionally applied statistical practices.

Solution: The only solution to adhere to compliance and regulation is to be informed and well-educated on the topic. There's no way around it other than learning because in this case, ignorance is most certainly not bliss as it carries both financial and reputational risk to your business. If you are unsure of any regulations or compliance you should consult expert legal and accounting firms specializing in those rules.

3.5 General Techniques for Handling Large Data

The multiple challenges have been discussed in the section above and the solutions for the defies are categorized based on the algorithms and out of memory errors. Never ending algorithms, out-of-memory errors, and speed issues are the most common

challenges you face when working with large data. In this section, we'll investigate solutions to overcome or alleviate these problems.

The solutions can be divided into three categories: using the correct algorithms, choosing the right data structure, and using the right tools. (Figure 3.1.4)

There is no such relationship between the problems discussed in the section above and the solutions to be incorporated. There are multiple solutions to a given problem which can also handle the issue of memory and computational overhead. This type of data science solutions can be generalized for challenges in the exploration of data. For instance, the compression and decompression of the data set help resolve the memory issues but this also affects computation speed with a shift from the slow hard disk to the fast CPU. Contrary to RAM (random access memory), the hard disc will store everything even after the power goes down, but writing to disc costs more time than changing information in the fleeting RAM.

When constantly changing the information, RAM is thus preferable over the (more durable) hard disc.

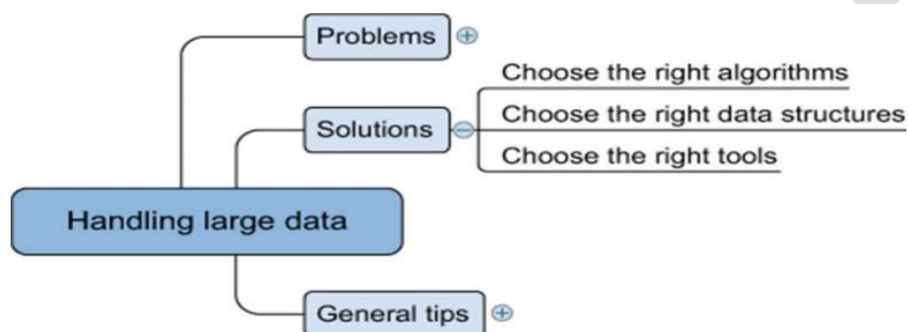


Fig. 3.1.4 Overview of solutions for handling large data sets

3.5.1 Choosing the Right Algorithm

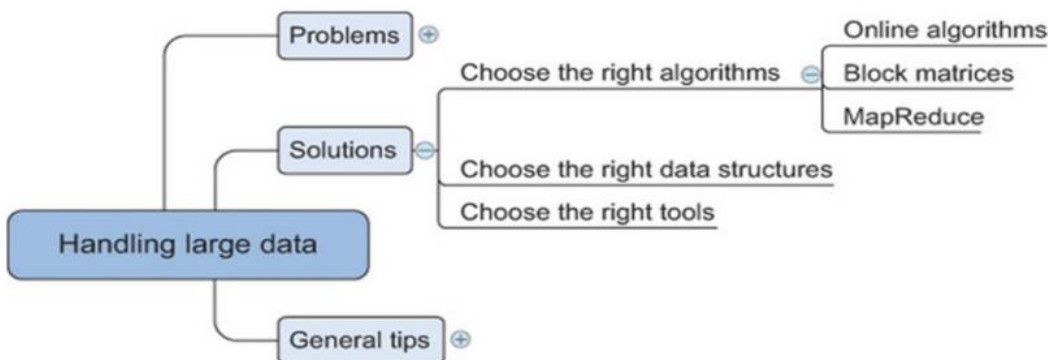


Fig. 3.1.5 The algorithms for handling the Big Data

The selection of an effective algorithm for the processing of the data can provide better results as compared to the enhanced hardware. In order to perform the predictive or selective analysis the best-chosen algorithm need not necessarily load the complete data into the memory or RAM, it rather supports the parallel computations with parallel or distributed databases. In this section three types of algorithms have been discussed that can perform the computations parallelly reducing the computation or memory overhead: online algorithms, block algorithms, and MapReduce algorithms, as shown in figure 3.1.5.

Several, but not all, machine learning algorithms can be trained using one observation at a time instead of taking all the data into memory. The model can be trained based on the current parameters and the previous parameter values can be made to forget by the algorithm.

This technique of “use and forget “ helps to attain high memory effectiveness in the system.

This way as the new data values is acquired by the algorithm, the previous values are forgotten or overwritten but the effect can be witnessed or observed in the performance metrics of the proposed model. Most online algorithms can also handle mini-batches; this way, the data science expert or the developer can feed the batches of 10 to 1,000 observations at one single instance and then applying the sliding window protocol to access the data. This learning can be handled by multiple means discussed as follows:

- Full batch learning (also called statistical learning) —Feed the algorithm all the data at once.
- Mini-batch learning —Feed the algorithm a spoonful (100, 1000, ..., depending on what your hardware can handle) of observations at a time.
- Online learning —Feed the algorithm one observation at a time.

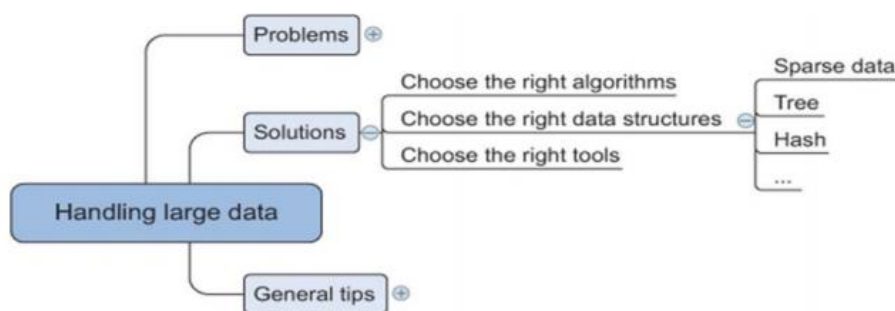


Fig. 3.1.6 Data structures applied in the data science

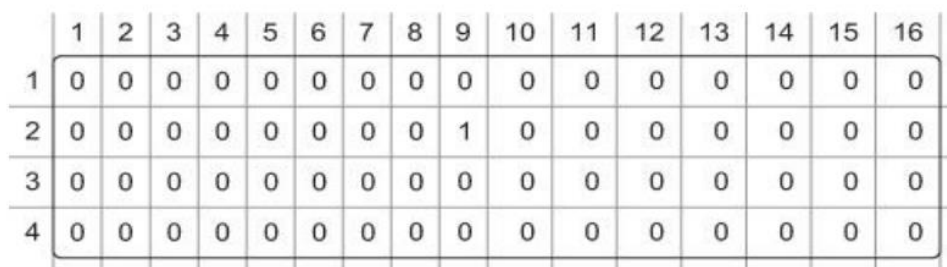
3.5.2 Right Data Structure

The algorithms as discussed can enhance the performance and execution for the manipulation of the data in the warehouse. This process in the data science field actually leads to fragmentation in the raw data so that is the reason the structures for the storage of the data is equally important for the data scientist or a data science researcher. Data structures have different storage requirements, but also influence the performance of CRUD (create, read, update, and delete) and other operations on the data set.

• Sparse Data

Most of the IT professional and the data scientist understand the representation through a sparse matrix. Similarly, the sparse data illustration can be done by giving a minimal detail about the data as compared to the total number of entries.

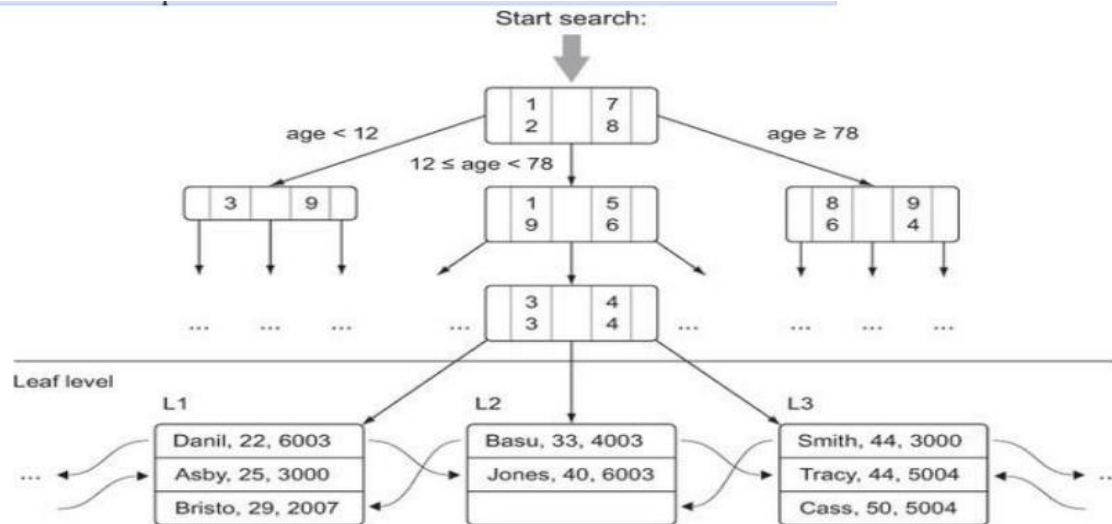
As in figure 3.1.7, the conversion of the data from the text to binary or an image to binary can be expressed as the sparse data. Imagine a set of 100,000 completely unrelated Twitter tweets. Most of them probably have fewer than 30 words, but together they might have hundreds or thousands of distinct words. For this reason, the text documents are processed for the stop words, cut into fragments and stored as vectors instead of the binary information. The basic idea is that any word present in the tweet is expressed as 1 and not in tweet is expressed as 0 resulting in sparse data indeed. But the matrix generated would require equal memory as compared to any other matrix even though it has a little information.



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

• Tree Structure

Trees is a special type of data structure that has faster retrieval of information in comparison to the table or sparse data. In these data structures the root node is the first directory for accessing the information and the child nodes are the sub directories of the root node. The information can be accessed form the child or leaf nodes by either using pointers or indexing of the tree structure. The figure 3.8 helps the researcher to understand the process of information retrieval in the tree structure.



• Hash Tables

The process of the calculation of the key for each data entry and allotting the key to the relevant bucket is the important process of the hash table structure for the storage of data. The process of storage and handling in the hash table makes it more reliable source for the retrieval of information based on the key value. This way you can quickly retrieve the information by looking in the right bucket when you encounter the data. Dictionaries in Python are a hash table implementation, and they're a close relative of key-value stores. You'll encounter them in the last example of this chapter when you build a recommender system within a database. Hash tables are used extensively in databases as indices for fast information retrieval.

3.5.2 Choosing the Right Tools

As discussed earlier in section 3.4.1 and 3.4.2 with the need to have a right algorithm and the right data structure, the requirement of a good tool is also important. The right tool can be a Python library or at least a tool that's controlled from Python, as shown figure 3.1.9.

Python has a number of libraries that can help you deal with large data. They range from smarter data structures over code optimizers to just-in-time compilers. **The following is a list of libraries we like to use when confronted with large data:**

Cython —The closer to the actual hardware of a computer, the more vital it is for the computer to know what types of data it has to process. For a computer, adding $1 + 1$ is different from adding $1.00 + 1.00$. The first example consists of integers and the second consists of floats, and these calculations are performed by different parts of the CPU. Python needs no specifications of the types of the data but the compiler can itself interpret the values based on the integer or float. This is although a slow process and so a superset of python can be used for the solution which forces the user to define the data type before the execution and hence can be implemented much faster. See <http://cython.org/> for more information on Cython.

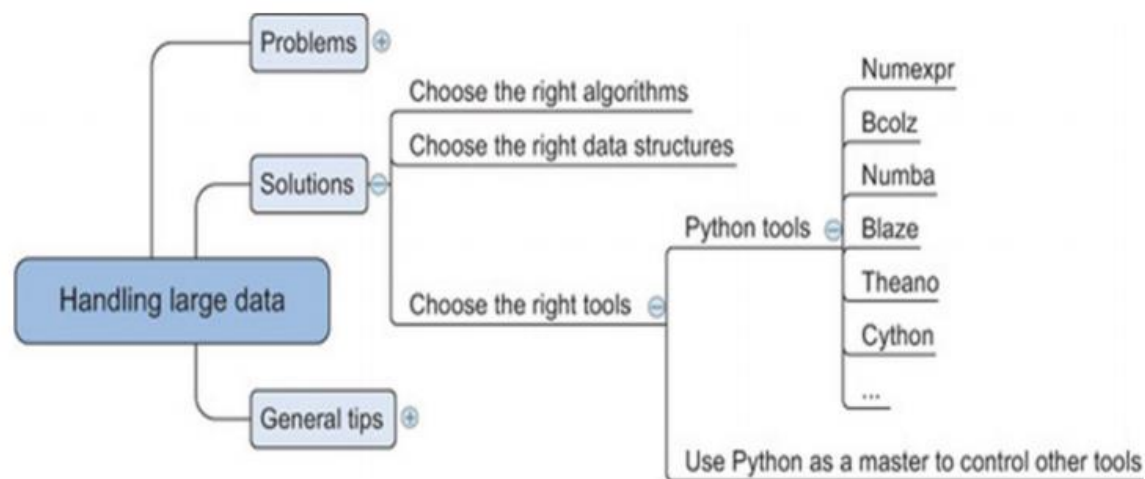


Fig. 3.1.7 Tools applied by data scientist for handing data

Numexpr —Numexpr is at the core of many of the big data packages, as is NumPy for in-memory packages. Numexpr is a numerical expression evaluator for NumPy but can be many times faster than the original NumPy. <https://github.com/pydata/numexpr>.

Numba —Numba is the package which compiles (just-in-time compiling) the code for the data manipulation and handling before actually executing it which makes it faster and less prone to errors. With the user gets a platform to develop a high level code with the speed of the compilation as in C. <http://numba.pydata.org/>.

Bcolz —Bcolz helps you overcome the out-of-memory problem that can occur when using NumPy. It can store and work with arrays in an optimal compressed form. It not only slims down your data need but also uses Numexpr in the background to reduce the calculations needed when performing calculations with bcolz arrays. <http://bcolz.blosc.org/>.

Blaze — Blaze is ideal in case the data scientist use the power of a database backend but like the —Pythonic way|| of working with data. Blaze will translate your Python code into SQL but can handle many more data stores than relational databases such as CSV, Spark, and others. Blaze delivers a unified way of working with many databases and data libraries.

<http://blaze.readthedocs.org/en/latest/index.html>.

Theano —Theano enables you to work directly with the graphical processing unit (GPU) and do symbolical simplifications whenever possible, and it comes with an excellent just-in-time compiler. On top of that it's a great library for dealing with an advanced but useful mathematical concept: tensors.

<http://deeplearning.net/software/theano>.

3.6 DISTRIBUTING DATA STORAGE AND PROCESSING WITH FRAMEWORKS

New big data technologies such as Hadoop and Spark make it much easier to work with and control a cluster of computers. Hadoop can scale up to thousands of computers, creating a cluster with petabytes of storage. This enables businesses to grasp the value of the massive amount of data available. —Big data Analytics|| is a phrase that was coined to refer to amounts of datasets that are so large, traditional data processing software simply can't manage them. For example, big data is used to pick out trends in economics, and those trends and patterns are used to predict what will happen in the future. These vast amounts of data require more robust computer software for processing, best handled by data processing frameworks. These are the top preferred data processing frameworks, suitable for meeting a variety of different needs of businesses.

3.6.1 Hadoop

This is an open-source batch processing framework that can be used for the distributed storage and processing of big data sets. Hadoop relies on computer clusters and modules that have been designed with the assumption that hardware will inevitably fail, and those failures should be automatically handled by the framework.

There are **four** main modules within Hadoop. Hadoop Common is where the libraries and utilities needed by other Hadoop modules reside. The Hadoop Distributed File System (HDFS) is the distributed file system that stores the data. Hadoop YARN (Yet Another Resource Negotiator) is the resource management platform that manages the computing resources in clusters, and handles the scheduling of users' applications. The

Hadoop MapReduce involves the implementation of the MapReduce programming model for large-scale data processing.

Hadoop operates by splitting files into large blocks of data and then distributing those datasets across the nodes in a cluster. It then transfers code into the nodes, for processing data in parallel. The idea of data locality, meaning that tasks are performed on the node that stores the data, allows the datasets to be processed more efficiently and more quickly.

Hadoop can be used within a traditional onsite datacenter, as well as through the cloud.

3.6.2 Apache Spark

Apache Spark is a batch processing framework that has the capability of stream processing, as well, making it a hybrid framework. Spark is most notably easy to use, and it's easy to write applications in Java, Scala, Python, and R. This open-source cluster computing framework is ideal for machine-learning, but does require a cluster manager and a distributed storage system. Spark can be run on a single machine, with one executor for every CPU core. It can be used as a standalone framework, and you can also use it in conjunction with Hadoop or Apache Mesos, making it suitable for just about any business.

Spark relies on a data structure known as the Resilient Distributed Dataset (RDD).

This is a read-only multiset of data items that is distributed over the entire cluster of machines. RDDs operate as the working set for distributed programs, offering a restricted form of distributed shared memory. Spark is capable of accessing data sources like HDFS, Cassandra, HBase, and S3, for distributed storage. It also supports a pseudo distributed local mode that can be used for development or testing.

The foundation of Spark is Spark Core, which relies on the RDD-oriented functional style of programming to dispatch tasks, schedule, and handle basic I/O functionalities. Two restricted forms of shared variables are used: broadcast variables, which reference read-only data that has to be available for all the nodes, and accumulators, which can be used to program reductions. Other elements included in Spark Core are: Spark SQL, which provides domain-specific language used to manipulate Data Frames.

Spark Streaming, which uses data in mini-batches for RDD transformations, allowing the same set of application code that is created for batch analytics to also be used for streaming analytics. Spark MLlib, a machine-learning library that makes the large-scale machine learning pipelines simpler. GraphX, which is the distributed graph processing framework at the top of Apache Spark.

3.6.3 Apache Storm

This is another open-source framework, but one that provides distributed, real-time stream processing. Storm is mostly written in Clojure, and can be used with any programming language. The application is designed as a topology, with the shape of a Directed Acyclic Graph (DAG). Spouts and bolts act as the vertices of the graph. The idea behind Storm is to define small, discrete operations, and then compose those operations into a topology, which acts as a pipeline to transform data.

3.6.4 Samza

Samza is another open-source framework that offers near a real-time, asynchronous framework for distributed stream processing. More specifically, Samza handles immutable streams, meaning transformations create new streams that will be consumed by other components without any effect on the initial stream. This framework works in conjunction with other frameworks, using Apache Kafka for messaging and Hadoop YARN for fault tolerance, security, and management of resources.

3.6.5 Flink

Flink is a hybrid framework, open-source, and stream processes, but can also manage batch tasks. It uses a high-throughput, low-latency streaming engine that is written in Java and Scala, and the runtime system that is pipelined allows for the execution of both batch and stream processing programs. The runtime also supports the execution of iterative algorithms natively. Flink's applications are all fault-tolerant and can support exactly-once semantics. Programs can be written in Java, Scala, Python, and SQL, and Flink offers support for event-time processing and state management.

3.8 QUESTIONS

1. Discuss the various representation techniques of different data types.
2. What are the issues that are faced in handling large data?
3. Explain briefly the techniques to handle large data.
4. What are the popular frameworks for big data storage?

Chapter Four- DATA ACQUISITION AND PROCESSING-2

Objectives

4.1 Data Science Ethics

4.2 Data Science – Good Aspects for Technology

4.3 Owners of Data

4.3.1 Responsibilities of the Data Owner

4.3.2 The Importance of Assigning Data Owners

4.3.3 Identification of Data Owners: Three Questions to Ask

4.4 Different Aspects of Privacy

4.5 Five C' s of Data Science

4.5.1 Consent

4.5.2 Clarity

4.5.3 Consistency and trust

4.5.4 Control and transparency

4.5.5 Consequences

4.6 Diversity – Inclusion

4.7 Future Trends

4.8 Questions

OBJECTIVE

1. To familiarize with the Data Science Ethics
2. To Introduce different aspects of privacy
3. To detect the future trends of data Science

4.1 DATA SCIENCE ETHICS

The skill to extract or mine the relevant patterns and the transforming capability to revolutionize the products in data science helps to bring a positive change in the social and technological sphere making it ethically neutral.

Analysts, data scientists, and information technology professionals must be concerned about data science ethics. Anyone who works with data must understand the fundamentals. Anyone dealing with any type of data must report any instances of data theft, unethical data collection, storage, use, etc.

For example, from the first time a consumer enters their email address on your website to the time they purchase your goods, your organization may gather and keep data about their trips. People in the marketing team might be dealing with the data. The data of the person must be preserved.

Protected data has been made public on the internet in the past, resulting in harm to persons whose information has been made available. Misconfigured databases, spyware, theft, or publishing on a public forum can all lead to data leaks. Individuals and organizations must use safe computing practices, conduct frequent system audits, and adopt policies to address computer and data security.

Companies must take appropriate cybersecurity steps to prevent the leakage of data and information. This is more important for banks and financial institutions which deal with customers' money. Protections must be maintained even when equipment is transferred or disposed of, according to policies.

4.2 DATA SCIENCE – GOOD ASPECTS FOR TECHNOLOGY

Data science involves a plethora of disciplines and expertise areas to produce a holistic, thorough and refined look into raw data. The professionals might be trained for the handling, manipulation and the interpretation of data but the most important and vital part of the data science is to perform the scrutiny of the relevant information from the disarrayed or unorganized form of raw data and only interconnect or forward only those data values which are of vital importance for the invention or the improvisation of the existing system. These reasons are sufficient enough for the data scientists to completely depend upon the existing technological advancement in the field of machine learning or deep learning with the ability to create the simulated models and predict the values in the models with the proposed algorithms and techniques.

To acclimatize the organization with the ethical data science, there's a need to understand what actually are the ethics about data science with the cost requirement for the implementation of ethical values and what can be the ways to execute such practices.

There are several ways and means with the respective solutions for the same.

Firstly, the data scientist needs to understand about the daintiness of the information in the data store. With this the relevant implementation or the operations can be performed on the data keeping in the view the consequences with improper or unacceptable access of the data for information. Indeed, the security perspective in

computer or network has proved many times that ignoring the consequences of accessing the data unethically might cause a trouble to the data professionals or the company as a whole in terms of loss of reputation or money.

With the available systems it is difficult for any data scientist to predict the inevitable unethical uses of the data but with the AI based techniques especially using machine learning and deep learning, a prediction can be made for the unintended consequences. For example, Facebook's —Year in Review|| that reminded people of deaths and other painful events. This can be handled by the data professionals in the field of data science by keeping in view the patterns of the data to be explored and how to think of better means to represent the data with a new or enhanced approach.

Another important step to stop the data mining if the administrator finds any kind of problem in the production line. This idea goes back to Toyota's Kanban: any assembly line worker can stop the line if they see something going wrong. The line doesn't restart until the problem is fixed. Workers don't have to fear consequences from management for stopping the line; they are trusted, and expected to behave responsibly.

The issue lurking behind all of these concerns is, of course, corporate culture. Corporate environments can be hostile to anything other than short-term profitability. That's a consequence of poor court decisions and economic doctrine, particularly in the U.S. But that inevitably leads us to the biggest issue: how to move the needle on corporate culture. Susan Etlinger has suggested that, in a time when public distrust and disenchantment is running high, ethics is a good investment. Upper-level management is only starting to see this; changes to corporate culture won't happen quickly. Users want to engage with companies and organizations they can trust not to take unfair advantage of them. Users want to deal with companies that will treat them and their

data responsibly, not just as potential profit or engagement to be maximized. Those companies will be the ones that create space for ethics within their organizations. We, the data scientists, data engineers, AI and ML developers, and other data professionals, have to demand change. We can't leave it to people that "do" ethics.

We can't expect management to hire trained ethicists and assign them to our teams. We need to live ethical values, not just talk about them. We need to think carefully about the consequences of our work. We must create space for ethics within our organizations. Cultural change may take time, but it will happen—if we are that change. That's what it means to do good data science.

4.3 OWNERS OF DATA

Data owners are either individuals or teams who make decisions such as who has the right to access and edit data and how it's used. Owners may not work with their data every day, but are responsible for overseeing and protecting a data domain.

A data owner is responsible for the data in a particular data domain. They may belong to the steering committee and ensure that the data under their view is governed throughout the organization. Data owners approve data glossaries and definitions as well as initiate data quality activities.

Owning' data – who owns data, what's capable of being owned, and what rights and responsibilities ownership attracts – is gaining a lot of attention.

4.3.1 Responsibilities of the Data Owner

The first responsibility of the data owner is to classify the data correctly. Once a

classification has been set it is up to the data owner to determine who has access to the data.

Usually, this access is based upon roles as opposed to individuals.

The data classification is one of the most important steps. Data classification has a different meaning for different organizations but at the basic level it is knowing the type of data a company has, determining its value, and categorizing it. For example, if your company has a secret sauce or original intellectual property it may be considered “top secret” or “confidential”. The reason to label or classify it as “top secret” or “confidential” is so it can be handled and ultimately protected appropriately. In addition to not putting the correct controls on data there is the potential to retain data for longer than needed or destroy data before it should be based on laws or contractual commitments. Many people have an opinion on how data should be classified or labeled but at the end of the day it is the responsibility of the data owner who is ultimately accountable for the data to make the final decision. The data owner will have the most knowledge of the use of the data and the value to the company. It is advisable for the data owner to get input from various sources like the data custodian or data users but the data owner has complete control over the data.

- Who has access to the data? Clarify the roles of people who can access the data.

Example: Employees can see an organization chart with departments, manager names, and titles but not salary information (Classification = internal). But a very limited audience like HR should only have access to salary data, performance data, or social security numbers (Classification = confidential).

- How is the data secured? Sensitive data elements within HR documentation have

been classified to be confidential and therefore it requires additional security controls to protect it. Some of the additional controls to secure confidential data stored in electronic medium could include being saved in a location on the network with appropriate safeguards to prevent unauthorized access (secure folders protected by passwords).

- How long the data is retained? Many industries require that data be retained for a certain length of time. For example, the finance industry requires a seven-year retention period and some health care industries requires a 100-year retention period. Data owners need to know the regulatory requirements for their data and if there is no clear guidance on retention then it should be based off the company's retention policy or best practices.
- How data should be destroyed? Based on the classification of the data there should be clear guidance on how to dispose or destroy the data. For example:

Information Medium	Public	Internal	Confidential
Hard Copy	Place in recycling bins or trash receptacles.	Place in secured shedding bins or manually shred.	Place in secured shedding bins or manually shred with a cross-cut shredder or pulped. A record must be maintained that indicates the records disposed of and the date of disposal.
Electronic records	Electronic records can be deleted normally.	Electronic records need to be overwritten to 1's and 0's or with a secure delete option.	Electronic records need to be degaussed off magnetic media after securely deleted or the physical media should be destroyed with a record maintained that indicates the records disposed of and the date of disposal.

- What data needs to be encrypted? Data owners should decide whether their data needs to be encrypted. To make this determination the data owner should know the applicable laws or regulation requirements set that must be complied with. A good example of a

regulation requirement is set by the Payment Card Industry (PCI) Data Security Standard and it requires that the transmission of cardholder data across open, public networks must be encrypted.

4.3.2 The Importance of Assigning Data Owners

In most organizations, as data passes through different teams and systems, assigning data owners can be cumbersome. However, this is a critical step for GDPR compliance.

Why assigning data owners is important:

- 1. Accountability** — Ownership creates accountability. Since GDPR introduces many controls on personal data, assigning responsibilities ensures that data will be continuously -monitored for compliance by the owners.
- 2. Defining policies** — As they have a vested interest in the integrity of their data, owners focus on defining policies (for example retention or deletion policies) and standards that ensure the alignment of their data to the GDPR.
- 3. Creating trusted data** — Data ownership is a key ingredient to gain customer trust and achieve measurable business benefits. Poor data could easily result in bad customer experiences and ultimately losing customers. In particular, when personal data is not reconciled into a data subject 360° view, compliance with data subject access rights, such as rights of portability or rights to be forgotten cannot be fully achieved.

4. Eliminating redundancies — As organizations strive to put the appropriate governance framework in place for GDPR, one common frustration is a loss of productivity. This issue stems from multiple teams addressing the same problem either because there isn't a clear understanding of data or they're not even aware that the problem has been resolved by another team. Federated ownership eliminates these painful issues.

4.3.3 Identification of Data Owners: Three Questions to Ask

The mandatory introduction of a data protection officer (DPO) role by GDPR in most organizations effectively creates a master data owner. Each and every element in a data taxonomy needs an individual owner, however, and there is little likelihood that a single DPO can hold this responsibility on such a large scale. In addition, this could create a security issue. Delegation and segregation of duties are needed.

Asking the right questions helps. Once these questions have been answered, the data owner should become clearer:

1. Who is most impacted by data accuracy?
2. Who has authority to decide the next step?
3. Who owns the related data attributes?

4.4 DIFFERENT ASPECTS OF PRIVACY

First there is the data where security and privacy has always mattered and for which there is already an existing and well galvanized body of law in place. Foremost among these is classified or national security data where data usage is highly regulated and enforced.

Other data for which there exists a considerable body of international and national law regulating usage includes:

Proprietary Data – specifically the data that makes up the intellectual capital of individual businesses and gives them their competitive economic advantage over others, including data protected under copyright, patent, or trade secret laws and the sensitive, protected data that companies collect on behalf of its customers;

Infrastructure Data - data from the physical facilities and systems – such as roads, electrical systems, communications services, etc. – that enable local, regional, national, and international economic activity; and

Controlled Technical Data - technical, biological, chemical, and military-related data and research that could be considered of national interest and be under foreign export restrictions.

It may be possible to work with publicly released annualized and cleansed data within these areas without a problem, but the majority of granular data from which significant insight can be gleaned is protected. In most instances, scientists, researchers, and other authorized developers take years to appropriately acquire the expertise, build the personal relationships, and construct the technical, procedural and legal infrastructure to work with the granular data before implementing any approach. Even using publicly released datasets within these areas can be restricted, requiring either registration, the recognition of or affiliation to an appropriate data governing body, background checks, or all three before authorization is granted. The second group of data that raises privacy and security concerns is personal data. Commonly referred to as Personally Identifiable

Information (PII), it is any data that distinguishes individuals from each other. It is also the data that an increasing number of digital approaches rely on, and the data whose use tends to raise the most public ire. Personal data could include but is not limited to an individual's:

- Government issued record data (social security numbers, national or state identity numbers, passport records, vehicle data, voting records, etc.);
- Law enforcement data (criminal records, legal proceedings, etc.);
- Personal financial, employment, medical, and education data;
- Communication records (phone numbers, texts data, message records, content of conversations, time and location, etc.);
- Travel data (when and where traveling, carriers used, etc.);
- Networks and memberships (family, friends, interests, group affiliations, etc.);
- Location data (where a person is and when);
- Basic contact information (name, address, e-mail, telephone, fax, twitter handles, etc.);
- Internet data (search histories, website visits, click rates, likes, site forwards, comments, etc.);
- Media data (which shows you're watching, music you're listening to, books or magazines you're reading, etc.);
- Transaction data (what you're buying or selling, who you're doing business with, where, etc.); and

- Bio and activity data (from personal mobile and wearable devices).

In industries where being responsible for handling highly detailed personal data is the established business norm – such as in the education, medical and financial fields – there are already government regulations, business practices and data privacy and security laws that protect data from unauthorized usage, including across new digital platforms. But in many other industries, particularly in data driven industries where personal data has been treated as proprietary data and become the foundation of business models, there is currently little to no regulation. In the new normal, the more that a data approach depends on data actively or passively collected on individuals, the more likely that consumers will speak up and demand privacy protection, even if they previously gave some form of tacit approval to use their data.

、

Despite this new landscape, there are lots of different ways to use personal data, some of which may not trigger significant privacy or security concerns. This is particularly true in cases where individuals willingly provide their data or data cannot be attributed to an individual. Whether individuals remain neutral to data approaches tends to be related to the level of control they feel they have over how their personal data is used. Some organizations that collect personal data extensively, such as Facebook and Google, work to increasingly provide their users with methods to control their own data. But for others, the lack of due diligence on data privacy in their approaches has already had their effect.

A third category of data needing privacy consideration is the data related to good people working in difficult or dangerous places. Activists, journalists, politicians, whistle-

blowers, business owners, and others working in contentious areas and conflict zones need secure means to communicate and share data without fear of retribution and personal harm.

4.5 FIVE C' S OF DATA SCIENCE

What does it take to build a good data product or service? Not just a product or service that's useful, or one that's commercially viable, but one that uses data ethically and responsibly.

Users lose trust because they feel abused by malicious ads; they feel abused by fake and misleading content, and they feel abused by —act first, and apologize profusely later|| cultures at many of the major online companies. And users ought to feel abused by many abuses they don't even know about. Why was their insurance claim denied? Why weren't they approved for that loan? Were those decisions made by a system that was trained on biased data? The slogan goes, "Move fast and break things." But what if what gets broken is society?

Data collection is a big business. Data is valuable: "the new oil," as the Economist proclaimed. We've known that for some time. But the public provides the data under the assumption that we, the public, benefit from it. We also assume that data is collected and stored responsibly, and those who supply the data won't be harmed. Essentially, it's a model of trust. But how do you restore trust once it's been broken? It's no use pretending that you're trustworthy when your actions have proven that you aren't. The

only way to get trust back is to be trustworthy, and regaining that trust once you've lost it takes time.

There's no simple way to regain users' trust, but a —golden rule|| for data as a starting point: —treat others' data as data scientist would like other to treat their data.|| However, implementing a “golden rule” in the actual research and development process is challenging.

The golden rule isn't enough by itself. There has to be certain guidelines to force discussions with the application development teams, application users, and those who might be harmed by the collection and use of data. Five framing guidelines help us think about building data products. We call them the five **Cs**: **consent**, **clarity**, **consistency**, **control** (and transparency), and **consequences** (and harm).

4.5.1 Consent

The trust between the people who are providing data and the people who are using it cannot be established without agreement about what data is being collected and how that data will be used . Agreement starts with obtaining consent to collect and use data. Unfortunately, the agreements between a service's users (people whose data is collected) and the service itself (which uses the data in many ways) are binary (meaning that you either accept or decline) and lack clarity. In business, when contracts are being negotiated between two parties, there are multiple iterations (redlines) before the

contract is settled. But when a user is agreeing to a contract with a data service, you either accept the terms or you don't get access. It's non-negotiable.

Data is frequently collected, used, and sold without consent. This includes organizations like Acxiom, Equifax, Experian, and Transunion, who collect data to assess financial risk, but many common brands also connect data without consent. In Europe, Google collected data from cameras mounted on cars to develop new mapping products. AT&T and Comcast both used cable set top boxes to collect data about their users, and Samsung collected voice recordings from TVs that respond to voice commands.

4.5.2 Clarity

Clarity is closely related to consent. Users must have clarity about what data they are providing, what is going to be done with the data, and any downstream consequences of how their data is used. All too often, explanations of what data is collected or being sold are buried in lengthy legal documents that are rarely read carefully, if at all. Observant readers of Eventbrite's user agreement recently discovered that listing an event gave the company the right to send a video team, and exclusive copyright to the recordings. And the only way to opt out was by writing to the company. The backlash was swift once people realized the potential impact, and Eventbrite removed the language.

Facebook users who played Cambridge Analytica's —This Is Your Digital Life|| game may have understood that they were giving up their data; after all, they were answering questions, and those answers certainly went somewhere. But did they understand how

that data might be used? Or that they were giving access to their friends' data behind the scenes? That's buried deep in Facebook's privacy settings. It really doesn't matter which service you use; you rarely get a simple explanation of what the service is doing with your data, and what consequences their actions might have. Unfortunately, the process of consent is often used to obfuscate the details and implications of what users may be agreeing to. And once data has escaped, there is no recourse.

4.5.3 Consistency and Trust

Trust requires consistency over time. You can't trust someone who is unpredictable.

They may have the best intentions, but they may not honor those intentions when you need them to. Or they may interpret their intentions in a strange and unpredictable way. And once broken, rebuilding trust may take a long time. Restoring trust requires a prolonged period of consistent behavior.

Consistency, and therefore trust, can be broken either explicitly or implicitly. An organization that exposes user data can do so intentionally or unintentionally. In the past years, we've seen many security incidents in which customer data was stolen: Yahoo!, Target, Anthem, local hospitals, government data, and data brokers like Experian, the list grows longer each day. Failing to safeguard customer data breaks trust—and safeguarding data means nothing if not consistency over time.

4.5.4 Control and Transparency

All too often, users have no effective control over how their data is used. They are given all-or-nothing choices, or a convoluted set of options that make controlling access overwhelming and confusing. It's often impossible to reduce the amount of data

collected, or to have data deleted later. A major part of the shift in data privacy rights is moving to give users greater control of their data. For example, Europe's General Data Protection Regulation (GDPR) requires a user's data to be provided to them at their request and removed from the system if they so desire.

4.5.5 Consequences

Data products are designed to add value for a particular user or system. As these products increase in sophistication, and have broader societal implications, it is essential to ask whether the data that is being collected could cause harm to an individual or a group. The unforeseen consequences and the —unknown unknowns|| about using data and combining data sets have been witnessed frequently. Risks can never be eliminated completely. However, many unforeseen consequences and unknown unknowns could be foreseen and known, if only people had tried. All too often, unknown unknowns are unknown because we don't want to know.

While Strava and AOL triggered a chain of unforeseen consequences by releasing their data, it's important to understand that their data had the potential to be dangerous even if it wasn't released publicly. Collecting data that may seem innocuous and combining it with other data sets has real-world implications. It's easy to argue that Strava shouldn't have produced this product, or that AOL shouldn't have released their search data, but that ignores the data's potential for good. In both cases, well-intentioned data scientists were looking to help others. The problem is that they didn't think through the consequences and the potential risks.

Many data sets that could provide tremendous benefits remain locked up on servers.

Medical data that is fragmented across multiple institutions limits the pace of research. And the data held on traffic from ride-sharing and gps/mapping companies could transform approaches for traffic safety and congestion. But opening up that data to researchers requires careful planning

4.6 DIVERSITY – INCLUSION

Reports of AI gone wrong abound, and Responsible AI has started to take a foothold in business — Gartner has even added Responsible AI as a new category on its Hype Cycle for Emerging Technologies. Yet when talking about solutions, increasing diversity and making data science a more inclusive field unfortunately don't often top the list. Noelle Silver, Head of Instruction, Data Science, Analytics, and Full Stack Web Development at HackerU, is looking to change that.

A 2018 study revealed that only 15% of data scientists are women, and sadly, a 2020 study found exactly the same results: it seems we haven't managed to move the needle. While diversity obviously encompasses more than just women, few studies have been able to quantify other types of representation in the field. Inclusivity is similarly difficult to quantify, both in terms of people working on technology and the ways in which technology can be accessed by all. But that doesn't mean there aren't solutions.

Problem

“The reality is that when we train machine learning models with a bunch of data, it's going to make predictions based on that data. If that data comes from a room of people

that look the same, talk the same, act the same, they're all friends — it's not a bad scenario. In the moment, you feel like things are good. No one is really seeing any problems; you don't feel any friction. It's very misleading, especially in artificial intelligence. So, you go to market.

The problem, though, is not everyone looks like you, or talks like you, or thinks like you. So even though you found a community of people that built this software that thinks the same, as soon as you go to market and someone other than that starts using it, they start to feel that friction.”

Solution

Of course, there's no easy, magic bullet solution to this problem, but foundations of a good start are:

- Committing the time and resources to practice inclusive engineering: This includes, but certainly isn't limited to, doing whatever it takes to collect and use diverse datasets.
- Create an experience that welcomes more people to the field: This might mean looking at everything from education to hiring practices.
- Think beyond regulations: Simply being compliant doesn't necessarily mean experiences are optimized.

4.7 QUESTIONS

1. Write a short note on Data Science Ethics.
2. What are the privacy aspects of data?
3. What are the five C's of data Science

Data Normalization

Data normalization is the practice of organizing data entries to ensure they appear similar across all fields and records, making information easier to find, group and analyze. There are many data normalization techniques and rules.

Normalizing the data refers to scaling the data values to a much smaller range such as $[-1, 1]$ or $[0.0, 1.0]$. There are different methods to normalize the data, as discussed below.

Normalization is an essential step in the **preprocessing** of data for machine learning models, and it is a feature scaling technique. Normalization is especially crucial for data manipulation, scaling down, or up the range of data before it is utilized for subsequent stages in the fields of soft computing, cloud computing, etc. Min-max scaling and **Z-Score Normalisation (Standardisation)** are the two methods most frequently used for normalization in feature scaling.

***Normalization** is scaling the data to be analyzed to a specific range such as $[0.0, 1.0]$ to provide better results.*

Data Normalization Techniques

1- Min-Max normalization:

This method of normalising data involves transforming the original data linearly. The data's minimum and maximum values are obtained, and each value is then changed using the formula that follows.

$$X_{\text{normalization}} = \frac{X - x_{\min}}{x_{\max} - x_{\min}}$$

This division scales the variable to a proportion of the entire range. As a result, the normalized value falls between 0 and 1.

This normalization guarantees all features will have exact same scale but does not handle outliers well.

2- Normalisation of Z-score or Zero Mean (Standardisation):

Using the **mean** and **standard deviation** of the data, values are normalised in this technique to create a standard normal distribution (mean: 0, standard deviation: 1). The equation that is applied is:

$$Z = \frac{x - \mu}{\sigma}$$

z-score handles outliers, but does not produce normalized data with the exact same scale.

Example:

Normalize the following group of data with min-max normalization 1000, 2000, 3000, 9000

Solution:

$$x_{\min}=1000, x_{\max}=9000$$

No.	x	x_{norm}
1	1000	$(1000-1000)/(9000-1000)=0$
2	2000	$(2000-1000)/(9000-1000)=0.125$
3	3000	$(3000-1000)/(9000-1000)=0.25$
4	9000	$(9000-1000)/(9000-1000)=1$

Example:

Perform z-score normalization with the data set: 3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 134

Solution:

Mean $\mu = \text{summation}/n = 21.2$, standard deviation $\sigma = \sqrt{\frac{\sum(x-\mu)^2}{n}} = 29.8$

No.	X	x_{norm}
1	3	$(3-21.2)/29.8 = -0.61$

2	5	$(5-21.2)/29.8=-0.54$
3	5	$(5-21.2)/29.8=-0.54$
4	8	$(8-21.2)/29.8=-0.44$
5	9	$(9-21.2)/29.8=-0.41$
6	12	$(12-21.2)/29.8=-0.31$
7	12	$(12-21.2)/29.8=-0.31$
8	13	$(13-21.2)/29.8=-0.28$
9	15	$(15-21.2)/29.8=-0.21$
10	16	$(16-21.2)/29.8=-0.17$
11	17	$(17-21.2)/29.8=-0.14$
12	19	$(19-21.2)/29.8=-0.07$
13	22	$(22-21.2)/29.8=0.03$
14	24	$(24-21.2)/29.8=0.09$
15	25	$(25-21.2)/29.8=0.13$
16	134	$(134-21.2)/29.8=3.79$

Pandas Package

The **pandas** package is the most important tool at the disposal of Data Scientists and Analysts working in Python today. **pandas** is the backbone of most data projects.

Through **pandas**, you get acquainted with your data by **cleaning**, **transforming**, and **analyzing** it.

For example, say you want to explore a dataset stored in a CSV on your computer. Pandas will extract the data from that CSV into a **DataFrame** — a table, basically — then let you do things like:

- **Calculate statistics** and answer questions about the data, like
 - What's the average, median, max, or min of each column?
 - Does column A correlate with column B?
 - What does the distribution of data in column C look like?
- **Clean** the data by doing things like removing missing values and filtering rows or columns by some criteria

- **Visualize** the data with help from **Matplotlib**. Plot bars, lines, histograms, bubbles, and more.
- Store the cleaned, transformed data back into a CSV, other file or database.

Pandas is built on top of the **NumPy** package, meaning a lot of the structure of NumPy is used or replicated in Pandas.

Data in pandas is often used to feed statistical analysis in **SciPy**, plotting functions from **Matplotlib**, and machine learning algorithms in **Scikit-learn**.

To import **pandas** we usually import it with a shorter name since it's used so much:

```
import pandas as pd
```

The primary two components of pandas are the **Series** and **DataFrame**.

A **Series** is essentially a **column**, and a **DataFrame** is a **multi-dimensional table** made up of a collection of Series.

Series			Series			DataFrame		
apples			oranges			apples	oranges	
0	3	+	0	0	=	0	3	0
1	2		1	3		1	2	3
2	0		2	7		2	0	7
3	1		3	2		3	1	2

There are *many* ways to create a **DataFrame** with pandas, but a great option is to just use a simple **dict**.

Let's say we have a fruit stand that sells apples and oranges. We want to have a column for each fruit and a row for each customer purchase. To organize this as a dictionary for pandas:

```
data = {
    'apples': [3, 2, 0, 1],
    'oranges': [0, 3, 7, 2]
}
```

and then pass it to the pandas DataFrame constructor:

```
purchases = pd.DataFrame(data)
```

```
print(purchases)
```

```
In [5]: data = {
        'apples': [3, 2, 0, 1],
        'oranges': [0, 3, 7, 2]
        }

        purchases = pd.DataFrame(data)

        print(purchases)
```

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Note: index of this dataframe is [0-3]

but we could also create our own when we initialize the DataFrame.

Let's have customer **names** as our index:

```
purchases = pd.DataFrame(data, index=['June', 'Robert', 'Lily', 'David'])
```

```
purchases
```

```
In [6]: purchases = pd.DataFrame(data, index=['June', 'Robert', 'Lily', 'David'])
        purchases
```

Out[6]:

	apples	oranges
June	3	0
Robert	2	3
Lily	0	7
David	1	2

So now we could **locate** a customer's order by using their name:

```
purchases.loc['June']
```

```
In [7]: purchases.loc['June']  
  
Out[7]: apples      3  
        oranges     0  
        Name: June, dtype: int64
```

```
In [ ]: |
```

Writing data into CSVs

df.to_csv('purchases.csv')

Reading data from CSVs

With CSV files all you need is a single line to load in the data:

```
df = pd.read_csv('purchases.csv')
```

df

```
In [9]: df.to_csv('purchases.csv')  
  
In [10]: df = pd.read_csv('purchases.csv')  
         df
```

```
Out[10]:
```

	Unnamed: 0	Name	Age
0	0	Amit	20
1	1	Cody	21
2	2	Drew	25

```
In [ ]:
```

create data frame with pandas

import pandas as pd

```
d = {'col1': [1, 2, 3, 4, 7], 'col2': [4, 5, 6, 9, 5], 'col3': [7, 8, 12, 1, 11]}
```

```
df = pd.DataFrame(data=d)
```

```
print(df)
```

Creating DataFrame to Export Pandas DataFrame to CSV

```
# importing pandas as pd
```

```
import pandas as pd
```

```
# list of name, degree, score
```

```
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
```

```
deg = ["MBA", "BCA", "M.Tech", "MBA"]
```

```
scr = [90, 40, 80, 98]
```

```
# dictionary of lists
```

```
dict = {'name': nme, 'degree': deg, 'score': scr}
```

```
df = pd.DataFrame(dict)
```

```
print(df)
```

```
# saving the dataframe
```

```
df.to_csv('file1.csv')
```

	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

```
In [2]: # importing pandas as pd
import pandas as pd

# List of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)

print(df)
```

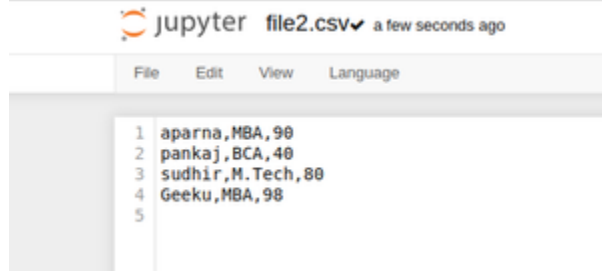
```
   name degree score
0  aparna   MBA   90
1  pankaj   BCA   40
2  sudhir  M.Tech  80
3   Geeku   MBA   98
```

```
In [ ]:
```

Saving CSV without *headers* and *index*.

saving the dataframe

```
df.to_csv('file2.csv', header=False, index=False)
```



Save the CSV file to a specified location

saving the dataframe

```
df.to_csv(r'C:\Users\Admin\Desktop\file3.csv')
```

Write a DataFrame to CSV file using tab separator

```
import pandas as pd
import numpy as np
```

```
users = {'Name': ['Amit', 'Cody', 'Drew'],
        'Age': [20, 21, 25]}
```

#create DataFrame

```
df = pd.DataFrame(users, columns=['Name', 'Age'])
```

```
print("Original DataFrame:")
```

```
print(df)
print('Data from Users.csv:')
```

```
df.to_csv('Users.csv', sep='\t', index=False, header=True)
new_df = pd.read_csv('Users.csv')
```

```
print(new_df)
```

```
In [4]: import pandas as pd
import numpy as np

users = {'Name': ['Amit', 'Cody', 'Drew'],
        'Age': [20, 21, 25]}

#create DataFrame
df = pd.DataFrame(users, columns=['Name', 'Age'])

print("Original DataFrame:")
print(df)
print('Data from Users.csv:')

df.to_csv('Users.csv', sep='\t', index=False, header=True)
new_df = pd.read_csv('Users.csv')

print(new_df)
```

Original DataFrame:

	Name	Age
0	Amit	20
1	Cody	21
2	Drew	25

Data from Users.csv:

	Name\tAge
0	Amit\t20
1	Cody\t21
2	Drew\t25

Or

Save dataframe to json file:

```
df.to_json('purchases.json')
```

To read it:

```
df = pd.read_json('purchases.json')
```

```
df
```

Reading data from a SQL database

If you're working with data from a SQL database you need to first establish a **connection** using an appropriate Python library, then [pass a query to pandas](#). Here we'll use **SQLite** to demonstrate.

```
import sqlite3
```

```
con = sqlite3.connect("database.db")
```

```
df.to_sql('purchases', con) # now the dataframe df is stored in a database file
```

In this SQLite database we have a table called *purchases*, and our index is in a column called "index".

By passing a **SELECT** query and our con, we can read from the *purchases* table:

```
import sqlite3
```

```
con = sqlite3.connect("database.db")
```

```
df = pd.read_sql_query("SELECT * FROM purchases", con)
```

```
df # to print
```

How To Convert Sklearn Dataset To Pandas Dataframe in Python?

```
from sklearn.datasets import load_iris
```

```
import pandas as pd
```

```
# Load the iris dataset from sklearn using load_iris() method
```

```
iris = load_iris()
```

```
# Convert the iris dataset to a pandas dataframe
```



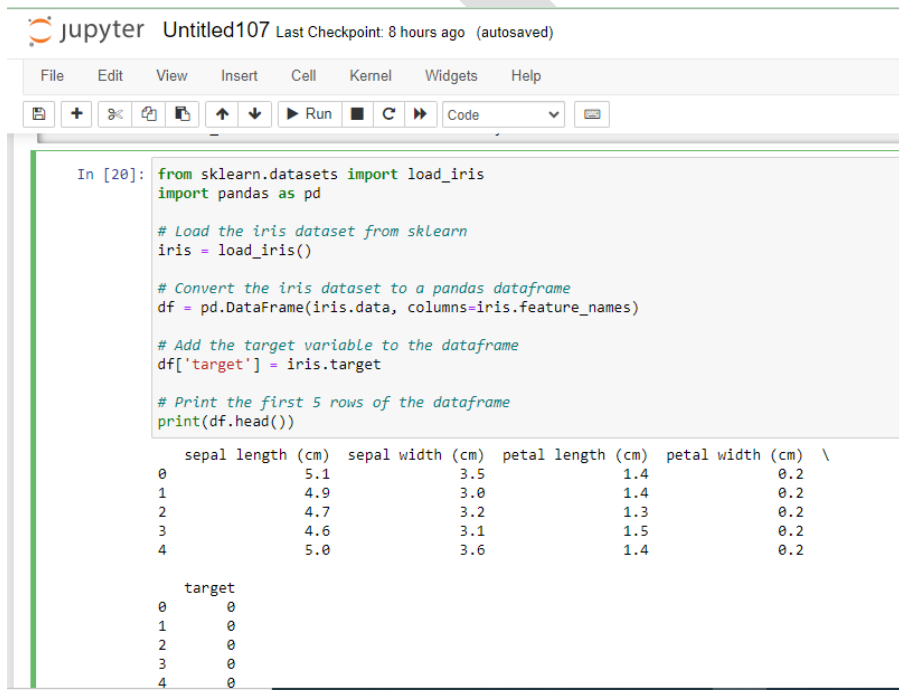
```
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add the target variable to the dataframe

df['target'] = iris.target

# Print the first 5 rows of the dataframe

print(df.head())
```



The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
In [20]: from sklearn.datasets import load_iris
import pandas as pd

# Load the iris dataset from sklearn
iris = load_iris()

# Convert the iris dataset to a pandas dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add the target variable to the dataframe
df['target'] = iris.target

# Print the first 5 rows of the dataframe
print(df.head())
```

The output of the code is displayed below the code cell, showing the first 5 rows of the DataFrame:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Explore the dataset:

```
print(df.info())
```

```
print(df.describe())
```

The **info()** function is useful to understand the overall structure of the data frame, the number of non-null values in each column, and the memory usage. While the summary statistics provide an overview of numerical features in your dataset.

```
[42]: print(df.info())
print(df.describe())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column              Non-Null Count  Dtype  
---  --
0   sepal length (cm)    150 non-null    float64
1   sepal width (cm)     150 non-null    float64
2   petal length (cm)    150 non-null    float64
3   petal width (cm)     150 non-null    float64
4   target               150 non-null    int32   
dtypes: float64(4), int32(1)
memory usage: 5.4 KB
None
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

	petal width (cm)	target
count	150.000000	150.000000
mean	1.199333	1.000000
std	0.762238	0.819232
min	0.100000	0.000000
25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

In []:

Removing Missing Values (if dataset has missing)

`df.dropna(inplace=True)`

or

by replacing:

`df = df.replace(1, 31) # Replace 1 with 31`

`df = df.replace(1, np.nan) # Replace 1 with "np.nan"`

Removing Duplicates

`duplicate_rows = df.duplicated()`

```
print("Number of duplicate rows:", duplicate_rows.sum())
```

```
# Number of duplicate rows: 1
```

The duplicates can be removed via the **drop_duplicates()** function.

```
df.drop_duplicates(inplace=True)
```

Data Normalization transforming numeric columns to a standard scale. In machine learning, some feature values differ from others multiple times. The features with higher values will dominate the learning process.

Example:

```
import pandas as pd
```

```
# create data
```

```
df = pd.DataFrame({'Column 1':[200,-4,90,13.9,5,  
                               -90,20,300.7,30,-200,400],  
                  'Column 2':[20,30,23,45,19,38,  
                               25,45,34,37,12]})
```

```
# view data
```

```
display(df)
```

```
In [27]: import pandas as pd

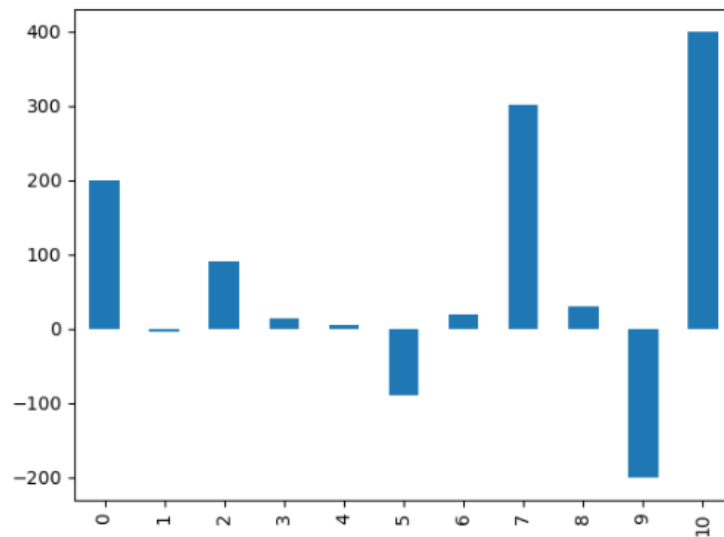
# create data
df = pd.DataFrame({'Column 1': [200, -4, 90, 13.9, 5,
                                -90, 20, 300.7, 30, -200, 400],
                   'Column 2': [20, 30, 23, 45, 19, 38,
                                25, 45, 34, 37, 12]})

# view data
display(df)
```

	Column 1	Column 2
0	200.0	20
1	-4.0	30
2	90.0	23
3	13.9	45
4	5.0	19
5	-90.0	38
6	20.0	25
7	300.7	45
8	30.0	34
9	-200.0	37
10	400.0	12

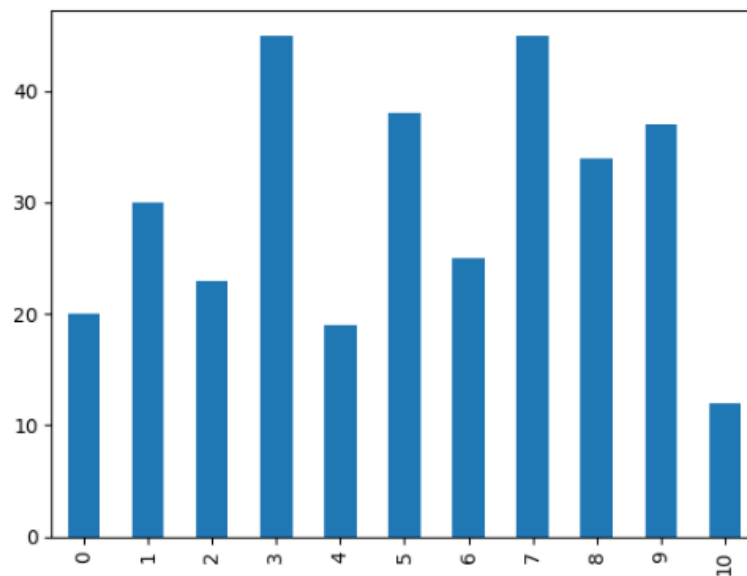
`df['Column 1'].plot(kind = 'bar')`

```
In [28]: df['Column 1'].plot(kind = 'bar')
Out[28]: <AxesSubplot:>
```



```
In [29]: df['Column 2'].plot(kind = 'bar')
```

```
Out[29]: <AxesSubplot:>
```



```
In [ ]: |
```

It is clear that **column2** is more normalized than **column1**.

Normalize Using The maximum absolute scaling:

The **maximum absolute scaling** rescales each feature between **-1** and **1** by dividing every observation by its **maximum absolute** value. We can apply the maximum absolute scaling in Pandas using the **.max()** and **.abs()** methods.

```
# copy the data
```

```
df_max_scaled = df.copy()
```

```
# apply normalization techniques on Column 1
```

```
column = 'Column 1'
```

```
df_max_scaled[column] = df_max_scaled[column] / df_max_scaled[column].abs().max()
```

```
# view normalized data
```

```
display(df_max_scaled)
```



```
In [30]: # copy the data
df_max_scaled = df.copy()

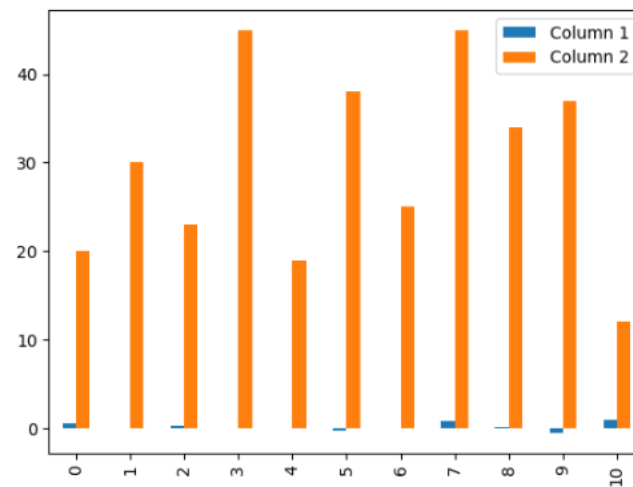
# apply normalization techniques on Column 1
column = 'Column 1'
df_max_scaled[column] = df_max_scaled[column] / df_max_scaled[column].abs().max()

# view normalized data
display(df_max_scaled)
```

	Column 1	Column 2
0	0.50000	20
1	-0.01000	30
2	0.22500	23
3	0.03475	45
4	0.01250	19
5	-0.22500	38
6	0.05000	25
7	0.75175	45
8	0.07500	34
9	-0.50000	37
10	1.00000	12

`df_max_scaled.plot(kind = 'bar')`

```
In [32]: df_max_scaled.plot(kind = 'bar')
Out[32]: <AxesSubplot:>
```



```
In [ ]:
```

Using The min-max feature scaling:

The min-max approach (often called normalization) rescales the feature to a hard and fast range of **[0,1]** by subtracting the minimum value of the feature then dividing by the range.

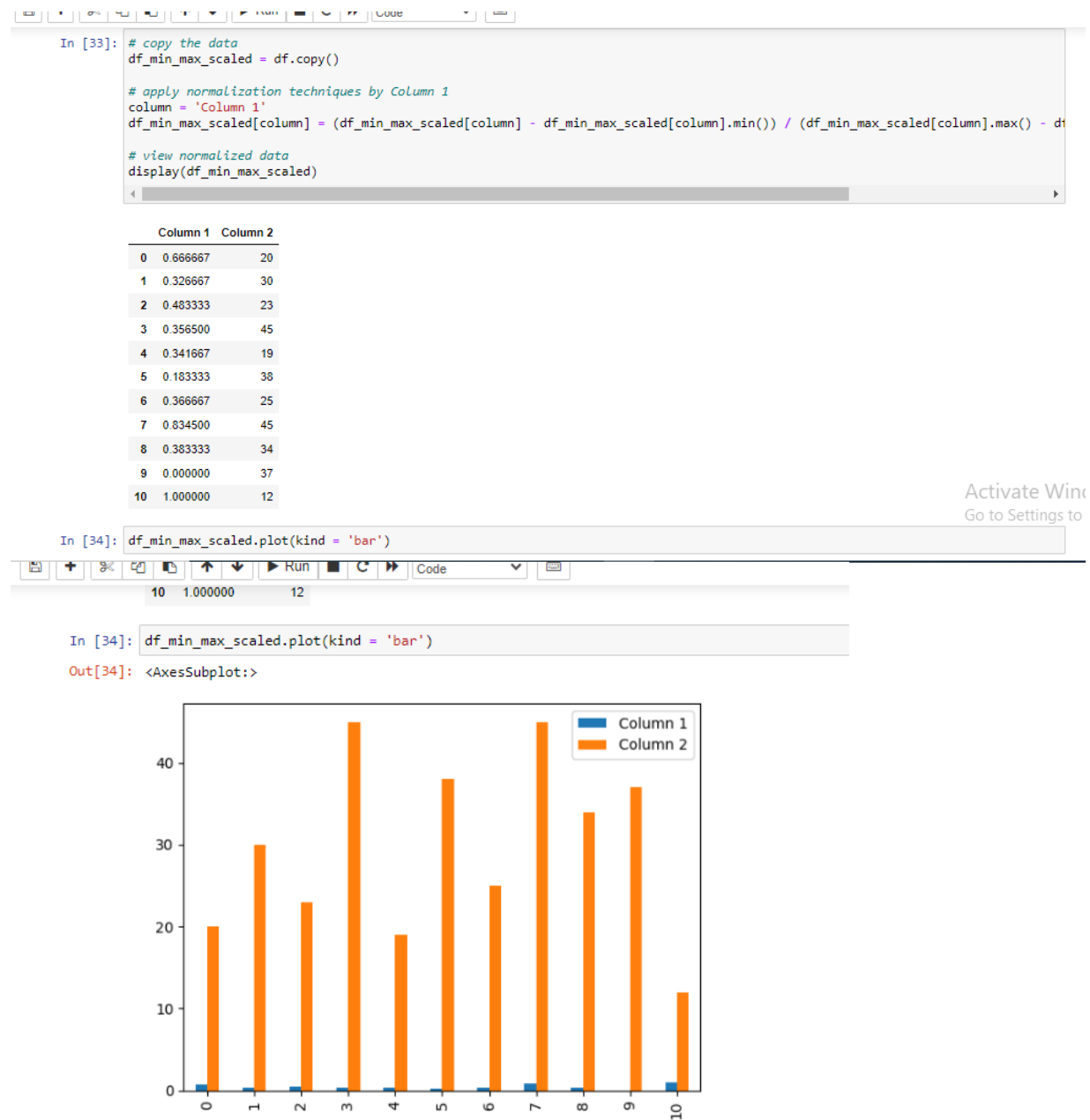
$$x = [1, 43, 65, 23, 4, 57, 87, 45, 45, 23]$$
$$\downarrow$$
$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad \begin{matrix} x_{min} = 1 \\ x_{max} = 87 \end{matrix}$$
$$\downarrow$$
$$x_{new} = [0, 0.48, 0.74, 0.25, 0.03, 0.65, 1, 0.51, 0.51, 0.25]$$

We can apply the min-max scaling in Pandas using the **.min()** and **.max()** methods.

```
# copy the data
df_min_max_scaled = df.copy()

# apply normalization techniques by Column 1
column = 'Column 1'
df_min_max_scaled[column] = (df_min_max_scaled[column] - df_min_max_scaled[column].min()) /
(df_min_max_scaled[column].max() - df_min_max_scaled[column].min())

# view normalized data
display(df_min_max_scaled)
```



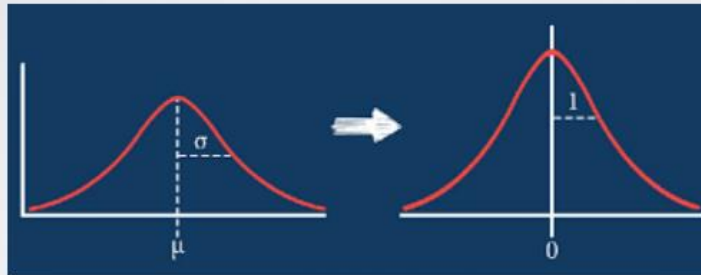
Using The z-score method:

The z-score method (often called standardization) transforms the info into distribution with a **mean of 0** and a **typical deviation of 1**. Each standardized value is computed by subtracting the mean of the corresponding feature then dividing by the quality deviation.

- **Data Standardisation:**

Standardisation typically means rescales data to have a mean of 0 and a standard deviation of 1 (unit variance).

$$x_{new} = \frac{x - \mu}{\sigma}$$



$x = [1, 43, 65, 23, 4, 57, 87, 45, 45, 23]$ $\mu = 39.3$ $x_{max} = 87$

↓
 $x_{new} = [-1.49, 0.14, 1.00, -0.63, -1.37, 0.69, 1.86, 0.22, 0.22, -0.63]$

```
# copy the data
```

```
df_z_scaled = df.copy()
```

```
# apply normalization technique to Column 1
```

```
column = 'Column 1'
```

```
df_z_scaled[column] = (df_z_scaled[column] - df_z_scaled[column].mean()) /  
df_z_scaled[column].std()
```

```
# view normalized data
```

```
display(df_z_scaled)
```

```
In [35]: # copy the data
df_z_scaled = df.copy()

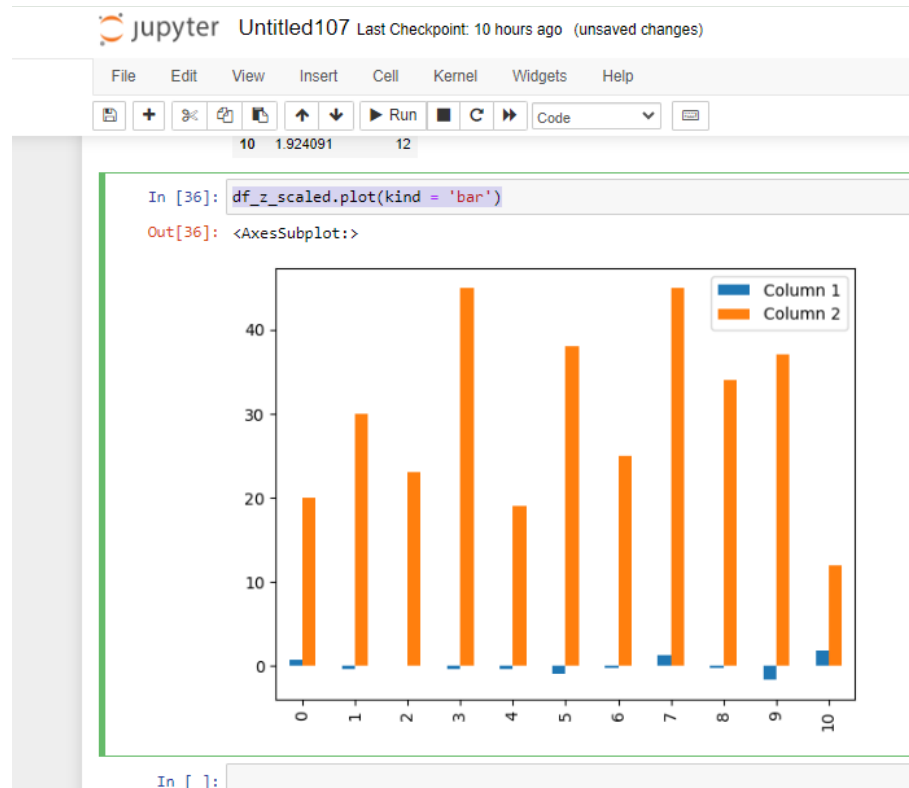
# apply normalization technique to Column 1
column = 'Column 1'
df_z_scaled[column] = (df_z_scaled[column] - df_z_scaled[column].mean()) / df_z_scaled[column].std()

# view normalized data
display(df_z_scaled)
```

	Column 1	Column 2
0	0.759387	20
1	-0.428611	30
2	0.118800	23
3	-0.324370	45
4	-0.376199	19
5	-0.929434	38
6	-0.288847	25
7	1.345815	45
8	-0.230611	34
9	-1.570021	37
10	1.924091	12

In []:

df_z_scaled.plot(kind = 'bar')



Example:

```
import pandas as pd

d = {"Name": ["Alisa", "Bobby", "Cat", "Madonna", "Rocky"],
     "Age": [1, 27, 25, 24, 31],
     "IQ": [100, 120, 95, 1300, 101]}

df = pd.DataFrame(d)

print(df.describe())
```

```
In [39]: import pandas as pd
d = {"Name": ["Alisa", "Bobby", "Cat", "Madonna", "Rocky"],
     "Age": [1, 27, 25, 24, 31],
     "IQ": [100, 120, 95, 1300, 101]}
df = pd.DataFrame(d)
print(df.describe())
```

	Age	IQ
count	5.000000	5.000000
mean	21.600000	343.200000
std	11.823705	534.952054
min	1.000000	95.000000
25%	24.000000	100.000000
50%	25.000000	101.000000
75%	27.000000	120.000000
max	31.000000	1300.000000

Example:

```
import pandas as pd

# list of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}
```

```
df = pd.DataFrame(dict)

print(df)

print()

print()

fm = df[(df.name == 'pankaj')]

print(fm)
```

```
In [53]: import pandas as pd
# list of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]
|
# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)
print(df)
print()
print()
fm = df[(df.name == 'pankaj')]
print(fm)
```

	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

	name	degree	score
1	pankaj	BCA	40

Example:

```
airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")

airline_safety.head()
```

```
In [41]: airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")
airline_safety.head()
```

```
Out[41]:
```

	airline	avail_seat_km_per_week	incidents_85_99	fatal_accidents_85_99	fatalities_85_99	incidents_00_14	fatal_accidents_00_14	fatalities_00_14
0	Aer Lingus	320906734	2	0	0	0	0	0
1	Aeroflot*	1197672318	76	14	128	6	1	88
2	Aerolineas Argentinas	385803648	6	0	0	1	0	0
3	Aeromexico*	596871813	3	1	64	5	0	0
4	Air Canada	1865253802	2	0	0	2	0	0

```
In [ ]:
```

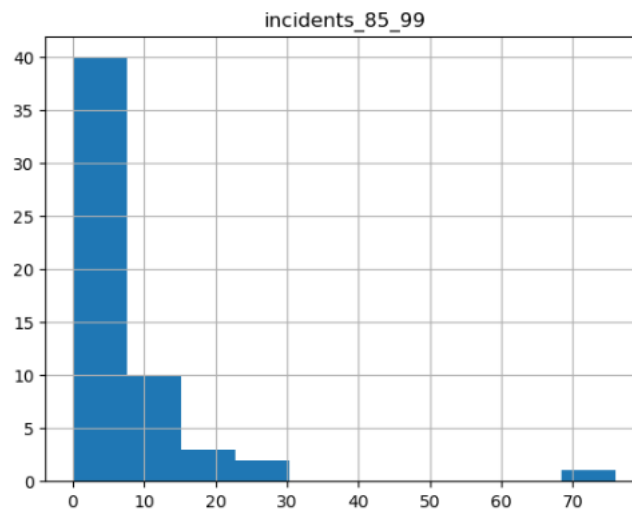
The data contains:

- airline: The new of the airline company.
- avail_seat: Passenger capacity. Available seat per km per week.
- incidents_85_99: Incidents between 1985 and 1999.
- fatal_accidents_85_99: Fatal accidents between 1985 and 1999.
- fatalities_85_99: Fatalities be between 1985 and 1999.
- incidents_00_14: Incidents between 2000 and 2014.
- fatal_accidents_00_14: Fatal accidents between 2000 and 2014.
- fatalities_00_14: Fatalities be between 2000 and 2014.

The data is stored in csv format and it appears to be structured (no missing data, no structural error).

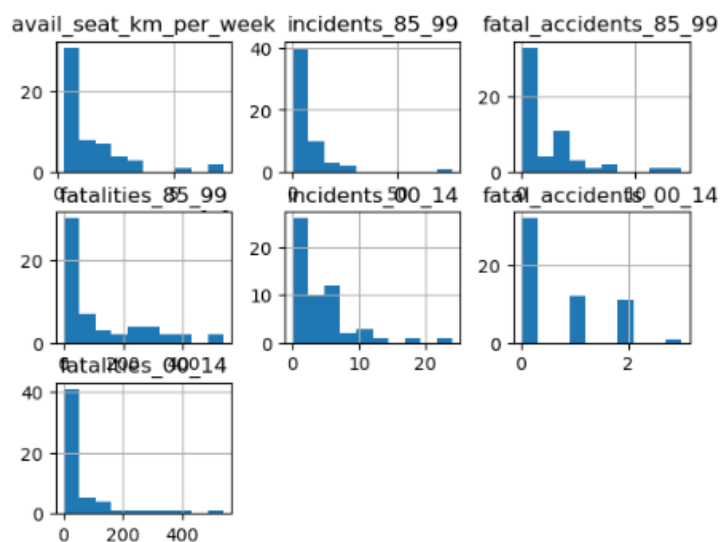
```
airline_safety.hist('incidents_85_99')
```

```
In [44]: airline_safety.hist('incidents_85_99')
Out[44]: array([[<AxesSubplot:title={'center':'incidents_85_99'}>]], dtype=object)
```



`airline_safety.hist()`

```
In [45]: airline_safety.hist()
Out[45]: array([[<AxesSubplot:title={'center':'avail_seat_km_per_week'}>,
<AxesSubplot:title={'center':'incidents_85_99'}>,
<AxesSubplot:title={'center':'fatal_accidents_85_99'}>],
[<AxesSubplot:title={'center':'fatalities_85_99'}>,
<AxesSubplot:title={'center':'incidents_00_14'}>,
<AxesSubplot:title={'center':'fatal_accidents_00_14'}>],
[<AxesSubplot:title={'center':'fatalities_00_14'}>],
<AxesSubplot:>, <AxesSubplot:>], dtype=object)
```



```
airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")
airline_safety.head()

# Filter out the airlines if incidents < 70
dfnan = df.mask(airline_safety["incidents_85_99"] < 70)

# Drop the irrelevant rows
df_filtered = dfnan.dropna()

# Print the results
print(df_filtered)
```

```
In [47]: airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")
airline_safety.head()
# Filter out the airlines if incidents < 70
dfnan = df.mask(airline_safety["incidents_85_99"] < 70)
# Drop the irrelevant rows
df_filtered = dfnan.dropna()
# Print the results
print(df_filtered)
```

	Name	Age	IQ
1	Bobby	27.0	120.0

```
In [ ]:
```

Chapter Five- DATA WRANGLING COMBINING AND MERGING DATA SETS

Objectives

5.1 Introduction

5.2 Data wrangling

5.2.1 Steps for data wrangling

5.2.2 Tools for data wrangling

5.3 Combining dataset

5.4 Concatenating dataset

5.5 Merging dataset

5.6 Reshaping dataset

5.6.1 Using melt () function

5.6.2 Using stack () and unstack () function

5.6.3 Using pivot () function

5.7 Data transformation

5.7.1 Data frame creation

5.7.2 Missing value

5.7.3 Encoding

5.7.4 Inset new column

5.7.5 Split column

5.8 String manipulation

5.9 Regular expression

5.9.1 The find all () function**5.9.2 The search () function****5.9.3 The split () function****5.9.4 The sub () function****OBJECTIVES**

The main goal of this chapter is to help students learn, understand and practice the data science approaches, which include the study of latest data science tools with latest programming languages. The main objectives of this module are data wrangling which includes data discovery, structuring, cleaning, enriching, validating and publishing, combining and merging datasets, data reshaping, pivoting, transformation, string manipulation operations and regular expression.

5.1 INTRODUCTION

Data science become a buzzword that everyone talks about the data science. Data science is an interdisciplinary field that combines different domain expertise, computer programming skills, mathematics and statistical knowledge to find or extract the meaningful or unknown patterns from unstructured and structure dataset.

Data science is useful for extraction, preparation, analysis and visualization of various information. Various scientific methods can be applied to get insight in data.

Data science is all about using data to solve problems. Data has become the fuel of industries.

It is most demandable field of 21st century. Every industry require data to functioning, searching, marketing, growing, expanding their business.

The application of areas of data science are health care, fraud detection, disease predicting, real time shipping routes, speech recognition, targeting advertising, gaming and many more.

How a Data Scientist works:

- **Ask the right questions** - To understand the business problem.
- **Explore and collect data** - From database, web logs, customer feedback, etc.
- **Extract the data** - Transform the data to a standardized format.
- **Clean the data** - Remove erroneous values from the data.
- **Find and replace missing values** - Check for missing values and replace them with a suitable value (e.g. an average value).
- **Normalize data** - Scale the values in a practical range (e.g. 140 cm is smaller than 1,8 m. However, the number 140 is larger than 1,8. - so scaling is important).
- **Analyze data**, find patterns and make future predictions.
- **Represent the result** - Present the result with useful insights in a way the "company" can understand.

5.2 DATA WRANGLING

Data wrangling is a **key component** of any data science project. Data wrangling is a process where one transforms “raw” data for making it more suitable for analysis and it will improve the quality of the data.

Data wrangling is the process of collecting, gathering and transforming of raw data into appropriate format for accessing, analyzing, easy understanding and further processing for better and quick decision making.

It is also known as **Data Munging** or **Data Pre-Processing**.

Data wrangling is a crucial first steps in the preparation of data for broad analysis of huge amount of data or big data. It requires significant amount of time and efforts. If data wrangling is properly conducted, it gives you insights into the nature of the data. It is not just a single time process but it is an iterative or repetitive process. Each step in the data wrangling process exposes the new potential ways for the data re-wrangling towards deriving the goal of complex data manipulation and analysis.

5.2.1 Steps for Data Wrangling

Data wrangling process includes **six** core activities:

- Discovery
- Structuring
- Cleaning
- Enriching
- Validating
- Publishing

- **Discovery**: Data discovering is an umbrella term. It describes the process to understand the dataset and insight into it. It involves the collection and evaluation of data from

various sources and is often used to identify the spot trends and detecting the patterns of the data and gain instant insight.

Now a days large business organization or companies have a large amount of data related to the customers, suppliers, production, sells, marketing etc. For example, if a company have a customer database, you can identify that the most of the customers are from which part of the city or state or country.

- **Structuring**: Data is coming from the various sources with the difference formats. Structuring is necessary because of the different size and shape of the data. Data structuring is the process or actions that change the form or schema of the dataset. Data splitting into the columns, deleting some fields from the dataset, pivoting rows are the form of data structuring.

- **Cleaning**: Before start the data manipulation or data analysis, you need to perform the data cleaning. Data cleaning is the process to identify the data quality related issues, such as missing or mismatched values, duplicate records etc. and apply the appropriate transformation to correct, replace or delete those values or records from the dataset to make high quality of data.

- **Enriching**: The data is useful for decision making process in the business. The data needed to take business related decision can be stored into multiple files.

To gather all necessary insights into single file and you need to enriched your existing dataset by performing joining and aggregating multiple data sources.

- **Validating**: After completion of data cleaning and data enriching, you need to check the accuracy of the data. If dataset is not accurate, it might be creating a problem. It is

necessary to do the data validation. This is the final check that any missing or mismatched data was not corrected during the transformation process. It is also need to validate that output dataset has the intended structure and content before publishing it.

- **Publishing:** After successful completion of data discovering, structuring, cleaning, enriching and validating, it's a time to published the wrangled output data for the further analytics processes, if any. The published data can be uploaded in the 55 organization"s software or store into the file in a specific location where organizations peoples knows it is ready to use.

5.2.2 Tools for Data Wrangling

There are some tools available for the data wrangling. Some of the popular tools are as follow:

Python

R

Tabula: Tabula is a tool for liberating data tables trapped inside the PDF files. It allows the user to upload the files in PDF format and extract the selected rows and columns from any tables available in the PDF file. It supports to extract this data from PDF to CSV or Microsoft Excel file format.

Data Wrangler:

It is an interactive tool for data cleaning. It takes the read word data and transform it into data tables which can be used for further processing or analysis. It also supports to export the data tables in Microsoft Excel, R, Tabula etc.

OpenRefine

OpenRefine, previously known as GoogleRefine. It is a Java based open source powerful tool for manipulates the huge data. It is used for data loading, understanding, cleaning and transforming from one format to another format. It also supports to extending the data with web services .

CSVKit

CSVKit is a suite for command line tools for converting and working with CSV file. It supports the covert the data from Excel to CSV, JSON to CSV, query with SQL etc.

5.3 COMBINING DATASET

Combining is the process to put two or more dataset together for further processing. The pandas library of Python provides easy functionality to combining the dataset together. Here we learn the combining dataset with concat, merge and join functions using **pandas** library.

- **Concat:** The `concat()` function is used for combining dataset across the rows or columns.

- **Merge:** The `merge()` function is used for combining the dataset on common columns.
- **Join:** The `join()` function is used for combining the dataset on key column or an index.

5.4 CONCATENATING DATASET

The “concat” function is used to perform the concatenation operation with the data frame along with an axis. The datasets are just stitched together along with axis (rows axis and column axis). Here we concatenate the datasets using **pandas**.

Syntax:

pd.concat(objs, axis, join, join_axes, ignore_index, keys)

objs: This is a sequence or mapping of Series, DataFrame, objects.

axis: This is an axis to concatenate. This value is {0, 1, ...}, default is 0.

join: This is used how to handle indexes on another axis(es). This value is {“inner”, “outer”}, default is “outer”. The outer is used for union operation and inner is used for intersection operation.

join_axes: This is the list of index objects. Its specific indexes to use for the other (n-1) axes instead of performing inner/outer set logic.

ignore_index: This value is Boolean type, default is False. If this value is True, do not use the index values on the concatenation axis. The resulting axis will be labeled 0, ..., n-1.

keys: This is a sequence to add an identifier to the result indexes, default is None.

Example:

Perform the concatenation operation using two different data frames **df1** and **df2**. The below code creates the two different data frame **df1** and **df2**.

```
import pandas as pd
```

```
# First dataframe creation
```

```
df1 = pd.DataFrame({  
    "Name":["Rahul","Shreya","Pankaj","Monika","Kalpesh"],  
    "Age":[20, 19, 24, 25, 25],  
    "Gender":["Male","Female","Male","Male","Female"],  
    "Course":["B.E.","B.Tech.","MCA","M.Tech.","M.E."]}, index = [1, 2, 3, 4, 5])
```

```
# Second dataframe creation
```

```
df2 = pd.DataFrame({  
    "Name":["Mayank","Jalpa","Sanjana","Vimal","Raj"],  
    "Age":[22, 21, 24, 26, 23],  
    "Gender":["Male","Female","Female","Male","Male"],  
    "Course":["MBA","MCA","B.E.","B.Tech.","M.Sc."]},  
    index = [1, 2, 3, 4, 5])
```

```
# Display first dataframe
```


df1

Output:

	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.
2	Shreya	19	Female	B.Tech.
3	Pankaj	24	Male	MCA
4	Monika	25	Male	M.Tech.
5	Kalpesh	25	Female	M.E.

Display second dataframe

df2

	Name	Age	Gender	Course
1	Mayank	22	Male	MBA
2	Jalpa	21	Female	MCA
3	Sanjana	24	Female	B.E.
4	Vimal	26	Male	B.Tech.
5	Raj	23	Male	M.Sc.

Concatenation of both dataframes

df3 = pd.concat([df1, df2]) df3

Output:

	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.
2	Shreya	19	Female	B.Tech.
3	Pankaj	24	Male	MCA
4	Monika	25	Male	M.Tech.
5	Kalpesh	25	Female	M.E.
1	Mayank	22	Male	MBA
2	Jalpa	21	Female	MCA
3	Sanjana	24	Female	B.E.
4	Vimal	26	Male	B.Tech.
5	Raj	23	Male	M.Sc.

Concatenation of both dataframe with axis as an argument

```
df3 = pd.concat([df1, df2], axis=1)
```

```
print(df3)
```

Output:

SN	Name	Age	Gender	Course	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.	Mayank	22	Male	MBA
2	Shreya	19	Female	B.Tech.	Jalpa	21	Female	MCA
3	Pankaj	24	Male	MCA	Sanjana	24	Female	B.E.
4	Monika	25	Male	M.Tech.	Vimal	26	Male	B.Tech.
5	Kalpesh	25	Female	M.E.	Raj	23	Male	M.Sc.

Concatenation of both dataframe with keys as an argument

```
df3 = pd.concat([df1, df2], keys=['x', 'y'])
```

```
df3
```

		Name	Age	Gender	Course
x	1	Rahul	20	Male	B.E.
	2	Shreya	19	Female	B.Tech.
	3	Pankaj	24	Male	MCA
	4	Monika	25	Male	M.Tech.
	5	Kalpesh	25	Female	M.E.
y	1	Mayank	22	Male	MBA
	2	Jalpa	21	Female	MCA
	3	Sanjana	24	Female	B.E.
	4	Vimal	26	Male	B.Tech.
	5	Raj	23	Male	M.Sc.

Concatenation of both dataframe using keys argument

```
df3 = pd.concat([df1, df2], keys=['x','y'], ignore_index=True)
```

df3

	Name	Age	Gender	Course
0	Rahul	20	Male	B.E.
1	Shreya	19	Female	B.Tech.
2	Pankaj	24	Male	MCA
3	Monika	25	Male	M.Tech.
4	Kalpesh	25	Female	M.E.
5	Mayank	22	Male	MBA
6	Jalpa	21	Female	MCA
7	Sanjana	24	Female	B.E.
8	Vimal	26	Male	B.Tech.
9	Raj	23	Male	M.Sc.

Concatenation of both dataframe using append

```
df1.append(df2)
```

	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.
2	Shreya	19	Female	B.Tech.
3	Pankaj	24	Male	MCA
4	Monika	25	Male	M.Tech.
5	Kalpesh	25	Female	M.E.
1	Mayank	22	Male	MBA
2	Jalpa	21	Female	MCA
3	Sanjana	24	Female	B.E.
4	Vimal	26	Male	B.Tech.
5	Raj	23	Male	M.Sc.

5.5 MERGING DATASET

The word “merge” and “join” both are used relatively interchangeable in SQL, R and Pandas. Both merge and join doing the similar things, but there are separate “merge” and “join” functions in Pandas. The merging/joining is the process of bringing two or more datasets together into single dataset and aligning the rows from each dataset based on the common attributes or columns. The „merge” function is used to perform the merging operation with the data frame. Here we merge the datasets using pandas.

Syntax:

```
pd.merge(left, right, how, on, left_on, right_on, left_index, right_index, sort)
```

left: This is the first data frame.

right: This is the second data frame.

how: This is the method how to perform the merge operation. The values of this field are one of „left“, „right“, „inner“, „outer“, default is „inner“.

On: This is the name of column in which action to be perform. This column must be available in both left and right data frame object.

left_on: This is the name of column from left data frame to use as keys.

right_on: This is the name of column from right data frame to use as keys.

left_index: This is using the index (row label) from left data frame as its join keys, if it is True.

right_index: This is used the index (row label) from right data frame as its join keys, if it is True.

sort: This is use to sort the result data frame by join keys in specific order. The value of this field is Boolean, default is True.

Example: Perform the merge operation using two different data frames i.e. left and right.

```
# Importing pandas library
```

```
import pandas as pd
```

```
# Left dataframe creation
```

```
left = pd.DataFrame({
```

```
    "Rno":[1, 2, 3, 4, 5],
```

```
"Name":["Rahul","Shreya","Pankaj","Monika","Kalpesh"],  
"Course":["B.E.","B.Tech.","MCA","M.Tech.","M.E."])
```

```
# Right dataframe creation
```

```
right = pd.DataFrame({  
    "Rno":[1, 2, 3, 4, 5],  
    "Name":["Mayank","Jalpa","Sanjana","Vimal","Raj"],  
    "Course":["B.E.","MBA","MCA","B.Tech.","M.Sc."])
```

```
# Display left dataframe
```

```
print(left)
```

	Rno	Name	Course
0	1	Rahul	B.E.
1	2	Shreya	B.Tech.
2	3	Pankaj	MCA
3	4	Monika	M.Tech.
4	5	Kalpesh	M.E.

```
# Display right dataframe
```

```
print(right)
```

	Rno	Name	Course
0	1	Mayank	B.E.
1	2	Jalpa	MBA
2	3	Sanjana	MCA
3	4	Vimal	B.Tech.
4	5	Raj	M.Sc.

perform the merge operation on both data frame using **on** as an argument.

Merge both left and right dataframe using single on key

```
pd.merge(left, right, on='Rno')
```

	Rno	Name_x	Course_x	Name_y	Course_y
0	1	Rahul	B.E.	Mayank	B.E.
1	2	Shreya	B.Tech.	Jalpa	MBA
2	3	Pankaj	MCA	Sanjana	MCA
3	4	Monika	M.Tech.	Vimal	B.Tech.
4	5	Kalpesh	M.E.	Raj	M.Sc.

Merge left and right dataframe using multiple on keys

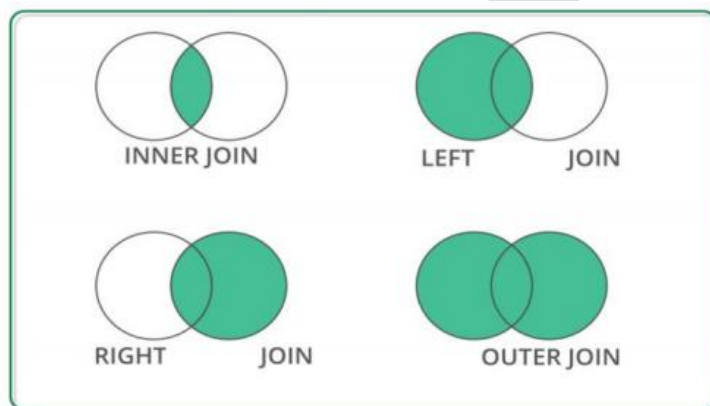
```
pd.merge(left, right, on=['Rno','Course'])
```

	Rno	Name_x	Course	Name_y
0	1	Rahul	B.E.	Mayank
1	3	Pankaj	MCA	Sanjana

The **merge** methods are same as SQL **join** equivalent as below:

Merge Method	SQL Join Equivalent	Description
left	Left Outer Join	Use keys from left object
right	Right Outer Join	Use keys from right object
inner	Inner Join	Use intersection of keys
outer	Full Outer Join	Use unions of keys

represented graphically as:



Merge both left and right dataframe using on and how

```
pd.merge(left, right, on='Course', how='left')
```

	Rno_x	Name_x	Course	Rno_y	Name_y
0	1	Rahul	B.E.	1.0	Mayank
1	2	Shreya	B.Tech.	4.0	Vimal
2	3	Pankaj	MCA	3.0	Sanjana
3	4	Monika	M.Tech.	NaN	NaN
4	5	Kalpesh	M.E.	NaN	NaN

Merge both left and right dataframe using on and how

```
pd.merge(left, right, on='Course', how='right')
```


	Rno_x	Name_x	Course	Rno_y	Name_y
0	1.0	Rahul	B.E.	1	Mayank
1	NaN	NaN	MBA	2	Jalpa
2	3.0	Pankaj	MCA	3	Sanjana
3	2.0	Shreya	B.Tech.	4	Vimal
4	NaN	NaN	M.Sc.	5	Raj

Merge both left and right dataframe using on and how

```
pd.merge(left, right, on='Course', how='inner')
```

	Rno_x	Name_x	Course	Rno_y	Name_y
0	1	Rahul	B.E.	1	Mayank
1	2	Shreya	B.Tech.	4	Vimal
2	3	Pankaj	MCA	3	Sanjana

Merge both left and right dataframe using on and how

```
pd.merge(left, right, on='Course', how='outer')
```

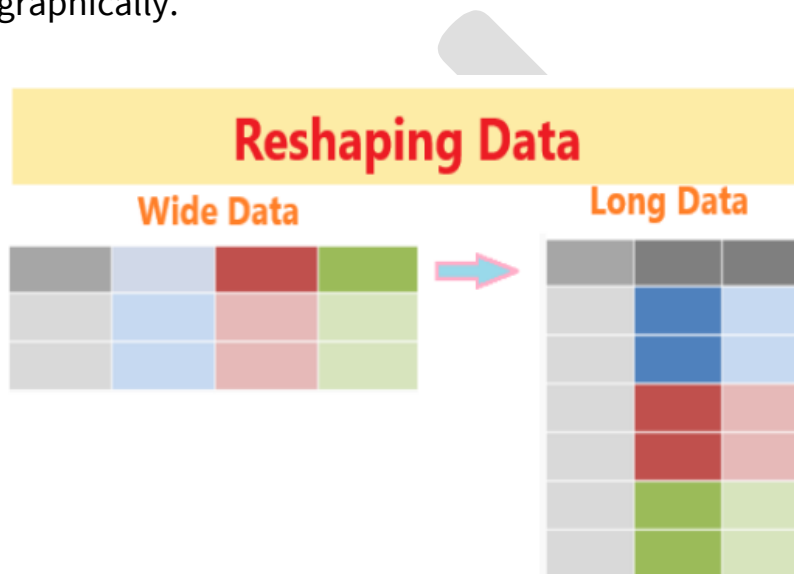
	Rno_x	Name_x	Course	Rno_y	Name_y
0	1.0	Rahul	B.E.	1.0	Mayank
1	2.0	Shreya	B.Tech.	4.0	Vimal
2	3.0	Pankaj	MCA	3.0	Sanjana
3	4.0	Monika	M.Tech.	NaN	NaN
4	5.0	Kalpesh	M.E.	NaN	NaN
5	NaN	NaN	MBA	2.0	Jalpa
6	NaN	NaN	M.Sc.	5.0	Raj

5.6 RESHAPING DATASET

The **dataset** is arranged into rows and columns is referred as the shape of data. In a dataset, each row represents one observation in a vertical or long data and each column is considered a variable with multiple distinct values.

It is needed to convert or transform the dataset from one format to another format, which is called reshaping of dataset.

Reshaping is the process to change the shape or structure of datasets, such as convert “**wide**” data tables into “**long**”. The below figure shows the reshaping process graphically.



The data frame will be reshaped by using **melt()**, **stack()**, **unstack()** and **pivot()** functions as follows:

```
# Importing pandas library
```

```
import pandas as pd
```

```
# Dataframe creation
```

```
df = pd.DataFrame({
```

```
"Rno":[1, 2, 3, 4, 5],
```

```
"Name":["Rajan","Shital","Mayur","Mittal","Mahesh"],
```

```
"Age": [25, 27, 24, 25, 21]])
```

```
# Display dataframe
```

```
print(df)
```

	Rno	Name	Age
0	1	Rajan	25
1	2	Shital	27
2	3	Mayur	24
3	4	Mittal	25
4	5	Mahesh	21

We can reshape the data frame using **melt()** function. This function is used to wide data frame columns into rows.

```
df.melt()
```

	Variable	value
0	Rno	1
1	Rno	2
2	Rno	3
3	Rno	4
4	Rno	5
5	Name	Rajan
6	Name	Shital
7	Name	Mayur
8	Name	Mittal
9	Name	Mahesh
10	Age	25
11	Age	27
12	Age	24
13	Age	25
14	Age	21

We can reshape the data frame using **stack()** and **unstack()** functions. The **stack()** function is used to increase the level of index in a data frame.

df.stack()

0	Rno	1
	Name	Rajan
	Age	25
1	Rno	2
	Name	Shital
	Age	27
2	Rno	3
	Name	Mayur
	Age	24

3	Rno	4
	Name	Mittal
	Age	25
4	Rno	5
	Name	Mahesh
	Age	21
dtype: object		

The **unstack()** function is used to do the revert back changes in a data frame was perform by the stack() function.

Rno	0	1
	1	2
	2	3
	3	4
	4	5
Name	0	Rajan
	1	Shital
	2	Mayur
	3	Mittal
	4	Mahesh
Age	0	25
	1	27
	2	24
	3	25
	4	21
dtype: object		

We can reshape the data frame using **pivot()** function. This function is used to reshape the data frame based on the specified column in a data frame.

Performing pivot function on dataframe

```
df.pivot(columns='Rno')
```

	Name					Age				
Rno	1	2	3	4	5	1	2	3	4	5
0	Rajan	NaN	NaN	NaN	NaN	25.0	NaN	NaN	NaN	NaN
1	NaN	Shital	NaN	NaN	NaN	NaN	27.0	NaN	NaN	NaN
2	NaN	NaN	Mayur	NaN	NaN	NaN	NaN	24.0	NaN	NaN
3	NaN	NaN	NaN	Mittal	NaN	NaN	NaN	NaN	25.0	NaN
4	NaN	NaN	NaN	NaN	Mahesh	NaN	NaN	NaN	NaN	21.0

5.7 DATA TRANSFORMATION

Data is collected from the various sources and combine it into a unified data frame. This data frame has large number of columns with different data types.

Data transformation is the process to transform or convert the data as per required format for further processing as and when needed.

The data transformation includes add new columns, find NaN values, drop NaN, replace with mean value, field encoding and decoding, column splitting etc.

5.7.1 Data Frame Creation

To perform the various transformation operation on data, first we have to create the data frame.

The below code will create and display a data frame **df** which contains the three columns such as “**Name**”, “**Gender**” and “**City**”.

```
#importing pandas library

import pandas as pd

# Dataframe creation

df = pd.DataFrame({

    "Name":["Jayesh Patel","Priya Shah","Vijay Sharma"],

    "Gender": ["Male","Female","Male"],

    "City": ["Rajkot","Delhi","Mumbai"]})

# Display dataframe

print(df)
```

	Name	Gender	City
0	Jayesh Patel	Male	Rajkot
1	Priya Shah	Female	Delhi
2	Vijay Sharma	Male	Mumbai

5.7.2 Missing Value

The dataset contains the many rows and columns. There are some cells in a dataset that have **NA** or empty cell. This is called **missing data** in a dataset. It is needed first to check that missing value before further processing. The common and very simple method to handle this missing value is to delete the rows which contain missing values.

```
df.isna().sum()
```

Here there are no any missing values in each column so it returns zero value in each column. If there are any missing values in each column, it returns the number of missing values.

Name 0

Gender 0

City 0

dtype: int64

drop all the rows which containing missing value.

```
df = df.dropna()
```

drop the columns where all elements are missing values.

```
df.dropna(axis=1, how='all')
```

drop the columns where any of the elements containing missing values

```
df.dropna(axis=1, how='any')
```

keep only the rows which contains maximum two missing values.:

```
df.dropna(thresh=2)
```

fill all missing values with mean value of the particular column.

```
df.fillna(df.mean())
```

To fill any missing values in specified column with **median** value of the particular column. Here we taken “Age” column for example.

```
df['Age'].fillna(df['Age'].median())
```

To fill any missing values in specified column with mode value of the particular column. Here we taken “Age” column for example.

```
df['Age'].fillna(df['Age'].mode())
```

5.7.3 Encoding

The dataset contains both numerical and categorical value. The categorical data is not much useful for data processing or analytics. It is needed to encoding the categorical value into numeric value.

Data encoding is the process to convert a categorical variable into a numerical form.

Here we discuss the label encoding which is simply converting each value in a column to a number. Our dataset has “**Gender**” column which has only two values “**male**” and “**female**”

It encodes like this:

Male→0

Female→1

```
df=df.Gender.replace({"Male":0,"Female":1})
```



```
print(df)
```

output:

```
0  0
1  1
2  0
Name: Gender, dtype: int64
```

5.7.4 Inset New Column

It is needed to add one or more columns in an existing dataset.

```
df.insert(1, "Age", [21, 23, 24], True)
```

```
print(df)
```

	Name	Age	Gender	City
0	Jayesh Patel	21	Male	Rajkot
1	Priya Shah	23	Female	Delhi
2	Vijay Sharma	24	Male	Mumbai

5.7.5 Split Column

To split one column into two or more different columns. The process to create two or more different columns from single column in a dataset is called column splitting. Sometimes the **“Full Name”** column of dataset may be need to split into **“First Name”** and **“Last Name”** as a separate column.

```
df[['First Name','Last Name']] = df.Name.str.split(expand=True)
```

```
df
```

	Name	Age	Gender	City	First Name	Last Name
0	Jayesh Patel	21	Male	Rajkot	Jayesh	Patel
1	Priya Shah	23	Female	Delhi	Priya	Shah
2	Vijay Sharma	24	Male	Mumbai	Vijay	Sharma

5.8 STRING MANIPULATION

String manipulation is the process of handling and analyzing the strings. The various operation can be performed on string such as string modification, parsing of string, string conversion etc. The various in-built functions available for string manipulation in different language. Here we perform some common string manipulation operations in Python.

We take the following strings as an example.

```
str1 = "Data Science"
```

```
str2 = "using"
```

```
str3 = "Python"
```

```
str4 = "2021"
```

```
str5 = " Data Science "
```

Function	Description	Example	Output
capitalize()	Converts the first character of string into upper case	str2.capitalize()	'Using'

casefold()	Converts the string into lower case	str1.casefold()	'data science'
center()	Returns the string in center of the specified size	str1.center(15)	' Data Science'
count()	Returns the number of times a specified value occurs in a string	str1.count("a")	2
endswith()	Returns true if the string ends with the specified value	str1.endswith("nce")	True
find()	Searches the string for a specified value and returns the position of where it is found.	str1.find("i")	7
format()	Formats the specified values in a string	str4.format()	'2021'
index()	Searches the string for a specified value and returns the position of where it was found	str1.index("c")	6

isalnum()	returns True if all the characters in a string are alphanumeric	str4.isalnum()	True
isalpha()	Returns True if all characters in a string are alphabet	str2.isalpha()	True
isdigit()	Returns True if all characters in a string are digits	str4.isdigit()	True
islower()	Returns True if all characters in a string are lower case	str2.islower()	True
isnumeric()	Returns True if all characters in a string are numeric	str4.isnumeric()	True
isprintable()	Returns True if all characters in a string are printable	str1.isprintable()	True
isspace()	Returns True if all characters in a string are whitespaces	str1.isspace()	False
istitle()	Returns True if the string follows the title case rules	str1.istitle()	True
isupper()	Returns True if all characters in a string are	str1.isupper()	False

	upper case letter		
len(string)	Returns the length of a string	len(str1)	12
lower()	Converts a string into lower case	str1.lower()	'data science'
lstrip()	Returns the string with left trim version	str5.lstrip()	'Data Science '
replace(old,new)	Replaces the old string with new string	Str1.replace("Science", "Analytics")	"Data Analytics"
rfind()	Searches the string for a specified value and returns the last position of where it was found	str1.rfind("e")	11
rindex()	Searches the string for a specified value and returns the last index position of where it is found	str1.rindex("a")	3
rstrip()	Returns the string with right trim version	str1.rstrip(5)	' Data Science'
split()	Splits the string with specified separator and returns a list	str1.split(" ")	['Data', 'Science']

startswith()	Returns true if the string starts with the specified value	str3.startswith("P")	True
strip()	Returns both left and right trim version	str5.strip()	'Data Science'
swapcase()	Returns the swaps cases, lower case becomes upper case and vice versa	str1.swapcase()	'dATA sCIENCE'
title()	Converts the first character of each word to upper case	str2.title()	'Using'
upper()	Converts a string into upper case	str1.upper()	'DATA SCIENCE'
zfill()	Returns the string with filled by 0 for specified number of times in a string at beginning	str3.zfill(10)	'0000Python'
+	Concatenates two strings	str1+" "+str2+" "+str3	"Data Science using Python"
*	Repeat the string n times	str3*3	"PythonPythonPython"
string[0]	Returns the first character of a string	str1[0]	'D'
string[7]	Returns the eighth	str1[7]	'i'

	character of a string		
string[2:8]	Returns the third to eighth characters	str1[2:8]	'ta Sci'
string[3:]	Returns characters from fourth to last	str1[3:]	's Science'
string[:8]	Returns characters from fourth to last	str1[:8]	'Data Sci'
string[-4:]	Returns the last four characters	str1[-4:]	'ence'
string[:-4]	Removes the last four characters	str1[:-4]	'Data Sci'

5.9 REGULAR EXPRESSION

Regular expression or RegEx is generally used to identify whether a sequence or character or pattern exists in a given string or not. It is also used to identify the position of such pattern in a string or file. It mainly used to find and replace specific patterns in a string or file. It helps to manipulate text-based datasets.

Python has a built-in package called **re**, to work with Regular Expression.

The **re** package of Python has set of functions to search the string for matching. The common Python RegEx functions are as follow:

Function	Description
findall	Returns a list of all match values
search	Returns a match object if math found anywhere in the string
split	Returns a list and split the string where each match found
sub	Replaces the one or more matches with a specified string

5.9.1 The findall() Function

This function returns a list which contains all match values.

Example:

```
import re

string = "Working with Data Science using Python"

result = re.findall("th",string)

print(result)
```

The list contains all match values in an order of they are found.

Output:

```
['th', 'th']
```

Example:

```
import re

string = "Working with Data Science using Python"

result = re.findall("ds",string)
```



```
print(result)
```

If no matches found, an empty list will return.

Output:

```
[]
```

5.9.2 The search() Function

This function returns a match object if match found anywhere in the string. Only first occurrence of match will be returned, if there are more than one match found. The none will return if no match found.

Example:

```
import re
```

```
string = "Working with Data Science using Python"
```

```
result = re.search("wi",string)
```

```
result
```

It found the match value “**wi**”.

Output:

```
<re.Match object; span=(8, 10), match='wi'>
```

Example:

```
import re
```

```
string = "Working with Data Science using Python"
```

```
result = re.search("\s",string)
```

```
result
```

It found the match value “\s” i.e. space.

Output:

```
<re.Match object; span=(7, 8), match=' '>
```

5.9.3 The split() Function

This function returns a list of where the string has been split at each match found.

Example:

```
import re
```

```
string = "Working with Data Science using Python"
```

```
result = re.split("\s",string)
```

```
result
```

It found the match value “\s” i.e. space.

Output:

```
['Working', 'with', 'Data', 'Science', 'using', 'Python']
```

Example:

```
import re
```

```
string = "Working with Data Science using Python"
```

```
result = re.split("\s",string,2)
```

```
result
```

It found the match value “\s” i.e. space. But here the string will split into first 2 occurrence of found only. The remaining string will print as it is.

Output:

```
['Working', 'with', 'Data Science using Python']
```

5.9.4 The sub() Function

This function replaces the string with the specified text at each match found.

It will print same string, if not match found.

Example:

```
import re
```

```
string = "Working with Data Science using Python"
```

```
result = re.sub("\s", "-",string)
```

```
result
```

It found the match value “\s” i.e. space. Every “\s” (space) will replace with the “-” (dash).

Output:

'Working-with-Data-Science-using-Python'

Example:

```
import re
```

```
string = "Working with Data Science using Python"
```

```
result = re.sub("\s","-",string,2)
```

```
result
```

It found the match value “\s” i.e. space. But here “\s” (space) will replace with the “-” (dash) in first two occurrence of found only. The remaining string will print as it is.

Output:

'Working-with-Data Science using Python'

QUESTIONS

1. What is data wrangling?
2. What is data cleaning?
3. List tools for data wrangling.
4. What is merging dataset?
5. What is reshaping dataset?
6. Explain steps for data wrangling process.
7. Explain concat function with example.

8. Explain merge operation with syntax and example.
9. Explain reshaping dataset different functions.
10. Explain string function with example.
11. Explain regular expression with example.
12. Create and display a data frame.
13. Create a data frame and find null values and remove it.
14. Create a data frame and insert new column in data frame.
15. Create a data frame and convert categorical data into numerical values.
16. Create a data frame and split the column.
17. Create data frames and combine using concat function.
18. Create data frames and merge with different arguments.
19. Create data frame and reshape using melt function.
20. Perform string manipulation operations on string.
21. Perform regular expression functions on string.

Chapter Six- AGGREGATION AND GROUP OPERATIONS GROUP BY MECHANICS

The main goal of this Chapter is to help students learn, understand and practice the data science approaches, which include the study of latest data science tools with latest programming . The main objectives of this chapter are data aggregation, group wise operation including data splitting, applying and combining, data transformation using lamda function, pivot table, cross tabulation using two-way and three-way cross table and data and time data type.

6.1 INTRODUCTION

Data science is an interdisciplinary field that combines different domain expertise, computer programming skills, mathematics and statistical knowledge to find or extract the meaningful or unknown patterns from unstructured and structure dataset.

Data science is useful for extraction, preparation, analysis and visualization of various information. Various scientific methods can be applied to get insight in data.

Data science is all about using data to solve problems. Data has become the fuel of industries. It is most demandable field of 21st century. Every industry require data to functioning, searching, marketing, growing, expanding their business.

The application of areas of data science are health care, fraud detection, disease predicting, real time shipping routes, speech recognition, targeting advertising, gaming and many more.

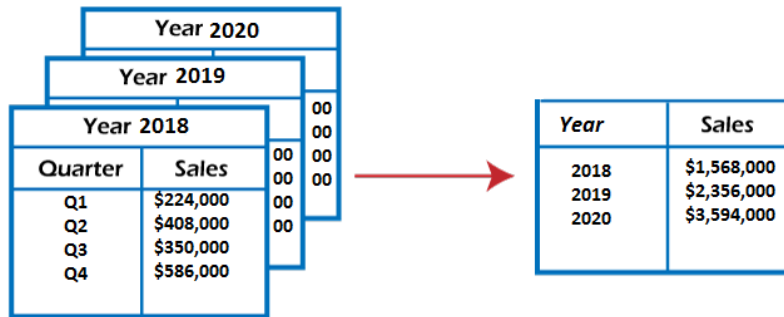
6.2 Data Aggregation

Data aggregation is the process to gather the raw data and express in a summarised form for statistical analysis. Data may be collected from various sources and combine into a summary format for data analysis.

A **dataset** contains large amount of data in a **rows** and **columns**. There are thousands or more data records are in a single dataset. Data aggregation will be useful to access and process the large amount of data quickly. Aggregate data can be access quickly to gain insight instead of accessing all the data records. A single raw of aggregated data can represent this large number of data records over a given time period to calculate the statistics such as sum, minimum, maximum, average and count.

- **Sum**: This function add all the specified data to get a total.
- **Min**: This function displays the lowest value of each specified category.
- **Max**: This function displays the highest value of each specified category.
- **Average**: This function calculates the average value of the specific data.
- **Count**: This function counts the total number of data entries for each category.

Data aggregation provides more insight information based on related cluster of data. For example, a company want to know the sales performance of different district, they would aggregate the sales data based on the district. Data can aggregate by date also, if you want to know the trends over a period of months, quarters, years, etc.



Aggregated Data

Example: Perform the aggregation operation on data frame.

#importing pandas library

import pandas as pd

Dataframe creation

```
df = pd.DataFrame({
    "Rno": [1,2,3,4,5,6,7,8,9,10],
    "Maths": [67,83,74,91,55,70,86,81,92,67],
    "Physics": [56,67,72,84,89,79,90,89,92,82],
    "Chemistry": [81,88,78,69,74,72,83,90,58,68],
    "Biology": [90,83,86,75,68,79,67,71,91,89],
    "English": [60,55,63,71,88,75,91,82,85,80]})
```

print(df)

output:

	Rno	Maths	Physics	Chemistry	Biology	English
0	1	67	56	81	90	60
1	2	83	67	88	83	55
2	3	74	72	78	86	63
3	4	91	84	69	75	71
4	5	55	89	74	68	88
5	6	70	79	72	79	75
6	7	86	90	83	67	91
7	8	81	89	90	71	82
8	9	92	92	58	91	85
9	10	67	82	68	89	8

Now, perform the aggregation using **min**, **max** and **sum**.

code to find min and max value of different subject of data frame df.

```
df.agg(["min","max"])
```

	Rno	Maths	Physics	Chemistry	Biology	English
Min	1	55	56	58	67	55
Max	10	92	92	90	91	91

To find min and max value and calculate average and sum value of different subject of data frame df:

```
df.agg(["min","max","average","sum"])
```

	Rno	Maths	Physics	Chemistry	Biology	English
min	1.0	55.0	56.0	58.0	67.0	55.0
max	10.0	92.0	92.0	90.0	91.0	91.0
average	5.5	76.6	80.0	76.1	79.9	75.0
sum	55.0	766.0	800.0	761.0	799.0	750.0

To perform the different aggregation functions on different columns.

Code to find min and max value of “Maths” subject max and sum of “Physics” subject and min, median and std of “English” subject of data frame df:

```
df.agg({"Maths":["min","max"],
      "Physics":["max","sum"],
      "English":["min","median","std"]})
```

	Maths	Physics	English
max	92.0	92.0	NaN
median	NaN	NaN	77.500000
min	55.0	NaN	55.000000
std	NaN	NaN	12.400717
sum	NaN	800.0	NaN

To calculate the sum of each column of data frame df:

df.sum()

Rno 55

Maths 766

Physics 800

Chemistry 761

Biology 799

English 750

dtype: int64

To find **min** value of each column of data frame df:

df.min()

Rno 1

Maths 55

Physics 56

Chemistry 58

Biology 67

English 55

dtype: int64

To find the **max** value of each column of data frame df:

df.max()

Rno 10

Maths 92

Physics 92

Chemistry 90

Biology 91

English 91

dtype: int64

To calculate the **mean** value of each column of data frame df:

df.mean()

Rno 5.5

Maths 76.6

Physics 80.0

Chemistry 76.1

Biology 79.9

English 75.0

dtype: float64

To **count** the numbers of values in each column of data frame df:

df.count()

Rno 10

Maths 10

Physics 10

Chemistry 10

Biology 10

English 10

dtype: int64

To calculate **standard deviation** of each column of data frame df:

df.std()

Rno 3.027650

Maths 11.992590

Physics 11.718931

Chemistry 9.859570

Biology 9.230986

English 12.400717

dtype: float64

To calculate the **basic statistics** of each column of data frame df:

df.describe()

	Rno	Maths	Physics	Chemistry	Biology	English
count	10.00000	10.00000	10.000000	10.00000	10.000000	10.000000
mean	5.50000	76.60000	80.000000	76.10000	79.900000	75.000000
std	3.02765	11.99259	11.718931	9.85957	9.230986	12.400717
min	1.00000	55.00000	56.000000	58.00000	67.000000	55.000000
25%	3.25000	67.75000	73.750000	69.75000	72.000000	65.000000
50%	5.50000	77.50000	83.000000	76.00000	81.000000	77.500000
75%	7.75000	85.25000	89.000000	82.50000	88.250000	84.250000
max	10.00000	92.00000	92.000000	90.00000	91.000000	91.000000

6.3 GROUP WISE OPERATION

The **groupby()** function is used to perform the group wise operation on a large dataset. This is a versatile and easy to use function which help to get summary of large dataset. The summary is easy to explore the dataset and shows the relationship between variables.

We can create a grouping of different categories and apply various functions to each category. This function is widely used in real data science projects which dealing with large mounts of data. It has ability to aggregate data efficiently. This function refers to a process of involving one or more of the following steps:

- **Splitting:** It is a process in which we split the dataset into different groups based on some criteria.
- **Applying:** It is a process in which we apply different functions to each group independently. To apply the function to each group, we perform some operations:

- **Aggregation:** It is a process to compute a statistical summary of the group such as sum, mean, median, etc.
- **Transformation:** It is a process to perform some group specific computations and return a like-indexed such as filling NA within group with a value derived from each group.
- **Filtration:** It is a process to remove some groups based on some criteria such as filtering out dataset based on group wise sum or mean.
- **Combining:** It is a process to combine different datasets after applying groupby function and results will store in a dataset.

The syntax of **groupby** function is:

DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=False, **kwargs)

- **by:** mapping, function, label, str
- **axis:** int, 0 or index and 1 or columns, default is 0, split along rows (0) or columns (1).
- **level:** int, level name, default is None, If the axis is a MultiIndex, group by a particular level or levels.

- **as_index:** boolean, default is True, for aggregated output, return object with group labels as the index.
- **sort:** boolean, default is True, sort group keys.
- **group_keys:** boolean, default is True, when calling apply, add group keys to index to identify pieces.
- **squeeze:** boolean, default is False, reduce the dimensionality of the return type, if possible

Example:

```
import pandas as pd

# Dataframe creation

df = pd.DataFrame({

    "Product":["Mango","Corn","Orange","Cabbage","Mango","Corn","Watermelon","Apple","Pumkin","Mango"],

    "Category":["Fruit","Vegetable","Fruit","Vegetable","Fruit","Vegetable","Fruit","Fruit","Vegetable","Fruit"],

    "Qty":[12, 5, 10, 2, 10, 3, 5, 8, 2, 10],

    "Price":[350, 80, 320, 50, 200, 50, 280, 380, 60, 400]})

print(df)
```


	Product	Category	Qty	Price
0	Mango	Fruit	12	350
1	Corn	Vegetable	5	80
2	Orange	Fruit	10	320
3	Cabbage	Vegetable	2	50
4	Mango	Fruit	10	200
5	Corn	Vegetable	3	50
6	Watermelon	Fruit	5	280
7	Apple	Fruit	8	380
8	Pumkin	Vegetable	2	60
9	Mango	Fruit	10	400

To perform the **groupby** operation on “Category” columns:

```
df.groupby("Category").sum()
```

the total Qty of “Fruit” category is 55 and total Price of “Fruit” category is 1930, while total Qty of “Vegetable” category is 12 and total Price of “Vegetable” category is 240.

	Qty	Price
Category		
Fruit	55	1930
Vegetable	12	240

To perform the groupby operation on “Product” columns.

```
df.groupby("Product").sum()
```

	Qty	Price
Product		
Apple	8	380
Cabbage	2	50
Corn	8	130
Mango	32	950
Orange	10	320
Pumkin	2	60
Watermelon	5	280

perform the **groupby** operation on “Category” columns.

```
df.groupby("Category")["Price"].mean()
```

```
Category
Fruit      321.666667
Vegetable   60.000000
Name: Price, dtype: float64
```

To find mean of “Qty” based on “Category” on data frame df.

```
df.groupby("Category")["Qty"].mean()
```

```
Category
Fruit      9.166667
Vegetable   3.000000
Name: Qty, dtype: float64
```

To perform the groupby operation on “Product” columns.

```
df.groupby("Product")["Price"].mean()
```

```
Product
Apple      380.000000
Cabbage     50.000000
Corn        65.000000
Mango      316.666667
Orange     320.000000
Pumkin      60.000000
Watermelon 280.000000
Name: Price, dtype: float64
```

To perform the groupby operation on “Category” columns.

```
df.groupby("Category")["Price"].median()
```

```
Category
Fruit      335
Vegetable   55
Name: Price, dtype: int64
```

To find standard deviation of “Price” based on “Category” on data frame df.

```
df.groupby("Category")["Price"].std()
```

```
Category
Fruit      73.325757
Vegetable  14.142136
Name: Price, dtype: float64
```

6.4 Transformation

Transformation is a process in which we perform some group-specific computations and return a like-indexed. Transformation perform on a group or a column which returns

an object that is indexed the same size of that is being grouped. Thus, the transform should return a result that is the same size as that of a group chunk.

Syntax:

DataFrame.transform(func, axis=0, *args, **kwargs)

- **func:** this is the function to use for data transformation.
- **axis:** the axis in which the transformation will perform, {0 or „index“, 1 or ‘columns’}, default is 0.
- ***args:** positional arguments to pass in func.
- ****kwargs:** keyword arguments to pass in func.

Example: perform some group specific operation and return a like-indexed.

```
import pandas as pd

# Creating the DataFrame

df = pd.DataFrame({

    "A":[8, 7, 15, 12, 15],

    "B":[None, 22, 32, 9, 7],

    "C":[10, 6, None, 8, 14]})

print(df)
```

out:

	A	B	C
0	8	NaN	10.0
1	7	22.0	6.0
2	15	32.0	NaN
3	12	9.0	8.0
4	15	7.0	14.0

To perform the transform operation using **lambda**.

```
result = df.transform(func = lambda x : x * 5)  
print(result)
```

	A	B	C
0	40	NaN	50.0
1	35	110.0	30.0
2	75	160.0	NaN
3	60	45.0	40.0
4	75	35.0	70.0

calculate the square root of value of all the columns A, B and C of data frame df.

```
result = df.transform(func = ["sqrt"])  
print(result)
```

	A	B	C
	sqrt	sqrt	sqrt
0	2.828427	NaN	3.162278
1	2.645751	4.690416	2.449490
2	3.872983	5.656854	NaN
3	3.464102	3.000000	2.828427
4	3.872983	2.645751	3.741657

calculate the exponential of value of all the columns A, B and C of data frame df.

```
result = df.transform(func = ["exp"])
```

```
print(result)
```

	A	B	C
	exp	exp	exp
0	2.980958e+03	NaN	2.202647e+04
1	1.096633e+03	3.584913e+09	4.034288e+02
2	3.269017e+06	7.896296e+13	NaN
3	1.627548e+05	8.103084e+03	2.980958e+03
4	3.269017e+06	1.096633e+03	1.202604e+06

calculate both square root and exponential together of value of all the columns A, B and C of data frame df.

```
result = df.transform(func = ["sqrt","exp"])
```

```
print(result)
```

	A		B		C	
sqrt	exp	sqrt	exp	sqrt	exp	

```

0  2.828427  2.980958e+03    NaN    NaN  3.162278  2.202647e+04
1  2.645751  1.096633e+03  4.690416  3.584913e+09  2.449490  4.034288e+02
2  3.872983  3.269017e+06  5.656854  7.896296e+13    NaN    NaN
3  3.464102  1.627548e+05  3.000000  8.103084e+03  2.828427  2.980958e+03
4  3.872983  3.269017e+06  2.645751  1.096633e+03  3.741657  1.202604e+06

```

Create dataframe:

```
import pandas as pd
```

```
# Creating the DataFrame
```

```
df = pd.DataFrame({
    "Team":["MI","CSK","RR","MI","KKR","KKR","MI","CSK","KKR",
    "RR"],
    "Score":[210,150,215,180,185,205,230,190,160,185]})

print(df)
```

	Team	Score
0	MI	210
1	CSK	150
2	RR	215
3	MI	180
4	KKR	185
5	KKR	205
6	MI	230
7	CSK	190
8	KKR	160
9	RR	185

To perform the transformation operation using groupby function.

groupby function will apply on “Team” and calculate the “sum” of “Score” using transform function.

```
df.groupby("Team")["Score"].transform("sum")
```

the sum of “Score” of each “Team” will be display:

```
0  620
1  340
2  400
3  620
4  550
5  550
6  620
7  340
8  550
9  400
Name: Score, dtype: int64
```

6.5 PIVOT TABLE

Pivot table is a statistical table which summarizes a substantial table like a big dataset. The summary in a pivot tables may include sum, min, max, mean, median or other statistical terms. The `pivot()` function provides general purpose pivoting with various data type such as string, numeric, etc. The `pivot_table()` function is used to create pivot table with aggregation of numeric data using data frame of pandas library of Python.

Syntax:

`DataFrame.pivot(data, index=None, columns=None, values=None, aggfunc)`

- **data:** dataframe object
- **index:** a column which has the same length as data. Keys to group by on the pivot table index.
- **columns:** a column which has the same length as data. Keys to group by on the pivot table column.

➤ **values:** column or list of columns to aggregate

➤ **aggfunc:** function to use for aggregation

Example :

```
import pandas as pd

# Dataframe creation

df = pd.DataFrame({

    "Product":["Mango","Corn","Orange","Cabbage","Mango","Corn","Watermelon","Apple","Pumkin","Mango"],

    "Category":["Fruit","Vegetable","Fruit","Vegetable","Fruit","Vegetable","Fruit","Fruit",

    "Vegetable","Fruit"],

    "Qty":[12, 5, 10, 2, 10, 3, 5, 8, 2, 10],

    "Price":[350, 80, 320, 50, 200, 50, 280, 380, 60, 400]})

print(df)
```

	Product	Category	Qty	Price
0	Mango	Fruit	12	350
1	Corn	Vegetable	5	80
2	Orange	Fruit	10	320
3	Cabbage	Vegetable	2	50
4	Mango	Fruit	10	200
5	Corn	Vegetable	3	50
6	Watermelon	Fruit	5	280
7	Apple	Fruit	8	380
8	Pumkin	Vegetable	2	60
9	Mango	Fruit	10	400

Page 105

Example: To create a pivot table of total sales of each product.

Pivot table of total sales of each product

```
tot_sales = df.pivot_table(index=["Product"], values=["Price"],aggfunc="sum")
```

Display pivot table of total sales

```
print(tot_sales)
```

	Price
Product	
Apple	380
Cabbage	50
Corn	130
Mango	950
Orange	320
Pumkin	60
Watermelon	280

Example: To create a pivot table of total sales of each category.

```
# Pivot table of total sales of each category
```

```
tot_sales = df.pivot_table(index=["Category"], values=["Price"], aggfunc="sum")
```

```
# Display pivot table of total sales
```

```
print(tot_sales)
```

we set the index as a “Category” and “sum” as an aggregate function to calculate the total sales of each category.

we set the index as a “Category” and “sum” as an aggregate function to calculate the total sales of each category. The above code will give the following output.

Price	
Category	
Fruit	1930
Vegetable	240

Example: To create a pivot table of total sales of each product.

```
# Pivot table of total sales of both product and category
```

```
tot_sales = df.pivot_table(index=["Category","Product"], values=["Price"],
```

```
aggfunc="sum")
```

```
# Display pivot table of total sales
```

```
print(tot_sales)
```

we set the index as a both “Category” and “Product” and “sum” as an aggregate function to calculate the total sales of each product.

		Price
Category	Product	
Fruit	Apple	380
	Mango	950
	Orange	320
	Watermelon	280
Vegetable	Cabbage	50
	Corn	130
	Pumkin	60

Example: To create a pivot table to find the minimum, maximum, mean and median of price of each category wise.

```
# Pivot table of min, max, mean and media of sales
```

```
tot_sales = df.pivot_table(index=["Category"], values=["Price"],
```

```
aggfunc={"min","max","mean","median"})
```

```
# Display pivot table of total sales
```

```
print(tot_sales)
```

we set the index as a “Category” and “min”, “max”, “mean” and “median” as an aggregate function to calculate the minimum, maximum, mean and median of price of each category wise.

Category	Price			
	max	mean	median	min
Fruit	400.0	321.666667	335.0	200.0
Vegetable	80.0	60.000000	55.0	50.0

Example: To create a pivot table of total product count of each category.

Pivot table of minimum, maximum and average sales

```
tot_sales = df.pivot_table(index=["Category", "Product"],  
values=["Price"],aggfunc=["count"])
```

Display pivot table of total sales

```
print(tot_sales)
```

we set the index as a both “Category” and “Product” and “count” as an aggregate function to count total product of each category. The above code will give the following output.

		count
		Price
Category	Product	
Fruit	Apple	1
	Mango	3
	Orange	1
	Watermelon	1
Vegetable	Cabbage	1
	Corn	2
	Pumkin	1

6.6 CROSS TABULATIONS

The cross-tabulation method is used to calculate the simple cross-tabulation of two or more factors. The pandas provide **crosstab()** function to build a cross-tabulation table which shows the frequency with which certain groups of data appear.

Syntax:

pd.crosstab(index, columns, values=None, rownames=None, colnames=None, aggfunc=None, margins=False, margins_name='All', normalize=False, dropna=True)

➤ **index:** array-like, values to group by in the rows.

- **columns:** array-like, values to group by in the columns.
- **values:** array-like, optional, array of values to aggregate according to the factors.
- **rownames:** sequence, must match number of row arrays passed, default is None
- **colnames:** sequence, must match number of column arrays passed if passed, default is None
- **aggfuncs:** function, optional, if no values array is passed, its computers a frequency table.
- **margins:** boolean, add row / column margins (i.e. subtotals), default is False.
- **margins_name:** string, name of the row / column that will contain the subtotals if margins is True, default is "All".
- **normalize:** boolean, {„all“, "index", "columns"}, or {0,1}, normalize by dividing all values by the sum of values, default is False.
- **dropna:** boolean, do not include columns whose all entries are NaN, default is True.

Example:

create a cross-table using crosstab() function of pandas library of Python.

```
import pandas as pd
```

```
# Dataframe creation
```

```
df = pd.DataFrame({
```

```
"Name":["Rahul", "Jyoti", "Rupal", "Rahul", "Jyoti", "Rupal",
```

```
"Rahul","Jyoti","Rupal","Rahul","Jyoti","Rupal","Rahul",  
"Jyoti","Rupal","Rahul","Jyoti","Rupal"],  
"Examination":["SEM-I","SEM-I","SEM-I","SEM-I","SEM-I",  
"SEM-I","SEM-I","SEM-I","SEM-I","SEM-II","SEM-II",  
"SEM-II","SEM-II","SEM-II","SEM-II","SEM-II","SEM-II",  
"SEM-II"],  
"Subject":["Physics","Physics","Physics","Chemistry",  
"Chemistry","Chemistry","Biology","Biology","Biology",  
"Physics","Physics","Physics","Chemistry","Chemistry",  
"Chemistry","Biology","Biology","Biology"],  
"Result":["PASS","PASS","FAIL","PASS","FAIL","PASS","FAIL",  
"PASS","FAIL","PASS","PASS","PASS","FAIL","PASS","PASS",  
"PASS","PASS","FAIL"]})  
  
print(df)
```


	Name	Examination	Subject	Result
0	Rahul	SEM-I	Physics	PASS
1	Jyoti	SEM-I	Physics	PASS
2	Rupal	SEM-I	Physics	FAIL
3	Rahul	SEM-I	Chemistry	PASS
4	Jyoti	SEM-I	Chemistry	FAIL
5	Rupal	SEM-I	Chemistry	PASS
6	Rahul	SEM-I	Biology	FAIL

7	Jyoti	SEM-I	Biology	PASS
8	Rupal	SEM-I	Biology	FAIL
9	Rahul	SEM-II	Physics	PASS
10	Jyoti	SEM-II	Physics	PASS
11	Rupal	SEM-II	Physics	PASS
12	Rahul	SEM-II	Chemistry	FAIL
13	Jyoti	SEM-II	Chemistry	PASS
14	Rupal	SEM-II	Chemistry	PASS
15	Rahul	SEM-II	Biology	PASS
16	Jyoti	SEM-II	Biology	PASS
17	Rupal	SEM-II	Biology	FAIL

Two-way cross table:

There are two columns is used to create a cross table is called two-way cross table.

To create a cross table of two columns “Subject” and “Result” as follow:

```
pd.crosstab(df.Subject, df.Result, margins=True)
```

set margin=True to display the row wise sum and column wise sum of the cross table.

Result	FAIL	PASS	All
Subject			
Biology	3	3	6
Chemistry	2	4	6
Physics	1	5	6
All	6	12	18

Three-way cross table:

There are three columns is used to create a cross table is called three-way cross table.

To create a cross table of three columns “Subject”, “Examination” and “Result” as follow:

Three-way cross table creation

```
pd.crosstab([df.Subject, df.Examination], df.Result, margins=True)
```

set margin=True to display the row wise sum and column wise sum of the cross table.

	Result	FAIL	PASS	All
Subject	Examination			
Biology	SEM-I	2	1	3
	SEM-II	1	2	3
Chemistry	SEM-I	1	2	3
	SEM-II	1	2	3
Physics	SEM-I	1	2	3
	SEM-II	0	3	3
All		6	12	18

6.7 DATE AND TIME DATA TYPE

Python does not have date and time data types, but it has a module named “datetime” can be imported to deal with the date and time. This is inbuilt module available in the Python. This module consists different classes to work with date and time. These classes provide different functions to work with dates, times and time intervals.

There are main **six** classes in datetime module:

Data Type	Description
Date	It is a date type object. It manipulates only date (i.e. day, month and year).
Time	It is a time object class. It manipulates only time of the any specific day (i.e. hour, minute, second, microsecond).
Datetime	It manipulates the combination of both time and date (i.e. day, month, year, hour, second, microsecond).
timedelta	It manipulates the duration expressing the different between two dates, times or datetime values in milliseconds.
Tzinfo	It is an abstract base class which provides time zone information.
timezone	It is a class that implements tzinfo abstract base class as a fixed offset from the UTC.

There are different format codes is used to formatting the data and time:

Directive	Description	Example
%a	Day of Week, short version	Fri
%A	Day of Week, full version	Friday
%w	Day of Week as a number from 0 to 6, Here 0 is Sunday	4
%d	Day of Month from 01 to 31	25
%b	Name of Month, short version	Mar
%B	Name of Month, full version	March
%m	Month as a number from 01 to 12	11
%y	Year, short version (in two digit)	21
%Y	Year, full version (in four digit)	2021
%H	Hour from 00 to 23 (in 24 hr format)	16
%I	Hour from 00 to 12 (in 12 hr format)	07
%p	AM or PM	AM
%M	Minute from 00 to 59	35
%S	Second from 00 to 59	45
%f	Microsecond from 000000 to 999999	234567
%z	UTC offset	+0100
%Z	Timezone	CST
%j	Day number of year from 001 to 365	325
%U	Week number of year, Sunday as the first day of week, from 00 to 53	40
%W	Week number of year, Monday as the first day of week, from 00 to 53	40
%c	Local version of date and time	Tue Mar 30 13:25:30 2021
%x	Local version of date (MM/DD/YY)	11/24/2021
%X	Local version of time (HH:MM:SS)	18:25:40

Example: To get current data and time.

```
import datetime as dt
```

```
d = dt.datetime.now()
```

```
print(d)
```

output.:

2021-05-15 00:53:08.925167

Example: To get the current date:

```
import datetime as dt
```

```
d = dt.date.today()
```

```
print(d)
```

output:

2021-05-15

Example: To get the todays date.

To get the today's date, month and year separately

```
import datetime as dt
```

```
today = dt.date.today()
```

```
print(today)
```

```
print("Day :",today.day)
```

```
print("Month :",today.month)
```

```
print("Year :",today.year)
```

output:

2021-05-15

Day : 15

Month : 5

Year : 2021

Example: To represent a date using date object.

To represent a date using date object

```
import datetime as dt
```

```
d = dt.date(2021, 1, 26)
```

```
print(d)
```

The above code will give the following output.

2021-01-26

Here, the date is passed as an argument.

Example: To represent a date using timestamp.

To get date from a timestamp

```
import datetime as dt
```

```
ts = dt.date.fromtimestamp(987654321)
```

```
print(ts)
```

output:

```
2001-04-19
```

example: To represent a time using time object.

To represent a time using time object

```
import datetime as dt
```

```
# time(hour=0, minute=0, second=0)
```

```
t = dt.time()
```

```
print("Time :",t)
```

```
# time(hour, minute, second)
```

```
t = dt.time(10, 40, 55)
```

```
print("Time :",t)
```

```
# time(hour, minute, second)
```

```
t = dt.time(hour=10, minute=40, second=55)
```

```
print("Time :",t)
```

```
# time(hour, minute, second, microsecond)
```

```
t = dt.time(10, 40, 55, 123456)
```

```
print("Time :",t)
```

```
# time(hour, minute, second, microsecond)
```

```
t = dt.time(10, 40, 55, 123456)
```

```
print("Hour :",t.hour)
```

```
print("Minute :",t.minute)
```

```
print("Second :",t.second)
```

```
print("Microsecond :",t.microsecond)
```

output:

Time : 00:00:00

Time : 10:40:55

Time : 10:40:55

Time : 10:40:55.123456

Hour : 10

Minute : 40

Second : 55

Microsecond : 123456

Example: To represent a datetime object.

To represent a datetime using datetime object

```
import datetime as dt
```



```
# datetime(year, month, day)

dtformat = dt.datetime(2021, 5, 15)

print(dtformat)

# datetime(year,month,day,hour,minute,second,microsecond)

dtformat = dt.datetime(2021, 5, 15, 16, 35, 25, 234561)

print(dtformat)
```

output:

2021-05-15 00:00:00

2021-05-15 16:35:25.234561

Example: To represent a datetime object using different format.

To represent a datetime using datetime object

```
import datetime as dt

dtformat = dt.datetime(2021, 5, 15, 16, 35, 25, 234561)

print("Year : ",dtformat.year)

print("Month : ",dtformat.month)

print("Day : ",dtformat.day)

print("Hour : ",dtformat.hour)

print("Minute : ",dtformat.minute)
```

```
print("Timestamp : ",dtformat.timestamp())
```

output:

Year : 2021

Month : 5

Day : 15

Hour : 16

Minute : 35

Timestamp : 1621076725.234561

Example: To find the difference between to dates and times.

Different between two dates and times

```
import datetime as dt
```

```
# date(year, month, day)
```

```
t1 = dt.date(year=2021, month=5, day=15)
```

```
t2 = dt.date(year=2020, month=7, day=25)
```

```
t3 = t1 - t2
```

```
print("Date Difference :",t3)
```

```
print("Type of t3 :",type(t3))
```

```
# date(year, month, day, hour, minute, second)
```

```
t1 = dt.datetime(year=2020, month=1, day=15, hour=8, minute=25, second=45)
t2 = dt.datetime(year=2021, month=4, day=25, hour=10, minute=30, second=50)
t3 = t1 - t2

print("Date Difference :",t3)

print("Type of t3 :",type(t3))
```

output:

Date Difference : 294 days, 0:00:00

Type of t3 : <class 'datetime.timedelta'>

Date Difference : -467 days, 21:54:55

Type of t3 : <class 'datetime.timedelta'>

Example: To use of timedelta object.

Different between two timedelta objects

```
import datetime as dt
```

```
t1 = dt.timedelta(weeks=6, days=5, hours=9, minutes=45, seconds=10)
```

```
t2 = dt.timedelta(weeks=4, days=3, hours=5, minutes=25, seconds=35)
```

```
t3 = t1 - t2
```

```
print("Time Delta Difference : ",t3)
```

```
t1 = dt.timedelta(weeks=3, hours=10, minutes=45)
```

```
t2 = dt.timedelta(days=4, minutes=15, seconds=35)
```

```
t3 = t1 - t2
```

```
print("Time Delta Difference : ",t3)
```

output:

Time Delta Difference : 16 days, 4:19:35

Time Delta Difference : 17 days, 10:29:25

Example: To represent the time in total seconds.

```
# Time duration in seconds
```

```
import datetime as dt
```

```
t = dt.timedelta(hours=9, minutes=35, seconds=15)
```

```
print("Time in Second :",t.total_seconds())
```

output:

Time in Second : 34515.0

Example: To use strftime() function for formatting.

```
# Use of strftime() function for date formatting
```

```
import datetime as dt
```

```
# current date and time

now = dt.datetime.now()

print("Current Date and Time :",now)

# time in HH:MM:SS format

f1 = now.strftime("%H:%M:%S")

print("Time format is :",f1)

# date and time format DD/MM/YY, HH:MM:SS format

f2 = now.strftime("%d/%m/%Y, %H:%M:%S")

print("Date :",f2)

# Date and time format MM/DD/YY, HH:MM:SS format

f3 = now.strftime("%m/%d/%Y, %H:%M:%S")

print("Date :",f3)
```

output:

Current Date and Time : 2021-05-15 17:16:52.794301

Time format is : 17:16:52

Date : 15/05/2021, 17:16:52

Date : 05/15/2021, 17:16:52

Example: To use strptime() function for formatting.

Use of strftime() function for date formatting

```
import datetime as dt
```

date in string format

```
dt_string = "15 May, 2021"
```

```
print("Date in string :",dt_string)
```

date in object format

```
dt_object = dt.datetime.strptime(dt_string, "%d %B, %Y")
```

```
print("Date in object :",dt_object)
```

output:

Date in string : 15 May, 2021

Date in object : 2021-05-15 00:00:00

Example: To use different timezone.

Use of time zone

```
import datetime as dt
```

```
import pytz
```

Local time zone

```
local = dt.datetime.now()
```

```
print("Local Time Zone:",local.strftime("%m/%d/%y, %H:%M:%S"))
```

London time zone

```
tz_London = pytz.timezone('Europe/London')
dt_London = dt.datetime.now(tz_London)
print("London Time Zone:",dt_London.strftime("%m/%d/%y, %H:%M:%S"))

# Newyork time zone
tz_NY = pytz.timezone('America/New_York')
dt_NY = dt.datetime.now(tz_NY)
print("New York Time Zone:",dt_NY.strftime("%m/%d/%y, %H:%M:%S"))
```

output:

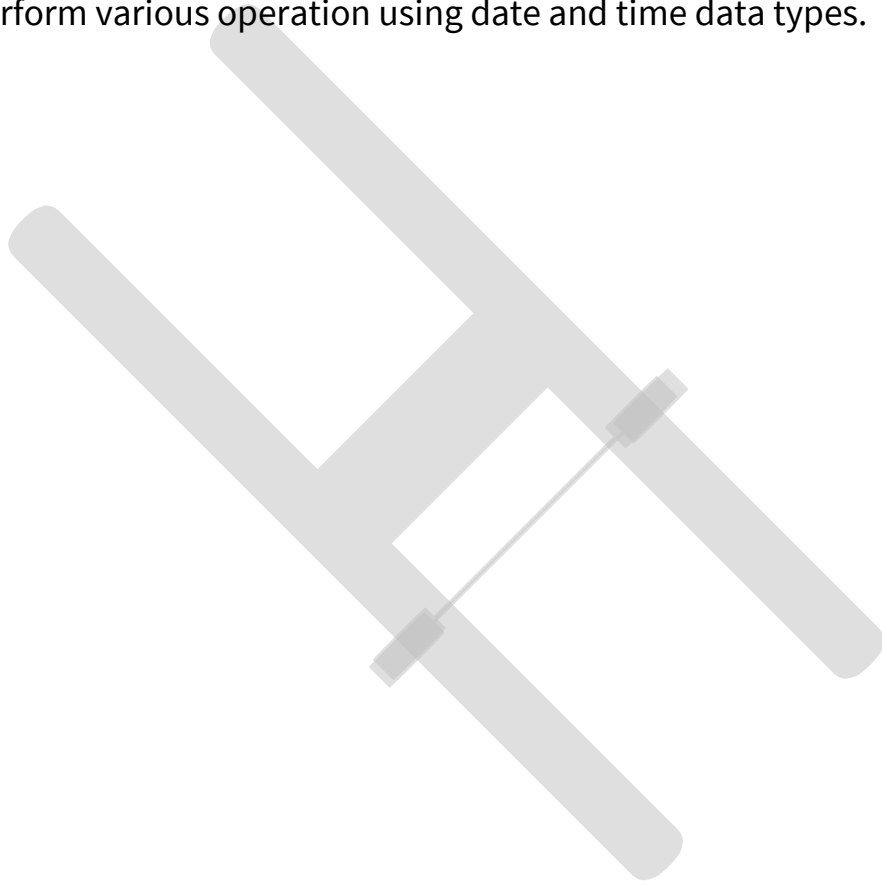
Local Time Zone: 05/15/21, 17:35:15

London Time Zone: 05/15/21, 13:05:15

New York Time Zone: 05/15/21, 08:05:15

Questions:

- 1- What is data aggregation?
- 2- What is data splitting?
- 3- What is transformation?
- 4- What is pivot table?
- 5- What is cross tabulation?
- 6- Explain data aggregation with different functions.
- 7- Explain groupby function with example.
- 8- Explain transform function with syntax and example.
- 9- Explain pivot table with example.
- 10- Explain cross tabulation with example.
- 11- Explain date and time data type with different format code.

- 12- Create a data frame and perform aggregation functions.
 - 13- Create data frames and perform groupby function with different arguments.
 - 14- Create data frames and perform transform function.
 - 15- Create data frame and perform pivot table with different arguments.
 - 16- Create data frame and prepare two-way and three-way cross table.
 - 17- Perform various operation using date and time data types.
- 

Chapter Seven- Data Modelling

7.1 Introduction

7.2 Generative Modeling

7.3 Predictive Modeling

7.3.1 Models

7.3.2 Predictive algorithms

7.4 Charts

7.4.1 Histogram

7.4.2 Scatter Plot

7.4.3 Line Chart

7.4.4 Bar Chart

7.5 Graph

7.6 3-D Visualization and Presentation

7.6.1 3D line plot

7.6.2 3D scatter plot

7.6.3 3D bar plot

7.6.4 wire plot

7.6.5 surface plot

7.1 INTRODUCTION

Data science is useful for extraction, preparation, analysis and visualization of various information. Various scientific methods can be applied to get insight in data.

Data science is all about using data to solve problems. Data has become the fuel of industries.

It is most demandable field of 21st century. Every industry require data to functioning, searching, marketing, growing, expanding their business.

7.2 INTRODUCTION TO GENERATIVE MODELING

Generative models are the family of machine learning models that are used to describe how data is generated. There are mainly two different types of problems to work with machine learning or deep learning algorithms such as supervised learning and unsupervised learning.

In **supervised learning** problem, we have two variables such as independent variables (x) and the target variable (y). The examples of supervised learning are classification, regression, object detection etc.

In **unsupervised learning** problem, we have only independent variables (x). there are no target variable or label. It aims is to find some underlying patterns from the dataset. The examples of unsupervised learning are clustering, dimensionality reduction etc.

The generative model is an unsupervised learning problem in machine learning. It

automatically discovers and learning the rules, regularities or patterns from the large input training dataset. This model learns to create a data that is look like as given. A generative model can be broadly defined as follows:

A generative model describes how a dataset is generated, in terms of a probabilistic model.

By sampling from this model, we are able to generate new data.

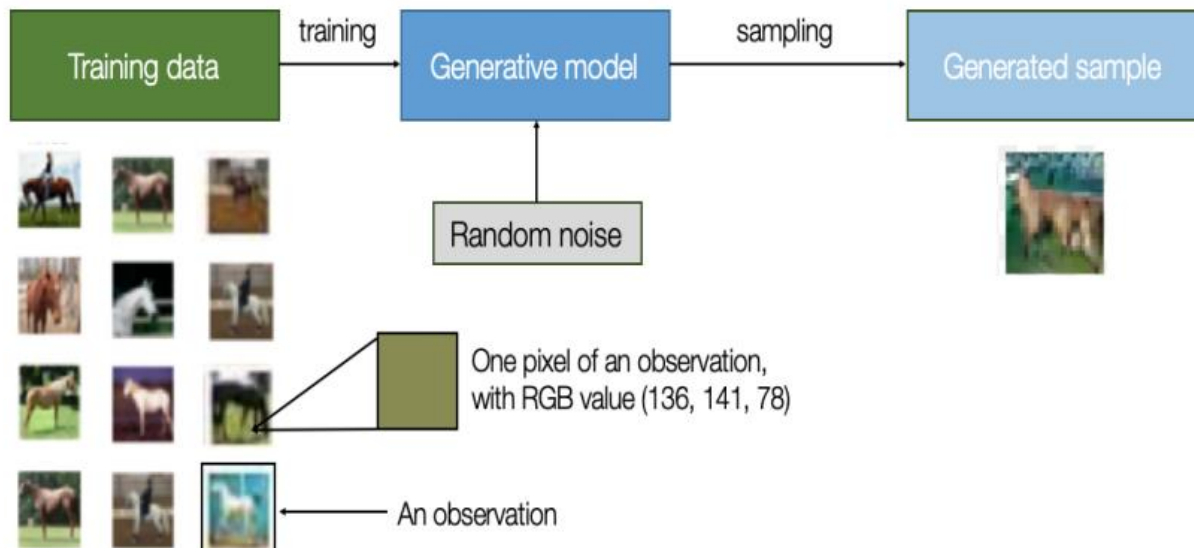
Generative model can generate new data instances. If we have a dataset containing images of any animal and we may develop a model which can generate a new image of same animal that is never existing but still it looks like as real animal. This model has learned the general

rules that govern the appearance of a specific animal. A generative modelling is used to solve this kind of problems. A generative model process are as follows:

We require a dataset which consists many instants of the entity which we want to generate.

This dataset is known as the training data and each data points is called as an observation.

The following diagram represent a horse animal dataset.



The existing dataset of horse images is used as a training dataset. Based on the training given to the existing dataset it built a generative model to create new images which look like as a real image.

Q:

Which of the following is a supervised learning algorithm?

- e) K-Means
- f) Linear Regression
- g) DBSCAN
- h) PCA

Answer: b) Linear Regression – because it uses labeled data.

7.3 INTRODUCTION TO PREDICTIVE MODELING

Predictive modeling is a mathematical approach to build models based on existing dataset, which will help to finding the future value or trend of a variable. The variety of

statistical techniques including data mining and machine learning are used to estimate or predict the future outcomes.

The predictive modelling is used for every area such as

- Weather forecasting
- Price forecasting
- Demand forecasting
- Sales forecasting
- Customer targeting
- Financial modeling
- Risk assessment
- Market analysis

7.3.1 Types of Models

There are different predictive analytics models are developed for specific applications as follows:

- Classification Model
- Clustering Model
- Forecasting Model
- Time Series Model
- Outlier Model

▪ Classification Model

This is a simplest and most commonly used predictive analytics model. It works on categorical information based on historical data.

This model is used or apply in many industrial applications because it can easily retrain with new data as per the needs.

▪ **Clustering Model**

This model is use to take the data and divide it into different nested smart groups based on some common attributes. It helps to divide or grouping things or data with shared characteristic or behaviors and take strategic decisions for each group.

For example, the customers can be divided based on common attributes like purchasing methods, purchasing power, etc. for targeted marketing campaign to the customers.

▪ **Forecast Model**

This is a very popular and most widely use model. It works with the metric value prediction, by estimating the value of new data based on learnings from historical data. It is also used to generate the numerical values and update where none or missing value found. This model can be applied wherever historical numerical data is available. It considers multiple input parameters.

This model is used in many different business and industries. For example, the company"s customer care department can predict how many supports calls they will receive per day.

▪ **Time Series Model**

This model is focusses on data where time is an input parameter. This model is applied by using different data points which is taken from the previous year's data to develop a numerical metric that will used to predict the trends within a specified

period of time.

This model is used in many industries which want to see how a particular variable change over a time period. It also takes care about extraneous factors that might be affect the variable such as seasons or seasonal variable. For example, the shopping mall owners want to know the how many customers may visit the mall in week or month.

▪ Outliers Model

This model is work with anomalous data entries in a dataset. It works by finding unusual data, either in isolation or in relation with different categories and numbers. It is more useful in industries were identifying anomalies can save organization corers of rupees such as finance and retail. It is more effective in fraud detection because it can find the anomalies. Since an incidence of fraud is a deviation from the norm, this model is more likely to predict it before it occurs. For example, when identifying a fraud transaction, this model can assess the amount of money lost, purchase history, time, location etc.

7.3.2 Predictive Algorithms

The predictive analytics algorithms can be separated into two things: machine learning and deep learning. These both are subsets of artificial intelligence (AI).

Machine Learning: It deals structural data such as table or spreadsheets. It has both linear and non-linear algorithms. Linear algorithms are quickly train, while non-linear are better optimized for the problems they are to face.

Deep Learning: It is a subset of machine learning. It deals with unstructured data such as images, social media posts, text, audio and videos.

There are several algorithms can be used for machine learning predictive modelling.

The most common algorithms are:

- Random Forest
- Generalized Linear Model (GLM) for Two Values
- Gradient Booster Model (GBM)
- K-Means
- Prophet

7.4 CHARTS

Charts is the representation of data in a graphical format. It helps to summarizing and presenting a large amount of data in a simple and easy to understandable formats. By placing the data in a visual context, we can easily detect the patterns, trends and correlations among them.

Python provides various easy to use multiple graphics libraries for data visualization with different features. These libraries are work with both small and large datasets.

Some of the most popular and commonly used Python data visualization libraries are:

- Matplotlib
- Pandas
- Seaborn
- ggplot
- Plotly

7.4.1 Histogram

Histogram is a graphical representation of the distribution of numerical data. It contains a rectangular area to display the statistical information which is proportional to the frequency of a variable. It is an estimate of probability distribution of a continuous variable.

In a histogram, the data are binned and the count for each bin is represent. The number of bins is selected so that it is comparable to the typical number of samples in a bin. The bins are specified as consecutive and non-overlapping intervals of a variable. The numbers of bin can be customized also.

There are three basic steps to construct a histogram:

- Bin the range of values
- Distribute the range of values in a series of intervals
- Count the numbers of value into each interval

The **hist()** function is used to plot a histogram. It computes and draw the histogram of x values. There is some parameter to construct the histogram are as follows:

- **bins:** numbers of bin in the plot, optional
- **range:** lower and upper range of the bins.
- **density:** density or count to populate the plot
- **histtype:** types of histogram plot such as bar, step and stepfilled, default is bar
- **align:** control to plot the histogram such as left, mid and right
- **rwidth:** relative width of a bar as a fraction of bin width
- **color:** it is a color spec or sequence of color specs
- **orientation:** horizontal or vertical representation, default is vertical

Example: Here we take an example of age of peoples.

The x-axis represents age group or bins and y-axis represents age.

Importing library

```
import matplotlib.pyplot as plt
```

Data values

```
age = [22, 55, 62, 45, 21, 22, 4, 12, 14, 64, 58, 68, 95, 85, 55, 38, 18, 37, 65, 59, 11,  
15, 80, 75, 65]
```

```
bins = [0,10,20,30,40,50,60,70,80,90,100]
```

Plotting the histogram with title and label

```
plt.hist(age, bins)
```

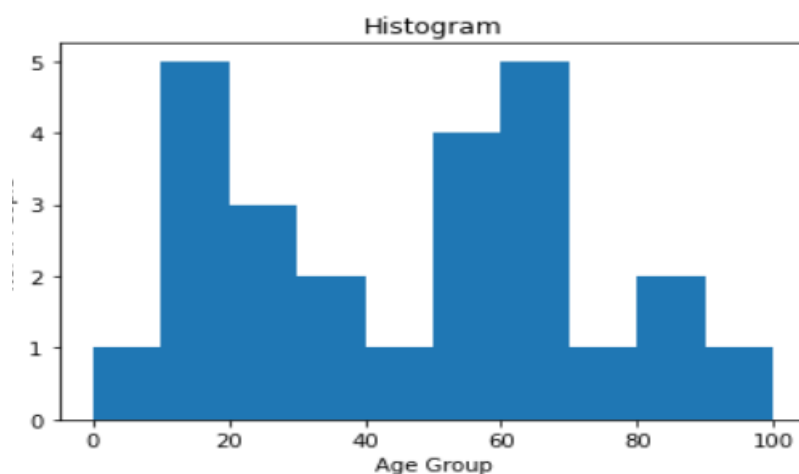
```
plt.xlabel("Age Group")
```

```
plt.ylabel("No. of People")
```

```
plt.title("Histogram")
```

Show plot

```
plt.show()
```



Here, the age is divided into different age group.

We can also set the type and color of histogram and using **histtype** and **color** as an argument.

```
import matplotlib.pyplot as plt
```

```
age = [22, 55, 62, 45, 21, 22, 34, 42, 42, 4, 2, 102, 95, 85, 55, 110, 120, 7, 65, 55,  
111, 115, 80, 75, 65, 4, 44, 43, 42, 48]
```

```
bins = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

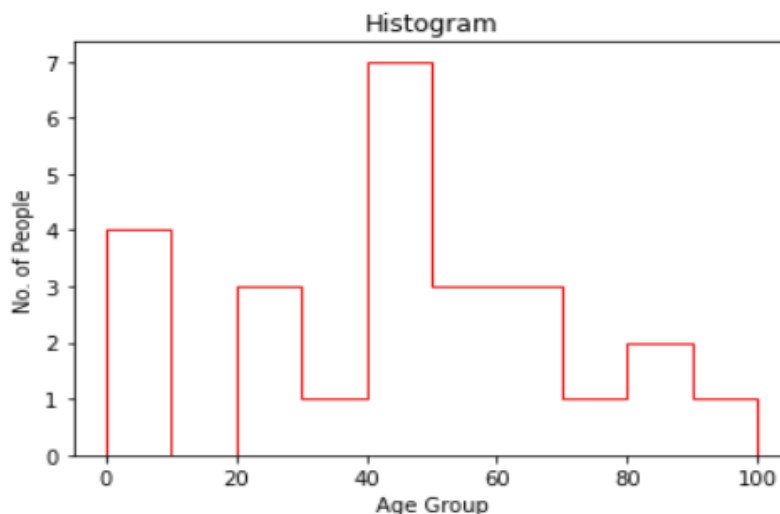
```
plt.hist(age, bins, histtype='step', rwidth=0.8, color="red")
```

```
plt.xlabel("Age Group")
```

```
plt.ylabel("No. of People")
```

```
plt.title("Histogram")
```

```
plt.show()
```



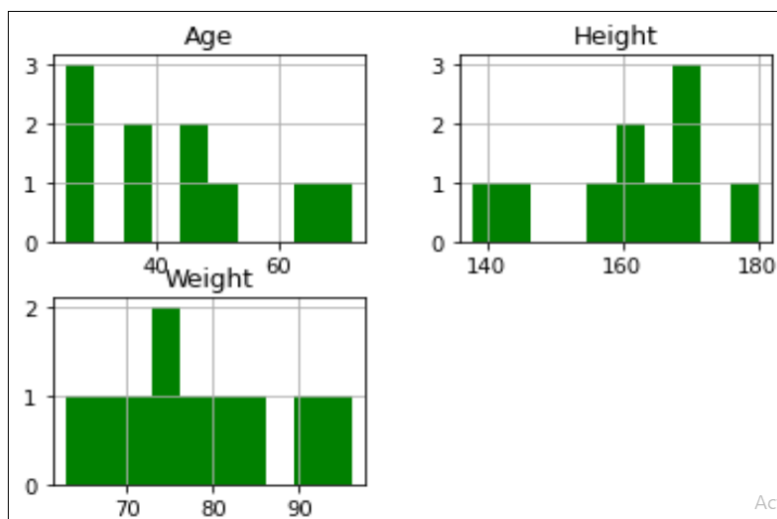
the type of histogram is set to **step** and color is set to **red**.

We can also create the multiple histogram of different columns as follows:

Here, we create a data frame with three different columns such as “**Age**”, “**Height**” and “**Weight**”.

```
import pandas as pd
```

```
df = pd.DataFrame({
    "Age": [25, 38, 45, 29, 65, 52, 46, 72, 28, 35],
    "Height": [145, 138, 160, 180, 165, 170, 158, 162, 171, 168],
    "Weight": [75, 90, 85, 72, 68, 76, 82, 96, 63, 79]})
hist = df.hist(bins=10)
```



7.4.2 Scatter Plot

Scatter plot is a diagram where each value in the dataset is represented by a dot. It is a set of dotted points to represent the individual data on both horizontal and vertical axes to reveal the distribution trends of data.

This plot is mostly used for large datasets to highlight the similarities in the dataset. It also shows the outliers and distribution of data.

The **scatter()** function is used to draw the scatter plot. This function plots one dot for each observation. It requires two different arrays of same length for both the x-axis and y-axis. we can also set the scatter plot title and labels on both the axis.

Example: Here we take an example of boys weight and girls weight. The x-axis represents “Boys_Weight” and y-axis represents “Girls_Weight”.

```
import matplotlib.pyplot as plt

# Data values

Boys_Weight = [67, 89, 72, 114, 65, 80, 91, 106, 60, 59]
Girls_Weight = [50, 59, 63, 40, 92, 88, 64, 45, 52, 59]

# Plotting scatter plot with title and label

plt.scatter(Boys_Weight, Girls_Weight)

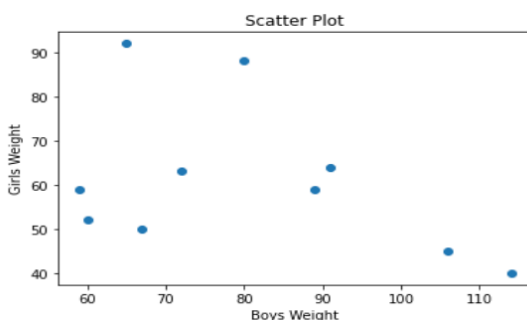
plt.title("Scatter Plot")

plt.xlabel("Boys Weight")

plt.ylabel("Girls Weight")

# Show plot

plt.show()
```



From the plot, we can see the relationship between boys and girls weight.

We can also compare the boys weight and girls weight of one class with another

class.

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
Boys_Weight = [67, 89, 72, 114, 65, 80, 91, 106, 60, 59]
```

```
Girls_Weight = [50, 59, 63, 40, 92, 88, 64, 45, 52, 68]
```

```
plt.scatter(Boys_Weight, Girls_Weight)
```

```
Boys_Weight = [54, 67, 92, 56, 83, 65, 89, 78, 50, 49]
```

```
Girls_Weight = [41, 79, 56, 74, 76, 73, 74, 87, 82, 63]
```

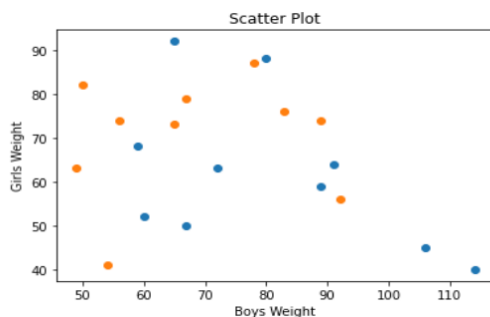
```
plt.scatter(Boys_Weight, Girls_Weight)
```

```
plt.title("Scatter Plot")
```

```
plt.xlabel("Boys Weight")
```

```
plt.ylabel("Girls Weight")
```

```
plt.show()
```



In above plot, we can see the relationship between boys and girls weight of one class with another class by using different colors.

We can also set or change the color of both the classes of data using **color** as an argument. Here we set the **red** color for first class students and **green** color for second class students.

```
import matplotlib.pyplot as plt

import numpy as np

Boys_Weight = [67, 89, 72, 114, 65, 80, 91, 106, 60, 59]
Girls_Weight = [50, 59, 63, 40, 92, 88, 64, 45, 52, 68]

plt.scatter(Boys_Weight, Girls_Weight, color="red")

Boys_Weight = [54, 67, 92, 56, 83, 65, 89, 78, 50, 49]
Girls_Weight = [41, 79, 56, 74, 76, 73, 74, 87, 82, 63]

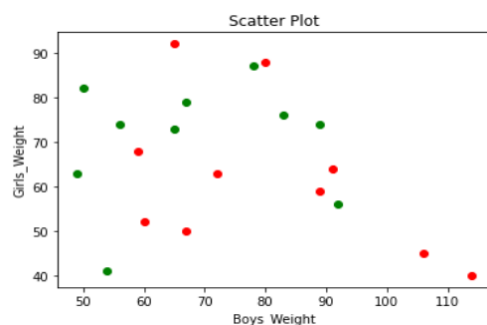
plt.scatter(Boys_Weight, Girls_Weight, color="Green")

plt.title("Scatter Plot")

plt.xlabel("Boys_Weight")

plt.ylabel("Girls_Weight")

plt.show()
```



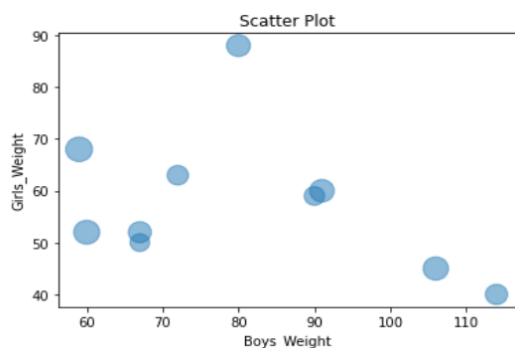
In above plot, we can see the relationship between both classes. Here, red color is used for first class students and green color is used for second class students.

We can also set or change the size of dots using **s** as an argument in scatter plot.

```
import matplotlib.pyplot as plt

import numpy as np
```

```
Boys_Weight = [67, 90, 72, 114, 67, 80, 91, 106, 60, 59]
Girls_Weight = [50, 59, 63, 40, 52, 88, 60, 45, 52, 68]
size = [200, 220, 240, 260, 280, 300, 320, 340, 360, 380]
plt.scatter(Boys_Weight, Girls_Weight, s=size, alpha=0.5)
plt.title("Scatter Plot")
plt.xlabel("Boys_Weight")
plt.ylabel("Girls_Weight")
plt.show()
```



In above plot, we can see the size of each dots.

We can also set the shape instead of dots in scatter plot. Here we set the **marker** and **edgecolor** as an argument in scatter function to set the shape with edge color in scatter plot.

```
import matplotlib.pyplot as plt
import numpy as np

Boys_Weight = [67, 89, 72, 114, 65, 80, 91, 106, 60, 59]
Girls_Weight = [50, 59, 63, 40, 92, 88, 64, 45, 52, 68]
```



```
plt.scatter(Boys_Weight, Girls_Weight, marker="s", edgecolor="green", s=50)

Boys_Weight = [54, 67, 92, 56, 83, 65, 89, 78, 50, 49]

Girls_Weight = [41, 79, 56, 74, 76, 73, 74, 87, 82, 63]

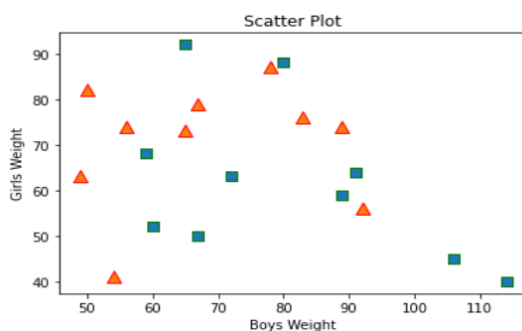
plt.scatter(Boys_Weight, Girls_Weight, marker="^", edgecolor="red", s=100)

plt.title("Scatter Plot")

plt.xlabel("Boys Weight")

plt.ylabel("Girls Weight")

plt.show()
```



In above plot, we can see the shape with edge color.

7.4.3 Line Chart

Line chart is used to shows the relation between two datasets on a different axis. There are multiple features available such as line color, line style, line width etc. It is also known as time series plot.

Matplotlib is most popular library for plotting different chart. Line chart is one of them. The **plot()** function is used to create a line chart. Here we will see some examples of line chart in Python.

Example: Here we take an example of numbers of students enroll in specific course in different year. The x-axis represents “**Year**” values and y-axis represents “**Student**”.

```
from matplotlib import pyplot as plt

# Data values

Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020]

Student = [88,76,61,68,92,85,62,58,75,83]

# Plotting the line

plt.plot(Year, Student)

# Show plot

plt.show()
```



n this chart, there is no label on both the axis and title of chart. Label is required to understand the dimensions of chart. The following code will create the line chart with **title** and **labeled** on both axes.

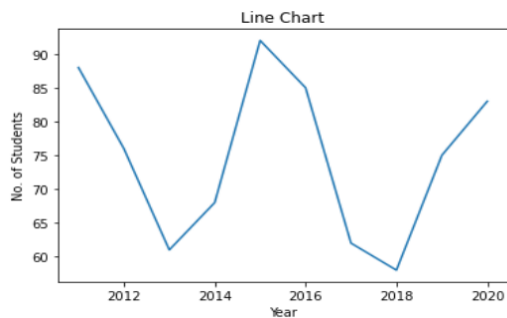
```
from matplotlib import pyplot as plt

Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020]

Student = [88,76,61,68,92,85,62,58,75,83]

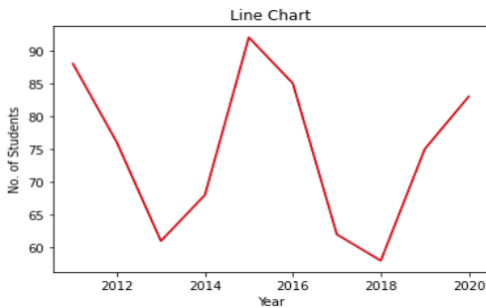
plt.title("Line Chart")
```

```
plt.xlabel("Year")  
plt.ylabel("No. of Students")  
plt.plot(Year, Student)  
plt.show()
```



We can set the line color also using **color** as an argument. Here we set **red** color to the line.

```
from matplotlib import pyplot as plt  
Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020]  
Student = [88, 76, 61, 68, 92, 85, 62, 58, 75, 83]  
plt.title("Line Chart")  
plt.xlabel("Year")  
plt.ylabel("No. of Students")  
plt.plot(Year, Student)  
plt.plot(Year, Student, 'red')  
plt.show()
```



We can set the line width also using **linewidth** or **lw** as an argument. Here we set **10** to the linewidth.

```
from matplotlib import pyplot as plt
```

```
Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020]
```

```
Student = [88,76,61,68,92,85,62,58,75,83]
```

```
plt.title("Line Chart")
```

```
plt.xlabel("Year")
```

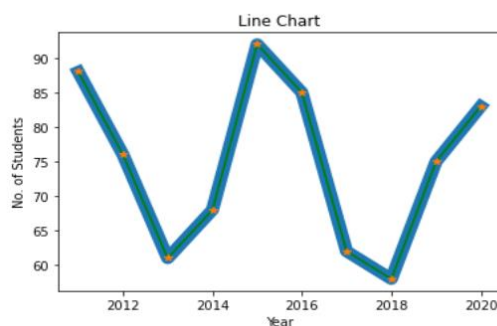
```
plt.ylabel("No. of Students")
```

```
plt.plot(Year, Student)
```

```
plt.plot(Year, Student, 'green', linewidth=10)
```

```
plt.plot(Year, Student, '*')
```

```
plt.show()
```



We can set the line style also using **linestyle** or **ls** as an argument. There are various types of style available such as solid, dotted, dashed and dashdot. Here we set **dotted** as a linestyle in line chart.

```
from matplotlib import pyplot as plt
```

```
Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020]
```

```
Student = [88,76,61,68,92,85,62,58,75,83]
```

```
plt.title("Line Chart")
```

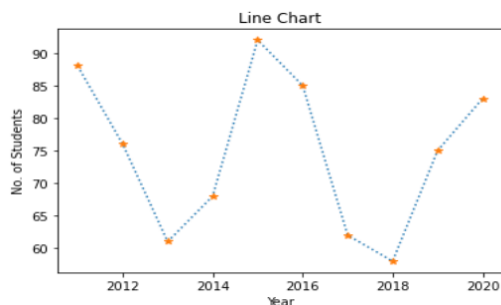
```
plt.xlabel("Year")
```

```
plt.ylabel("No. of Students")
```

```
plt.plot(Year, Student, linestyle = 'dotted')
```

```
plt.plot(Year, Student, '*')
```

```
plt.show()
```



We can set the multiple lines in a single line chart. Here x-axis and y-axis represent the different values. We plot the line chart separately for both the axis. Here we set the **dotted** as a linestyle in x-axis.

```
from matplotlib import pyplot as plt
```

```
X = [36,41,56,82,64,38,73,59,38,78]
```

```
Y = [73,46,73,68,54,56,63,80,54,67]
```

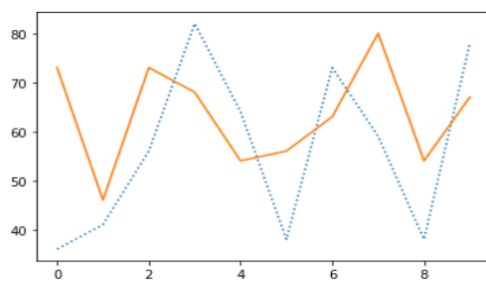
```
plt.plot(X, linestyle = 'dotted')
```

```
plt.plot(Y)
```

```
plt.show()
```

The above code will create line chart as follow:

Here the dotted line represents the x-axis and solid line represent the y-axis.



7.4.4 Bar Chart

Bar chart or **bar plot** is representing the category of data with rectangular bars with different heights and lengths with reference to the values that they present. The **bar()** function is used to create a bar chart. The bar chart can be plotted both horizontally and vertically.

The bar chart describes the comparisons between distinct categories. One axis represents the particular categories being compared and another axis represent the measured values respected to those categories. The numerical values of variables in a dataset represent the height or length of bar.

Example: Here we take an example of students name and age. The x-axis represents “**Name**” and y-axis represents “**Age**”. Here we also set the chart title and labels on both the axis.

```
from matplotlib import pyplot as plt
```

```
import numpy as np

Name = np.array(["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"])

Age = np.array([20, 30, 24, 15, 12])

# Labelling the axes and title

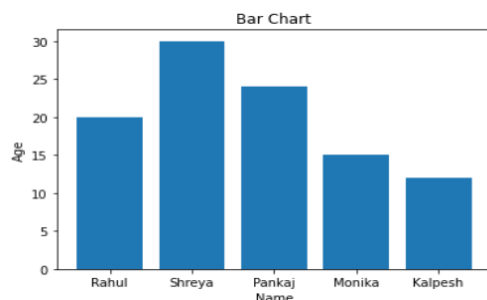
plt.title("Bar Chart")

plt.xlabel("Name")

plt.ylabel("Age")

# Plotting the bar

plt.bar(Name, Age)
```



We can change the bar color also. We set **color** as an argument to change the color of bar. Here we set the **red** as a color of bar.

```
from matplotlib import pyplot as plt

import numpy as np

Name = np.array(["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"])

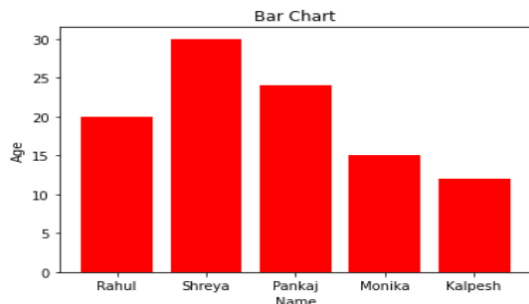
Age = np.array([20, 30, 24, 15, 12])

plt.title("Bar Chart")

plt.xlabel("Name")
```

```
plt.ylabel("Age")
```

```
plt.bar(Name, Age, color="red")
```



We can set the bar width also. We set ***width*** as an argument to set the width of bar.

Here we set ***0.2*** as a width of bar.

```
from matplotlib import pyplot as plt
```

```
import numpy as np
```

```
Name = np.array(["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"])
```

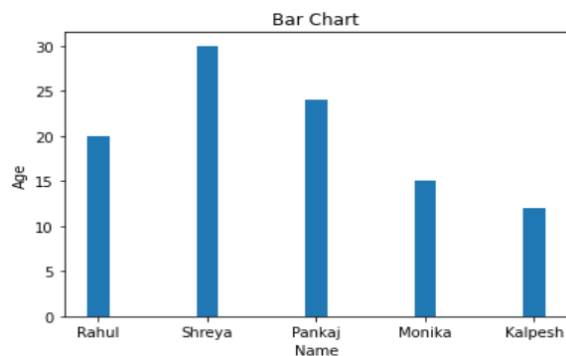
```
Age = np.array([20, 30, 24, 15, 12])
```

```
plt.title("Bar Chart")
```

```
plt.xlabel("Name")
```

```
plt.ylabel("Age")
```

```
plt.bar(Name, Age, width=0.2)
```



We can display bar horizontally also instead of vertically. We set the bar horizontally by using **barh()** function.

```
from matplotlib import pyplot as plt
```

```
import numpy as np
```

```
Name = np.array(["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"])
```

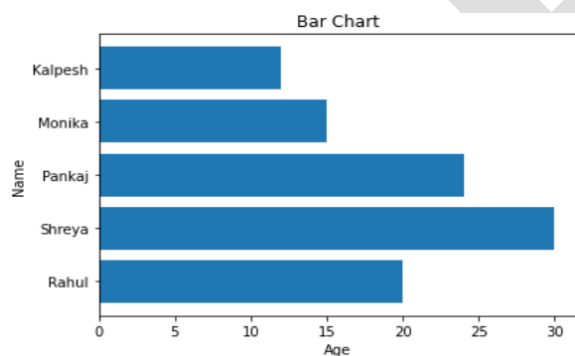
```
Age = np.array([20, 30, 24, 15, 12])
```

```
plt.title("Bar Chart")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Name")
```

```
plt.barh(Name, Age)
```



The multiple bar chart is used to represent the comparison among the different variables in a dataset. We can set the thickness and positions of bars also.

Here, we take an example of marks of different subject with the name of students.

X-axis represents the students and y-axis represents the marks of different subjects.

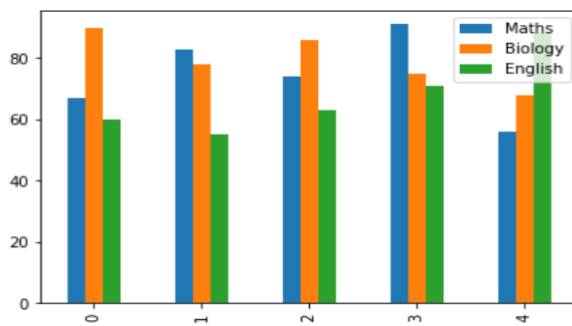
```
from matplotlib import pyplot as plt
```

```
import pandas as pd
```

```
df = pd.DataFrame({
```

```
"Name":["Rahul","Shreya","Pankaj","Monika","Kalpesh"],  
"Maths":[67,83,74,91,56],  
"Biology":[90,78,86,75,68],  
"English":[60,55,63,71,88])  
df.plot.bar()
```

Here, we show the comparisons of marks of different subjects of the students.



7.5 GRAPH

Graph is a pictorial representation of a set of objects. Some pairs of objects are connected through links. The interaction of link is denoted by points which is known as **vertices**. The link which is used to connect the vertices is called **edges**. We can perform some operation on graph such as:

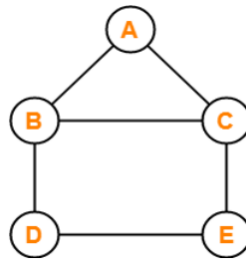
- Display vertices
- Display edges
- Add new vertex
- Add new edge
- Create graph

The dictionary data type is used to present a graph in Python. The vertices of a graph are

representing as the keys of dictionary and the links between the vertices also called edges which represent as the values of dictionary

Take the following graph as an example.

$V = \{A, B, C, D, E\}$ #vertices



$E = \{AB, AC, BC, BD, CE, DE\}$ #edges

The above graph represents using Python as below.

Create the dictionary with graph elements

```
graph = { "a" : ["b","c"],
```

```
"b" : ["a","c", "d"],
```

```
"c" : ["a","b", "e"],
```

```
"d" : ["b","e"],
```

```
"e" : ["c","d"]
```

```
}
```

Print the graph

```
print(graph)
```

Output:

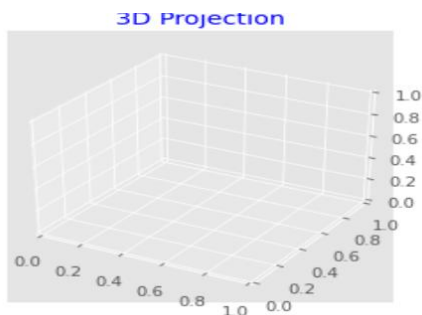
```
{'a': ['b', 'c'], 'b': ['a', 'c', 'd'], 'c': ['a', 'b', 'e'], 'd': ['b', 'e'], 'e': ['c', 'd']}
```

7.6 3D VISULIZATION AND PRESENTATION

The matplotlib library is most popular for data visualization in Python. It was initially designed for two-dimension plotting, but some three-dimension plotting utilities were built on top matplotlib's two-dimension display in later versions. Three dimensional plots are enabling by importing the mplot3d toolkit, which included with the main matplotlib.

A three-dimensional axis can be created by using the keyword ***projection="3d"*** as follows.

```
import matplotlib.pyplot as plt  
  
# 3D projection plot  
  
fig = plt.figure()  
  
ax = plt.axes(projection="3d")  
  
plt.title("3D Projection", color="blue")
```



7.6.1 3D Line Plot

This is a most basic three-dimensional plot created using set of (x, y, z) triples. It is also known as time series plot. This plot is plotted using ***ax.plot3D*** function as follow:

```
from mpl_toolkits import mplot3d  
  
import numpy as np  
  
import matplotlib.pyplot as plt
```

```
# 3D projection

fig = plt.figure()

ax = plt.axes(projection = "3d")

# All three axis

z = np.linspace(0, 1, 100)

x = z * np.sin(50 * z)

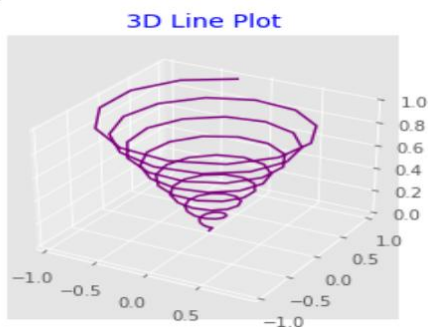
y = z * np.cos(50 * z)

# 3D Line plotting

ax.plot3D(x, y, z, "purple")

ax.set_title("3D Line Plot", color="blue")

plt.show()
```



ot

7.6.2 3D Scatter Plot

This is a basic three-dimensional plot created using set of (x, y, z) triples. It represents the data points on three axes to show the relationship between three variables. This plot is plotted using **ax.scatter3D** function as follow:

```
from mpl_toolkits import mplot3d
```

```
import numpy as np

import matplotlib.pyplot as plt

# 3D projection

fig = plt.figure()

ax = plt.axes(projection = "3d")

# All three axis

z = np.linspace(0, 1, 100)

x = z * np.sin(25 * z)

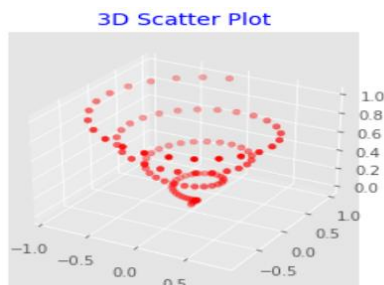
y = z * np.cos(25 * z)

# 3D scatter plotting

ax.scatter3D(x, y, z, color="red")

ax.set_title("3D Scatter Plot", color="blue")

plt.show()
```



7.6.3 3D Bar Plot

The three-dimensional bar plot is used to compare the relationship between three variables. This plot is plotted using ***ax.bar3d*** function as follow:

```
from mpl_toolkits.mplot3d import axes3d
```

```
import matplotlib.pyplot as plt

import numpy as np

from matplotlib import style

# 3D projection

fig = plt.figure()

ax = fig.add_subplot(111, projection="3d")

# All three axis

x = [1,3,5,7,9,11,7,3,5,6]

y = [5,7,2,6,4,6,5,3,6,7]

z = np.zeros(10)

dx = np.ones(10)

dy = np.ones(10)

dz = [1,3,5,7,9,11,7,5,3,7]

# 3D Bar plotting

ax.bar3d(x, y, z, dx, dy, dz, color="orange")

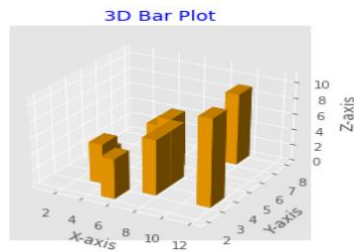
ax.set_title("3D Bar Plot", color="blue")

ax.set_xlabel("X-axis")

ax.set_ylabel("Y-axis")

ax.set_zlabel("Z-axis")

plt.show()
```



7.6.4 3D Wire Plot

This plot takes a grid of values and draws the lines between nearby points on three dimensional surface. This plot is plotted using **`ax.plot_wireframe`** method as follow:

```
import numpy as np

import matplotlib.pyplot as plt

# 3D projection

fig = plt.figure()

ax = plt.axes(projection="3d")

# Function

def func(x, y):

    return np.sin(np.sqrt(x * x + y * y))

# All three axis

x = np.linspace(-5, 5, 25)

y = np.linspace(-5, 5, 25)

X, Y = np.meshgrid(x, y)

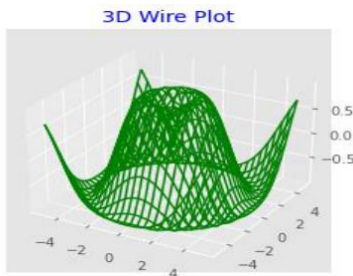
Z = func(X, Y)

# 3D wireframe plotting

ax.plot_wireframe(X, Y, Z, color="Green")
```



```
ax.set_title("3D Wire Plot", color="blue")  
plt.show()
```



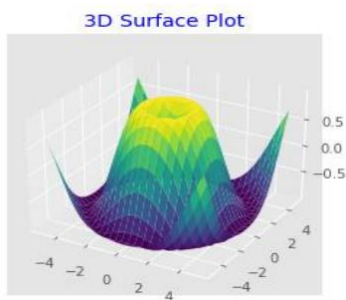
7.6.5 3D Surface Plot

This plot is like as wireframe plot, but each face of the wireframe is filled polygon.

This plot shows the functional relationship between one dependent variable and two independent variables. This plot is plotted using **`ax.plot_surface`** method as follow:

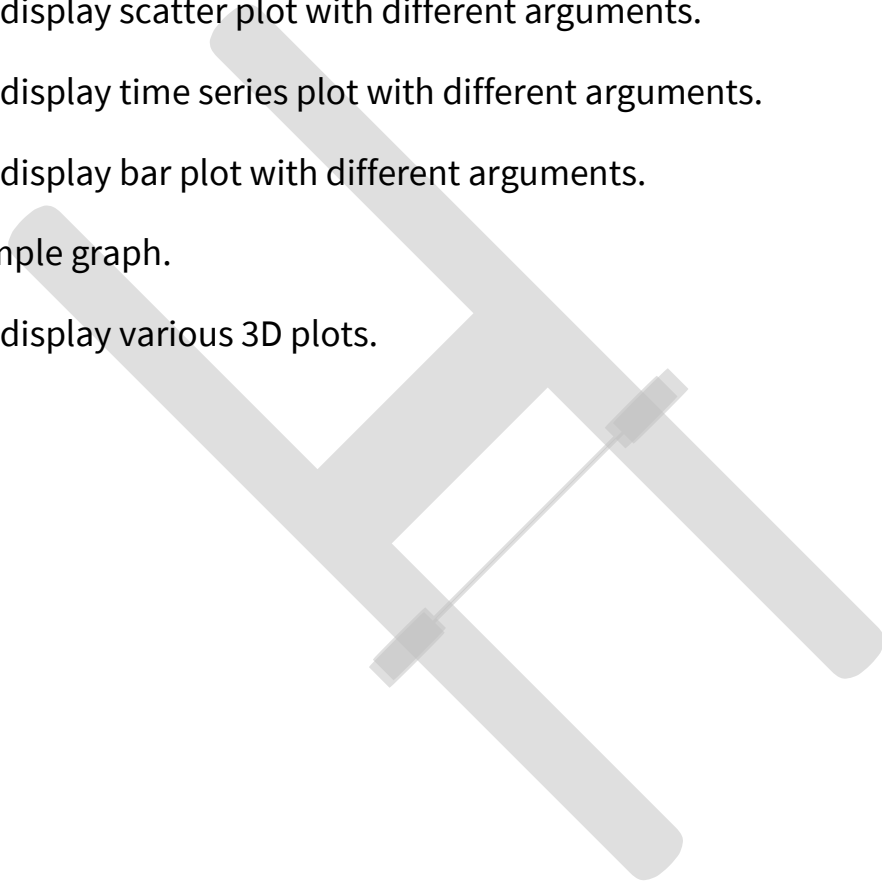
```
import numpy as np  
import matplotlib.pyplot as plt  
  
# 3D projection  
fig = plt.figure()  
ax = plt.axes(projection="3d")  
  
# Function  
def func(x, y):  
    return np.sin(np.sqrt(x * x + y * y))  
  
# All three axis  
x = np.linspace(-5, 5, 25)  
y = np.linspace(-5, 5, 25)
```

```
X, Y = np.meshgrid(x, y)
Z = func(X, Y)
# 3D surface plotting
ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='viridis', edgecolor='none')
ax.set_title("3D Surface Plot", color="blue")
plt.show()
```



QUESTIONS

1. What is generative modeling?
2. What is predictive modeling?
3. List the types of predictive modeling.
4. What is histogram?
5. List types of charts.
6. What is graph?
7. List the 3D plots.
8. Explain predictive modeling in details.
9. Explain histogram with example.
10. Explain scatter plot with example.

11. Explain line chart or time series plot with example.
 12. Explain bar plot with example.
 13. Explain three-dimensional plot with example.
 14. Create and display histogram with different arguments.
 15. Create and display scatter plot with different arguments.
 16. Create and display time series plot with different arguments.
 17. Create and display bar plot with different arguments.
 18. Create a simple graph.
 19. Create and display various 3D plots.
- 

Chapter Eight- Data Science Applications

8.1 Objectives

8.2 Data Science in Business

8.3 Data Science in Clean Energy

8.4 Data Science in Health Care

8.5 Insurance

8.6 Travel

8.7 Transport

8.8 Manufacturing

8.9 Telecommunication

8.10 Supply Chain Management

8.11 Gaming

8.12 Governance

8.1 OBJECTIVES

The objective of this unit is to illustrate the applications of Data Science. After completing this chapter, learners will be able to understand the fundamental applications of the latest technology.

This Unit will cover the applications in the following areas:

- Data Science in Clean Energy, Health Care
- Insurance
- Travel

- Transport
- Gaming, Supply Chain Management

8.2 DATA SCIENCE IN BUSINESS

In the modern era, various companies are using Data science to ease their regular processing.

Most companies use data to make better decisions and that data will be implemented for their growth.

There are many different ways by which Data Science is helping businesses to run in a better way:

1. Making best decisions:

The traditional way of business was more detailed and fixed in nature but that data cannot be used in business operations and decision-making strategies. In this processing, various factors are involved to get the best outcome such as:

1. Aware of the context and behavior of the problem.
2. Elaborate and measuring the quality of data.
3. Best algorithms and tools will be used to find the best solutions.
4. Using stories about success for the understanding of teams.

With the help of these steps, businesses need data science to speed up the decision-making process.

2. Manufacturing Better Products

Most companies should be able to engage their consumers towards products. They find the best alternates for their customers and assure them related to their products. Thus, companies need data to develop their product in the best possible way. The process involves such as feedback, use advanced analytical tools, market trends, and innovative ideas.

3. Efficiently Manage the Business

Modern business is fully based on data and that will help to make meaningful analysis with events of prediction. With the use of data science, a businessman can manage their business more efficiently either business is on a large scale and a small startup. Moreover, companies can predict the growth rate of their implementation. Thus, it can help in summarizing the overall performance of the company and the distribution of the products worldwide. Data science helps in managing business efficiency by some factors such as tracking the performance, the success rate of the product, and other important metrics.

4. Predict Outcomes for Future Outcome

Prediction factors are the most crucial part of businesses. With the advanced tools and technologies, industries have elaborated their capability to provide the best training to their employee. In simple terms, predictive analytics is that which involves several machine learning algorithms and tools for future outcomes using previous year's data and these tools are SAS, IBM, SPSS, etc.

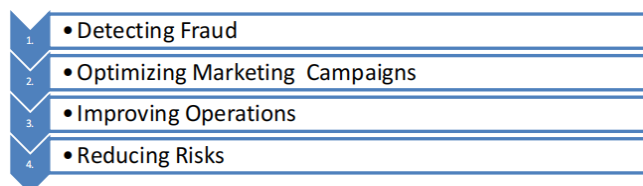


Figure 2: Predictive Processes

5. Assessing Business Decisions

After prediction, it is important to understand how these predictions will affect the performance of the business and growth. If the decision leads to face any onsequences, then analyze the decision and eliminate the problems. Thus, business can make profits with the help of data science.

8.3 DATA SCIENCE IN CLEAN ENERGY

Clean energy is also known as renewable energy. It continuously grows with the advancement of the Internet of Things (IoT). The connectivity and sensor advancement collect more innumerable data from outside resources. Earlier energy companies did not work with a large amount of data.

But, with the approach of data science, one can derive important insight from the excessive data.

Data science to be used in the industry of clean energy to improve and optimizes the processes.

One of the significant approaches is to improve the day-to-day operations in the energy renewable sector. Data plays an efficient role in the management and clean energy regulation.

Data science is to be used in many ways in clean energy. One of the principal examples is the solar plant that collects data for optimizing power performance, also predicts the maintenance required after the particular interval, etc. These applications include the widespread collection of data and its analysis.

There exist several ways of using clean energy with data science. For example, the data to be collected from solar to optimize the performance, reduce the maintenance times, analyze the expected maintenance time, and make solar implementation more compact. These applications of clean energy are extensively used for collection and data analysis.

8.3.1. Clean Energy for Most Suitable Environment and Economy

It can be a great asset to the enterprise that makes optimizations in the day-to-day enforcements in the wind or solar farm. In this way, renewable energy is an environmentally friendly and cost-effective option to fossil fuels that improve efficiency. With the advancement of renewable energy sources, it is possible to get more mileage from wind farms or solar plants in a desirable manner.

8.4 DATA SCIENCE IN HEALTH CARE

Data science plays a vital role in all industries and it will help to transform the health care sector also.

Medicine and healthcare facilities both are a crucial part of everyone's life and the treatment and the medicine are fully relied on doctor prescription but this wasn't always right. On the other hand, data science will help them with their diagnoses. In the healthcare sector, several fields such as medical imaging, drug discovery, genetics, predictive diagnosis, and some other areas where data science has been used.

a) **Medical Imaging:** Technology has improved the health sector day by day and there are

various imaging techniques to visualize the internal parts of the human body such as X-Ray,

MRI, and CT-Scan. In the past few years, doctors treat patients manually and this technique creates a problem for both of them, however, it has been possible through deep learning technologies in data science. In the past few years ago, doctors check the patients manually but due to deep learning of data science, the trend has to change day by day.

b) **Genomics:** It is the study of sequencing and analysis of genomes. A genome is made of DNA. It is a time-consuming and expensive process to analyze the sequence of genomes.

However, with the advanced tools of data science, it is possible to analyze and understanding human genes at a low cost. The Gene system has helped to find the connections between genetics and the health of the person. Moreover, it also involves finding the proper drug and how it will affect the particular genetic structure and this combination is called Bioinformatics. Several tools are related to data science such as MapReduce, SQL, Galaxy, etc.

c) **Drug discovery:** Drug Discovery is a highly complicated task. Thus, the pharmaceutical

industries are based on data science to solve their problems and create the best outcome for

humanity. For example: In Covid -19 researcher of various countries share their data so they produce the best vaccine for the social cause. Drug discovery is a time-consuming process that involves heavy finance and heavy testing. Companies use the patient's information such as mutation profiles from the previous year's data and they can design drugs that address the key mutations in the genetic sequences. Furthermore, researchers study chemical compounds and test the combinations of different cells and genetic mutations. After that, they will produce the drug to solve the health issues.

d) **Monitoring patient health:** Data science plays a major role in IoT devices and these devices assist to check the heartbeat, temperature and check other medical parameters of the person. The data is collected with the help of analytical tools and it helps the doctors track the patient biological rhythm. Furthermore, it helps the doctors to take compulsory decisions for their patients.

e) **Tracking & Preventing Diseases:** Data science plays a significant role in monitoring patient's health and takes important steps to detect chronic diseases at an early level with the best use of Artificial Intelligence.

8.5 INSURANCE

Insurance is one of the most competitive and unpredictable industry and it has been dependent on statistics however data science has changed the scenario. These days, insurance companies have a large range of information sources for covering the risk assessment. Big data helps to find risks and maintain effective strategies for customers.



Figure 3: Effective strategies for customer

a) **Fraud detection:** Insurance fraud is faced by insurance companies every year. Data science is the best way to detect fraudulent activity with the help of software and various techniques.

This software relies on the previous history of such activities and uses the sampling method to consider solving this issue.

b) **Price optimization:** It is a complex approach thus it can handle combinations of various methods and algorithms. Insurance companies use these algorithms to compare the changes between previous years and customer policy, because price optimization is closely related to the customer's price awareness. Price optimization assists to improve the customer's bond with the company for a long period.

c) **Risk assessment:** Risk assessment tools in the insurance industry encourage the forecasting of risks. Risks are of two major types: pure and speculative. A risk assessment identifies the risk amount and various reasons for the risk and these are the basis for data analysis and calculations.

d) **Healthcare insurance:** Health insurance is the best insurance throughout the world and it covers the costs caused by any major diseases, accidents, disability, or death. In most countries, health policies are strongly recommended by governments. Healthcare facilities are improving rapidly due to which companies face to provide better services and reduce the customer's costs. As a result, companies store a wide range of data includes claims data, membership information, medical records, etc. so, companies will

provide quality of care, fraud detection, and prevention and consumers get better facilities from the companies.

e) **Claims prediction:** Insurance companies are interested in the prediction of the coming future, so these companies predict the financial loss earlier by the use of models such as a decision tree, a random forest, a binary logistic regression, and a support vector machine.

Forecasting, the demand of the future to charge premiums that are not too high and not low.

Price models also contribute to the improvement, which helps the company to be one step beyond its opponents.

8.6 TRAVEL

Data Science is growing with multiple paradigms, tools, and machine learning approaches to acquire predictive, descriptive, and perspective goals to derive hidden data from the raw facts.

Nowadays, traveling is a thriving industry that regularly increases the number of consumers and requires the highly processing of data. This approach is highly adapting the data science algorithms. Travel industries such as hotel, reservation, airline, booking sites, etc. are the broad areas that manage multiple requests from customers every day. Data Science plays an efficient role and is used for hospitality and marketable uses in travel.

a) Viewpoint Analysis: The area where data science has proved valuable is viewpoint analysis. This analysis depends on the reviews of customers on the website. Organizations also track the remarks and emojis that are used by the customers for brands. It is critical to understand the view of the customer towards the brand and how they recommend the services to others. In this, Data science provides the resolution to do so.

b) **Tour Planning:** Data Science helps in building agendas and planning that assist in time management, cost-cutting, and quick decision making. The new trends in AI are trained by data science that requires constant learning about customer requests and requirements. It helps the customers by spending less time and numerous cost-effective plans. Every customer's history is considered by the AI machine learning tools to discover the most suitable plans.

8.7 TRANSPORT

Data science is also used in transportation actively making its mark in construct safer ways for drivers. In this sector, data science introduced a self-driving car for the new generation. It also analysis of fuel consumption patterns, driver behaviour and actively monitor the vehicle has established a strong bond between the company and the user. Driverless cars are the most advanced topic currently available in the market. Same as the driverless metro even cycle also has invented by Google engineers. Moreover, industries can also create the best logical routes and this is possible through data science. For example, most of the transportation companies follow data science for providing a better experience to their customers so that customers give the best feedback for their company and they also predict the best way for the customers without inconveniences like traffic and road conjunction.

8.8 MANUFACTURING

Manufacturing is the pillar of every other industry because it helps to analyze the performance, reduction of faults in the production adapts to the changes in the market trends, and upgrades the production system by the use of new technologies. The trend should be changed with the help of data science in manufacturing companies to boost production and generate annual revenue. To illustrate: the car industry is the real cause of a manufacturing company. In the last year, 2020 major food companies are using AI machines and this technology is capable of performing the given tasks such as:

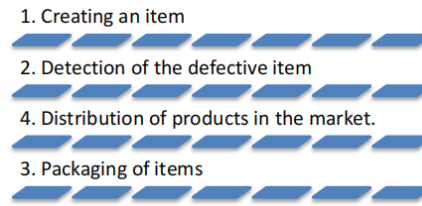


Figure 3: Steps

Product efficiency and performance of the machines are calculated by data science techniques such as visualization, which investigates the number of products per day as well as the number of defective product items and reasons for the defects. Data science helps to predict the annual growth on the behalf of current production systems.

Applications of Data Science in Manufacturing

The major application of data science in manufacturing industries is given below:

- Price Optimization
- Predictive Analytics
- Demand Forecasting and Inventory Management
- Supply Chain Management

a. **Price Optimization:** The main factor of Competition between the companies is the price of the product and numerous factors on which the end price of a product depends. These factors are raw materials, machines, labor cost, and electricity. All these affect the price of the final product. If the price of the product is high then it will affect the demand for the product in the market. At this stage, techniques of data science help to understand the requirement of products and identify the unnecessary costs of the product. It helps to optimize the cost of the product that would be responsible for their customers as well as, this cost helps them to in the highly corporative world with affordable cost. By this, the companies can groom their profitability for their future betterment.

b. **Predictive Analysis:** The overall growth of the company depends on its command of the trend of the market, the need of customers, and business rivals. Predictive analysis

is one of the major factors that can help the organization's goals as per the customers' needs.

Data science predicts the growth, customer demands, and future requirements due to these predictions', organizations set their goals for future manufacturing. During this process, data science can protect from errors in the products and develop more secure technologies that can help to increase production. Data science tools help to figure out the business and make changes in the product according to requirements. Production staff can build planning to cover uncertain situations using predictive analysis tools.

c. Demand Forecasting and Inventory Management: One major key factor for successful manufacturing is on-time production. Manufacturing companies face high-priority tasks in packaging and supplies. In a competitive market, it is important to forecast the demand of the customers in advance. Moreover, data science can help to analyze and predict customer demands. It helps the companies to manage the production and supply chain. In addition, it prevents unneeded production. At last, this feature is to manage inventory management, customer requirements, and business needs.

Some key factors about demand forecasting and inventory management:

- 1.** It helps to overcome the need for irrelevant storage.
- 2.** It controls inventory management.
- 3.** It covers the gap between the suppliers and the production companies.
- 4.** It manages the supply process.

In this way, production companies can perform well in the corporate world and create new strategies for future development.

8.9 TELECOMMUNICATION

Telecommunication companies are using data science applications that help in profit maximization. These companies play an efficient role in building an effective marketing strategy and business strategy. It analyzes and visualizes data that performs transferring and many other tasks. The basic functionality of these companies is strongly related to transfer, import, and exchange. The older techniques are not considered these days, because the amount of data that passes through the communications links are growing larger with every passing minute. The most relevant and efficient use cases of data science in the field of telecommunication.

a. Fraud Detection: The telecommunication industry is affected by frauds, illegal access, theft, fake policies, authorization, cloning, etc. Frauds directly influence the relationship between the user and the company. Thus, the approaches of fraud detection, tools, and techniques are widely available. By using machine learning algorithms, the operator can identify and categorize normal traffic and fraud. If it identifies any fraud, it generates an alert and informs the system administrator. This technique is efficient because it provides a real-time response to anomalous activities.

b. Predictive analytics: It is applied in telecommunication companies for predicting valuable information faster, better and helps in making decisions. This approach uses historical data to build predictions. The better the quality and for a long time in history, the better will be predictability.

c. Consumer Segmentation: The success of telecommunication companies is to create market segments and content can be divided according to each group. There exist four segmentation schemes such as customer value segmentation, customer lifecycle, behavior, migration segmentation.

8.10 SUPPLY CHAIN MANAGEMENT

The supply chain process involved in manufacturing has always been complicated. From start to end risk has been involved in all stages. Some complicated stages are a collection of requirements, gathering raw data, market data, analysis of the various resources, well-trained staff and machines, quality and distribution of the product in the market.

While production, some other factors are involved during company running time. Supply chain impact on the business loss due to organization faces several problems. Data science helps to manage the supply chain, detects the events where overloading is happening, predict the future possibilities of delay in manufacturing and supply. By this, manufacturers store the data in the form of backup for future emergencies and it will help all the businesses.

Finally, the applications of Data Science in manufacturing will change the current scenario of older methods used by the production companies. It will enhance the revenue generation of companies and contribute to the overall economic growth of industrialization.

8.11 GAMING

The gaming industry is one of the greatest industries in modern days. These days approximately two billion users all over the world have been involved in this industry due to more users data to processed is enhancing day by day. Data science helps to improve the business of gaming with an understanding of the data.

a. Game development: Most important task in gaming is developing the game. In this step, the developer considers the idea behind the game, its functionality, design plays an analytical part of gaming and interesting features in-game playing. In which performance should be measured, result, and it may be adapted according to the need of the customer.

In which data science is used to develop models, to analyze and identify expansion

points, make predictions, algorithms, pattern identification, maps in which players should move.

b. Game monetization: Another main factor of game planning is linking with companies' growth. Well, design games will help the developer to make money, thus the developer will concentrate on the growth of the company. Big data is used to predict the behavior so that users will come to play the same game again and again and will be ready to pay money for playing.

c. Game design: Game designing is an art and it will be the best career for developers. It is a complex task requiring several programming languages. Developers show their creativity to help to create interactive and complex sides for the games. Gaming analytics are used to obtain about user's requirements, to understand the various barriers, reasoning, and timing. Advanced gaming concepts, storylines, and create a strategy to find the previous data.

d. Object identification: Latest graphics features, artificial intelligence applications are the key factors of developers and creative designers. Image recognition technologies are remodeling the gaming industry. Developer's uses object detection models, so they create the natural scenes with the perfect movement of the objects. For example, these models are used to differentiate between teams, commands, obstacles, and figures become easier and much faster for interactive games.

e. Visual effects and graphics: During video game development, there were various graphic techniques invented. Due to these advanced techniques, gaming effects have to change such as motion capture in games, real-time rendering, and many others. While using these motion techniques allows the creation of characters with more human features and attributes. It helps in display facial emotions and expressions with natural movements. Animation's developers try to use the latest algorithms to push the video boundaries of the game reached up to one more advanced level. Real-time tools are used for this purpose.

f. Personalized marketing: Personalized marketing is applied by those companies which are linked with animations, video, and gaming. Companies avoid useless and unprofitable advertisements. In which marketers and game developers are interacted with customers and lead by the creation of successful market messages and results will be provided to the exact people. Personalized marketing is helping to find the activities of the users and side by side attract new users.

g. Fraud detection: In Gaming, all the actions and recommendations in the world are fast.

Companies face the problem of finding fraudulent activity, so it is a challenging task. To solve this problem, companies provide verifications of users by law, finding doubtful accounts, and stop these fraud accounts as soon as possible. Payment fraud is common in gaming due to which companies require a high level of security system for detection of such users. Machine learning algorithms come to the security of gaming industries by applying fraud detection methods.

h. Social and customer analysis: The gaming industry proves to be very innovative and successful in the modern era. Its growing popularity is depending upon, after a new minute a new user is attracting towards games. Millions of users take part in video games actively all over the world. Most youngsters use gaming as a platform and leave a significant amount of valuable data that are used by developers because data helps them to understand the customer's perception and develop the latest games with advanced features.

At last, data science is helping to improve in the gaming industry by using techniques and methods. Data science helps the developers to invent the latest games and companies gain profit from such games and users enjoy playing real games and reduced their stress level.

8.12 GOVERNANCE

Governance is the application of data science that consists of rules, policies, processes, and organizational structure to support the data management task of the enterprise. The hierarchy of the organization data provides guidance, understanding, security, and trust among its stakeholders. With the growth of companies' data, the organization needs to create an appropriate big data environment that makes them available across the organization.

This integration is an essential part of deciding workflows and making decisions by various teams. Data governance is an essential task of the organization's data management. With the help of this governance, one can easily know the kind of data availability, where the data resides, and the method of data usage.

8.13 BIOTECHNOLOGY

Biotechnology is a term that represents the manipulation of crops, breed the animals to produce specific features or simply biotechnology modifies the products for specific use. In the modern era, our technological tools are mathematics, statistics, computational resources, availability of data sources, etc. In data science, biotechnology plays a vital role to improve medical treatment, agriculture, animal breeding, and industrialization and to solve in environmental issues.

Types of biotechnology

The basic term of biotechnology can be broken down into small chunks bases on common use and various applications.

1. **Red Biotechnology:** It is the latest technology that will use in medical processes such as getting an organism to produce new drugs and stem cells to revive damaged human tissues or regenerate entire organs.
2. **White Biotechnology:** It has to be used in industry such as the invention of new chemicals or fuels for vehicles.

3. **Green Biotechnology:** It is for living creatures means green biotechnology implement in agricultural processes such as removing pests, so everybody uses pest's free crops and creates environmentally friendly development.

4. **Gold Biotechnology:** It is also known as Bioinformatics and it will use in biological data.

5. **Blue Biotechnology:** It will help in defense such as encompasses processes in marine and aquatic environments.

6. **Yellow Biotechnology:** It is the oldest branch of biotechnology and is used to process more nutrition-rich food products.

Yellow Biotechnology can be said as the oldest branch of Biotechnology. It uses microorganisms or insects for the production of more nutrition-rich food products for us so Yellow Biotechnology is also called Nutritional Biotechnology or Insect Biotechnology.

Biotechnology and Data Science

Biotechnologists are researchers and they use statistical analyses to redesign based on experiments. Data scientists focus on math and stats background however biotechnologists work on biostatistics and they have to used quantitative data and shifting through which factors are more likely to produce a particular effect requires major computational effort.

8.14 PHARMACEUTICALS

The launching of new pharmaceutical products to market is a long process with many consequences. A thousand trials need to be taken regularly to meet the objective that delays getting the output and also increases the cost with this expensive process. There exist numerous data points, experiments, benefits, and risks that must be analyzed which help in making the industry logically fit for the big data.

The cost of clinical trials can also be reduced by the data scientist:

- **Patient selection based on data:** Multiple data sources including social media and public databases related to health that identifies the population best for the trials.
- **Monitoring:** The companies can monitor on a real-time basis to identify operational risk.
- **Safety Assurance:** The data scientist can also consider the side effects before actual trials.

8.15 GEOSPATIAL ANALYTICS AND MODELLING

Geospatial analytics helps in gathering, manipulating, and displaying geographic data that includes GPS and satellite information. This data relies on coordinates and specific terms like a street address, postal address, etc. These techniques help in creating geographic models and visualization patterns for accurate modeling and predictions. The data can be collected from different technologies such as GPS, social media, location sensors, mobile, satellite, etc. to build better visualization of data for understanding the complex relationships between places and people.

The visualization includes the maps, graphs, cartograms, statistics, etc. that shows the history and current changes also. It helps in making predictions more accurate and easier.

Geospatial analytics also adds timing and location for the data that creates a complete picture of events. Geospatial analytics process a large amount of geographic data. It helps the users to interact with billions of mapped locations by looking at real-time geospatial visualizations. Users can traverse the data using time and space by instantly checking changes from days to years.

The benefits of geospatial analytics include:

- **Appealing insights:** The data displayed in the form of maps are easier to understand by any user.

- **Better vision:** With the help of visual images, one can understand how the spatial conditions are changing in real-time that helps an organization to understand the changes and determine actions for the future.
- **Targeted report:** The location-based data helps the organizations to understand, why some locations and countries, like the United States, are more prosperous for business than others.

Geospatial Analytics Use Cases

Telecommunications: It quickly visualizes the record related to call details and network records to fix the issues before the customer notices. This tool helps in the identification of anomalies for signal fluctuation and assists how to resolve them.

Military: In a military operation, logistics provides an aspect of situational awareness. This tool helps the military to optimize the placement of resources by predicting the infrastructure usage, and maintenance needs.

Weather: It helps in getting a quick response to weather by getting alerts. This data also helps the airlines with routing and insurance companies to better access the property risk.

Urban Planning/Development: It helps in determining the growing population's effect on energy, transportation, and other resources. This data helps to analyze the large datasets with high speed and scale.

Chapter Nine- Hypothesis Testing

Hypothesis testing involves formulating assumptions about population parameters based on sample statistics.

Example: You say an average height in the class is 30 or a boy is taller than a girl. All of these is an assumption that we are assuming, and we need some statistical way to prove these. We need some mathematical conclusion whatever we are assuming is true.

Defining Hypotheses

- **Null hypothesis (H0):** it is a basic assumption or made based on the problem knowledge.

Example: A company's mean production is 50 units/per da $H_0: \mu = 50$.

- **Alternative hypothesis (H1):** The alternative hypothesis is the hypothesis used in hypothesis testing that is contrary to the null hypothesis.

Example: A company's production is not equal to 50 units/per day i.e. $H_1: \mu \neq 50$.

- **Level of significance:** It refers to the degree of significance in which we accept or reject the null hypothesis. 100% accuracy is not possible for accepting a hypothesis, so we, therefore, select a level of significance that is usually 5%. This is normally denoted with α and generally, it is 0.05 or 5%, which means your output should be 95% confident to give a similar kind of result in each sample.

- **P-value:** The **P value**, or calculated probability, is the probability of finding the observed/extreme results when the null hypothesis(H_0) of a study-given problem is true.

If your P-value is less than the chosen significance level then you reject the null hypothesis i.e. accept that your sample claims to support the alternative hypothesis.

One-Tailed and Two-Tailed Test

One tailed test focuses on one direction, either greater than or less than a specified value. We use a one-tailed test when there is a clear directional expectation based on

prior knowledge or theory. The critical region is located on only one side of the distribution curve. If the sample falls into this critical region, the null hypothesis is rejected in favor of the alternative hypothesis.

One-Tailed Test

There are two types of one-tailed test:

- **Left-Tailed (Left-Sided) Test:** The alternative hypothesis asserts that the true parameter value is less than the null hypothesis. Example: $H_0: \mu \geq 50$ and $H_1: \mu < 50$
- **Right-Tailed (Right-Sided) Test:** The alternative hypothesis asserts that the true parameter value is greater than the null hypothesis. Example: $H_0: \mu \leq 50$ and $H_1: \mu > 50$

Two-Tailed Test

A two-tailed test considers both directions, greater than and less than a specified value. We use a two-tailed test when there is no specific directional expectation, and want to detect any significant difference.

Example: $H_0: \mu = 50$ and $H_1: \mu \neq 50$

Steps:

- 1- Define Null and Alternative Hypothesis
- 2- Choose Significance level

Select a significance level (α), typically 0.05, to determine the threshold for rejecting the null hypothesis. It provides validity to our hypothesis test, ensuring that we have sufficient data to back up our claims. Usually, we determine our significance level beforehand of the test. The [p-value](#) is the criterion used to calculate our significance value.

- 3- Calculate test statistic

There are various hypothesis tests, each appropriate for various goal to calculate our test. This could be a [Z-test](#), [Chi-square](#), [T-test](#), and so on.

1. **Z-test:** If population means and standard deviations are known. Z-statistic is commonly used.
2. **t-test:** If population standard deviations are unknown. and sample size is small then t-test statistic is more appropriate.
3. **Chi-square test:** Chi-square test is used for categorical data or for testing independence in contingency tables

4. **F-test:** F-test is often used in analysis of variance (ANOVA) to compare variances or test the equality of means across multiple groups.

We have a smaller dataset, So, T-test is more appropriate to test our hypothesis.

T-statistic is a measure of the difference between the means of two groups relative to the variability within each group. It is calculated as the difference between the sample means divided by the standard error of the difference. It is also known as the t-value or t-score.

4- **Comparing Test Statistic:**

In this stage, we decide where we should accept the null hypothesis or reject the null hypothesis. There are two ways to decide where we should accept or reject the null hypothesis.

Method A: Using Critical values

Comparing the test statistic and tabulated critical value we have,

- If Test Statistic > Critical Value: Reject the null hypothesis.
- If Test Statistic $\leq$ Critical Value: Fail to reject the null hypothesis.

Note: Critical values are predetermined threshold values that are used to make a decision in hypothesis testing. To determine [critical values](#) for hypothesis testing, we typically refer to a statistical distribution table, such as the normal distribution or t-distribution tables based on.

Method B: Using P-values

We can also come to a conclusion using the p-value,

- If the p-value is less than or equal to the significance level i.e. ($p \leq \alpha$), you reject the null hypothesis. This indicates that the observed results are unlikely to have occurred by chance alone, providing evidence in favor of the alternative hypothesis.
- If the p-value is greater than the significance level i.e. ($p \geq \alpha$), you fail to reject the null hypothesis. This suggests that the observed results are consistent with what would be expected under the null hypothesis.

5- Interpret the results

Calculating test statistic

1. Z-statistics:

When population means and standard deviations are known.

$$Z = \frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}}$$

where,

- $\bar{x}$ is the sample mean,
- μ represents the population mean,
- σ is the standard deviation
- and n is the size of the sample.

2. T-Statistics

T test is used when $n < 30$,

t-statistic calculation is given by:

$$t = \frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}}$$

where,

- ' $\bar{x}$ ' bar is the mean of the sample,
- μ is the assumed mean,
- σ is the standard deviation
- and n is the number of observations

3. Chi-Square Test

Chi-Square Test for Independence categorical Data (Non-normally distributed) using:

$$\chi^2 = \sum \frac{(\text{Observed value} - \text{Expected value})^2}{\text{Expected value}}$$

$$\chi^2 = \sum (O_i - E_i)^2 / E_i$$

where O_i is the observed value and E_i is the expected value.

Examples:

Does a New Drug Affect Blood Pressure?

Imagine a pharmaceutical company has developed a new drug that they believe can effectively lower blood pressure in patients with hypertension. Before bringing the drug to market, they need to conduct a study to assess its impact on blood pressure.

Data:

- Before Treatment: 120, 122, 118, 130, 125, 128, 115, 121, 123, 119
- After Treatment: 115, 120, 112, 128, 122, 125, 110, 117, 119, 114

Step 1: Define the Hypothesis

- **Null Hypothesis:** (H_0) The new drug has no effect on blood pressure.
- **Alternate Hypothesis:** (H_1) The new drug has an effect on blood pressure.

Step 2: Define the Significance level

Let's consider the Significance level at 0.05, indicating rejection of the null hypothesis.

If the evidence suggests less than a 5% chance of observing the results due to random variation.

Step 3: Compute the test statistic

Using [paired T-test](#) analyze the data to obtain a test statistic and a p-value.

The test statistic (e.g., T-statistic) is calculated based on the differences between blood pressure measurements before and after treatment.

$$t = m/(s/\sqrt{n})$$

Where:

- **m** = mean of the difference i.e $X_{\text{after}}, X_{\text{before}}$
- **s** = standard deviation of the difference (d) i.e $d_i = X_{\text{after},i} - X_{\text{before},i}$
- **n** = sample size,

then, $m = -3.9$, $s = 1.8$ and $n = 10$

we, calculate the , T-statistic = -9 based on the formula for paired t test

Step 4: Find the p-value

The calculated t-statistic is -9 and degrees of freedom $df = 9$, you can find the p-value using statistical software or a t-distribution table.

thus, $p\text{-value} = 8.538051223166285e-06$

Step 5: Result

- If the p-value is less than or equal to 0.05, the researchers reject the null hypothesis.
- If the p-value is greater than 0.05, they fail to reject the null hypothesis.

Conclusion: Since the p-value ($8.538051223166285e-06$) is less than the significance level (0.05), the researchers reject the null hypothesis. There is statistically significant evidence that the average blood pressure before and after treatment with the new drug is different.

```
i]: import numpy as np
    from scipy import stats

    # Data
    before_treatment = np.array([120, 122, 118, 130, 125, 128, 115, 121, 123, 119])
    after_treatment = np.array([115, 120, 112, 128, 122, 125, 110, 117, 119, 114])

    # Step 1: Null and Alternate Hypotheses
    # Null Hypothesis: The new drug has no effect on blood pressure.
    # Alternate Hypothesis: The new drug has an effect on blood pressure.
    null_hypothesis = "The new drug has no effect on blood pressure."
    alternate_hypothesis = "The new drug has an effect on blood pressure."

    # Step 2: Significance Level
    alpha = 0.05

    # Step 3: Paired T-test
    t_statistic, p_value = stats.ttest_rel(after_treatment, before_treatment)

    # Step 4: Calculate T-statistic manually
    m = np.mean(after_treatment - before_treatment)
    s = np.std(after_treatment - before_treatment, ddof=1) # using ddof=1 for sample standard deviation
    n = len(before_treatment)
    t_statistic_manual = m / (s / np.sqrt(n))

    # Step 5: Decision
    if p_value <= alpha:
        decision = "Reject"
```

```

—»decision = "Reject"
else:
—»decision = "Fail to reject"

# Conclusion
if decision == "Reject":
—»conclusion = "There is statistically significant evidence that the average blood pressure before and after treatment with the
else:
—»conclusion = "There is insufficient evidence to claim a significant difference in average blood pressure before and after tre

# Display results
print("T-statistic (from scipy):", t_statistic)
print("P-value (from scipy):", p_value)
print("T-statistic (calculated manually):", t_statistic_manual)
print(f"Decision: {decision} the null hypothesis at alpha={alpha}.")
print("Conclusion:", conclusion)

```

```

T-statistic (from scipy): -9.0
P-value (from scipy): 8.538051223166285e-06
T-statistic (calculated manually): -9.0
Decision: Reject the null hypothesis at alpha=0.05.
Conclusion: There is statistically significant evidence that the average blood pressure before and after treatment with the new
drug is different.

```

Activate Windows
Go to Settings to activate

In the above example, given the T-statistic of approximately -9 and an extremely small p-value, the results indicate a strong case to reject the null hypothesis at a significance level of 0.05.

- The results suggest that the new drug, treatment, or intervention has a significant effect on lowering blood pressure.
- The negative T-statistic indicates that the mean blood pressure after treatment is significantly lower than the assumed population mean before treatment.

Example:

Cholesterol level in a population

Data: A sample of 25 individuals is taken, and their cholesterol levels are measured. Cholesterol Levels (mg/dL): 205, 198, 210, 190, 215, 205, 200, 192, 198, 205, 198, 202, 208, 200, 205, 198, 205, 210, 192, 205, 198, 205, 210, 192, 205.

Populations Mean = 200

Population Standard Deviation (σ): 5 mg/dL(given for this problem)

Step 1: Define the Hypothesis

- **Null Hypothesis (H_0):** The average cholesterol level in a population is 200 mg/dL.
- **Alternate Hypothesis (H_1):** The average cholesterol level in a population is different from 200 mg/dL.

Step 2: Define the Significance level

As the direction of deviation is not given, we assume a two-tailed test, and based on a normal distribution table, the critical values for a significance level of 0.05 (two-tailed) can be calculated through the [z-table](#) and are approximately -1.96 and 1.96.

Step 3: Compute the test statistic

The test statistic is calculated by using the z formula $Z = (203.8 - 200) / (5 \div \sqrt{25})$ and we get accordingly, $Z = 2.0399999999999992$.

Step 4: Result

Since the absolute value of the test statistic (2.04) is greater than the critical value (1.96), we reject the null hypothesis. And conclude that, there is statistically significant evidence that the average cholesterol level in the population is different from 200 mg/dL

Python Implementation of Hypothesis Testing:

```
import scipy.stats as stats
import math
import numpy as np

# Given data
sample_data = np.array(
    [205, 198, 210, 190, 215, 205, 200, 192, 198, 205, 198, 202, 208, 200, 205, 198, 205, 210, 192, 205, 198, 205, 210, 192, 205]
)
population_std_dev = 5
population_mean = 200
sample_size = len(sample_data)

# Step 1: Define the Hypotheses
# Null Hypothesis (H0): The average cholesterol level in a population is 200 mg/dL.
# Alternate Hypothesis (H1): The average cholesterol level in a population is different from 200 mg/dL.

# Step 2: Define the Significance Level
alpha = 0.05 # Two-tailed test

# Critical values for a significance level of 0.05 (two-tailed)
critical_value_left = stats.norm.ppf(alpha/2)
critical_value_right = -critical_value_left
```

Activate Windows

```
# Step 3: Compute the test statistic
sample_mean = sample_data.mean()
z_score = (sample_mean - population_mean) / \
    (population_std_dev / math.sqrt(sample_size))

# Step 4: Result
# Check if the absolute value of the test statistic is greater than the critical values
if abs(z_score) > max(abs(critical_value_left), abs(critical_value_right)):
    print("Reject the null hypothesis.")
    print("There is statistically significant evidence that the average cholesterol level in the population is different from 200 mg/dL.")
else:
    print("Fail to reject the null hypothesis.")
    print("There is not enough evidence to conclude that the average cholesterol level in the population is different from 200 mg/dL.")
```

Reject the null hypothesis.

There is statistically significant evidence that the average cholesterol level in the population is different from 200 mg/dL.

Activate Windows

Example:**The Sports Watch Data Set**

Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	Hours_Work	Hours_Sleep
30	80	120	240	10	7
30	85	120	250	10	7
45	90	130	260	8	7
45	95	130	270	8	7
45	100	140	280	0	7
60	105	140	290	7	8
60	110	145	300	7	8
60	115	145	310	8	8
75	120	150	320	0	8
75	125	150	330	8	8

The data set above consists of 6 variables, each with 10 observations:

- **Duration** - How long lasted the training session in minutes?
- **Average_Pulse** - What was the average pulse of the training session? This is measured by beats per minute
- **Max_Pulse** - What was the max pulse of the training session?
- **Calorie_Burnage** - How much calories were burnt on the training session?
- **Hours_Work** - How many hours did we work at our job before the training session?
- **Hours_Sleep** - How much did we sleep the night before the training session?

```
Average_pulse_max = max(80, 85, 90, 95, 100, 105, 110, 115, 120, 125)
```

```
print (Average_pulse_max)
```

```
Average_pulse_min = min(80, 85, 90, 95, 100, 105, 110, 115, 120, 125)
```

```
print (Average_pulse_min)
```

```
import numpy as np
```

```
Calorie_burnage = [240, 250, 260, 270, 280, 290, 300, 310, 320, 330]
```

```
Average_calorie_burnage = np.mean(Calorie_burnage)
```

```
print(Average_calorie_burnage)
```

Extract data:

```
import pandas as pd
```

```
health_data = pd.read_csv("data.csv", header=0, sep=",")
```

```
print(health_data)
```

Consider data:

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	Hours_Work	Hours_Sleep
0	30.0	80	120	240.0	10.0	7.0
1	45.0	85	120	250.0	10.0	7.0
2	45.0	90	130	260.0	8.0	7.0
3	60.0	95	130	270.0	8.0	7.0
4	60.0	100	140	280.0	0.0	7.0
5	NaN	NaN	NaN	NaN	NaN	NaN
6	60.0	105	140	290.0	7.0	8.0
7	60.0	110	145	300.0	7.0	8.0
8	45.0	NaN	AF	NaN	8.0	8.0
9	45.0	115	145	310.0	8.0	8.0
10	60.0	120	150	320.0	0.0	8.0
11	60.0	9 000	130	NaN	NaN	8.0
12	45.0	125	150	330.0	8.0	8.0

- There are some blank fields
- Average pulse of 9 000 is not possible
- 9 000 will be treated as non-numeric, because of the space separator
- One observation of max pulse is denoted as "AF", which does not make sense

So, we must clean the data in order to perform the analysis.

Remove Blank Rows

We see that the non-numeric values (9 000 and AF) are in the same rows with missing values.

Solution: We can remove the rows with missing observations to fix this problem.

When we load a data set using Pandas, all blank cells are automatically converted into "NaN" values.

So, removing the NaN cells gives us a clean data set that can be analyzed.

We can use the `dropna()` function to remove the NaNs. `axis=0` means that we want to remove all rows that have a NaN value:

Example:

```
health_data.dropna(axis=0,inplace=True)
```

```
print(health_data)
```

So,

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	Hours_Work	Hours_Sleep
0	30.0	80	120	240.0	10.0	7.0
1	45.0	85	120	250.0	10.0	7.0
2	45.0	90	130	260.0	8.0	7.0
3	60.0	95	130	270.0	8.0	7.0
4	60.0	100	140	280.0	0.0	7.0
6	60.0	105	140	290.0	7.0	8.0
7	60.0	110	145	300.0	7.0	8.0
9	45.0	115	145	310.0	8.0	8.0
10	60.0	120	150	320.0	0.0	8.0
12	45.0	125	150	330.0	8.0	8.0

Data Categories

To analyze data, we also need to know the types of data we are dealing with.

Data can be split into two main categories:

1. **Quantitative Data** - Can be expressed as a number or can be quantified. Can be divided into two sub-categories:
 - **Discrete data**: Numbers are counted as "whole", e.g. number of students in a class, number of goals in a soccer game
 - **Continuous data**: Numbers can be of infinite precision. e.g. weight of a person, shoe size, temperature
2. **Qualitative Data** - Cannot be expressed as a number and cannot be quantified. Can be divided into two sub-categories:

- **Nominal data:** Example: gender, hair color, ethnicity
- **Ordinal data:** Example: school grades (A, B, C), economic status (low, middle, high)

By knowing the type of your data, you will be able to know what technique to use when analyzing them.

Data Types

We can use the `info()` function to list the data types within our data set:

```
print(health_data.info())
```

```
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Duration    12 non-null    float64
1   Average_Pulse 11 non-null    object
2   Max_Pulse    12 non-null    object
3   Calorie_Burnage 10 non-null    float64
4   Hours_Work   11 non-null    float64
5   Hours_Sleep  12 non-null    float64
dtypes: float64(4), object(2)
```

We see that this data set has two different types of data:

- Float64
- Object

We cannot use objects to calculate and perform analysis here. We must convert the type object to float64 (float64 is a number with a decimal in Python).

We can use the **`astype()`** function to convert the data into float64.

The following example converts "Average_Pulse" and "Max_Pulse" into data type float64 (the other variables are already of data type float64):

```
health_data["Average_Pulse"] = health_data['Average_Pulse'].astype(float)
health_data["Max_Pulse"] = health_data["Max_Pulse"].astype(float)
```

```
print (health_data.info())
```

```
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Duration     10 non-null     float64
1   Average_Pulse 10 non-null     float64
2   Max_Pulse     10 non-null     float64
3   Calorie_Burnage 10 non-null     float64
4   Hours_Work    10 non-null     float64
5   Hours_Sleep   10 non-null     float64
dtypes: float64(6)
```

Analyze the Data

When we have cleaned the data set, we can start analyzing the data.

We can use the describe() function in Python to summarize data:

```
print(health_data.describe())
```

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	Hours_Work	Hours_Sleep
Count	10.0	10.0	10.0	10.0	10.0	10.0
Mean	51.0	102.5	137.0	285.0	6.6	7.5
Std	10.49	15.4	11.35	30.28	3.63	0.53
Min	30.0	80.0	120.0	240.0	0.0	7.0
25%	45.0	91.25	130.0	262.5	7.0	7.0
50%	52.5	102.5	140.0	285.0	8.0	7.5
75%	60.0	113.75	145.0	307.5	8.0	8.0
Max	60.0	125.0	150.0	330.0	10.0	8.0

- **Count** - Counts the number of observations
- **Mean** - The average value
- **Std** - Standard deviation (explained in the statistics chapter)
- **Min** - The lowest value
- **25%, 50% and 75%** are percentiles (explained in the statistics chapter)
- **Max** - The highest value

Chapter TEN - Data Transformation-Data Reduction-

We discussed in *Chapter 2* Data Transformation we will return back now:

Data transformation is a technique used to **convert** the raw data into a suitable format that efficiently eases data mining and retrieves strategic information.

Data transformation includes **data cleaning techniques** and a **data reduction technique** to convert the data into the appropriate form.

Data transformation **changes** the **format, structure, or values of the data** and converts them into **clean, usable data**.

Data may be transformed at two stages of the data pipeline for data analytics projects.

Data Transformation Techniques:

1. Data smoothing
2. Attribute construction
3. Data aggregation
4. Data normalization
5. Data discretization
6. Data generalization

1. Data Smoothing

Data smoothing is a process that is used to **remove noise** from the dataset using some algorithms.

The **noise** is removed from the data using the techniques such as **binning, regression, clustering**.

- **Binning:** This method splits the sorted data into the number of bins and smoothens the data values in each bin considering the neighborhood values around it.
- **Regression:** This method identifies the **relation** among two dependent attributes so that if we have one attribute, it can be used to predict the other attribute.
- **Clustering:** This method **groups similar data values** and form a **cluster**. The values that lie outside a cluster are known as **outliers**.

2. Attribute Construction

In the attribute construction method, the new attributes consult the existing attributes to construct a new data set that eases data mining. New attributes are created and applied to assist the mining process from the given attributes. This simplifies the original data and makes the mining more efficient.

Ex: we may have the *height* and *width* of each plot. So here, we can construct a new attribute '**area**' from attributes 'height' and 'weight' or **BMI**, using these two features.

3. Data Aggregation

Data collection or aggregation is the method of storing and presenting data in a summary format.

The data may be obtained from multiple data sources to integrate these data sources into a data analysis description.

For example, we have a data set of sales reports of an enterprise that has quarterly sales of each year. We can aggregate the data to get the enterprise's **monthly** or **annual** sales report.

4. Data Normalization

Normalizing the data refers to scaling the data values to a much smaller range such as $[-1, 1]$ or $[0.0, 1.0]$. Discussed before.

5. Data Discretization

This is a process of **converting continuous** data into **a set of data intervals**. Continuous attribute values are substituted by small interval labels. This makes the data easier to study and analyze.

Data discretization can be **classified** into **two** types: **supervised discretization**, where the class information is used, and **unsupervised discretization**, which is based on which direction the process proceeds, i.e., 'top-down splitting strategy' or 'bottom-up merging strategy'.

For example, the values for the **age** attribute can be **replaced by the interval labels** such as (0-10, 11-20...) or (kid, youth, adult, senior).

6. Data Generalization

It converts **low-level** data attributes to **high-level data attributes** using concept **hierarchy**. This conversion from a lower level to a higher conceptual level is useful to get a clearer picture of the data.

Data generalization can be divided into two approaches:

- Data cube process (OLAP) approach.

- Attribute-oriented induction (AOI) approach.

For example, age data can be in the form of (20, 30) in a dataset. It is transformed into a higher conceptual level into a categorical value (young, old).

Data Reduction

Data reduction techniques ensure the integrity of data while reducing the data. Data reduction is a process that reduces the volume of original data and represents it in a much smaller volume.

Techniques of Data Reduction:

1. Dimensionality reduction
2. Numerosity reduction
3. Data cube aggregation
4. Data compression
5. Discretization operation

1. Dimensionality Reduction

Whenever we encounter weakly important data, we use the attribute required for our analysis.

Dimensionality reduction eliminates the attributes from the data set under consideration, thereby reducing the volume of original data.

When working with machine learning models:

datasets with **too many features** can cause issues like **slow computation** and **overfitting**. Dimensionality reduction helps by reducing the number of features while retaining key information.

Example:

*when you are **building a model to predict house prices** with **features** like **bedrooms**, **square footage**, and **location**. If you add too many features, such as room condition or flooring type, the dataset becomes large and complex.*

With too many features, training can slow down and the model may focus on irrelevant details, like flooring type, which could lead to inaccurate predictions.

Categories of Dimensionality Reduction Techniques

- Feature Selection
- Filter Methods
- Wrapper Methods
- Embedded Methods
- Feature Extraction
- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Uniform Manifold Approximation and Projection (UMAP)

Here are **three** methods of **dimensionality reduction**.

- **Wavelet Transform:** In the wavelet transform, suppose a data vector A is transformed into a numerically different data vector A' such that both A and A' vectors are of the same length.

- **Principal Component Analysis (PCA):** Suppose we have a data set to be analyzed that has tuples with n attributes. The principal component analysis identifies k independent tuples with n attributes that can represent the data set.

Principal Component Analysis (PCA) help in identifying and removing irrelevant or redundant features.

- **Attribute Subset Selection:** The large data set has many attributes, some of which are irrelevant to data mining or some are redundant. The core attribute subset selection reduces the data volume and dimensionality.

2. Numerosity Reduction

The numerosity reduction reduces the original data volume and represents it in a much smaller form. This technique includes **two types** **parametric** and **non-parametric** numerosity reduction.

- i) **Parametric:** Parametric numerosity reduction incorporates storing only data parameters instead of the original data. One method of parametric numerosity reduction is the regression and log-linear method.

- o **Regression and Log-Linear:** Linear regression models a relationship between the two

attributes by modeling a linear equation to the data set. Suppose we need to model a linear function between two attributes.

$$y=wx+b$$

Here, y is the response attribute, and x is the predictor attribute

Log-linear model discovers the relation between two or more discrete attributes in the database.

Suppose we have a set of tuples presented in n-dimensional space. Then the log-linear model is used to study the probability of each tuple in a multidimensional space.

ii) Non-Parametric: A non-parametric numerosity reduction technique does not assume any model. The non-Parametric technique results in a more uniform reduction, irrespective of data size, but it may not achieve a high volume of data reduction like the parametric.

Non-Parametric data reduction techniques, **Histogram**, **Clustering**, **Sampling**, **Data Cube Aggregation**, and **Data Compression**.

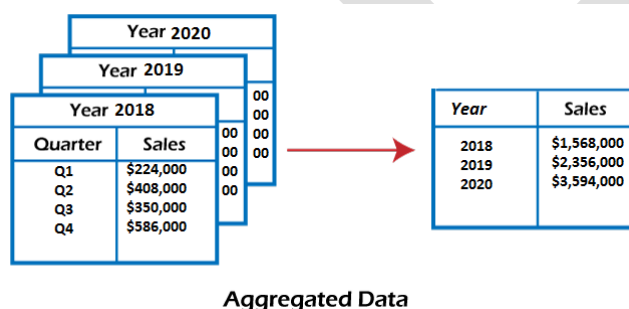
- **Histogram:** A histogram is a graph that represents frequency distribution which describes how often a value appears in the data. Histogram uses the binning method to represent an attribute's data distribution. It uses a disjoint subset which we call bin or buckets.
- **Clustering:** Clustering techniques groups similar objects from the data so that the objects in a cluster are similar to each other, but they are dissimilar to objects in another cluster.
- **Sampling:** One of the methods used for data reduction is sampling, as it can reduce the large data set into a much smaller data sample.
- **Cluster sample:** The tuples in data set D are clustered into M mutually disjoint subsets.

The data reduction can be applied by implementing SRSWOR on these clusters. A simple random sample of size s could be generated from these clusters where $s < M$.

• **Stratified sample:** The large data set D is partitioned into mutually disjoint sets called 'strata'. A simple random sample is taken from each stratum to get stratified data. This method is effective for skewed data.

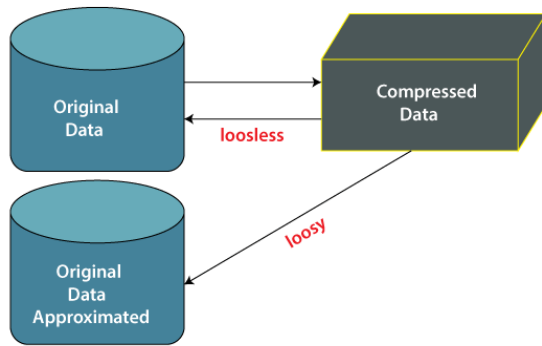
3. Data Cube Aggregation

This technique is used to aggregate data in a simpler form. Data Cube Aggregation is a multidimensional aggregation that uses aggregation at various levels of a data cube to represent the original data set, thus achieving data reduction.



4. Data Compression

Data compression employs modification, encoding, or converting the structure of data in a way that consumes **less space**. Data compression involves building a compact representation of information by removing redundancy and representing data in binary form. Data that can be restored successfully from its compressed form is called Lossless compression.



i. **Lossless Compression:** Encoding techniques (Run Length Encoding) allow a simple and minimal data size reduction. Lossless data compression uses algorithms to restore the precise original data from the compressed data.

يقلل حجم الملف عن طريق إزالة بعض البيانات الوصفية غير الضرورية

ii. **Lossy Compression:** In lossy-data compression, the decompressed data may differ from the original data but are useful enough to retrieve information from them. For example, the JPEG image format is a lossy compression, but we can find the meaning equivalent to the original image. Methods such as the Discrete Wavelet transform technique PCA (principal component analysis) are examples of this compression.

يقلل حجم الملف عن طريق إزالة بعض البيانات بشكل دائم

5. Discretization Operation

The **data discretization** technique is used to **divide the attributes of the continuous nature into data with intervals**. We replace many constant values of the attributes with labels of small intervals.

This means that mining results are shown in a concise and easily understandable way.

i. **Top-down discretization:** If you first consider one or a couple of points (so-called

breakpoints or split points) to divide the whole set of attributes and repeat this method up to the end, then the process is known as top-down discretization, also known as splitting.

ii. **Bottom-up discretization:** If you first consider all the constant values as split-points, some are discarded through a combination of the neighborhood values in the interval. That process is called bottom-up discretization.

Data discretization refers to a method of **converting a huge number of data values into smaller ones** so that the evaluation and management of data become easy.

In other words, **data discretization** is a method of converting attributes values of continuous data into a finite set of intervals with minimum data loss.

There are **two** forms of data discretization: **supervised discretization** and **unsupervised discretization**.

Supervised discretization refers to a method in which the **class data is used**.

Unsupervised discretization refers to a method depending upon the way which operation proceeds. It means it works on the top-down splitting strategy and bottom-up merging strategy.

Now, we can understand this concept with the help of an example.

Example:

Suppose we have an attribute of **Age** with the given values.

Age :

1,5,9,4,7,11,14,17,13,18, 19,31,33,36,42,44,46,70,74,78,77

before discretization:			
Attribute	Age	Age	Age
	1,5,4,9,7	11,14,17,13,18,19	31,33,36,42,44,46
After Discretization	Child	Young	Mature

Some Famous techniques of data discretization

Histogram analysis

Histogram refers to a plot used to represent the underlying frequency distribution of a continuous data set. Histogram assists the data inspection for data distribution.

For example, Outliers, skewness representation, normal distribution representation, etc.

Binning

Binning refers to a data smoothing technique that helps to group a huge number of continuous values into smaller values. For data discretization and the development of idea hierarchy, this technique can also be used.

Cluster Analysis

Cluster analysis is a form of data discretization. A clustering algorithm is executed by dividing the values of x numbers into clusters to isolate a computational feature of x.

Data discretization using decision tree analysis

data discretization refers to a decision tree analysis in which a top-down slicing technique is used.

It is done through a supervised procedure. In a numeric attribute discretization, first, you need to select the attribute that has the least entropy, and then you need to run it with the help of a recursive process.

Data discretization and concept hierarchy generation

The term **hierarchy** represents an organizational structure or mapping in which items are ranked according to their levels of importance. In other words, we can say that a hierarchy concept refers to a sequence of mappings with a set of more general concepts to complex concepts. It means mapping is done from low-level concepts to high-level concepts.

Top-down mapping

Top-down mapping generally starts with the top with some general information and ends with the bottom to the specialized information.

Bottom-up mapping

Bottom-up mapping generally starts with the bottom with some specialized information and ends with the top to the generalized information.

Chapter Eleven – Statistics for Data Science

- **Statistics** is the science of **collecting**, **analyzing**, and **interpreting** data to uncover patterns and make decisions.
- In **data science**, it acts as the backbone for understanding data and building reliable models.

It helps analysts to understand the data better.

- **Summarizes data** using measures like **mean**, **median**, and **variance**
- **Models** uncertainty with **probability** and **distributions**
- **Tests hypotheses** (e.g., A/B testing)
- **Finds relationships** through **regression** and **correlation**.

Types of Statistics

There are commonly two **types of statistics**, which are discussed below:

- **Descriptive Statistics:** *Descriptive Statistics* helps us simplify and organize big chunks of data. This makes large amounts of data easier to understand.
- **Inferential Statistics:** *Inferential Statistics* is a little different. It uses smaller data to conclude a larger group. It helps us predict and draw conclusions about population.

What is Data in Statistics?

Data is a **collection of observations**, it can be in the form of numbers, words, measurements, or statements.

Types of Data

- **1. Qualitative Data:** This data is descriptive. For example - She is beautiful, He is tall, etc.
- **2. Quantitative Data:** This is numerical information. For example- A horse has four legs.
- **Discrete Data:** It has a particular fixed value and can be counted.

- **Continuous Data:** It is not fixed but has a range of data and can be measured.

Descriptive statistics describe, show, and summarize the basic features of a dataset found in a given study, presented in a summary that describes the data sample and its measurements.

It helps analysts to understand the data better.

Descriptive statistics represent the available data sample and does not include theories, inferences, probabilities, or conclusions. That's a job for **inferential statistics**.

A good example of descriptive statistics:

look to a student's grade point average (GPA).

A GPA gathers the data points created through a large selection of grades, classes, and exams, then averages them together and presents a general idea of the student's mean academic performance.

Note that the GPA doesn't predict future performance or present any conclusions.

Types of Descriptive Statistics

Descriptive statistics break down into several types, characteristics, or measures. Some authors say that there are two types. Others say three or even four. In the spirit of working with averages, we will go with three types.

- **Distribution**, which deals with each value's frequency
- **Central tendency**, which covers the averages of the values
- **Variability** (or **dispersion**), which shows how spread out the values are **Distribution** (also called **Frequency Distribution**)

Datasets consist of a distribution of scores or values. Statisticians use **graphs** and **tables** to summarize the frequency of every possible value of a variable, rendered in percentages or numbers.

For instance, if you held a poll to determine people's favorite Beatle, you'd set up one column with all possible variables (John, Paul, George, and Ringo), and another with the number of votes.

Statisticians depict frequency distributions as either a **graph** or as a **table**.

Measures of Central Tendency

Measures of central tendency estimate a dataset's average or center, finding the result using three methods: **mean**, **mode**, and **median**.

Mean. The mean is also known as “M” and is the most common method for finding averages.

It is one of the first measurements we use to have a look at the data is to obtain sample statistics from the data, such as the sample mean. Given a sample of n values, $\{x_i\}, i = 1, \dots, n$, the mean, μ , is the sum of the values divided by the number of values,

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i.$$

For instance, say someone is trying to figure out how many hours a day they sleep in a week. So, the data set would be the hour entries (e.g., 6,8,7,10,8,4,9), and the sum of

those values is 52. There are seven responses, so $N=7$. You divide the value sum of 52 by N , or 7, to find M , which in this instance is 7.3.

```
import pandas as pd

# list of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}
df = pd.DataFrame(dict)
print(df)
print()
print()
fm['score'].mean()
```

```
In [56]: import pandas as pd
# List of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)
print(df)
print()
print()
fm['score'].mean()
```

	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

Out[56]: 40.0

Mode. The mode is just the most frequent response value. Datasets may have any number of modes, including “zero.” You can find the mode by arranging your dataset's order from the lowest to highest value and then looking for the most common response. So, in using our sleep study from the last part: 4,6,7,8,8,9,10. As you can see, the mode is eight.

Median. Finally, we have the median, defined as the value in the precise center of the dataset.

Arrange the values in ascending order (like we did for the mode) and look for the number in the set’s middle. In this case, the median is **eight**.

Sample median

The **mean** of the samples is a good descriptor, but it has an important drawback: what will happen if in the sample set there is an error with a value very different from the rest?

For example, considering hours worked per week, it would normally be in a range between 20 and 80; but what would happen if by mistake there was a value of 1000? An

item of data that is significantly different from the rest of the data is called an **outlier**. In this case, the mean, μ , will be drastically changed towards the **outlier**. One solution to this drawback is offered by the statistical median, μ_{12} , which is an order statistic giving the middle value of a sample

```
print(fm['score'].median())
```

Variability (also called Dispersion)

The measure of variability gives the statistician an idea of how spread out the responses are. The spread has three aspects — **range**, **standard deviation**, and **variance**.

Range. Use range to determine how far apart the most extreme values are. Start by subtracting the dataset's lowest value from its highest value. Once again, we turn to our sleep study:

4,6,7,8,8,9,10. We subtract four (the lowest) from ten (the highest) and get six. There's your range.

Standard Deviation. This aspect takes a little more work. The standard deviation (s) is your dataset's average amount of variability, showing you how far each score lies from the mean. The larger your standard deviation, the greater your dataset's variable. Follow these six steps:

1. List the scores and their means.
2. Find the deviation by subtracting the mean from each score.
3. Square each deviation.
4. Total up all the squared deviations.
5. Divide the sum of the squared deviations by $N-1$.

6. Find the result's square root.

Example: we turn to our sleep study: 4,6,7,8,8,9,10.

Raw number/data	Deviation from mean	Deviation Squared
4	$4-7.3 = -3.3$	10.89
6	$6-7.3 = -1.3$	1.69
7	$7-7.3 = -0.3$	0.09
8	$8-7.3 = 0.7$	0.49
8	$8-7.3 = 0.7$	0.49
9	$9-7.3 = 1.7$	2.89
10	$10-7.3 = 2.7$	7.29
M=7.3	Sum = 0.9	Square sums= 23.83

When you divide the sum of the squared deviations by 6 (N-1): $23.83/6$, you get 3.971, and the square root of that result is 1.992.

As a result, we now know that each score deviates from the mean by an average of 1.992 points.

Variance: Variance reflects the dataset's degree spread. The greater the degree of data spread, the larger the variance relative to the mean. You can get the variance by just squaring the standard deviation. Using our above example, we square 1.992 and arrive at 3.971.

Sample variance

The mean is not usually a sufficient descriptor of the data. We can go further by knowing two numbers: mean and variance.

$$\sigma^2 = \frac{1}{n} \sum_i (x_i - \mu)^2.$$

```
print(fm['score'].var())
```

Measuring Asymmetry: Skewness and Pearson's Median Skewness Coefficient

Skewness:

Skewness is the measure of the **asymmetry** of an ideally symmetric probability distribution and is given by the third standardized moment.

In simple words, skewness is the measure of how much the probability distribution of a random variable deviates from the normal distribution.

For univariate أحادي المتغير data, the formula for **skewness** is a statistic that measures the asymmetry of the set of n data samples, x_i :

$$g_1 = \frac{1}{n} \frac{\sum_i (x_i - \mu)^3}{\sigma^3},$$

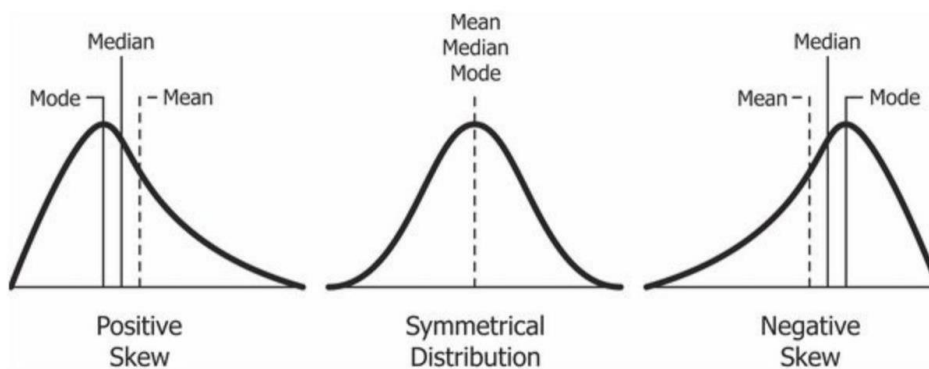
where μ is the mean, σ is the standard deviation, and n is the number of data points.

Negative deviation indicates that the distribution “**skews left**” (it extends further to the left than to the right).

Note: the skewness for a **normal distribution** is **zero**, and any **symmetric** data must have a skewness of zero.

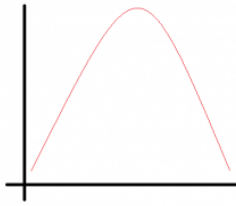
The **normal distribution** is the probability distribution **without any skewness**. You can look at the image below which shows symmetrical distribution that's basically a normal distribution and you can see that it is symmetrical on both sides of the dashed line. Apart from this, **there are three types of skewness**:

- Positive Skewness
- Negative Skewness
- Zero skewness



The probability distribution with its tail on the right side is a positively skewed distribution and the one with its tail on the left side is a negatively skewed distribution.

- **Normal Distribution:** Normal Distribution is a probability distribution that is symmetric about the mean. It is also known as Gaussian Distribution. The distribution appears as a Bell-shaped curve which means the mean is the most frequent data in the given data set.



In Normal Distribution :

Mean = Median = Mode

- **Standard Normal Distribution:** When in the Normal Distribution **mean = 0** and the Standard Deviation = 1 then Normal Distribution is called as Standard Normal Distribution.
- Normal Distributions are symmetrical in nature it doesn't imply that every symmetrical distribution is a Normal Distribution.
- Normal Distribution is the probability distribution without any skewness.

Unlike the Normal Distribution (mean = median = mode), in positive as well as negative skewness mean, median, and mode all are different.

Positive Skewness

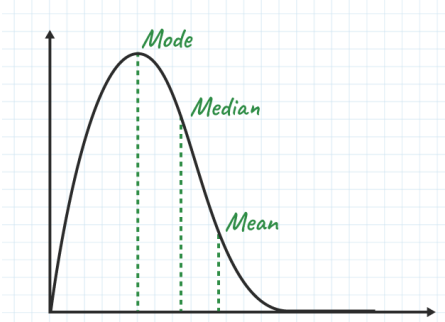
In positive skewness, the extreme data values are larger, which in turn increase the mean value of the data set, or in the simple term in positive skew distribution is the distribution having the tail on the right side.

Positive Skewness means the **tail** on the **right** side of the distribution is **longer**.

The mean and median will be greater than the mode.

Condition for positive skewness = **Mean > Median > Mode**

Positive Skew



Let's take an example of the income distribution where a **few people earn very high incomes and the majority earn lower incomes**. so, this is often **positively** skewed. Analyzing skewed data can provide valuable insights into the underlying causes and potential solutions or interventions.

In Positive Skewness:

Mean > Median > Mode

Negative Skewness

In negative skewness, the extreme data values are smaller, which decreases the mean value of the dataset or the negative skew distribution is the distribution having the tail on the left side.

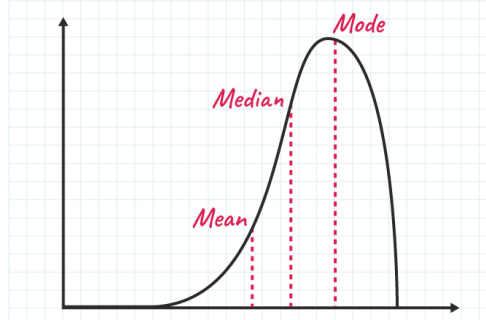
Negative Skewness means when the **tail** of the **left** side of the distribution is **longer** than the tail on the right side.

The **mean** and **median** will be **less** than the **mode**.

Condition for negative skewness is **Mode > Median > Mean**

The curve shows negative skewness in the image below,

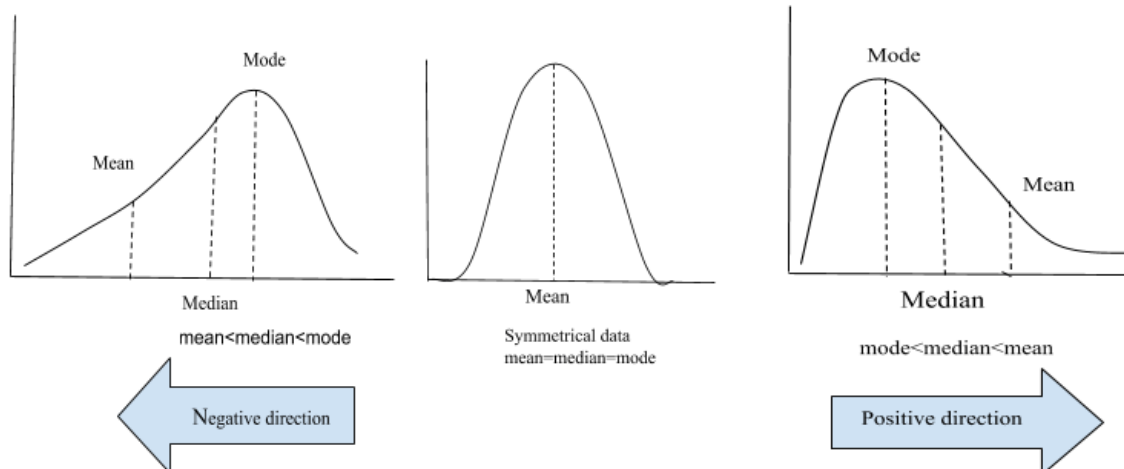
Negative Skew



Let's take an example of a match, during the match most of the players of a particular team scored runs above 50 and only a few of them scored below 10. In such a case, the data is generally represented with the help of a negatively skewed distribution. And this data is helpful to analyze the game's performance.

In Negative Skewness:

Mean < Median < Mode



Zero Skewness

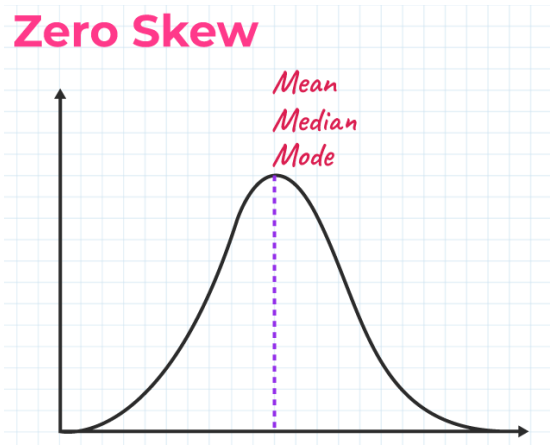
It is also known as a “symmetric distribution”.

It signifies that distribution of data is **evenly distributed around the mean**, with no long tails on either end of the distribution.

Condition for zero skewness is **Mean = Mode = Median**

The curve for zero skewness is shown in the image below,

Zero Skew



Calculate the skewness coefficient of the sample

Pearson's first coefficient of skewness

Subtract a mode from a mean, then divides the difference by standard deviation.

$$\text{Pearson's first coefficient} = \frac{\text{Mean} - \text{Mode}}{\text{Standard Deviation}}$$

As **Pearson's correlation coefficient** differs from **-1** (perfect negative linear relationship) to **+1** (perfect positive linear relationship), including a value of 0 indicating no linear relationship.

When we divide the covariance values by the standard deviation, it truly scales the value down to a limited range of **-1 to +1**. That accurately the range of the correlation values.

Pearson's second coefficient of skewness

Multiply the difference by 3, and divide the product by standard deviation.

$$\text{Pearson's second coefficient} = \frac{3 (\text{Mean} - \text{Median})}{\text{Standard Deviation}}$$

$$\text{Mean} - \text{Mode} \approx 3 (\text{Mean} - \text{Median})$$

If the **skewness** is between -0.5 & 0.5, the data are **nearly symmetrical**.

If the skewness is between -1 & -0.5 (negative skewed) or between 0.5 & 1 (positive skewed), the data are **slightly skewed**.

If the skewness is lower than -1 (**negative skewed**) or greater than 1 (**positive skewed**), the data are **extremely skewed**.

The **Pearson's median skewness coefficient** is a more robust alternative to the skewness coefficient and is defined as follows:

$$g_p = 3(\mu - \mu_{12})\sigma$$

μ mean, μ_{12} median

def **pearson**(x):

 return 3*(x.mean() - x.median())*x.std()

print "Pearson's coefficient of x = ", pearson(x)

Kurtosis:

Kurtosis measures the "heaviness of the tails" of a distribution (in compared to a normal

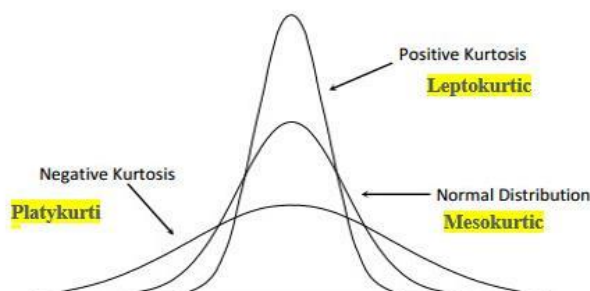
distribution). Kurtosis is positive if the tails are "heavier" than for a normal distribution, and negative if the tails are "lighter" than for a normal distribution. The normal distribution has kurtosis of zero.

Kurtosis characterizes the shape of a distribution - that is, its value does not depend on an arbitrary change of the scale and location of the distribution. For example, kurtosis of a sample (or population) of temperature values in Fahrenheit will not change if you transform the values to Celsius (the mean and the variance will, however, change).

The kurtosis of a distribution or sample is equal to the 4th central moment divided by the 4th power of the standard deviation, minus 3.

To calculate the kurtosis of a sample:

- i) subtract the mean from each value to get a set of deviations from the mean;
- ii) divide each deviation by the standard deviation of all the deviations;
- iii) average the 4th power of the deviations and subtract 3 from the result.



Excess Kurtosis

The excess kurtosis is used in statistics and probability theory to compare the kurtosis coefficient with that normal distribution. Excess kurtosis can be positive (Leptokurtic

distribution), negative (Platykurtic distribution), or near to zero (Mesokurtic distribution).

Since normal distributions have a kurtosis of 3, excess kurtosis is calculating by subtracting kurtosis by 3.

$$\text{Excess kurtosis} = \text{Kurt} - 3$$

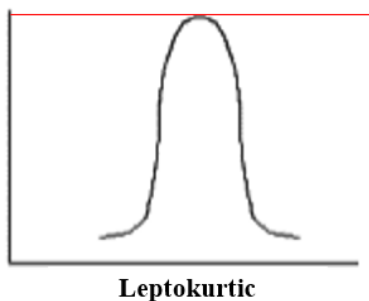
Types of excess kurtosis

1. *Leptokurtic or heavy-tailed distribution* (kurtosis more than normal distribution).
2. *Mesokurtic* (kurtosis same as the normal distribution).
3. *Platykurtic or short-tailed distribution* (kurtosis less than normal distribution).

Leptokurtic (kurtosis > 3)

Leptokurtic is having very long and skinny tails, which means there are more chances of outliers.

Positive values of kurtosis indicate that distribution is peaked and possesses thick tails. An extreme positive kurtosis indicates a distribution where more of the numbers are located in the tails of the distribution instead of around the mean.

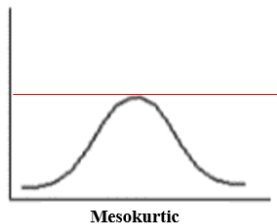


platykurtic (kurtosis < 3)

Platykurtic having a lower tail and stretched around center tails means most of the data points are present in high proximity with mean. A platykurtic distribution is flatter (less peaked) when compared with the normal distribution.

Mesokurtic (kurtosis = 3)

Mesokurtic is the same as the normal distribution, which means kurtosis is near to 0. In Mesokurtic, distributions are moderate in breadth, and curves are a medium peaked height.



$$\text{Mesokurtic} = 3 - 3 = 0$$

Covariance, and Pearson's and Spearman's Rank Correlation

Variables of data can express relations. For example, countries that tend to invest in research also tend to invest more in education and health. This kind of relationship is captured by the covariance.

يمكن لمتغيرات البيانات التعبير عن العلاقات. على سبيل المثال، تميل البلدان التي تميل إلى الاستثمار في البحوث إلى زيادة الاستثمار في التعليم والصحة. يتم التقاط هذا النوع من العلاقات من خلال التغيرات.

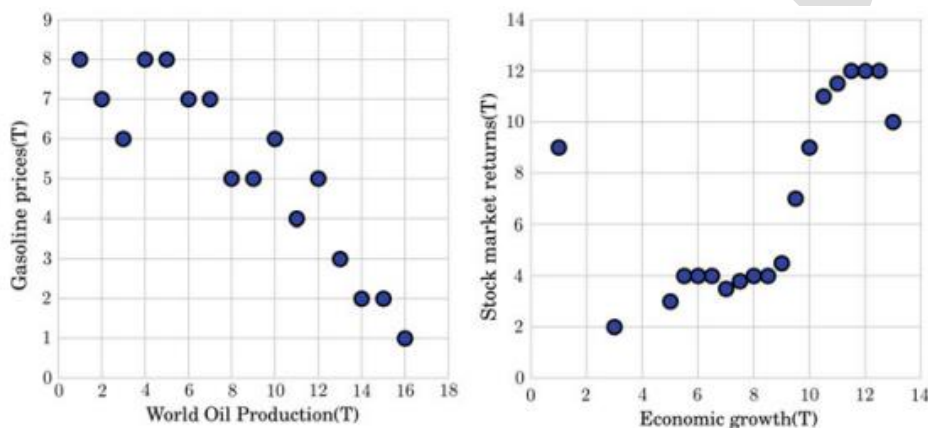


Fig. 3.9 Positive correlation between economic growth and stock market returns worldwide (*left*). Negative correlation between the world oil production and gasoline prices worldwide (*right*)

Covariance is a measure of the relationship between two random variables and to what extent, they change together. Or we can say, in other words, it defines the changes between the two variables, such that change in one variable is equal to change in another variable. This is the property of a function of maintaining its form when the variables are linearly transformed. Covariance is measured in units, which are calculated by multiplying the units of the two variables.

Population Covariance Formula

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

Sample Covariance

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N - 1}$$

Types of Covariance

Covariance can have both **positive** and **negative** values. Based on this, it has two types:

- 6- Positive Covariance
- 7- Negative Covariance

Positive Covariance

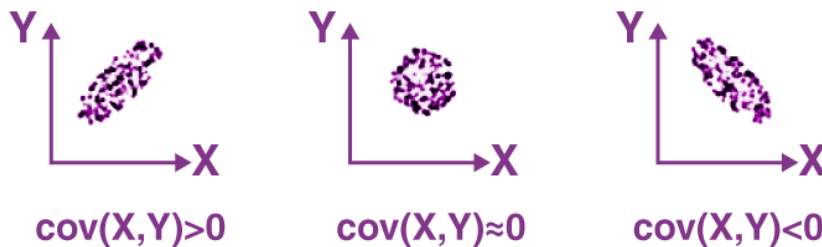
If the covariance for any two variables is positive, that means, both the variables move in the same direction.

Here, the variables show similar behavior.

That means, if the values (greater or lesser) of one variable corresponds to the values of another variable, then they are said to be in positive covariance.

Negative Covariance

If the covariance for any two variables is negative, that means, both the variables move in the opposite direction. It is the opposite case of positive covariance, where greater values of one variable correspond to lesser values of another variable and vice-versa.



If $\text{cov}(X, Y)$ is greater than zero, then we can say that the covariance for any two variables is positive and both the variables move in the same direction.

If $\text{cov}(X, Y)$ is less than zero, then we can say that the covariance for any two variables is negative and both the variables move in the opposite direction.

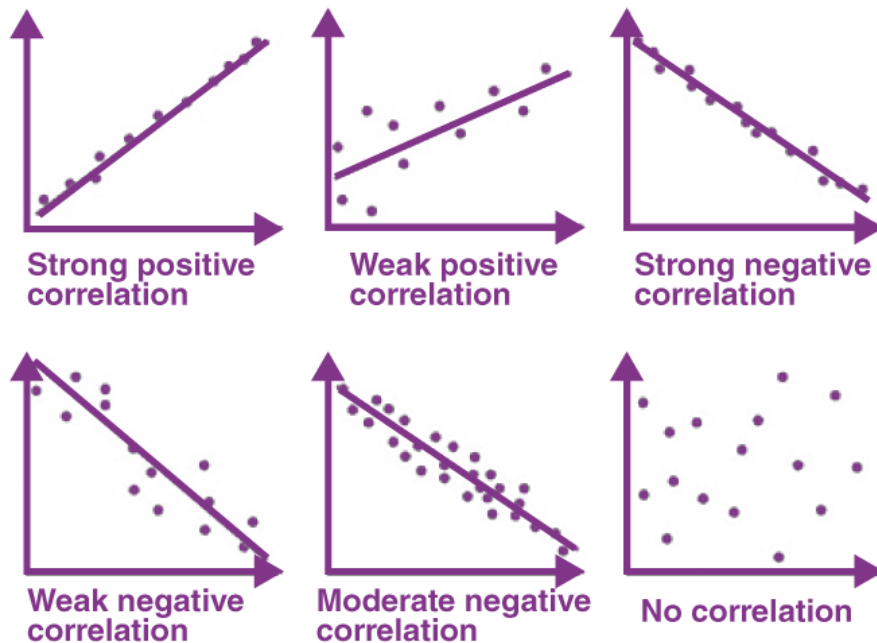
If $\text{cov}(X, Y)$ is zero, then we can say that there is no relation between two variables.

Correlation Coefficient Formula

We have already discussed covariance, which is the evaluation of changes between any two variables. Correlation estimates the depth of the relationship between variables. It is the estimated measure of covariance and is dimensionless. In other words, the correlation coefficient is a constant value always and does not have any units. The relationship between the correlation coefficient and covariance is given by;

$$\text{Correlation}, \rho(X, Y) = \text{Cov}(X, Y) / \sigma_X \sigma_Y$$

Based on the value of correlation coefficient, we can estimate the type of correlation between the given two variables. Also, the graphical representation of correlation among two variables is given in the below figure.



Note that the **Pearson's correlation** is always between -1 and $+1$, where the magnitude depends on the degree of correlation.

If the Pearson's correlation is 1 (or -1), it means that the variables are **perfectly** correlated (positively or negatively)

Spearman's Rank Correlation

The Spearman's rank correlation comes as a solution to the robustness problem of Pearson's correlation when the data contain outliers.

The main idea is to use the **ranks** of the **sorted** sample data, instead of the values themselves.

For example, in the list $[4, 3, 7, 5]$, the rank of 4 is 2 , since it will appear second in the ordered list $([3, 4, 5, 7])$.

Spearman's correlation computes the correlation between the ranks of the data.

For example, considering the data: $X=[10, 20, 30, 40, 1000]$, and $Y=[-70, -1000, -50, -10, -20]$, where we have an outlier in each one set.

If we compute the ranks, they are $[1.0, 2.0, 3.0, 4.0, 5.0]$ and $[2.0, 1.0, 3.0, 5.0, 4.0]$.

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

As value of the Pearson's coefficient, we get 0.28, which does not show much correlation between the sets. However, the Spearman's rank coefficient, capturing the correlation between the ranks, gives as a final value of 0.80, confirming the correlation between the sets. As an exercise, you can compute the Pearson's and the Spearman's rank correlations for the different Anscombe configurations given in Fig. 3.10. Observe if linear and nonlinear correlations can be captured by the Pearson's and the Spearman's rank correlations.

Example:

consider the following example data regarding the marks achieved in a maths and English exam:

	Marks									
	5	7	4	7	6	6	5	8	7	6
English	6	5	5	1	1	4	8	0	6	1
Maths	6	7	4	6	6	5	5	7	6	6
	6	0	0	0	5	6	9	7	7	3

The procedure for ranking these scores is as follows:

First, create a table with four columns and label them as below:

English (mark)	Maths (mark)	Rank (English)	Rank (maths)
56	66	9	4
75	70	3	2
45	40	10	10
71	60	4	7

English (mark)	Maths (mark)	Rank (English)	Rank (maths)
61	65	6.5	5
64	56	5	9
58	59	8	8
80	77	1	1
76	67	2	3
61	63	6.5	6

You need to rank the scores for maths and English separately. The score with the highest value should be labelled "1" and the lowest score should be labelled "10" (if your data set has more than 10 cases then the lowest score will be how many cases you have). Look carefully at the two individuals that scored 61 in the English exam (highlighted in bold). Notice their joint rank of 6.5. This is because when you have two identical values in the data (called a "tie"), you need to take the average of the ranks that they would have otherwise occupied. We do this because, in this example, we have no way of knowing which score should be put in rank 6 and which score should be ranked 7. Therefore, you will notice that the ranks of 6 and 7 do not exist for English. These two ranks have been averaged ($(6 + 7)/2 = 6.5$) and assigned to each of these "tied" scores.

An example of calculating Spearman's correlation

To calculate a Spearman rank-order correlation on data without any ties we will use the following data:

	Marks									
English	56	75	45	71	62	64	58	80	76	61
Maths	66	70	40	60	65	56	59	77	67	63

We then complete the following table:

English (mark)	Maths (mark)	Rank (English)	Rank (maths)	d	d ²
56	66	9	4	5	25
75	70	3	2	1	1
45	40	10	10	0	0
71	60	4	7	3	9
62	65	6	5	1	1
64	56	5	9	4	16
58	59	8	8	0	0
80	77	1	1	0	0
76	67	2	3	1	1
61	63	7	6	1	1

Where d = difference between ranks and d² = difference squared.

We then calculate the following:

$$\sum d_i^2 = 25 + 1 + 9 + 1 + 16 + 1 + 1 = 54$$

We then substitute this into the main equation with the other information as follows:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

$$\rho = 1 - \frac{6 \times 54}{10(10^2 - 1)}$$

$$\rho = 1 - \frac{324}{990}$$

$$\rho = 1 - 0.33$$

$$\rho = 0.67$$

as $n = 10$. Hence, we have a ρ (or r_s) of 0.67. This indicates a strong positive relationship between the ranks individuals obtained in the maths and English exam.

That is, the higher you ranked in maths, the higher you ranked in English also, and vice versa.

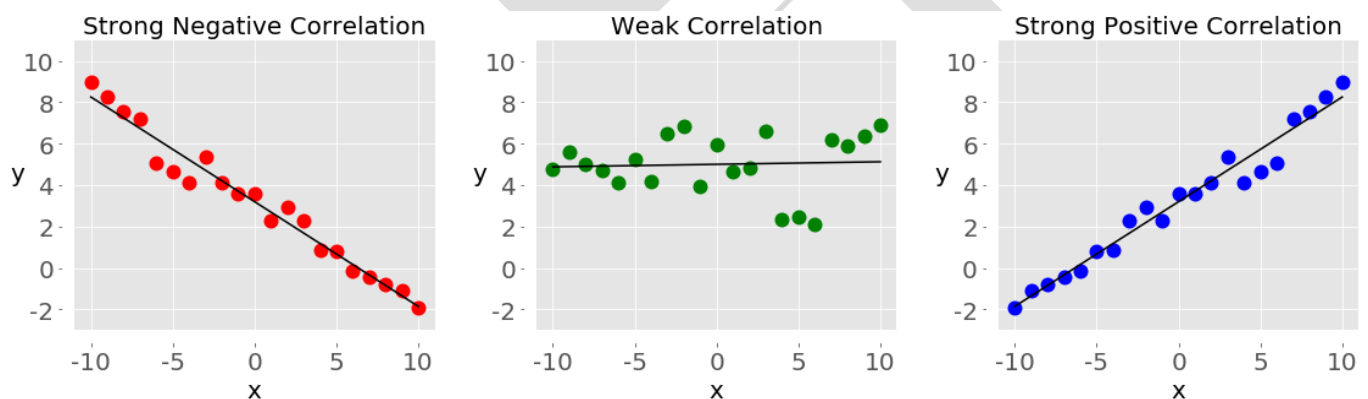
Example:

When data is represented in the form of a table, the rows of that table are usually the observations, while the columns are the features. Take a look at this employee table:

Name	Years of Experience	Annual Salary
Ann	30	120,000
Rob	21	105,000
Tom	19	90,000
Ivy	10	82,000

In this table, each row represents one observation, or the data about one employee (either Ann, Rob, Tom, or Ivy). Each column shows one property or feature (name, experience, or salary) for all the employees.

If you analyze any two features of a dataset, then you'll find some type of **correlation** between those two features. Consider the following figures:

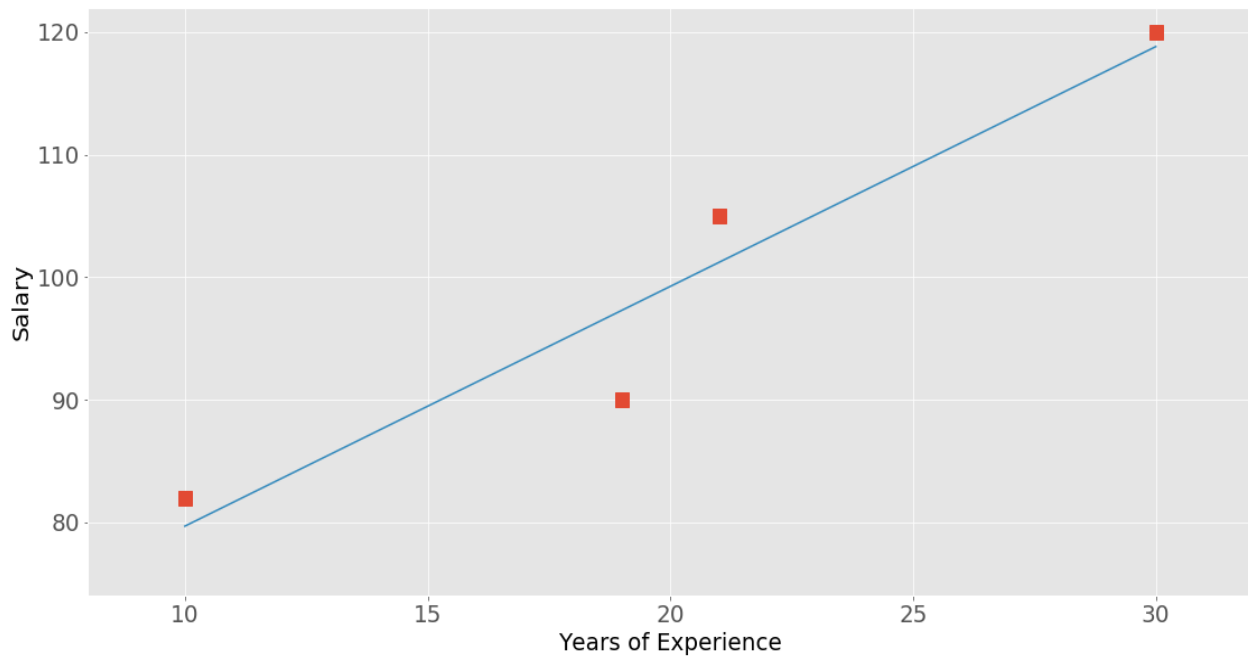


Each of these plots shows one of three different forms of correlation:

1. **Negative correlation (red dots):** In the plot on the left, the y values tend to decrease as the x values increase. This shows strong negative correlation, which occurs when *large* values of one feature correspond to *small* values of the other, and vice versa.
2. **Weak or no correlation (green dots):** The plot in the middle shows no obvious trend. This is a form of weak correlation, which occurs when an association between two features is not obvious or is hardly observable.

3. **Positive correlation (blue dots):** In the plot on the right, the y values tend to increase as the x values increase. This illustrates strong positive correlation, which occurs when *large* values of one feature correspond to *large* values of the other, and vice versa.

The next figure represents the data from the employee table above:



The correlation between experience and salary is positive because higher experience corresponds to a larger salary and vice versa.


```
In [21]: import numpy as np
x = np.arange(10, 20)
print("x=",x)
# array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
y = np.array([2, 1, 4, 5, 8, 12, 18, 25, 96, 48])
print("y=",y)
# array([ 2,  1,  4,  5,  8, 12, 18, 25, 96, 48])
r = np.corrcoef(x, y) # Pearson correlation coefficient
print("r=",r)
print("r[0, 1]",r[0, 1])
print("r[1, 0]",r[1, 0])
```

```
x= [10 11 12 13 14 15 16 17 18 19]
y= [ 2  1  4  5  8 12 18 25 96 48]
r= [[1.          0.75864029]
     [0.75864029 1.          ]]
r[0, 1] 0.7586402890911867
r[1, 0] 0.7586402890911866
```

```
In [ ]:
```

The values on the main diagonal of the correlation matrix (upper left and lower right) are equal to 1. The upper left value corresponds to the correlation coefficient for x and x , while the lower right value is the correlation coefficient for y and y . They are always equal to 1.

Kendal tau Rank Correlation

The Kendall tau-b correlation coefficient, τ_b , is a nonparametric measure of association based on the number of concordances and discordances in paired observations.

Suppose two observations (X_i, Y_i) and (X_j, Y_j) are **concordant** if they are in the same order with respect to each variable. That is, if

1. $X_i < X_j$ and $Y_i < Y_j$, or if
2. $X_i > X_j$ and $Y_i > Y_j$

They are **discordant** if they are in the reverse ordering for X and Y , or the values are arranged in opposite directions. That is, if

1. $X_i < X_j$ and $Y_i > Y_j$, or if
2. $X_i > X_j$ and $Y_i < Y_j$

The two observations are **tied** if $X_i = X_j$ and/or $Y_i = Y_j$

The total number of pairs that can be constructed for a sample size of n is

$$N = \binom{n}{2} = \frac{1}{2}n(n-1)$$

N can be decomposed into these five quantities:

$$N = P + Q + X_0 + Y_0 + (XY)_0$$

where P is the number of concordant pairs, Q is the number of discordant pairs, X_0 is the number of pairs tied only on the X variable, Y_0 is the number of pairs tied only on the Y variable, and $(XY)_0$ is the number of pairs tied on both X and Y .

The Kendall tau-b for measuring order association between variables X and Y is given by the following formula:

$$t_b = \frac{P - Q}{\sqrt{(P + Q + X_0)(P + Q + Y_0)}}$$

-1:1

Example:

Age and **percentage** body fat were measured in 18 adults. Calculate each of the Pearson, Spearman, and Kendall correlation coefficients. AS:

No. Age fat%

01	23	9.5
02	23	27.9
03	27	7.8
04	27	17.8
05	39	31.4
06	41	25.9
07	45	27.4
08	49	25.2
09	50	31.1
10	53	34.7

11	53	42.0
12	54	29.1
13	56	32.5
14	57	30.3
15	58	33.0
16	58	33.8
17	60	41.1
18	61	34.5

Question:

Calculate the coefficient of covariance for the following data:

X	2	8	18	20	28	30
Y	5	12	18	23	45	50

Solution:

Number of observations = 6

Mean of X = 17.67

Mean of Y = 25.5

$$\begin{aligned} \text{Cov}(X, Y) &= \left(\frac{1}{n}\right) [(2 - 17.67)(5 - 25.5) + (8 - 17.67)(12 - 25.5) + (18 - 17.67)(18 - 25.5) + \\ &\quad (20 - 17.67)(23 - 25.5) + (28 - 17.67)(45 - 25.5) + (30 - 17.67)(50 - 25.5)] \\ &= 157.83 \end{aligned}$$

Quartiles (measure of dispersion): divides the dataset into **four** equal parts:

- **Q1** (First Quartile): Median of the lower 50% of the dataset (25th percentile).
- **Q2** (Second Quartile / Median): Median of the entire dataset (50th percentile).
- **Q3** (Third Quartile): Median of the upper 50% of the dataset (75th percentile).

Mean Absolute Deviation: The average of the absolute differences between each data point and the mean. It provides a measure of the average deviation from the mean.

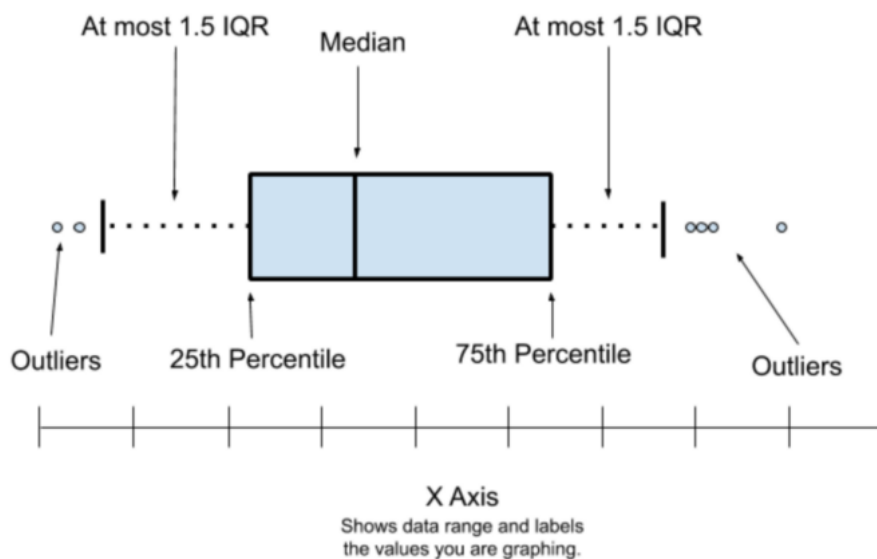
$$\text{Formula: MeanAbsoluteDeviation} = \frac{\sum_{i=1}^n |X - \mu|}{n}$$

Box plot:

A box plot also known as **Five Number Summary**, summarizes data using the median, upper quartile, lower quartile, and the minimum and maximum values. It allows you to see important characteristics of the data at a glance(visually). This also help us to visualize outliers in the data set.

Box plot or Five Number Summary has below five information.

1. Median
2. Lower Quartile (25th Percentile)
3. Upper Quartile (75th Percentile)
4. Minimum Value
5. Maximum Value

**Example of Box**

Step 1 — take the set of numbers given

14, 19, 100, 27, 54, 52, 93, 50, 61, 87, 68, 85, 75, 82, 95

Arrange the data in increasing(ascending) order

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 95, 100

Step 2 — Find the median of this data set. Median is mid value in this ordered data set.

14, 19, 27, 50, 52, 54, 61, **68**, 75, 82, 85, 87, 93, 95, 100

Here it is 68.

Step 3 — Lets find the Lower Quartile.

Lower Quartile is the median from the left of the, medium found in the Step 2(ie. 68)

(14, 19, 27, **50**, 52, 54, 61), 68, 75, 82, 85, 87, 93, 95, 100

Lower Quartile is 50

Step 4 — Lets find the Upper Quartile.

Upper Quartile is the median from the Right of the medium found in the Step 2(ie. 68)

14, 19, 27, 50, 52, 54, 61, 68, (75, 82, 85, **87**, 93, 95, 100)

Upper Quartile is 87

step 5 — Lets find the Minimum Value

It is value lies in the extreme left from this data set or first value in the data set after ordering.

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 95, 100

Minimum Value is 14

Step 6 — Lets find the Maximum Value

It is value lies in the extreme Right from this data set or last value in the data set after ordering.

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 95, **100**

Maximum Value is 100

Median	=> 68
Lower Quartile	=> 50
Upper Quartile	=> 87
Minimum Value	=> 14
Maximum Value	=> 100

Range :

Range is basically spread of our data set. Range can be found as difference between Maximum Value and Minimum Values.

Range=Maximum Value-Minimum Value

Range= 100-14 = 86

Quantiles and Percentiles

we can order the samples $\{x_i\}$, then find the x_p so that it divides the data into two parts, where

- a fraction p of the data values is less than or equal to x_p and
- the remaining fraction $(1 - p)$ is greater than x_p .

That value, x_p , is the p -th quantile, or the $100 \times p$ -th percentile.

For example, a 5-number summary is defined by the values $x_{\min}$, Q1, Q2, Q3, $x_{\max}$, where Q1 is the 25 × p -th percentile, Q2 is the 50 × p -th percentile and Q3 is the 75 × p -th percentile.

Interquartile Range(IQR):

Interquartile Range(IQR) is difference between Upper quartile and Lower quartile.

As per picture above,our Lower quartile is 50 and Upper quartile is 87

$$\text{IQR} = \text{Upper Quartile} - \text{Lower Quartile}$$

$$\text{IQR} = 87 - 50 = 27$$

Box plot with Even numbers of data set :**Step 1 :**

We have 14 records below.

14, 19, 100, 27, 54, 52, 93, 50, 61, 87, 68, 85, 75, 82

Arrange the data in increasing(ascending) order

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 100

Step 2: Since we have the even number take the middle two values add them and divide them by 2.

Here Values at position 7 & 8 are middle values.

							64.5							
							↑							
Numbers =>	14,	19,	27,	50,	52,	54,	61,	68,	75,	82,	85,	87,	93,	100
Position =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14

So our new median value is 64.5

Numbers =>	14,	19,	27,	50,	52,	54,	61,	64.5,	68,	75,	82,	85,	87,	93,	100
Position =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Continue Step 3 to Step 6 to get the values mention in **Working Example of Box**

Plot section.Final Result as below

Median	=> 64.5
Lower Quartile	=> 50
Upper Quartile	=> 85
Minimum Value	=> 14
Maximum Value	=> 100

Pivot table:

A pivot table is a **summary tool** that wraps up or summarizes information sourced from bigger tables. These bigger tables could be a database, an Excel spreadsheet, or any data that is or could be converted in a table-like form. The data summarized in a pivot table might include sums, averages, or other statistics which the pivot table groups together in a meaningful way.

Wide vs. Long Data

Pivoting is the act of converting “long data” into “wide data”. Wide and long data formats serve different purposes. It’s often helpful to think of how data might be collected in the first place.

Representing potentially multi-dimensional data in a single 2-dimensional table may require some compromises. Either some data will be repeated (long data format) or your data set may require blank cells (wide data).

Examples of pivot table:

- **Pivoting in Python with Pandas**

Starting with the raw dataset loaded into df_long_data


```
df_wide_data = df_long_data.pivot_table(  
    index = ["GEO", ],  
    columns = ["Vehicle type", "Year"],  
    values = ["VALUE"],  
    aggfunc = "sum"  
)
```

1. **Index** determines what will be unique in the leftmost column of the result.
2. **Columns** creates a column for each unique value in the “Vehicle type” and “Year” columns of the input table.
3. **Values** defines what to put in the cells of the output table.
4. **aggfunc** defines *how* to combine the values (commonly sum, but other aggregation functions are also used: min, max, mean, 95th percentile, mode). You can also define your own aggregation functions and pass in the function.

• Pivoting in Google Sheets

Pivoting in Google sheets is doing the exact same thing as our python code above but in a graphical way. Instead of specifying arguments for the pivot_table function, we select from dropdowns. But the important thing to remember is pivoting doesn’t “belong” to any particular software, it’s a generalized approach for dealing with data.

• Grouping in PostgreSQL

Many relational databases don’t have a built in pivot function. We can write a query that approximates the desired results but it does require some manual intervention to define the possible groups.

Heat map:

Heatmaps visualize the data in a 2-dimensional format in the form of colored maps. The color maps use hue, saturation, or luminance to achieve color variation to display various details. This color variation gives visual cues to the readers about the magnitude of numeric values.

Heatmaps can describe the density or intensity of variables, visualize patterns, variance, and even anomalies. Heatmaps show relationships between variables. These variables are plotted on both axes.

Uses of HeatMap

Business Analytics: A heat map is used as a visual business analytics tool. A heat map gives quick visual cues about the current results, performance, and scope for improvements. Heatmaps can analyze the existing data and find areas of intensity that might reflect where most customers reside, areas of risk of market saturation, or cold sites and sites that need a boost. Heat maps can be continued to be updated to reflect the growth and efforts. These maps can be integrated into a business's workflow and become a part of ongoing analytics.

- **Website:** Heatmaps are used in websites to visualize data of visitors' behavior. This visualization helps business owners and marketers to identify the best & worst-performing sections of a webpage. These insights help them with optimization.

- **Exploratory Data Analysis:** EDA is a task performed by data scientists to get familiar with the data. All the initial studies are done to understand the data are known as EDA. Exploratory Data Analysis (EDA) is the process of analyzing datasets before the modeling task. It is a tedious task to look at a spreadsheet filled with numbers and determine

essential characteristics in a dataset

- **Molecular Biology:** Heat maps are used to study disparity and similarity patterns in DNA, RNA, etc.
- **Geovisualization:** Geospatial heatmap charts are useful for displaying how geographical areas of a map are compared to one another based on specific criteria. Heatmaps help in cluster analysis or hotspot analysis to detect clusters of high concentrations of activity; For example, Airbnb rental price analysis.
- **Marketing and Sales:** The heatmap's capability to detect warm and cold spots is used to improve marketing response rates by targeted marketing. Heatmaps allow the detection of areas that respond to campaigns, under-served markets, customer residence, and high sale trends, which helps optimize product lineups, capitalize on sales, create targeted customer segments, and assess regional demographics.

Types of HeatMaps

Typically, there are two types of Heatmaps:

- **Grid Heatmap:** The magnitudes of values shown through colors are laid out into a matrix of rows and columns, mostly by a density-based function. Below are the types of Grid Heatmaps.
- ✓ **Clustered Heatmap:** The goal of Clustered Heatmap is to build associations between both the data points and their features. This type of heatmap implements clustering as part of the process of grouping similar features. Clustered Heatmaps are widely used in biological sciences for studying gene similarities across individuals.

The order of the rows in Clustered Heatmap is determined by performing hierarchical cluster analysis of the rows. Clustering positions similar rows together on the map. Similarly, the order of the columns is determined.

✓ **Correlogram:** A correlogram replaces each of the variables on the two axes with numeric variables in the dataset. Each square depicts the relationship between the two intersecting variables, which helps to build descriptive or predictive statistical models.

- **Spatial Heatmap:** Each square in a Heatmap is assigned a color representation according to the nearby cells' value. The location of color is according to the magnitude of the value in that particular space. These Heatmaps are data-driven “paint by numbers” canvas overlaid on top of an image. The cells with higher values than other cells are given a hot color, while cells with lower values are assigned a cold color.

Correlation statistics:

Correlation

➤ Correlation measures the relationship between two variables.

We mentioned that a function has a purpose to predict a value, by converting input (x) to output ($f(x)$). We can also say that a function uses the relationship between two variables for prediction.

Correlation Coefficient

The correlation coefficient measures the relationship between two variables.

The correlation coefficient can never be less than -1 or higher than 1.

- 1 = there is a perfect linear relationship between the variables (like Average_Pulse against Calorie_Burnage)
- 0 = there is no linear relationship between the variables
- -1 = there is a perfect negative linear relationship between the variables (e.g. Less hours worked, leads to higher calorie burnage during a training session)

Example of a Perfect Linear Relationship (Correlation Coefficient = 1)

- We will use scatterplot to visualize the relationship between Average_Pulse and Calorie_Burnage (we have used the small data set of the sports watch with 10 observations).
- This time we want scatter plots, so we change kind to "scatter":

#Three lines to make our compiler able to draw:

```
import sys
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
health_data = pd.read_csv("data.csv", header=0, sep=",")
```

```
health_data.plot(x='Average_Pulse', y='Calorie_Burnage', kind='scatter'),
```

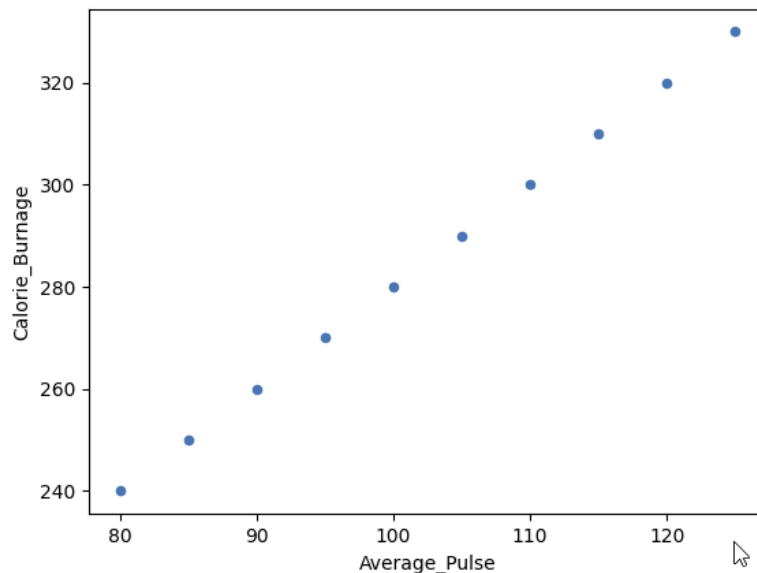
```
plt.show()
```

#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
```

```
sys.stdout.flush()
```

output:



Example of a Perfect Negative Linear Relationship (Correlation Coefficient = -1)

#Three lines to make our compiler able to draw:

```
import sys
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
negative_corr = {'Hours_Work_Before_Training': [10,9,8,7,6,5,4,3,2,1],
```

```
'Calorie_Burnage': [220,240,260,280,300,320,340,360,380,400]}
```

```
negative_corr = pd.DataFrame(data=negative_corr)
```

```
negative_corr.plot(x='Hours_Work_Before_Training',y='Calorie_Burnage',  
kind='scatter')
```

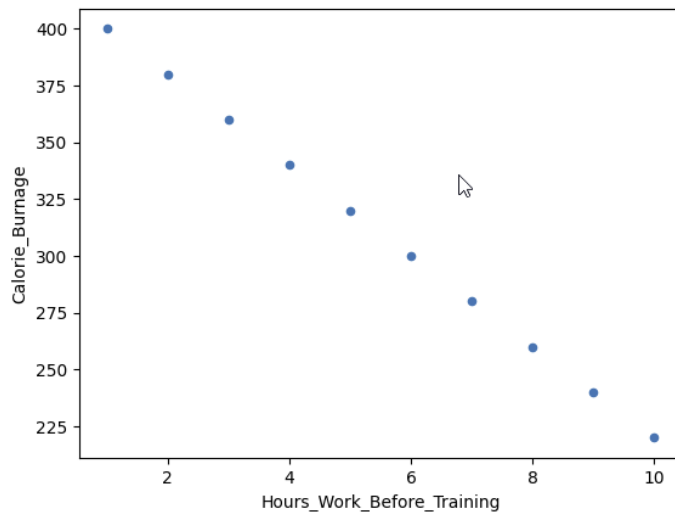
```
plt.show()
```

#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
```

```
sys.stdout.flush()
```

output:



We have plotted fictional data here. The x-axis represents the amount of hours worked at our job before a training session. The y-axis is Calorie_Burnage.

If we work longer hours, we tend to have lower calorie burnage because we are exhausted before the training session.

The correlation coefficient here is -1.

Example of No Linear Relationship (Correlation coefficient = 0)

#Three lines to make our compiler able to draw:

```
import sys
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
full_health_data = pd.read_csv("data.csv", header=0, sep=",")
```

```
full_health_data.plot(x='Duration', y='Max_Pulse', kind='scatter')
```

```
plt.show()
```

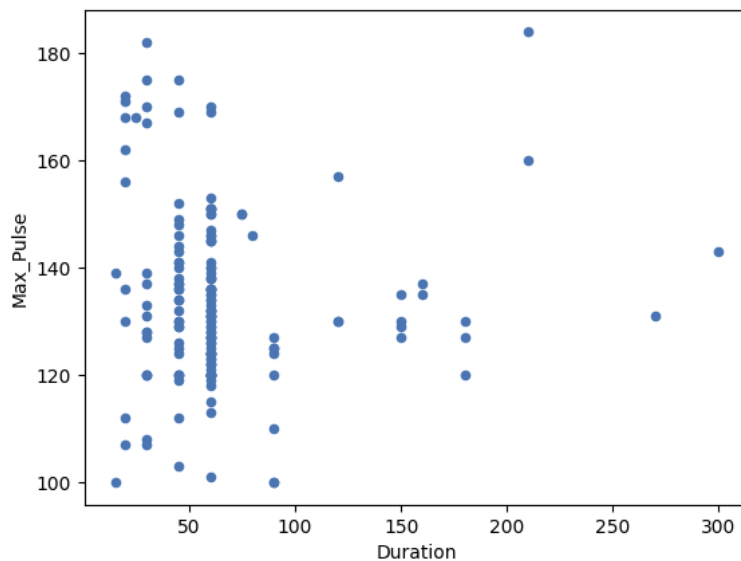
#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
```

```
sys.stdout.flush()
```

output:





Population is all elements in a group.

- a **population** is a collection of objects, items (“units”) about which information is sought;

For example,

- College students in US is a population that includes all of the college students in US.
- 25-year-old people in Europe is a population that includes all of the people that fits the description.

It is not always feasible or possible to do analysis on population because we cannot collect all the data of a population. Therefore, we use samples.

Sample is a subset of a population that is observed.

Variables

A variable is an attribute of an object of study that may vary for different cases. Thus, a variable varies for different case studies in research. Considering the previous example

of the survey on the temperature of many cities worldwide on the same day, the variables are "Temperature" and "City" because both the attributes vary for different cases.

Now variables can be of two types.

1. **Numerical variable:** They represent values that have numbers. For Example, age, weight, height.

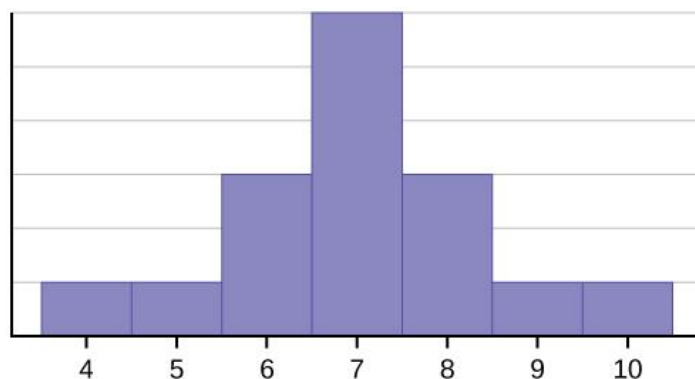
2. **Categorical variable**

These variables represent values that have words, for example, name, nationality, sport, etc. For our previous example of the survey, "Temperature" is a numerical variable.

Consider the following data set.

4; 5; 6; 6; 6; 7; 7; 7; 7; 7; 7; 8; 8; 8; 8; 9; 10

This data set can be represented by following histogram. Each interval has width one, and each value is located in the middle of an interval.



For example,

- 1000 college students in US is a subset of “college students in US” population.

When we compare two samples (or groups), we can use **t-test** to see if there is any difference in means of groups. When we have more than two groups, t-test is not the optimal choice because we need to apply t-test to pairs separately. Consider we have groups A, B and C. To be able to compare the means, we need to apply a t-test to A-B, A-C and B-C. As the number of groups increase, this becomes harder to manage.

In the case of comparing three or more groups, ANOVA is preferred. There are two elements of

ANOVA:

ANOVA stands for analysis of variance and, as the name suggests, it helps us understand and compare variances among groups. Before going in detail about ANOVA, let's remember a few terms in statistics:

- **Mean:** The average of all values.
- **Variance:** A measure of the variation among values. It is calculated by adding up squared differences of each value and the mean and then dividing the sum by the number of samples.

$$Variance = \frac{\sum(x_i - \text{mean})^2}{N}$$

X_i = value i

N = number of values

Mean = average of all values

- **Standard deviation:** The square root of variance.

In order to understand the motivation behind ANOVA, or some other statistical tests, we should learn two simple terms: population and sample.

- Variation within each group
- Variation between groups

ANOVA test result is based on F ratio which is the ratio of the variation between groups to the variation within groups.

$$F = \frac{\text{Variation between groups}}{\text{Variation within groups}}$$

F ratio shows how much of the total variation comes from the variation between groups and how much comes from the variation within groups.

EXERCISES:

Use NumPy to:

- 1) Create an arbitrary one-dimensional array called "v".

```
import numpy as np
```

```
v= np.array([3,8,12,18,7,11,30])
```

- 2) Create a new array which consists of the odd indices of previously created array "v".

```
odd_elements = v[1::2]
```

- 3) Create a new array in backwards ordering from v.

```
reverse_order = v[::-1]
```

- 4) What will be the output of the following code:

```
a = np.array([1, 2, 3, 4, 5])
```

```
b = a[1:4]
```

```
b[0] = 200
```

```
print(a[1])
```

The output will be 200, because slices are views in numpy and not copies.

5) Create a two-dimensional array called "m".

```
m = np.array([ [11, 12, 13, 14], [21, 22, 23, 24], [31, 32, 33, 34] ])
```

6) Create a new array from m, in which the elements of each row are in reverse order.

```
m[:,::-1]
```

7) Another one, where the rows are in reverse order.

```
m[::-1]
```

8) Create an array from m, where columns and rows are in reverse order.

```
m[::-1,::-1]
```

9) Cut off the first and last row and the first and last column

```
m[1:-1,1:-1]
```

10) Define an int16 data type and call this type i16. The elements of the list 'lst' are turned into i16 types to create the two-dimensional array A.

```
import numpy as np
```

```
i16 = np.dtype(np.int16)
```

```
print(i16)
```

```
lst = [ [3.4, 8.7, 9.9], [1.1, -7.8, -0.7], [4.1, 12.3, 4.8] ]
```

```
A = np.array(lst, dtype=i16)
```

```
print(A)
```

```
import numpy as np
```

```
dt = np.dtype([('country', 'S20'), ('density', 'i4'), ('area', 'i4'), ('population', 'i4')])

population_table = np.array([
    ('Netherlands', 393, 41526, 16928800),
    ('Belgium', 337, 30510, 11007020),
    ('United Kingdom', 256, 243610, 62262000),
    ('Germany', 233, 357021, 81799600),
    ('Liechtenstein', 205, 160, 32842),
    ('Italy', 192, 301230, 59715625),
    ('Switzerland', 177, 41290, 7301994),
    ('Luxembourg', 173, 2586, 512000),
    ('France', 111, 547030, 63601002),
    ('Austria', 97, 83858, 8169929),
    ('Greece', 81, 131940, 11606813),
    ('Ireland', 65, 70280, 4581269),
    ('Sweden', 20, 449964, 9515744),
    ('Finland', 16, 338424, 5410233),
    ('Norway', 13, 385252, 5033675)],
    dtype=dt)

print(population_table[:4])
```

```
In [1]: import numpy as np
dt = np.dtype([('country', 'S20'), ('density', 'i4'), ('area', 'i4'), ('population', 'i4')])
population_table = np.array([
('Netherlands', 393, 41526, 16928800),
('Belgium', 337, 30510, 11007020),
('United Kingdom', 256, 243610, 62262000),
('Germany', 233, 357021, 81799600),
('Liechtenstein', 205, 160, 32842),
('Italy', 192, 301230, 59715625),
('Switzerland', 177, 41290, 7301994),
('Luxembourg', 173, 2586, 512000),
('France', 111, 547030, 63601002),
('Austria', 97, 83858, 8169929),
('Greece', 81, 131940, 11606813),
('Ireland', 65, 70280, 4581269),
('Sweden', 20, 449964, 9515744),
('Finland', 16, 338424, 5410233),
('Norway', 13, 385252, 5033675)],
dtype=dt)
print(population_table[:4])
```

```
[(b'Netherlands', 393, 41526, 16928800)
(b'Belgium', 337, 30510, 11007020)
(b'United Kingdom', 256, 243610, 62262000)
(b'Germany', 233, 357021, 81799600)]
```

```
print(population_table['density'])

print(population_table['country'])

print(population_table['area'][2:5])
```

```
In [2]: print(population_table['density'])
print(population_table['country'])
print(population_table['area'][2:5])
```

```
[393 337 256 233 205 192 177 173 111  97  81  65  20  16  13]
[b'Netherlands' b'Belgium' b'United Kingdom' b'Germany' b'Liechtenstein'
 b'Italy' b'Switzerland' b'Luxembourg' b'France' b'Austria' b'Greece'
 b'Ireland' b'Sweden' b'Finland' b'Norway']
[243610 357021    160]
```

```
In [ ]:
```

12) Attendance of all classes of a school are as follows find their skewness?
1st (35), 2nd(32), 3rd(38), 4th(39), 5th(43)

lass Name	Number of students
1 st	35
2 nd	32
3 rd	38
4 th	39
5 th	45

Q) What does positive skewness mean?

Answer:

Positive skewness means the distribution is skewed to the right. There are more values on the right side of the mean than on the left side.

Condition for Positive Skewness = Mean > median > mode

Q: What does negative skewness mean?

Answer:

Negative skewness means the distribution is skewed to the left. There are more values on the left side of the mean than on the right side.

Condition for negative skewness = Mode > median > mean

Q: What is the formula for calculating skewness?

$$g_1 = \frac{1}{n} \frac{\sum_i (x_i - \mu)^3}{\sigma^3},$$

Chapter Twelve– Regression Analysis

Regression analysis is a predictive modelling technique that assesses the relationship between **dependent** (i.e., the goal/target variable) and **independent** factors. Forecasting, time series modelling, determining the relationship between variables, and predicting continuous values can all be done using regression analysis. Just to give you an Analogy, Regression is the best way to study the relationship between household areas and a driver's household electricity cost.

Regression has 2 Categories Namely,

- **Simple Linear Regression:** The association between two variables is established using a straight line in Simple Linear Regression. It tries to create a line that is as near to the data as possible by determining the slope and intercept, which define the line and reduce regression errors. There is a single x and y variable

Equation: $Y = mX + c$

- **Multiple Linear Regression:** Multiple linear regressions are based on the presumption that both the dependent and independent variables, or Predictor and Target variables, have a linear relationship. There are two types of multilinear regressions: linear and nonlinear. It has one or more x variables and one or more y variables, or one dependent variable and two or more independent variables

- Equation: $Y = m_1X_1 + m_2X_2 + m_3X_3 + \dots c$

- Where,

- Y = Dependent Variable

m = Slope

X = Independent Variable

c = Intercept

- Now, let us understand both Simple and Multiple Linear Regression implementation with the below sample datasets!!

Simple Linear Regression

Given the experience let us predict the salary of the employees using a simple Regression model

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

Read the data from the csv file

```
data = pd.read_csv('/content/Salary_Data.csv') #reading data
```

```
data.head()
```

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

➤ Plot Years of Experience vs Salary based on Experience

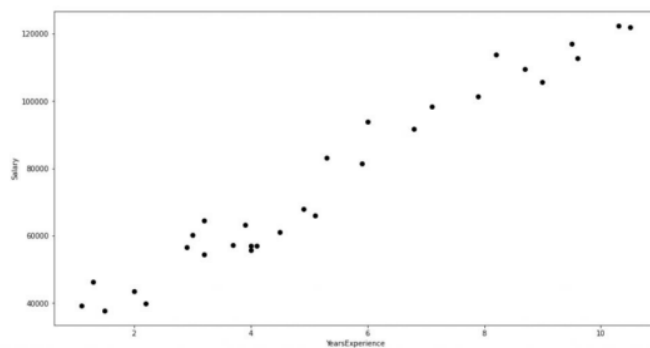
```
plt.figure()
```

```
plt.scatter(data['YearsExperience'],data['Salary'],c='black')
```

```
plt.xlabel("YearsExperience")
```

```
plt.ylabel("Salary")
```

```
plt.show()
```



Reshape and fit the data into simple linear regression

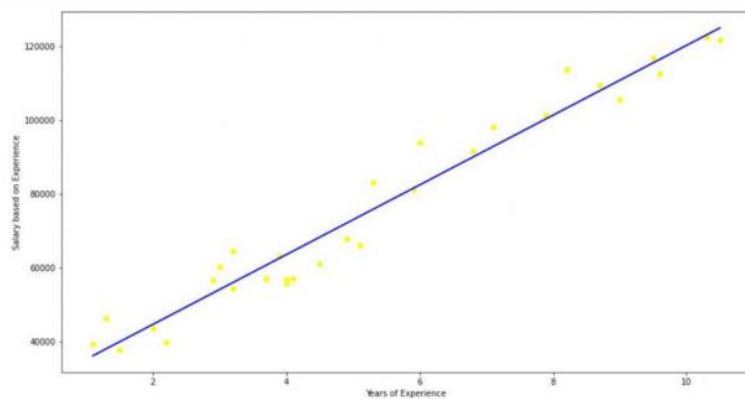
```
X = data['YearsExperience'].values.reshape(-1,1)
```

```
y = data['Salary'].values.reshape(-1,1)
```

```
reg = LinearRegression()
```

```
reg.fit(X, y)
```

Print R-squared value



Multiple Linear Regression

Let us the dataset set of advertising sales based on Tv and Newspaper

Import required libraries

```
%matplotlib inline
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
plt.rcParams['figure.figsize'] = (15.0, 8.0)
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error
```

```
from sklearn.metrics import r2_score
```

Read dataset from csv file

```
data = pd.read_csv('/content/Advertising Agency.csv') #reading data
```

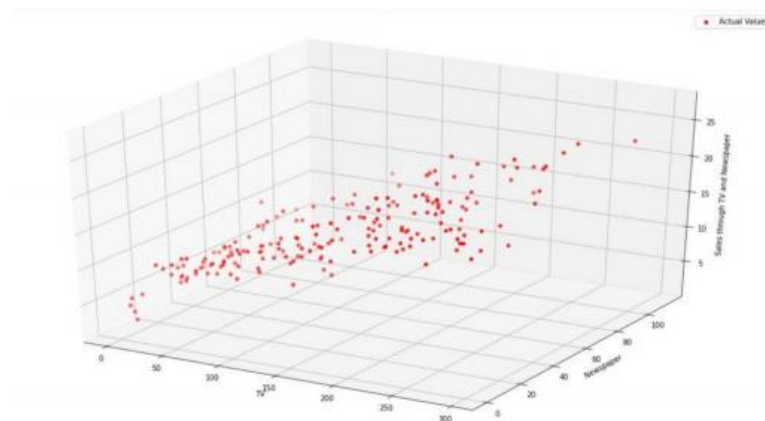
```
data.head()
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```
tv = data['TV'].values #storing dataframe values as variables
```

```
newspaper = data['Newspaper'].values
```

```
sales = data['Sales'].values
```



Plotting Actual vs Predicted values

```
fig = plt.figure(2)

ax = Axes3D(fig)

ax.scatter(X_train[:,0], X_train[:,1], y_train, color='r', label='Actual Values')

ax.scatter(X_test[:,0], X_test[:,1], y_pred, color='b', label='Predicted Values')

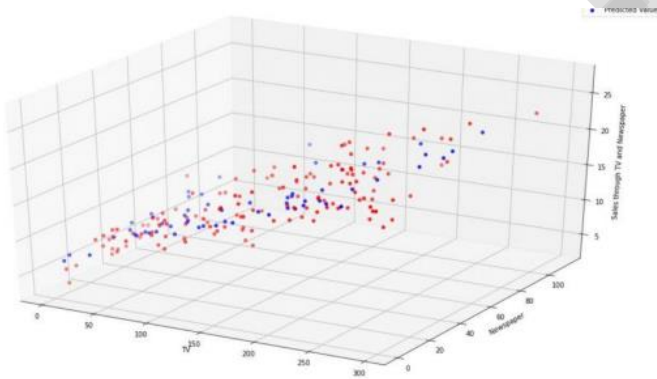
ax.set_xlabel('TV')

ax.set_ylabel('Newspaper')

ax.set_zlabel('Sales through TV and Newspaper')

ax.legend()

plt.show()
```



Multiple regression in machine learning:

Multiple Linear Regression is one of the important regression algorithms which models the linear relationship between a **single dependent continuous variable** and **more than one independent variable**.

Example:

Prediction of CO₂ emission based on engine size and number of cylinders in a car.

MLR equation:

In Multiple Linear Regression, the target variable(Y) is a linear combination of multiple predictor variables $x_1, x_2, x_3, \dots, x_n$. Since it is an enhancement of Simple Linear Regression, so the same is applied for the multiple linear regression equation, the equation becomes:

$$Y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Where,

Y= Output/Response variable

$b_0, b_1, b_2, b_3, b_n, \dots$ = Coefficients of the model

$x_1, x_2, x_3, x_4, \dots$ = Various Independent/feature variable

Data Science and Supervised and unsupervised Learning

Decision Tree

GENERALIZATION ERROR

EVALUATION METRICS

Model Evaluation Metrics define the evaluation metrics for evaluating the performance of a machine learning model, which is an integral component of any data science project. It aims to estimate the generalization accuracy of a model on the future (unseen/out-of-sample) data.

A **confusion matrix** is a matrix representation of the prediction results of any binary testing that is often used to describe the performance of the classification model (or “classifier”) on a set of test data for which the true values are known.

The confusion matrix itself is relatively simple to understand, but the related terminology can be confusing.

		Predicted Values	
		Negative	Positive
Actual Values	Negative	TN True Negative	FP False positive
	Positive	FN False Negative	TP True Positive

Each prediction can be one of the four outcomes, based on how it matches up to the actual value:

- True Positive (TP): Predicted True and True in reality.

- True Negative (TN): Predicted False and False in reality.
- False Positive (FP): Predicted True and False in reality.
- False Negative (FN): Predicted False and True in reality

Now let us understand this concept using hypothesis testing.

A Hypothesis is speculation or theory based on insufficient evidence that lends itself to further testing and experimentation. With further testing, a hypothesis can usually be proven true or false.

A **Null Hypothesis** is a hypothesis that says there is no statistical significance between the two variables in the hypothesis. It is the hypothesis that the researcher is trying to disprove.

We would always reject the null hypothesis when it is false, and we would accept the null hypothesis when it is indeed true.

Even though hypothesis tests are meant to be reliable, there are two types of errors that can occur.

These errors are known as Type 1 and Type II errors.

For example, when examining the effectiveness of a drug, the null hypothesis would be that the drug does not affect a disease.

Type I Error:- equivalent to False Positives(FP).

The first kind of error that is possible involves the rejection of a null hypothesis that is true.

Let's go back to the example of a drug being used to treat a disease. If we reject the null

hypothesis in this situation, then we claim that the drug does have some effect on a disease. But if the null hypothesis is true, then, in reality, the drug does not combat the disease at all. The drug is falsely claimed to have a positive effect on a disease.

Type II Error:- equivalent to False Negatives(FN).

The other kind of error that occurs when we accept a false null hypothesis. This sort of error is

called a type II error and is also referred to as an error of the second kind.

CROSS VALIDATION

Cross validation is a technique for assessing how the statistical analysis generalises to an independent data set. It is a technique for evaluating machine learning models by training several models on subsets of the available input data and evaluating them on the complementary subset of the data. Using cross-validation, there are high chances that we can detect over-fitting with ease.

K-Fold Cross Validation

First I would like to introduce you to a golden rule — “*Never mix training and test data*”. Your first step should always be to **isolate the test data-set** and use it only for final evaluation. Cross-validation will thus be performed on the training set.

Split 1	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 1
Split 2	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 2
Split 3	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 3
Split 4	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 4
Split 5	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 5

Training data

Test data

Initially, the entire training data set is broken up in k equal parts. The first part is kept as the hold out (testing) set and the remaining $k-1$ parts are used to train the model. Then the trained model is then tested on the holdout set. The above process is

repeated k times, in each case we keep on changing the holdout set. Thus, every data point get an equal opportunity to be included in the test set.

Usually, k is equal to 3 or 5. It can be extended even to higher values like 10 or 15 but it becomes extremely computationally expensive and time-consuming. Let us have a look at how we can implement this with a few lines of Python code and the Sci-kit Learn API

```
from sklearn.model_selection import cross_val_score  
  
print(cross_val_score(model, X_train, y_train, cv=5))
```

We pass the **model** or classifier object, the features, the labels and the parameter **cv** which indicates the **K** for K-Fold cross-validation. The method will return a list of k accuracy values for each iteration. In general, we take the average of them and use it as a consolidated cross-validation score.

```
import numpy as np  
  
print(np.mean(cross_val_score(model, X_train, y_train, cv=5)))
```

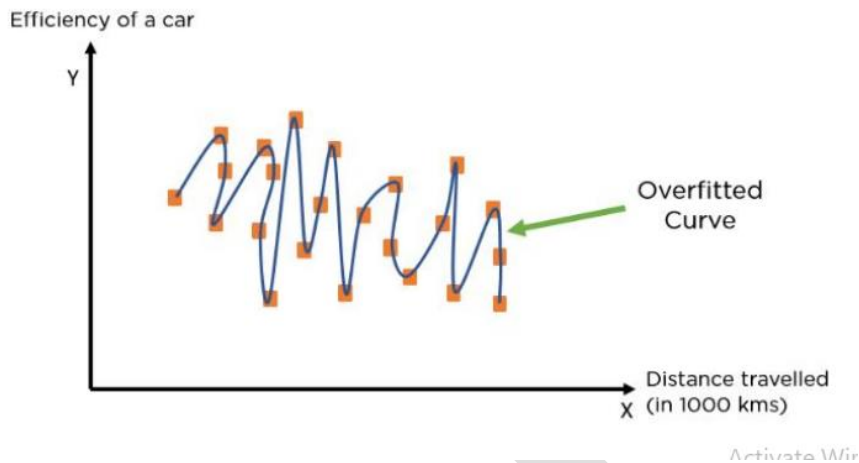
Although it might be computationally expensive, cross-validation is essential for evaluating the performance of the learning model.

Overfitting and Underfitting :

What is Overfitting?

When a model performs very well for training data but has poor performance with test data (new data), it is known as overfitting. In this case, the machine learning model learns the details and noise in the training data such that it negatively affects the performance of the model on test data.

Overfitting can happen due to low bias and high variance.



Reasons for Overfitting

- Data used for **training** is not cleaned and contains noise (garbage values) in it
- The model has a high variance
- The size of the training dataset used is not enough
- The model is too complex

Ways to Tackle Overfitting

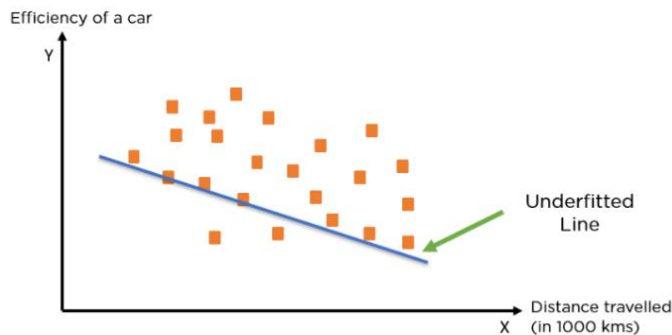
- Using K-fold cross-validation
- Using Regularization techniques such as Lasso and Ridge
- Training model with sufficient data
- Adopting ensembling techniques

What is Underfitting?

When a model has not learned the patterns in the training data well and is unable to generalize

well on the new data, it is known as underfitting. An underfit model has poor performance on the

training data and will result in unreliable predictions. Underfitting occurs due to high bias and low variance.



Reasons for Underfitting

- Data used for training is not cleaned and contains noise (garbage values) in it
- The model has a high bias
- The size of the training dataset used is not enough
- The model is too simple

Ways to Tackle Underfitting

- Increase the number of features in the dataset
- Increase model complexity
- Reduce noise in the data
- Increase the duration of training the data

Ridge Regression

Ridge regression is a model tuning method that is used to analyse any data that suffers from multicollinearity. This method performs L2 regularization. When the issue of multicollinearity occurs, least-squares are unbiased, and variances are large, this results in predicted values being far away from the actual values.

The cost function for ridge regression:

$$\text{Min}(\|Y - X(\theta)\|^2 + \lambda \|\theta\|^2)$$

Lambda is the penalty term. λ given here is denoted by an alpha parameter in the ridge function.

Ridge Regression Models

For any type of regression machine learning model, the usual regression equation forms the base which is written as:

$$Y = XB + e$$

Where Y is the dependent variable, X represents the independent variables, B is the regression coefficients to be estimated, and e represents the errors or residuals.

Ridge Regression Predictions

We now show how to make predictions from a Ridge regression model. In particular, we will make predictions based on the Ridge regression model created for Example 1 with $\lambda = 1.6$.

The raw input data is repeated in range A1:E19 of Figure 1 and the unstandardized regression coefficients calculated in Figure 2 of Ridge Regression Analysis Tool is repeated in range G2:H6 of Figure 1.

	A	B	C	D	E	F	G	H	I	J	K
1	X1	X2	X3	X4	Y			coeff		Pred	Res
2	3	6	2	8	3		Intercept	11.42071		13.48634	-10.4863
3	7	7	11	14	15		X1	0.264684		18.20573	-3.20573
4	11	11	23	33	19		X2	0.315482		22.73489	-3.73489
5	15	12	26	34	27		X3	0.508153		25.42886	1.571137
6	21	16	12	5	23		X4	-0.2047		27.10111	-4.10111
7	23	17	16	10	23					28.95506	-5.95506
8	28	22	22	15	31		MSE	42.68541		33.8813	-2.8813
9	31	16	28	24	39		Lambda	1.6		33.98906	5.010938
10	38	21	34	31	47					39.03526	7.964736
11	39	27	27	8	51					42.34392	8.656083
12	42	24	31	16	47					42.58652	4.413481
13	49	32	40	25	51					49.69422	1.305778
14	57	29	42	21	55					52.70036	2.299637
15	68	36	35	9	63					56.71961	6.280387
16	71	42	39	15	67					60.21096	6.789043
17	89	51	51	23	71					72.27483	-1.27483
18	95	53	60	29	71					77.83906	-6.83906
19	97	55	68	40	75					80.8129	-5.8129
20											554.9103
21	50	20	30	25	41.09159						
22	30	30	20	20	34.89471						

The predictions for the input data are shown in column J. In fact, the values in range J2:J19 can be

calculated by the array formula

=H2+MMULT(A2:D19,H3:H6).

Alternatively, they can be calculated by the array formula

=RidgePred(A2:D19,A2:D19,E2:E19,H9)

Real Statistics Function: The Real Statistics Resource Pack provides the following functions.

RidgeMSE(Rx, Ry, *lambda*) = MSE of the Ridge regression defined by the x data in Rx, y data in Ry and the given lambda value.

RidgePred(Rx0, Rx, Ry, *lambda*): returns an array of predicted y values for the x data in range Rx0 based on the Ridge regression model defined by Rx, Ry and *lambda*; if Rx0 contains only one row then only one y value is returned.

References

- 1- Joel Grus, “Data Science from Scratch First Principals with Python”, 2nd edition, O’REILLY, USA, 2019.
- 2- Laura Lagual and Santi Segui, “**Introduction to Data Science** A Python Approach to Concepts, Techniques and Applications”, Springer 2017

